Constructor Fabric

A Practical Guide for Students and Practitioners

Giancarlo Succi Alexander Tormasov Sergey Ulasen

2026

Contents

Preface iii

I Using Constructor Fabric 1

1 Getting Started 2
1.1 What Constructor Fabric Is . . . 2
1.1.1 The three parts, and the problem all of this solves . . . 3
1.1.2 The four pieces . . . 3
1.1.3 What each side is for . . . 4
1.1.4 When it is overhead . . . 4
1.2 Installing . . . 5
1.2.1 What you need first . . . 5
1.2.2 Installing the command line tool . . . 5
1.2.3 If `pipx` itself is missing . . . 6
1.2.4 Connecting a repository . . . 6
1.2.5 What ends up in the repository . . . 7
1.2.6 When install goes wrong . . . 7
1.3 The SDLC Kit . . . 7
1.3.1 Other places a kit can come from . . . 8
1.3.2 Keeping kits current . . . 8
1.4 Talking To It . . . 8
1.4.1 One rule that saves you an hour . . . 8
1.4.2 Turning it on in chat . . . 8
1.4.3 The naming convention . . . 9
1.4.4 Aliases and sub-routes . . . 9
1.4.5 A request that works, and one that wastes your afternoon . . . 10
1.4.6 Gates you will be asked about . . . 10
1.5 The Ten Stages at a Glance . . . 11
1.6 Validating . . . 12
1.6.1 `cfs validate` . . . 12
1.6.2 The smaller checks . . . 12
1.6.3 Looking at the graph . . . 13
1.6.4 The loop . . . 13
1.6.5 What the loop does not give you . . . 13
1.7 Habits and Traps . . . 13
1.7.1 Habits that pay off . . . 13
1.7.2 Traps . . . 14
1.7.3 Two traps worth a closer look . . . 14
1.8 More Than One Repository . . . 15
1.9 Your First Thirty Minutes . . . 15

Exercises . . . 15

2 Choosing and Setting Up a Free or Cheap LLM **17**
2.1 Where Does the Model Come From? . . . 17
2.2 The Five Hosts . . . 17
2.2.1 So the real question . . . 18
2.2.2 Four ways to pay nothing . . . 18
2.3 Frontier APIs . . . 18
2.3.1 Google Gemini: the one standing free API . . . 18
2.3.2 Anthropic: free chat, not free API . . . 18
2.3.3 OpenAI: the same story . . . 19
2.3.4 Mistral: a free tier that means it . . . 19
2.3.5 The frontier providers side by side . . . 19
2.4 Open Weights on Your Own Machine . . . 19
2.4.1 Which model to try . . . 19
2.4.2 Ollama in four commands . . . 20
2.4.3 The local server speaks a familiar dialect . . . 20
2.4.4 LM Studio . . . 20
2.4.5 How much laptop do you need . . . 20
2.4.6 Why an Apple Silicon Mac wins here . . . 21
2.4.7 What you give up by going local . . . 21
2.5 Free Cloud Inference . . . 21
2.6 Student Programmes, and Why This Section Names a Country . . . 22
2.6.1 GitHub Student Developer Pack — yes . . . 22
2.6.2 Google AI Plus for students — read this twice . . . 22
2.6.3 OpenAI student programmes — no . . . 22
2.6.4 Azure for Students and AWS Educate — no . . . 22
2.7 Three Setups Worth Doing . . . 23
2.7.1 Path A: Copilot through the Student Pack . . . 23
2.7.2 Path B: Claude Code on a model you own . . . 23
2.7.3 Codex, or any other OpenAI-API host, on the same local model . . . 24
2.7.4 Path C: a Gemini or Mistral key . . . 24
2.7.5 Which one should you pick . . . 25
2.8 Matching the Model to the Operation . . . 25
2.9 These Numbers Will Rot . . . 26
Exercises . . . 26

3 The Full Pipeline, Walked Through **28**
3.1 The Project . . . 28
3.1.1 Registering the system . . . 28
3.2 Stages 1 and 2: Explore, Then Open the Options . . . 29
3.3 Stage 3: The Product Requirements Document . . . 30
3.3.1 Actors get identifiers too . . . 30
3.3.2 A functional requirement . . . 30
3.3.3 Non-functional requirements, with IDs . . . 31
3.3.4 How you know it worked . . . 31
3.3.5 And the parts that admit what is missing . . . 31
3.3.6 Review, then check . . . 32
3.4 Stage 4: One Decision, Written Down . . . 32
3.5 Stage 5: The DESIGN Document . . . 33
3.5.1 Drivers: requirement in, solution out . . . 33
3.5.2 Three layers . . . 34

3.5.3 Principles and constraints, also with IDs . . . 34
3.5.4 Domain model, components, sequences, data . . . 35
3.6 Stage 6: Decompose . . . 36
3.7 Stage 7: The FEATURE Specification . . . 37
3.7.1 Context: pinning the numbers down . . . 38
3.7.2 An actor flow in CDSL . . . 38
3.7.3 The algorithm . . . 38
3.7.4 The state machine . . . 39
3.7.5 The definition of done . . . 39
3.8 Stage 8: The Code, and Its Markers . . . 40
3.8.1 The marker vocabulary . . . 40
3.8.2 The domain model . . . 41
3.8.3 The algorithm, marked line by line . . . 42
3.8.4 A state machine in code . . . 43
3.8.5 The tick loop . . . 44
3.8.6 Where the DoD markers land . . . 45
3.8.7 Config defaults, discharged . . . 46
3.8.8 The unmarked boundary . . . 46
3.9 Following One Identifier . . . 47
3.10 Stages 9 and 10: Review and the Whole Picture . . . 48
3.10.1 Reviewing the code . . . 48
3.10.2 Deterministic validation . . . 48
3.10.3 The dependency map . . . 49
3.10.4 Asking about one identifier . . . 49
3.10.5 And the smaller checks . . . 49
3.11 What This Example Is, and Is Not . . . 50
Exercises . . . 50

II How Constructor Fabric Is Built 51

4 Studio and PDSL 52
4.1 The Federation . . . 52
4.2 What Studio Is . . . 53
4.2.1 Two responsibilities, two philosophies . . . 53
4.2.2 Exactly two workflows . . . 53
4.2.3 One canonical source of control flow . . . 54
4.2.4 Handoff as an explicit envelope . . . 54
4.3 The Kit System . . . 54
4.3.1 The kit model that got deleted . . . 55
4.3.2 Four levels of extensibility . . . 55
4.3.3 Manifest formats: three plus one . . . 56
4.3.4 Resolving a kit: version authority . . . 56
4.3.5 Display version versus content identity . . . 57
4.3.6 The mirror override layer . . . 57
4.4 The Four Layers That Are Not Layers . . . 57
4.4.1 What “Layered Systems” actually requires . . . 58
4.4.2 The verdict . . . 58
4.4.3 Why this is worth four pages . . . 58
4.4.4 To be fair to Studio . . . 59
4.4.5 A promise for the next chapter . . . 59
4.5 PDSL . . . 59

4.5.1 What it is for . . . 60
4.5.2 Design goals, as written . . . 60
4.5.3 The core shape . . . 60
4.5.4 The keyword set . . . 61
4.5.5 A real unit . . . 61
4.5.6 Compile once, adapt per host: the first sighting . . . 63
4.5.7 The same motif, somewhere unrelated . . . 63
4.5.8 When is an LLM allowed to decide anything? . . . 63
4.5.9 One hundred and fifty routers . . . 64
4.6 The Missing Interpreter . . . 64
4.6.1 What PDSL-aware code exists . . . 64
4.6.2 So where does execution happen? . . . 65
4.6.3 Judging it against the criteria . . . 65
4.6.4 A useful contrast . . . 66
4.6.5 What the absence actually costs . . . 66
4.6.6 The organisation's own self-assessment . . . 66
4.6.7 Open questions . . . 67
4.7 Where That Leaves Us . . . 67
Exercises . . . 67

5 Gears, the Plugin Runtime 69
5.1 What Gears Is . . . 69
5.1.1 The problem being solved . . . 69
5.2 Capabilities and the Registry . . . 70
5.2.1 The capability set . . . 70
5.2.2 A capability trait, for real . . . 70
5.2.3 Advice buried in a doc comment . . . 71
5.2.4 The registry's view of a capability . . . 71
5.2.5 Type-safe querying, without downcasting . . . 71
5.2.6 The capability set type . . . 72
5.2.7 How a gear declares itself . . . 72
5.2.8 Link-time discovery . . . 73
5.2.9 The price of cleverness . . . 74
5.2.10 Keeping the domain layer clean . . . 74
5.3 The Host Runtime Lifecycle . . . 74
5.3.1 pre_init and migrations . . . 75
5.3.2 init, proxy-wiring, post_init . . . 75
5.3.3 rest, grpc, start, stop . . . 75
5.3.4 One ordering constraint, spelled out . . . 76
5.3.5 Stopping properly . . . 76
5.4 Is This Template Method? . . . 76
5.4.1 The same mistake, twice more . . . 76
5.5 ClientHub: Telling Patterns Apart . . . 77
5.5.1 ClientHub, in its own words . . . 77
5.5.2 Candidate one: Singleton . . . 77
5.5.3 Candidate two: Registry . . . 78
5.5.4 Candidate three: Service Locator . . . 78
5.5.5 How the claim got confirmed . . . 78
5.5.6 Scopes . . . 78
5.5.7 Why this exercise is worth your time . . . 78
5.6 Gears-frontx . . . 79

5.6.1 Plugin composition, frontend edition . . . 79
5.6.2 What .build() does, in order . . . 80
5.6.3 A second composition axis . . . 80
5.6.4 The layered SDK, and whether it is real . . . 80
5.6.5 The callback . . . 81
5.6.6 The EventBus . . . 81
5.6.7 Observer, not Mediator . . . 82
5.6.8 A doc comment that is not true . . . 82
5.7 Federation at a Different Granularity . . . 82
5.7.1 A default that got reversed . . . 83
5.7.2 The directory service . . . 83
5.7.3 Two careful details in that API . . . 83
5.7.4 Where the routes get served . . . 84
5.7.5 API versioning at the type level . . . 84
5.7.6 Trait inheritance, considered and rejected . . . 84
5.7.7 Backward compatibility, enforced by CI . . . 84
5.8 Two More Frontend Details Worth Keeping . . . 84
5.8.1 The blob URL trick . . . 84
5.8.2 Publishing nine packages . . . 85
5.9 Where That Leaves Us . . . 85
Exercises . . . 85

6 Insight, Data and Lineage 87
6.1 What Insight Is . . . 87
6.1.1 Why a data platform belongs in an architecture study . . . 87
6.2 Connectors . . . 88
6.2.1 A real connector manifest . . . 88
6.2.2 Four fields stamped on every record . . . 88
6.2.3 Where the engineering actually is . . . 89
6.2.4 Three tiers of connector authorship . . . 89
6.3 Bronze, Silver, Gold . . . 90
6.3.1 What actually lives in Gold . . . 90
6.3.2 Identity resolution is the hard part . . . 90
6.3.3 The macro library . . . 90
6.4 The Invariant . . . 91
6.4.1 Why the invariant exists . . . 91
6.4.2 The documented exception . . . 92
6.4.3 Tenant isolation, chosen deliberately . . . 92
6.5 Pipes and Filters, For Real . . . 92
6.5.1 Why the medallion pipeline passes . . . 93
6.5.2 Why the evidence is unusually strong . . . 93
6.5.3 Three pipelines that did not pass . . . 93
6.5.4 A style that is simply absent . . . 93
6.5.5 Naming, scored . . . 94
6.6 Traceability Markers . . . 94
6.6.1 A marker in the wild . . . 94
6.6.2 The marker family . . . 95
6.6.3 One awkward fact . . . 95
6.7 The Unifier . . . 95
6.7.1 What the documentation describes . . . 96
6.7.2 The first verification pass . . . 96

6.7.3 The second pass . . . 96
6.7.4 What actually does the normalising . . . 97
6.7.5 Three lessons, in order of importance . . . 97
6.7.6 Why report it rather than quietly fix it . . . 97
6.7.7 How to tell, quickly . . . 97
6.7.8 Names that turned out to be empty . . . 98
6.8 Where That Leaves Us . . . 98
Exercises . . . 98

7 Security and the Bigger Picture 100
7.1 The Problem, Stated Properly . . . 100
7.1.1 Three requirements from the decision record . . . 100
7.1.2 The four roles . . . 101
7.2 The Problem a Yes/No Answer Cannot Solve . . . 101
7.2.1 Five options, one chosen . . . 102
7.2.2 The idea: return a predicate, not a boolean . . . 102
7.2.3 The consequences, as written . . . 102
7.2.4 Failing closed, eleven ways . . . 103
7.2.5 One of those rules is worth a closer look . . . 103
7.3 Tenant Scoping . . . 104
7.3.1 Bypassing the scope is a build failure . . . 104
7.3.2 The empty scope . . . 104
7.3.3 Hierarchies without recursion . . . 104
7.3.4 The sentence worth framing . . . 105
7.4 Identity Across Boundaries . . . 105
7.4.1 Re-validate at every hop . . . 105
7.4.2 A vendor-neutral identity type . . . 106
7.4.3 The token is not in the type . . . 106
7.4.4 Cryptography that fails closed . . . 106
7.4.5 The security section, in one idea . . . 106
7.5 Ten Other Platforms . . . 107
7.5.1 How the comparators were chosen . . . 107
7.5.2 One rebrand, mid-study . . . 107
7.5.3 The four axes . . . 107
7.5.4 The comparison . . . 107
7.5.5 The evidence caveat, in bold . . . 108
7.5.6 The split running through the category . . . 108
7.5.7 Which side is Constructor Fabric on? . . . 109
7.5.8 Why straddling might be right . . . 109
7.6 Recurring Motifs . . . 109
7.6.1 Compile once, adapt per host: the third sighting . . . 109
7.6.2 A second convergence . . . 110
7.7 What We Learned . . . 110
7.7.1 The scoreboard . . . 110
7.7.2 Four subsystems, one design problem . . . 110
7.7.3 The habit that mattered most . . . 111
7.7.4 Five things to keep . . . 111
7.7.5 What is still open . . . 111
Exercises . . . 111

III Extending Constructor Fabric **113**

8 The Kit-Authoring Mechanism **114**
8.1 Why Anything Needs Extending . . . 114
8.1.1 Two bad options and one good one . . . 114
8.2 A Kit, Concretely . . . 115
8.2.1 Directory names are conventions, not law . . . 116
8.2.2 Where kits come from . . . 116
8.2.3 What belongs in a kit . . . 116
8.3 Register Mode Versus Copy Mode . . . 117
8.3.1 Who owns what . . . 118
8.4 `.cf-studio-kit.toml` Is the Whole Contract . . . 118
8.4.1 The smallest legal kit . . . 118
8.4.2 The kit header, field by field . . . 119
8.4.3 A resource entry, in full . . . 119
8.4.4 `source` against `install_path` . . . 120
8.4.5 What the manifest must not contain . . . 120
8.4.6 Validation rules worth memorising . . . 120
8.5 The Nine Resource Kinds . . . 121
8.5.1 `template` . . . 121
8.5.2 `checklist` . . . 122
8.5.3 `rule` . . . 122
8.5.4 `constraints` . . . 122
8.5.5 `skill` . . . 123
8.5.6 `agent` . . . 123
8.5.7 `script` and `directory` . . . 124
8.5.8 `other` . . . 124
8.6 Names and Collisions . . . 124
8.6.1 The prefix rule . . . 124
8.6.2 Opting out . . . 125
8.6.3 A naming convention worth adopting . . . 125
8.6.4 Why identifiers and not paths . . . 125
8.7 The Flow, End to End . . . 125
8.7.1 Step 1: normalize . . . 126
8.7.2 Step 2: read what it wrote . . . 126
8.7.3 Step 3: install in register mode . . . 126
8.7.4 Step 4: generate the integration files . . . 127
8.7.5 Step 5: validate . . . 127
8.7.6 One deterministic pass is not enough . . . 127
8.7.7 Updating a registered kit . . . 127
8.8 More Than One Repository . . . 128
8.9 Migrating Off the Old Model . . . 128
8.10 A Gear Is Not a Kit . . . 129
8.11 Where That Leaves Us . . . 129
Exercises . . . 130

9 A Real Kit, Walked Through **132**
9.1 The Kit's Shape . . . 132
9.1.1 First surprise: there is no `.cf-studio-kit.toml` here . . . 132
9.1.2 The root, and three counts worth checking . . . 133
9.1.3 The layout the whole kit is built on . . . 134
9.1.4 What the manifest declares . . . 135

9.1.5 The design principles, stated by the kit itself . . . 135
9.2 `RESEARCH-REQUEST`, Top to Bottom . . . 136
9.2.1 The template . . . 137
9.2.2 The rules . . . 138
9.2.3 The omissions unit, and the value of an identifier . . . 140
9.2.4 The validation unit . . . 140
9.2.5 Error handling, and asking instead of guessing . . . 141
9.2.6 The workflow . . . 141
9.2.7 The six steps . . . 142
9.2.8 The validation criteria, restated as a checklist . . . 143
9.2.9 The deterministic contract . . . 144
9.3 Installing It . . . 145
9.3.1 The development gate . . . 145
9.4 The Deterministic Half . . . 146
9.4.1 The CLI . . . 147
9.4.2 One coverage check, in full . . . 147
9.4.3 Where a checker beats a rule . . . 148
9.5 What to Take From This . . . 148
Exercises . . . 149

10 Building Your Own Kit 151
10.1 Step 0: Decide Whether You Need a Kit . . . 151
10.2 Step 1: Name the Artifact Kinds . . . 153
10.3 Step 2: Choose the Resource Kinds . . . 153
10.4 Step 3: Write the Manifest . . . 154
10.5 Step 4: The Four Files, Per Artifact Kind . . . 154
10.5.1 `template.md` . . . 154
10.5.2 `rules.md` . . . 155
10.5.3 `checklist.md` . . . 155
10.5.4 `examples/example.md` . . . 156
10.6 Step 5: Constraints — What a Machine Can Check . . . 156
10.7 Step 6: Scripts — What Only a Script Can Check . . . 157
10.8 Step 7: Register, Generate, Validate . . . 157
10.9 Step 8: Ship It, Then Add the Checks . . . 158
10.10 Two Design Briefs Worth Trying . . . 158
10.10.1 A paper-writing kit . . . 158
10.10.2 A house overlay on the SDLC kit . . . 159
10.11 Failure Modes to Expect . . . 159
10.12 Where That Leaves Us . . . 160
Exercises . . . 160

11 Mental Models and the Shape of a Kit 162
11.1 Three Books About What Is In Your Head . . . 162
11.1.1 Craik: the mind runs a model . . . 162
11.1.2 Norman: your model and my model are not the same model . . . 163
11.1.3 Johnson-Laird: reasoning is running models, not applying rules . . . 163
11.2 What a Kit Is, Read Through Norman . . . 164
11.2.1 Why this generalises . . . 165
11.3 Writing a Paper . . . 165
11.3.1 The PRD role: the claim, written before the prose . . . 165
11.3.2 The DESIGN role: the argument as a structure . . . 166
11.3.3 The CPT ID role: a claim and its evidence, connected . . . 166

11.3.4 What a validate step would check . . . 167
11.3.5 Where this analogy strains, and it does . . . 168
11.4 Writing a Grant Proposal . . . 169
11.5 What This Chapter Is, and Is Not . . . 169
Exercises . . . 170

Solutions to Exercises **172**
A.1 Chapter 1: Getting Started . . . 172
A.2 Chapter 2: Choosing and Setting Up a Free or Cheap LLM . . . 174
A.3 Chapter 3: The Full Pipeline, Walked Through . . . 176
A.4 Chapter 4: Studio and PDSL . . . 180
A.5 Chapter 5: Gears, the Plugin Runtime . . . 183
A.6 Chapter 6: Insight, Data and Lineage . . . 187
A.7 Chapter 7: Security and the Bigger Picture . . . 190
A.8 Chapter 8: The Kit-Authoring Mechanism . . . 193
A.9 Chapter 9: A Real Kit, Walked Through . . . 197
A.10 Chapter 10: Building Your Own Kit . . . 202
A.11 Chapter 11: Mental Models and the Shape of a Kit . . . 211

Preface

This book teaches Constructor Fabric in three parts. Part I gets you from a clean machine to a completed pipeline run, using one real example project from start to finish. Part II goes underneath the tool and explains how its three main subsystems – Studio, Gears, and Insight – are actually built, in enough depth to reason about the system rather than just operate it. Part III teaches you to extend Constructor Fabric yourself, walking a real, shipped kit end to end before asking you to design one of your own.

Every command, file path, and code excerpt in this book is taken from the real Constructor Fabric source and its example projects. Where the source material was ambiguous or inconsistent, the book says so directly rather than smoothing it over – you will find a few of those notes along the way, and they are worth reading as carefully as the rest.

Each chapter ends with exercises. They are not decorative. Several ask you to run a real command against a real repository and read what comes back; a few ask you to break something on purpose and explain why it broke. That is the fastest way to learn a system like this one.

Part I

Using Constructor Fabric

Chapter 1

Getting Started

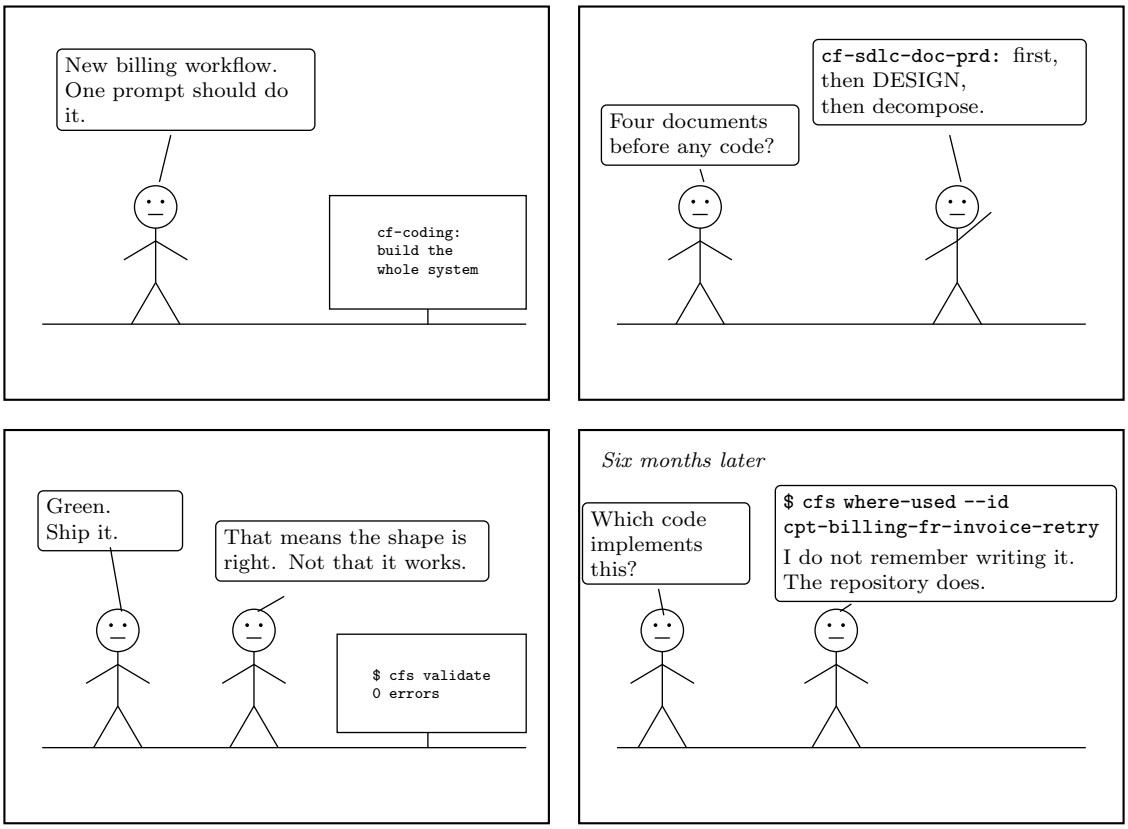


Figure 1.1: The case for the extra documents.

1.1 What Constructor Fabric Is

Constructor Fabric is an open-source, AI-native platform that connects the full software lifecycle — plan, build, run — into one traceable, governed flow from requirements to production. That is the one-sentence version, and like most one-sentence versions it hides the part that matters to you on the first day.

The part that matters on the first day is this: the tool you install is called Constructor Studio, its command is `cfs`, and it does not contain a language model. It has no API key of its own, no endpoint, no model name. It never talks to a model provider.

What it gives you instead is document templates, named workflows, review checklists, deterministic checks, and stable identifiers that link a requirement to the code that implements

it. It runs *inside* an AI coding tool you were already using — Claude Code, Cursor, OpenAI Codex, GitHub Copilot, or Windsurf — and that tool brings the model.

This split is the single most important thing to understand before you install anything, because almost every early confusion traces back to forgetting it. When something goes wrong, the first useful question is always: was that the host tool's model being weak, or was that Constructor Fabric routing me somewhere I did not intend? Those two failures look similar in a chat window and have completely different fixes.

1.1.1 The three parts, and the problem all of this solves

Constructor Fabric is organised into three parts: Studio, which this book is about; Gears, the plugin runtime that lets software built this way actually run; and Insight, the data and lineage layer underneath it. This book stays inside Studio until Part II, where Gears and Insight each get their own chapter.

Studio itself, today, is mostly a collection of skills plus a small orchestrator written in Python. That is not a placeholder description; it is what the tool actually is. A skill is a named prompt, and the orchestrator's job is to pick the right one and load the right context for it, nothing more. Each skill still has to reach the model you are using, and that wiring is host-specific: for Claude Code it lands in `CLAUDE.md`, for other agentic tools in `AGENTS.md` or the equivalent file that tool reads on startup. `cfs generate-agents`, a few pages ahead, is what writes those files. Day to day, you drive the system from chat, by name, not from the terminal — `cfs` itself is mostly there for setup, validation, and work that spans more than one repository.

A kit is a bundle of skills, templates, and rules built for one purpose. The one used throughout this book is the SDLC kit, built for taking a piece of software from a requirement to reviewed code; `cfs init` will ask you in a moment whether to install it.

Here is why any of this exists. Real software is not built against one rule; it is built against dozens, sometimes hundreds, at once: security policy, coding style, regulatory requirements, release process. A company the size of Acronis runs well over a hundred documents that constrain how its software gets written. No developer holds all of that in their head at once, and a language model does not either — a context window is a smaller, more literal version of the same limit. The same problem shows up outside code, wherever a rulebook governs the output: a scientific paper has to satisfy a venue's format, its page limit, and its idea of acceptable evidence, which is the same kind of constraint in different clothes. Kits, templates, and rules are where Constructor Fabric keeps track of which constraints apply, so neither you nor the model has to hold all of them in mind at once.

That help has a cost. Loading a large set of rules and templates into context on every step can run several times the tokens of an unstructured prompt — often two to one, sometimes closer to ten to one on a request-heavy kit. Chapter 2 is partly about managing that cost, including when it is worth sending the small, low-stakes steps to a cheaper model and keeping your best one for the steps that actually need it.

1.1.2 The four pieces

You will interact with four things, and it helps to name them early.

`cfs` A command line tool you run in a terminal. Setup, validation, updates, dependency maps, multi-repository work.

`cf-*` **skills** Names you type into the chat of your AI coding tool to start a workflow. `cf-explore`, `cf-plan`, `cf-sdlc-doc-prd`, and so on.

Kits Installable bundles of skills, templates, checklists, and rules. The SDLC kit is the default one and the one this book uses.

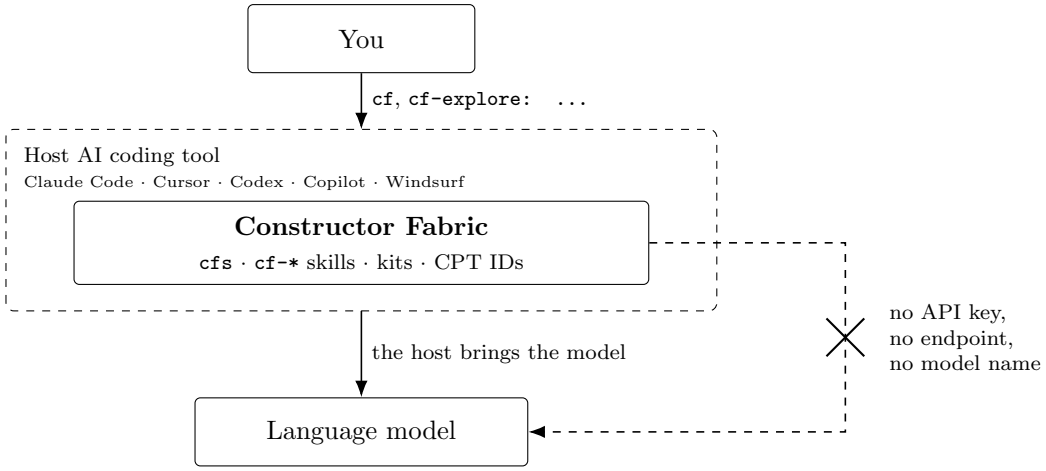


Figure 1.2: Constructor Fabric sits inside the host tool and never calls a model.

CPT IDs Canonical Provenance Trace identifiers. The strings that let you follow one requirement into design, into a feature specification, into code, and into tests.

Two of these are tools you run, one is a package you install, and one is just a naming convention for strings inside your own documents. The fourth is the one that does the real work, and it is worth saying plainly that it is not clever technology. A CPT ID is a short string like `cpt-ex-ovwa-fr-notify-on-limit` that appears verbatim in a requirements document, then verbatim in a design document, then verbatim in a comment above a Python function. That repetition is the entire traceability mechanism. It works because it is boring.

1.1.3 What each side is for

Use your AI coding tool for the things models are good at: reasoning, writing, transformation, implementation judgement.

Use Constructor Fabric for the things models are bad at:

- choosing the workflow instead of guessing one,
- loading the right context instead of whatever is in scroll range,
- applying rules and templates consistently across sessions,
- running deterministic checks that give the same answer every time,
- keeping requirements, design, and code linked,
- cutting large tasks into phases somebody can actually review.

Nothing in that list is about generating better text. All of it is about making the generation happen inside a process you can inspect afterwards.

1.1.4 When it is overhead

The documentation says out loud when this is the wrong tool, which is a good sign in a project, so we will repeat it. Constructor Fabric is usually not the best first move when:

- the task is one typo in one file;
- you want free-form exploration and a structured output is not the goal;

- you want a throwaway prototype as fast as possible;
- you have no source document, no scope, and no acceptance criteria to anchor on;
- you do not need validation, traceability, rules, or planning at all;
- your token budget is tight and the task is small enough that loading workflow context, rules, and validation logic costs more than it saves.

That is not a defect. It is the trade-off of using a structured system. If you try to run the full pipeline on a two-line fix, you will spend twenty minutes producing artifacts nobody needs and conclude, reasonably, that the whole thing is bureaucracy.

1.2 Installing

1.2.1 What you need first

- Python 3.11 or newer.
- Git.
- One supported AI coding tool.
- `pipx`, only if you are bootstrapping the repository yourself.
- `gh`, only if you want the pull request review and status workflows.

That is the whole list. There is no database, no service to run, no account to create.

One shortcut worth knowing before you install anything: if the repository you are joining *already* has a Constructor Studio setup directory and generated host integration files, you usually do not need to install `cfs` globally just to try it. Make sure Python 3.11 or newer is available, open the repository in your AI coding tool, and activate in chat. Install the CLI when you need to bootstrap a repository yourself, run terminal commands directly, or manage setup across several repositories.

1.2.2 Installing the command line tool

```
$ pipx install git+https://github.com/constructorfabric/studio.git
  installed package constructor-studio, installed using Python 3.11.2
  These apps are now globally available
    - cfs
done!

$ cfs --version
cfs 0.x.y
```

If `cfs --version` prints a version, you are done with this step. The command also reports the local Studio cache and the project-pinned Studio copy when those are available.

If `cfs` is not found, open a new terminal and try again before touching anything else. This resolves the problem more often than any other action, and people skip it because it feels too simple to be the answer.

A PyPI release is planned but not the current path, so the Git URL above is what you use today.

1.2.3 If `pipx` itself is missing

On macOS, with Homebrew:

```
$ brew install pipx
$ pipx ensurepath
$ source ~/.zshrc
```

On Windows, with Scoop:

```
> scoop install pipx
> pipx ensurepath
```

Then open a new terminal so the updated `PATH` is picked up. On macOS with the default `zsh` setup, reloading `~/.zshrc` is usually enough.

1.2.4 Connecting a repository

If you are starting from a repository that already exists on GitHub or another Git remote, get a local copy first:

```
$ git clone <url-of-your-repository>
$ cd <repository-name>
```

Then bring Constructor Fabric in:

```
$ cfs init
$ cfs generate-agents
```

Run `cfs init` from the repository root, not from a subdirectory. It is interactive and it will ask you a few questions:

- **The project root.** Accept the default.
- **The setup directory.** Keep `.cf-studio/` unless you have a concrete reason not to.
- **Whether generated runtime and agent files stay gitignored.** Yes.
- **Whether to install the SDLC kit.** Say yes. This is the kit named a few pages back: it carries the product-to-code chain, and everything in Chapter 3 depends on it.

For a first trial, every default is the right answer.

Running `cfs init` again in an already-initialised repository does not start over. It repairs the generated runtime files and host integrations using the engine version already pinned in that project, which makes it a safe recovery command rather than a destructive one.

`cfs generate-agents` writes the integration files your host tool reads. It may show you a preview of what it is about to write and wait for confirmation, so do not assume it has stalled when it goes quiet. It can also report partial success when some host-specific capabilities are skipped or some generated outputs are deliberately preserved — that is normal and not an error.

Rerun `cfs generate-agents` after installing a kit, after changing rules or workflows, and whenever a host stops picking up the skills. If you want only one host's files rewritten, name it:

```
$ cfs generate-agents --agent claude
$ cfs generate-agents --agent cursor
$ cfs generate-agents --agent copilot
$ cfs generate-agents --agent openai
$ cfs generate-agents --agent windsurf
```

There is also a read-only counterpart, `cfs agents`, which reports the same integration surface without writing any files. Use it when you want to know what generation *would* do.

1.2.5 What ends up in the repository

```
.cf-studio/
    .core/ # generated runtime, gitignored
    .gen/ # generated runtime, gitignored
    config/ # yours: AGENTS.md, SKILL.md, rules/
    core.toml
    version.toml # pinned engine version, used by repair runs
    whatsnew.toml
.claude/ .cursor/ .codex/ .windsurf/ .github/ .agents/
```

The footprint is deliberately small. Generated runtime files under `.cf-studio/.core/` and `.cf-studio/.gen/` are gitignored by default, and so are the host integration folders.

The part your team reviews and evolves is `config/`: project task guidance in `config/AGENTS.md` and `config/SKILL.md`, and project rules under `config/rules/`. Kit content is tracked, ignored, or registered depending on how you installed the kit.

Two files are worth knowing by name. `version.toml` records the pinned Studio engine request used by later repair runs, which is why re-running `cfs init` is predictable. `whatsnew.toml` records the core What's New entries already shown to you by `cfs update`, so you do not read the same release notes twice.

You do not need to open any of the generated files by hand on a first trial.

1.2.6 When install goes wrong

Table 1.1 lists the failures you are actually likely to hit, with the fix that works.

Table 1.1: Common setup failures and their fixes.

Symptom	Fix
`pipx: command not found`	Install `pipx`, run `pipx ensurepath`, open a new terminal.
`cfs: command not found` after a successful install	New terminal, or `source ~/.zshrc` on macOS.
Setup ran in the wrong place	You were not in the repository root. Run `cfs init` again from it.
Not sure what to answer during `cfs init`	Defaults: default project root, `.cf-studio/`, ignored generated files, SDLC kit accepted.
`cfs generate-agents` looks stalled	It may be previewing files and waiting for your confirmation.
Generated files exist but the AI coding tool ignores them	Reload or reopen the repository in the tool.
Typing `cf` gets a normal chatty answer	Wrong repository, or `cfs generate-agents` never ran.
You expected slash commands and only `cf-*` names work	That is normal. Slash commands are host-specific aliases.
Cross-repository checks feel noisy on day one	Start with `cfs validate --local-only`.

1.3 The SDLC Kit

A kit is an installable bundle: skills, templates, checklists, process instructions, and rules. The SDLC kit is the usual starting point because it carries the product-to-code chain. It adds the document skills — PRD, ADR, DESIGN, DECOMPOSITION, FEATURE, implementation — plus change impact analysis, reverse engineering, and pull request review.

Accepting it during `cfs init` is the path of least resistance. If you declined, or you want it in a second repository:

```
$ cfs kit install constructorfabric/studio-kit-sdlc
$ cfs generate-agents
```

The second command is not optional. A new kit means new skills, and the host integration files have to be rewritten before your chat can see them.

1.3.1 Other places a kit can come from

```
$ cfs kit install <owner/repo[@ref]>
$ cfs kit install git/<url>[//<subdir>][@<kit>] --version <ref>
$ cfs kit install --path <path> --install-mode copy
$ cfs kit install --path <path> --install-mode register
```

The first two forms pull from GitHub or a generic Git remote, which is how a platform or architecture team publishes a shared standard across an organisation. The last two install from a local path, and the difference matters:

`copy` The kit's files are copied into the setup directory. You get a snapshot you can edit.

`register` The kit stays where it is, and only a binding is recorded in `core.toml`. Effective resource paths are re-derived from the kit manifest at runtime. Use this when the kit lives inside the same project or workspace and you want one copy, not two.

1.3.2 Keeping kits current

```
$ cfs kit check-updates
$ cfs kit update
$ cfs update --with-kits yes
```

`cfs kit check-updates` shows you what changed upstream and changes nothing locally — run it freely. A plain top-level `cfs update` repairs the Studio runtime and generated integrations but deliberately leaves kits alone; you opt in with `--with-kits yes`. After any kit change, run `cfs generate-agents` again.

1.4 Talking To It

1.4.1 One rule that saves you an hour

- **Terminal**: `cfs` and its subcommands. Setup, validation, updates, maps, workspaces.
- **Chat**: `cf` and the `cf-*` skill names. Exploration, planning, writing, review.

Do not type `cf-*` skill names into the terminal. Nothing will happen. Do not type `cfs validate` into the chat and expect a real run; you will get a description of what it might have printed, which is worse than nothing because it looks like a result.

1.4.2 Turning it on in chat

Open the repository you just initialised in your AI coding tool and type:

```
cf
```

You should see Constructor Fabric load and offer a workflow routing menu. Some hosts will also show you the resolved Constructor Studio path or the loaded context. If the chat answers like a normal assistant instead, the setup did not reach the host: check that you opened the same repository you initialised, check that `cfs generate-agents` ran, reload the repository, and retry.

Then ask for the tour:

```
cf-help
```

1.4.3 The naming convention

Every workflow has a name starting with `cf-`. Some take a request after a colon; some are command-like prompts on their own.

```
cf-help
cf-auto-config
cf-explore: find the artifacts and code paths for the billing workflow
cf-coding: implement the next safe phase of this migration
```

`cf-<skill>: <request>` is the portable form and should be your default. The skill name selects the workflow; the text after the colon is the job.

Table 1.2 lists the base skills. Table 1.3 lists the SDLC kit skills, which are the ones Chapter 3 uses throughout.

Both tables reflect the skill set at the time of writing. Skills are added, renamed, and reorganised as Constructor Fabric evolves; if a name here does not match what `cf-help` shows you, trust `cf-help`.

Table 1.2: Base Constructor Fabric skills.

Skill	Use
`cf-help`	What is available and where to start.
`cf-explore: ...`	Find relevant files, artifacts, rules, project context.
`cf-brainstorm: ...`	Explore options before scope is fixed.
`cf-auto-config`	Infer or refresh project rules for an existing repository.
`cf-plan: ...`	Split large or risky work into reviewable phases.
`cf-planning: ...`	Build a reusable phase plan and definition of done for any domain.
`cf-coding: ...`	Write, review, fix, or test code.
`cf-write-docs: ...`	Write or review a human-facing document.
`cf-write-skills: ...`	Author prompts, workflows, skills.
`cf-testing: ...`	Write or run tests.
`cf-explain: ...`	Walk through a document, pull request, code area, or decision.
`cf-map: ...`	Render or inspect the dependency map.
`cf-git-commit: ...`	Stage and commit scoped changes with a compliant message.
`cf-workspace: ...`	Configure or work across several repositories.
`cf-kit: ...`	Create, normalise, validate, or update a kit.
`cf-debug-prompts: ...`	Watch a workflow execute step by step.
`cf-brave-new-world`	Let Studio pick safe, reversible defaults for this session.

1.4.4 Aliases and sub-routes

Three of the names in Table 1.2 are compatibility aliases. `cf-coding` loads `cf-coding-gen`; `cf-write-docs` loads `cf-documenting-gen`; `cf-write-skills` loads `cf-prompting-gen`. When you already know what you want, call the sub-route directly and skip a routing step:

```
cf-coding-gen: cf-coding-review: cf-coding-fix:
cf-coding-ci: cf-coding-tests: cf-code-planning:

cf-documenting-gen: cf-documenting-review: cf-documenting-fix:
cf-prompting-gen: cf-prompting-review: cf-prompting-fix:
```

`cf-analyze` and `cf-generate` are stable router entrypoints. They preserve the older verbs and resolve the best matching skill for your request; they are not the workers themselves.

Table 1.3: SDLC kit skills.

Skill	Use
`cf-sdlc-doc-prd: ...`	Write or revise a Product Requirements Document.
`cf-sdlc-doc-adr: ...`	Record one Architecture Decision Record.
`cf-sdlc-doc-design: ...`	Write or revise the DESIGN document.
`cf-sdlc-decompose: ...`	Break a design into ordered feature work.
`cf-sdlc-doc-feature: ...`	Write or revise one FEATURE specification.
`cf-sdlc-implement: ...`	Implement an approved FEATURE in code with traceability.
`cf-sdlc-change-impact-analysis: ...`	Analyse downstream impact from changed artifacts or code.
`cf-sdlc-reverse-engineer: ...`	Reconstruct SDLC artifacts from existing code and markers.
`cf-sdlc-pr-review: ...`	Review a pull request against project and SDLC expectations.
`cf-sdlc-pr-status: ...`	Summarise pull request state, checks, and unresolved review work.

1.4.5 A request that works, and one that wastes your afternoon

These work:

```
cf-plan: break this auth migration into safe implementation phases

cf-explore: trace where auth configuration is defined and loaded

cf-coding: review this module for correctness, regression risk,
          and missing tests

cf-write-docs: review this requirement and list the top 5 unclear
              or missing points before implementation
```

Each one names a target, bounds the scope, and makes the success criterion visible.

These do not:

```
cf-coding: build the whole system
cf-coding: make this project enterprise grade
cf-coding: improve everything
cf-coding: tell me if this code is good
cf-coding: rewrite the app based on best practices
cf-coding: add the feature, markers are not important
```

No scope, no target, no source of truth, no success criteria. The workflow has nothing to route on, so you get a guess dressed up as a process.

1.4.6 Gates you will be asked about

Concrete workflows pause at shared gates before the main work starts. You will see some subset of:

- **Interaction mode**: `assistant`, `normal`, or `debug`. Pick `assistant` the first few times — it narrates where the workflow is, why the current step is happening, and which option is recommended. `debug` reuses `cf-debug-prompts` in a lighter run mode, pausing only on breakpoints, menus, or errors.
- **Context discovery** with `cf-explore` first.
- **Brainstorming** before scope is fixed.
- **Plan-first** when the work is multi-step.

- **Sub-agent dispatch**.

Every gate is yours to approve. Nothing large starts silently. This is slower than a single prompt and it is the reason the result is reviewable.

1.5 The Ten Stages at a Glance

Here is the pipeline the SDLC kit exists to support. Imagine a team adding a new billing workflow to an existing product: a product manager, an architect, two developers, a QA engineer. Nobody rewrites the process; they use the one they already have, with Constructor Fabric holding the artifacts together.

Table 1.4 is the whole thing on one page. Chapter 3 walks every stage in detail against a real project, so treat this as orientation rather than instruction.

Table 1.4: The ten pipeline stages and their entry points.

#	Stage	How you start it
1	Gather what exists	`cf-explore: gather existing product notes, customer constraints, and related artifacts`
2	Open the options	`cf-brainstorm: map the options for the new billing workflow`
3	Requirements	`cf-sdlc-doc-prd: write the Product Requirements Document for the billing workflow`
4	Record the decision	`cf-sdlc-doc-adr: record the architecture decision for payment provider isolation`
5	Design	`cf-sdlc-doc-design: write the DESIGN document from the approved Product Requirements Document and Architecture Decision Records`
6	Decompose	`cf-sdlc-decompose: decompose the DESIGN document into reviewable FEATURE work`
7	Feature specification	`cf-sdlc-doc-feature: write the FEATURE spec for invoice retry handling`
8	Implement	`cf-sdlc-implement: implement the approved FEATURE specification with CPT markers and unit tests`
9	Review the code	`cf-coding: review the changed code for correctness, regression risk, and missing tests`
10	Look at the whole thing	`cfs map`, then `cf-map: find dangling references`

A few observations that are easy to miss from a table.

Stage 1 is the cheapest and the most often skipped. The product manager starts with `cf-explore`, not with the PRD. What comes out is a short map: which notes exist, which are stale, which constraints are already written down somewhere nobody remembers.

Stage 2 has an expiry date. `cf-brainstorm` is for the period while scope is still open. Once the PRD exists, brainstorming the shape of the thing is too late to be useful.

Stage 5 names its inputs. Look carefully at the stage 5 request: it says "from the approved Product Requirements Document and Architecture Decision Records." The design skill transforms approved documents. It does not invent an architecture from a one-line prompt, and if you give it a one-line prompt that is what it will have to do.

Stages 3 through 8 each end in a review and a check. The document or the code is reviewed against the configured methodology and checklist, you see the findings *before* anything is rewritten, and you approve the fix scope. Then the deterministic part runs.

Stage 9 wants a fresh chat. Writing context leaks assumptions into review. Better still, let the host dispatch a sub-agent, so the reviewer never sees the writer's reasoning.

The chain, end to end, is:

PRD → ADR → DESIGN → DECOMPOSITION → FEATURE → code → tests → review

Linked by CPT IDs, shaped by the kit, checked by `cfs`. Each team reads the part it owns with the checklist that matches its job. QA, performance, security, DevOps, and delivery use the same files, not copies of them.

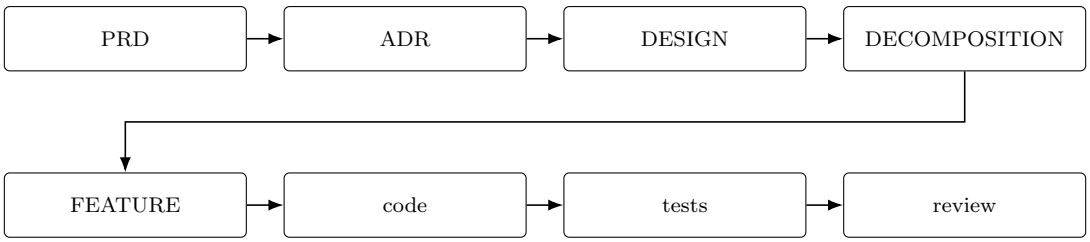


Figure 1.3: The chain, end to end.

The claim being made here is narrow, and it is worth stating precisely because the broad version is not true. The result is not "AI wrote the project." The result is a delivery system you can inspect. Someone can ask "which code implements this approved requirement?" and get an answer out of the repository instead of out of the one developer who remembers.

1.6 Validating

Everything up to this point has been probabilistic — a model wrote something and you read it. The `cfs` checks are the other half: same files in, same answer out, every time.

1.6.1 `cfs validate`

```
$ cfs validate
$ cfs validate --local-only
```

This checks the configured artifacts: structure, required sections, references, CPT ID format, and consistency rules. Specifically, on identifiers it checks that every ID matches the naming template for its kind; that cross-references exist where coverage is mandatory; that task checkboxes are consistent, so a reference marked `[x]` cannot point at a definition still marked `[]`; that priority markers are present where required; and, where code traceability is on, that a `[x]` definition has a matching code marker while a `[]` definition does not yet.

`--local-only` skips cross-repository resolution, which is what you want while iterating and on day one.

Run it locally before you open a pull request, and keep it in CI so that review is not the first place it ever runs.

1.6.2 The smaller checks

```
$ cfs validate-toc guides/USAGE-GUIDE.md guides/CONFIGURATION.md

$ cfs check-language README.md guides/USAGE-GUIDE.md
```

`cfs validate-toc` confirms that a document's table of contents matches its actual headings. `cfs check-language` confirms the document is written in an allowed language. Small, boring checks. They catch exactly the drift that nobody notices in review because nobody re-reads a table of contents.

1.6.3 Looking at the graph

```
$ cfs map
$ cfs spec-coverage
$ cfs where-defined --id cpt-billing-fr-invoice-retry
$ cfs where-used --id cpt-billing-fr-invoice-retry
$ cfs list-ids --source docs
```

`cfs map` builds the dependency graph across documents and code by scanning links, code references, and `@cpt-*` identifiers. It is the fastest way to see how artifacts, rules, workflows, and code wire together, and where the wiring is broken. `cfs spec-coverage` reports which identifiers that are supposed to reach code actually have markers. `cfs where-defined` tells you which file defines an ID; `cfs where-used` finds every reference to it; `cfs list-ids` dumps everything one source declares.

Use these before a refactor, not after it.

1.6.4 The loop

For any non-trivial work, on documents or on code, the safe default loop is:

1. plan or generate,
2. validate,
3. review,
4. read the findings and approve the fix scope,
5. fix,
6. validate again,
7. repeat until nothing known is outstanding.

Same loop for a PRD and for a Python module. That symmetry is deliberate and it is most of what makes the system learnable.

1.6.5 What the loop does not give you

It improves quality. It does not prove correctness.

A green `cfs validate` means the shape is right, not that the software works. A passed checklist means the review was consistent, not that the review was correct. Final human review is still required, the guide says so explicitly, and it says so for a reason: a system that produces a lot of green checkmarks is very good at persuading tired people that it has thought for them.

1.7 Habits and Traps

1.7.1 Habits that pay off

- Name the target: the artifact, the code area, the outcome.
- Give it source documents and constraints, not intent.

- Plan before the conversation is already overloaded.
- Validate early and keep validating.
- Say what must not change, explicitly.
- Use the smallest workflow that still keeps you in control.
- Fresh chat for each substantial writing or review task.
- Make the review repeatable, then make the final call yourself.

1.7.2 Traps

- Using it as a generic chat tool, which bypasses everything that makes it useful.
- Starting with a writing skill on a huge, ambiguous task.
- Skipping validation until the end, by which point upstream errors have been amplified through four documents.
- Treating a green check or a long fix loop as proof of correctness.
- Reaching for it for wide-open brainstorming when structure is not what you want.
- Running a full structured workflow for a change a direct edit would have covered.
- Asking for an outcome without naming the document it comes from, so the tool guesses.
- Carrying stale context from one task into an unrelated one.

1.7.3 Two traps worth a closer look

Two failure modes are common enough, and expensive enough, to deserve more than a bullet point.

Over-engineering a one-off. The whole point of validation, review, and checklists is to push toward a thorough, defensible result, and for production software that is exactly the right push. It is the wrong push for code that will run once — a script behind a single scientific result, a throwaway analysis, a proof of concept nobody will maintain. Left unchecked, the loop happily generates a test for every small operation, then a fix for a failing test, then a test for the fix, and removes nothing, because nothing in the loop asks whether the thing being protected needed protecting in the first place. The codebase grows, and so does the token bill, since every further step has to read all of it back into context first. The fix is not to switch validation off; it is to say, in the PRD or the request itself, what kind of project this is — "this is a research script, the result matters and production hardening does not" — and to say plainly that scope should not grow beyond what was asked. Naming the goal is what stops the loop from optimising for the wrong thing.

Careless naming has consequences. A term you introduce casually — in a PRD, a rule, a variable name — is not necessarily read the way you meant it, and not necessarily read the same way twice. Two patterns follow from this, and both are worth naming precisely because both have actually happened. First, borrowing vivid, violent, or biological language for ordinary engineering ideas — calling a cleanup routine a "kill," a retention policy a "death," an isolation mechanism a "pollution" control — can, after enough repetition, read to the underlying model like a description of something it should refuse, and a session can stall on a safety refusal for a request that has nothing to do with the words it happens to be using. Second, a term defined precisely once can drift: a data-isolation rule introduced as "seal this data so a test cannot

modify it" was, a few iterations later, quietly reinterpreted as an instruction to encrypt the data, because the model no longer had the original definition in view and fell back on the word's ordinary sense. Plain, literal, non-metaphorical names cost nothing and avoid both problems. If a term matters, put its definition in a rule or a constraints file where it persists, rather than trusting that one early sentence will still be in scope five documents later.

1.8 More Than One Repository

Do not clone the full setup into every small service repository. Keep one orchestration repository and connect the rest as a workspace:

```
$ cfs workspace-init
$ cfs workspace-add --name docs --path ../docs-repo --role artifacts
$ cfs workspace-add --name services --path ../services-repo --role codebase
$ cfs workspace-info
```

One place for setup, shared rules and kits, and traceability that crosses repository boundaries. If each repository also needs its own generated host files, run `cfs generate-agents` in each one after the kit or workspace configuration changes.

1.9 Your First Thirty Minutes

1. Install, then `cfs init`, then `cfs generate-agents`.
2. In chat: `cf`, then `cf-help`.
3. Pick one small real input: one requirement, one design note, one focused change.
4. Run one of `cf-explain: ...`, `cf-coding: review ...`, or `cf-plan: ...`.

Not a repository-wide review, and not a broad implementation request. Success at this point looks like three things: activation is confirmed in the right repository, one useful result appeared that you can act on, and the next step is clearer than it was thirty minutes ago.

That is the whole of the first session. The rest of this book is about what happens when you keep going: Chapter 2 solves the problem this chapter quietly assumed away — that you already have a working AI coding tool with a model behind it — and Chapter 3 walks the ten stages end to end on a project you can clone and read.

Exercises

1. **Install and verify.** Install `cfs`, run `cfs init` and `cfs generate-agents` in a scratch Git repository, and then list what appeared. For each of `.cf-studio/.core/`, `.cf-studio/config/`, `.cf-studio/version.toml`, and one host folder such as `.claude/`, write one sentence saying whether it is gitignored and whether you would ever edit it by hand.
2. **Terminal or chat?** Take these eight invocations and sort them into two columns, terminal and chat, without looking back at this chapter: `cfs validate`, `cf-help`, `cfs kit check-updates`, `cf-explore: ...`, `cfs map`, `cf-map: find dangling references`, `cfs generate-agents --agent claude`, `cf-sdlc-doc-prd: ...`. Then check yourself. Anything you got wrong is a mistake you will make again under time pressure, so note it.
3. **Rewrite four bad requests.** Take four of the failing requests from this chapter ("build the whole system", "make this project enterprise grade", "improve everything", "tell me

if this code is good") and rewrite each one against a repository you actually have, so that it names a target, bounds the scope, and states a visible success criterion. Keep each rewrite under two lines.

4. **Clone and orient.** Clone the Constructor Studio repository and open `examples/overwork_alert/`. Without reading Chapter 3, list the four documents under `architecture/` and the nine Python modules under `src/overwork_alert/`, and guess for each module which document you think describes it. You will check your answers in Chapter 3.

5. **Predict the check.** `cfs validate` enforces that a reference marked `[x]` cannot point at a definition marked `[]`. Explain in three or four sentences what real-world mistake that rule is designed to catch, and why the reverse case — a `[]` reference pointing at an `[x]` definition — is allowed.

Chapter 2

Choosing and Setting Up a Free or Cheap LLM

2.1 Where Does the Model Come From?

Chapter 1 installed Constructor Fabric and walked a whole pipeline with it. At no point did any of it call a language model.

That is not an oversight in the description. Constructor Fabric has no API key, no endpoint, and no model name of its own. It is an orchestration layer that activates inside an AI coding tool you were already running. The host tool brings the model; Constructor Fabric brings the process.

Which means the last chapter quietly assumed something expensive: that you already have a working AI coding tool with a capable model behind it. For a student, that assumption normally costs money. This chapter is about how to make it cost nothing, or nearly nothing, and about which of the obvious-looking free options are actually dead ends.

2.2 The Five Hosts

There are five documented integration targets: Claude Code, Cursor, OpenAI Codex, GitHub Copilot, and Windsurf. These are what `cfs generate-agents` writes files for, and where `cf` activation in chat is expected to work.

They are not interchangeable. Table 2.1 is the support matrix, and the differences in it are host limitations, not Constructor Fabric limitations.

Table 2.1: What each host tool provides.

Capability	Claude	Cursor	Copilot	Codex	Windsurf
Skill integration	Yes	Yes	Yes	Yes	Yes
Host sub-agents	Yes	Yes	Yes	Yes	No
Read-only review	Strong	Strong	Partial	Weaker	None
Worktree isolation	Yes	No	No	No	No
Model selection	Yes	Yes	No	Partial	N/A
Multi-model host	No	Yes	Yes	No	Yes

Three rows deserve a sentence each, because they change how the pipeline of Chapter 1 feels in practice.

Host sub-agents are what let a review run in a process that never saw the writing conversation. Without them, "use a fresh chat for review" becomes a discipline you have to remember rather than something the workflow can arrange for you. Windsurf has none, so on Windsurf the separation between generation and review is entirely manual.

Worktree isolation means an implementation run can happen in a separate Git worktree, so a half-finished multi-file change is not sitting in your working directory while you decide whether you like it. Only Claude Code has this.

Multi-model host is the row that will matter most to you if you end up on Copilot or Cursor. A host that can switch models lets you use one family for generation and a different one for review, which is a real quality gain and costs nothing extra.

2.2.1 So the real question

The question is not "how do I get a free LLM for Constructor Fabric." It is "how do I get free access to one of those five tools, and what model sits behind it."

Those are two separate questions and they get collapsed into one constantly. A free chat subscription is not free API access. A free student program is not always a free model. Keeping those apart will save you an afternoon.

2.2.2 Four ways to pay nothing

1. A frontier provider with a standing free API tier.
2. An open-weight model running on your own machine.
3. A free cloud inference service that hosts open-weight models for you.
4. A student programme that gives you a host tool outright.

We go through all four, and then at the end this chapter names the two or three setups actually worth doing.

2.3 Frontier APIs

2.3.1 Google Gemini: the one standing free API

Google's Gemini API has a free tier that is simply always there. No trial clock, no expiry date. It covers the Flash family of models, priced at $0 per million tokens on that tier. You get a key from Google AI Studio with an ordinary Google account.

Among the frontier providers, this is the only one with a real free API tier today. That is worth stating plainly because the general mood suggests otherwise.

What the free tier allows, at time of writing: roughly 10 to 15 requests per minute for the Flash models, and roughly 1,000 to 1,500 requests per day. That is enough for exploration, single-document generation, and review passes. It is not enough for an agent that fires dozens of tool calls per minute for an hour, which is exactly what a long Constructor Fabric implementation run does. Check the pricing page before you rely on any of those numbers; they move.

What the free tier costs you is not money. Free-tier prompts may be used to improve Google's models. That is the trade: you pay in data instead of euros. Fine for a course exercise on a public repository. Not fine for your employer's code, a thesis under embargo, or anything under an NDA. The paid tiers do not carry the same term, which tells you the data clause is a deliberate part of the free deal rather than an accident of the legal text.

2.3.2 Anthropic: free chat, not free API

Claude has a free plan in the chat application, with usage caps. Claude Code — the host tool we actually care about — sits on a paid plan or on API billing. At the API level there is no standing free tier, only credits that expire.

So "I have a free Claude account" does not give you a model for Constructor Fabric. It gives you a web chat window, which is a different product.

There is a way to run Claude Code for free, and it does not involve Anthropic at all. That is Path B later in this chapter.

2.3.3 OpenAI: the same story

ChatGPT has a free tier in the chat product. The API is pay as you go, priced per million tokens, with no standing free allowance. New accounts sometimes get trial credit, which expires. Codex, the host tool, follows the paid plans.

Same conclusion as Anthropic: a free consumer account is not an API key.

2.3.4 Mistral: a free tier that means it

Mistral's La Plateforme has an "Experiment" tier that is genuinely free. It covers all their models, including Codestral, the code-focused one. It is rate limited and explicitly intended for experimentation rather than production.

Mistral is a European provider, which matters to some people for data residency reasons and matters not at all to others. Practically, it is less widely wired into the five host tools than Gemini is, so expect more manual setup and check your host's provider settings before you commit to it.

2.3.5 The frontier providers side by side

Table 2.2: Standing free API access by frontier provider.

Provider	**Standing free API?**	**What you get**
Google	Yes	Flash models at $0, roughly 10–15 requests/min, prompts may train the models
Mistral	Yes	The Experiment tier, all models including Codestral, rate limited
Anthropic	No	Free chat plan only; API is paid, trial credit expires
OpenAI	No	Free chat plan only; API is pay as you go

Two out of four, which is better than the general mood suggests.

2.4 Open Weights on Your Own Machine

The other direction is to stop renting a model.

An open-weight model is one whose parameters you can download. You run it on your own hardware, so there is no key, no quota, and no request counter. The cost moves from a monthly bill to your laptop's memory, disk, and battery.

Two years ago the small open-weight models were toys for this kind of work. That has changed enough to be worth taking seriously, though not enough to pretend the gap has closed.

2.4.1 Which model to try

Qwen3-Coder is currently the strongest open-weight line for agentic coding on local hardware. It was trained with tool use and multi-step editing in mind, which is precisely what a Constructor Fabric workflow does to a model: call a tool, read a file, edit it, validate, read the validation

output, edit again. It comes in several sizes so you can match it to the machine you own. If you only try one local family for this, try this one first.

Gemma is Google's open-weight family, tuned for running on device. Small, efficient, well behaved on a laptop, and noticeably weaker at long agentic chains.

DeepSeek publishes distilled small variants — a large model's behaviour compressed into a size you can host.

Both of the latter are reasonable second choices, and both are worth trying, because "best" here depends more on your machine than on any benchmark.

2.4.2 Ollama in four commands

```
$ brew install ollama # or download from ollama.com
$ ollama serve # starts the local server on :11434
$ ollama pull qwen3-coder:30b # downloads the weights, once
$ ollama run qwen3-coder:30b
>>> Send a message (/? for help)
```

Ollama is the easiest of the local runners and the one to start with. The download is large — tens of gigabytes for the bigger sizes. Do it on the university network, not on a phone tether, and not fifteen minutes before a lab session.

2.4.3 The local server speaks a familiar dialect

This is the part that makes local models useful rather than merely interesting:

```
$ curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "qwen3-coder:30b",
    "messages": [{"role": "user", "content": "hello"}]
  }'
```

Ollama exposes an OpenAI-compatible endpoint at `/v1`. Anything that can be pointed at a custom base URL can be pointed here. That is the hook Path B uses later.

2.4.4 LM Studio

LM Studio is the same idea with a graphical interface: browse models, download them, chat, and serve. It also exposes a local OpenAI-compatible server.

It is nicer than Ollama if you want to watch memory use and fiddle with quantisation settings while you experiment, because it shows you both. Ollama is easier to script and easier to write down in a book, so the commands here use Ollama. Either is fine.

2.4.5 How much laptop do you need

A 7 to 8 billion parameter model needs about 6 to 8 GB of memory when quantised. That runs on any modern laptop, including a 16 GB Windows machine. It is good enough for explaining code, small edits, reviewing a single file, and first-pass documentation. It is not good enough for architecture decisions, decomposition, or long multi-file implementation.

A 27 to 32 billion parameter model needs about 20 to 24 GB. This is the size where local models start to be usable for real agentic work. A Mac with 32 GB or more handles it comfortably. On a typical Windows laptop it usually does not fit, and the reason is structural rather than a matter of tuning.

2.4.6 Why an Apple Silicon Mac wins here

Apple Silicon uses unified memory: the GPU can address all system RAM. A 32 GB MacBook can therefore give a model something like 20 GB or more of usable GPU memory.

A typical Windows laptop has 16 GB of system RAM plus a discrete GPU with 6 or 8 GB of its own VRAM. The model has to fit in that smaller dedicated pool, or it spills to the CPU and crawls — and "crawls" here means tokens per second slow enough that an agentic workflow with fifty tool calls becomes an afternoon.

This is the one situation in this book where the Mac in the lecture hall has a real, structural advantage rather than a stylistic one.

2.4.7 What you give up by going local

Raw capability, first. A 30B model is not a frontier model, and pretending otherwise wastes your afternoon and then makes you distrust the tooling for the model's mistakes.

Speed on long contexts, second — which is unfortunately where Constructor Fabric workflows live, since a workflow loads templates, rules, and a source document before it does anything.

Battery, fan noise, and about 20 GB of disk, third.

What you gain: no quota, no key, no network, and your code never leaves the machine. For a thesis on private data, or an internship project under an NDA, that last point can decide the entire question on its own.

2.5 Free Cloud Inference

The third route: somebody else hosts the open-weight models and lets you use them for free. You get an API key and a base URL, and the models are the same open weights you could have run locally — but faster than your laptop and costing you no disk.

Each provider caps you differently, and the caps are the interesting part.

Cerebras gives roughly 1 million tokens per day on the free tier, with a daily reset and no credit card required to start. That is a generous daily budget by the standards of this list, and it is the one to try first when you do not want to install anything.

Groq gives free-tier access to a set of open-weight models, capped both per minute and per day. It is known for very fast token generation, which genuinely changes how an agent feels to use — a workflow that takes fifty steps feels different at 500 tokens per second than at 30. Sign-up is quick and the caps are tighter than Cerebras.

OpenRouter is a router in front of many providers, with a set of models carrying a `:free` suffix. Without any payment you get 20 requests per minute and 50 requests per day, which is very little. After a single non-expiring purchase of $10 or more in credit, the daily cap goes to 1,000 requests per day.

That second OpenRouter line is the most interesting number in this chapter. Ten euros once — not per month — and the daily cap goes up twentyfold, on a key that reaches many different models. If you want to compare model families on the same Constructor Fabric workflow, this is the cheapest way to do it.

Table 2.3: Free cloud inference services.

Service	Free allowance	Card needed?
Cerebras	about 1M tokens/day, resets daily	No
Groq	open-weight models, per-minute and per-day caps	No
OpenRouter	20 req/min and 50 req/day; 1,000/day after a one-off $10 credit purchase	No, until you want the higher cap

All three serve open weights, so the model-quality question is the same one from the previous section. What you are buying with these services is speed and disk, not capability.

2.6 Student Programmes, and Why This Section Names a Country

Student programmes are the most generous route and the most geographically uneven one. The same programme is free in one country, unavailable in the next, and useless in a third. Everything that follows was checked for Italy, because that is where this course is taught; if you are reading this elsewhere, treat the structure of the argument as portable and every specific eligibility claim as something to re-verify.

Three of the following help you. Two do not. One will actively mislead you.

2.6.1 GitHub Student Developer Pack — yes

Available worldwide, Italy included. You verify with your university email and student status.

It includes the student plan of GitHub Copilot, with a monthly allowance of AI credits, and it also gives command-line access through Copilot CLI. GitHub Copilot is one of the five documented Constructor Fabric hosts, which is the whole point.

Why this is the shortest path: no credit card, no local model to download, no API key to wire into anything, and a supported host tool working today. Copilot is also a multi-model host, so you can pick a stronger model for review and a faster one for bulk work — see Section 2.8. The limit is the monthly credit allowance, which is finite but not small.

2.6.2 Google AI Plus for students — read this twice

Google offers students a free 12-month "Google AI Plus" subscription, and Italy is covered.

It is a consumer subscription. It does not include Gemini API access. API usage is billed separately, under the pricing page discussed earlier in this chapter.

So it gives you a nicer chat product and nothing your AI coding tool can use. This is the entry on the list most likely to cost you an hour, because the offer's name contains "AI" and "student" and your brain fills in the rest.

The free Gemini API tier is still there and still free. It just has nothing to do with this offer.

2.6.3 OpenAI student programmes — no

OpenAI runs "Codex for Students" and a "Student Collective" programme. Both explicitly exclude Italy from eligibility.

There is no workaround worth attempting. Violating the terms of a programme to get free inference credit is a bad way to start a career, and it is a much worse trade than it looks. If you want a Codex-shaped workflow, you are paying for it.

2.6.4 Azure for Students and AWS Educate — no

Both give students cloud credit and both are available in Italy. Both look, at first glance, like a way to get an LLM budget.

Their free and starter tiers carve out exactly the services you want: Azure OpenAI Service is excluded from the free allowance, and AWS Bedrock is excluded from the free allowance.

They are useful for a virtual machine to run something on. They are not useful for a model.

Table 2.4: Student programme scorecard for Italy.

Programme	Helps?	Why
GitHub Student Pack	Yes	Copilot student plan, a supported host, works in Italy
Google AI Plus (student)	No	Consumer subscription, excludes API access
Gemini API free tier	Yes	Not a student programme at all, just free
Codex for Students	No	Italy is excluded
Azure for Students	No	Credit excludes Azure OpenAI Service
AWS Educate	No	Credit excludes Bedrock

2.7 Three Setups Worth Doing

Everything above is a survey. This section is the part to actually follow.

2.7.1 Path A: Copilot through the Student Pack

Claim the Pack, enable Copilot, and open your repository in an editor with Copilot active. Then:

```
$ cd path/to/your/repository
$ cfs init
$ cfs generate-agents --agent copilot
```

And in chat:

```
cf
cf-help
```

Nothing else to install, nothing to configure, no key to keep secret. This is the recommendation for most students, and if you are unsure which path to take, take this one and move on to Chapter 3.

The one thing to be aware of is that Copilot's generated agent files do not carry a model-selection surface the way Claude Code's do. Choose the right model in the host environment or the session *before* you invoke a workflow, rather than expecting the workflow to choose for you.

2.7.2 Path B: Claude Code on a model you own

Claude Code is the strongest host for Constructor Fabric — sub-agents, worktree isolation, model hints in the generated agent files. Anthropic will not give it to you for free.

Since Ollama version 0.14.0, Ollama exposes an endpoint that speaks the Anthropic Messages API. So Claude Code can be pointed at a model running on your own laptop. Two environment variables and one model pull:

```
$ ollama pull gpt-oss:20b

$ export ANTHROPIC_AUTH_TOKEN=ollama
$ export ANTHROPIC_BASE_URL=http://localhost:11434
$ claude --model gpt-oss:20b
```

The token is a placeholder. Ollama does not check it, and Claude Code refuses to start without one, so it has to be set to something. No account, no key, no quota, no rate limit, and no network traffic leaving the machine.

Check it before you trust it:

```
$ ollama list
NAME ID SIZE MODIFIED
gpt-oss:20b aa4295ac10c3 13 GB 2 minutes ago

$ curl -s http://localhost:11434/api/tags | head -3
```

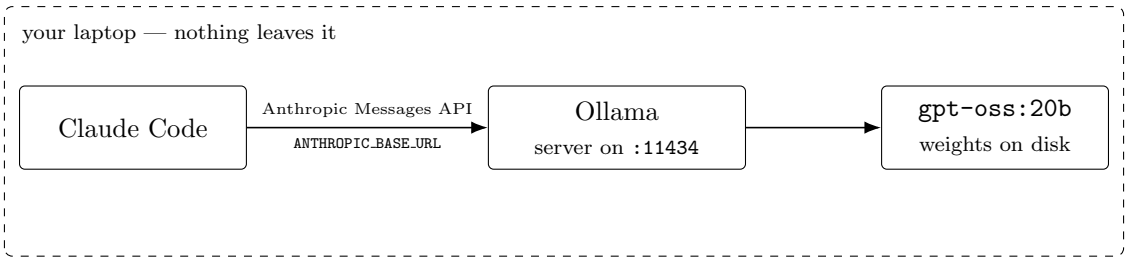


Figure 2.1: Path B: the host tool talks to your own machine.

```
{"models":[{"name":"gpt-oss:20b", ...

$ echo $ANTHROPIC_BASE_URL
http://localhost:11434
```

If `claude` still asks you to log in, the two variables did not reach the shell that launched it. That is the failure mode, essentially always. Exporting them in one terminal and launching Claude Code from an editor's integrated terminal in another is the usual way to get there.

Then bring Constructor Fabric in:

```
$ cd path/to/your/repository
$ cfs init
$ cfs generate-agents --agent claude
$ claude --model gpt-oss:20b
```

```
cf
cf-explore: find the main artifacts and code paths in this repository
```

From here the whole of Chapter 1 applies unchanged. One adjustment: keep requests small. A local model has less patience than a frontier one, and a workflow that would have run to completion on a hosted model will sometimes lose the thread halfway through on a 20B one. Cut the work into more, smaller phases than you would otherwise.

2.7.3 Codex, or any other OpenAI-API host, on the same local model

The same local server works for Codex, or for any tool that speaks the OpenAI API rather than Anthropic's. Ollama's and LM Studio's local servers both expose an OpenAI-compatible endpoint at `/v1` — the same one the `curl` example earlier in this chapter used — so the mechanism is the same one Path B used: point the host's OpenAI client at your local server instead of at OpenAI's, with a placeholder key the local server does not check.

Codex reads its model-provider configuration from its own configuration file rather than from environment variables, and the exact field names have changed as the tool has matured, so check Codex's current configuration reference before you wire this up rather than trusting a specific example here to still be accurate. What you are looking for, whatever it is called this month, is a way to declare a named provider with a custom base URL pointing at `http://localhost:11434/v1` for Ollama, or at your LM Studio server's address, and a profile that uses that provider. Once the provider is declared, the rest of this chapter applies unchanged: run `cfs init`, then `cfs generate-agents --agent openai`, and talk to it in chat exactly as Path B does with Claude Code.

2.7.4 Path C: a Gemini or Mistral key

Get a free key from Google AI Studio or from Mistral's La Plateforme.

This is useful in two places: hosts that let you choose the provider, and the scripts you write around the edges of your work — a quick summariser, a batch classifier, a one-off transformation that does not need a whole workflow.

Two cautions. Do not assume that all five hosts will accept an arbitrary third-party key; check your host's own provider settings first. And watch the per-minute limit rather than the per-day one. An agent burns requests far faster than a human chatting, and 10 to 15 requests per minute is a ceiling you will hit in the first few minutes of a real implementation run, not on day three.

2.7.5 Which one should you pick

- **Any laptop, want it working in ten minutes**: Path A, Copilot through the Student Pack.
- **Mac with 32 GB or more, want no limits at all**: Path B, Claude Code on Ollama.
- **Private code you cannot send anywhere**: Path B, and it is not a close call.
- **Want frontier quality on a small budget**: the Gemini free tier, inside its limits.
- **Want to compare many models**: OpenRouter, and seriously consider the one-off $10.

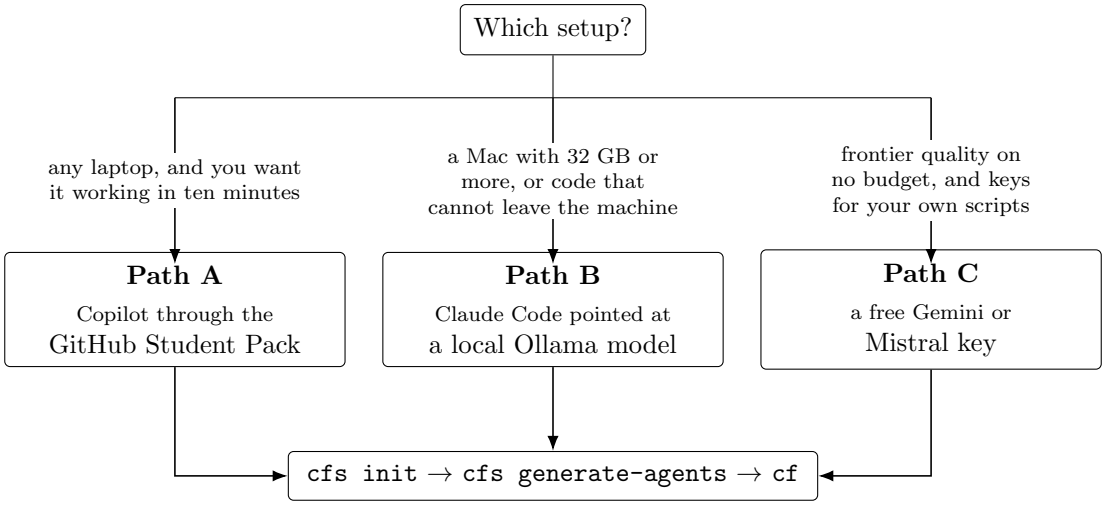


Figure 2.2: Which of the three paths to set up, and where they all meet.

2.8 Matching the Model to the Operation

Once you have more than one model available — which happens the moment you are on a multi-model host or holding an OpenRouter key — the question changes from "which model" to "which model for which step." Constructor Fabric's own guidance uses three tiers:

fast-tier High-volume, lower-complexity work where speed matters more than deep reasoning.

default / inherit The current session model. This should usually be your default choice.

strong reasoning Your best available tier, for tasks with ambiguity, architecture judgement, long dependency chains, or expensive mistakes.

Table 2.5 maps the operations from Chapter 1 onto those tiers.

The family-level tendency, stated as a practical default and not a law: GPT-style thinking models tend to be better at *review* and at artifact-heavy work, mostly because they make fewer small mistakes; Claude-style models tend to be better at *generation*, especially turning an

Table 2.5: Recommended model tier by operation.

Operation	Tier
Lookups: `where-defined`, `list-ids`, shallow repository navigation	fast-tier or inherit
Small formatting fixes, marker recovery, narrow edits with an explicit target	fast-tier or inherit, if scope is tight
Structured pull request review, first pass	fast-tier; GPT-style review model if available
Validation triage and checklist scanning	fast-tier or inherit
`cf-plan` on large, risky, multi-phase work	strong reasoning
Artifact transformation: PRD → DESIGN → DECOMPOSITION → FEATURE	strong reasoning; GPT-style often preferred
Brownfield understanding, reverse engineering, architecture review	strong reasoning
Code generation from an approved FEATURE or DESIGN	inherit; Claude-style models often stronger here
Final acceptance decision	human

approved specification into code. The gap in either direction is not dramatic. It is large enough to be worth a two-second model switch on a multi-model host, and not large enough to justify paying for a second subscription.

The last row of Table 2.5 is not a joke entry. No model tier replaces final human judgement, and this is the same point Chapter 1 made about green checkmarks, arriving from a different direction.

2.9 These Numbers Will Rot

Everything in the middle of this chapter — the request limits, the token allowances, the country eligibility lists — was checked in August 2026.

Free tiers, rate limits, and student eligibility change every few months, and sometimes without an announcement. The structure of the argument holds much longer than the numbers do: there will continue to be one or two frontier providers with a real free tier, local open weights will continue to trade capability for privacy and quota-freedom, cloud inference services will continue to compete on daily caps, and student programmes will continue to be generous and geographically arbitrary.

Before you rely on a specific limit, open the provider's own page and check it. The relevant sources are: the Gemini API pricing page, Anthropic's plans and pricing page, OpenAI's API pricing page, the Mistral La Plateforme tier documentation, the Ollama model library, Ollama's post on running Claude Code against a local model, OpenRouter's rate limit reference, and the GitHub Student Developer Pack page.

You now have the process from Chapter 1 and something to run it on. Chapter 3 spends both of them on a real project.

Exercises

1. **Get one host working end to end.** Pick Path A, B, or C, set it up, and confirm activation: open an initialised repository in your host tool, type `cf`, and then `cf-help`. Write down which path you chose, how long it took, and the first thing that went wrong. If nothing went wrong, write down which step you were least sure about.

2. **Re-verify three numbers.** Take three specific claims from this chapter — for example

the Gemini free-tier request rate, the OpenRouter free daily cap, and whether Codex for Students is available in your country — and check each one against the provider's current page. Note the date you checked and whether the claim still holds. If any has changed, write the corrected version in one line.

3. **Measure your own laptop's ceiling.** Install Ollama, pull one model in the 7–8B range and, if your machine allows it, one in the 27–32B range. Run the same short prompt against both and record the wall-clock time and whether the larger one fit in memory or spilled to CPU. Then say, in two sentences, which of the Chapter 1 operations you would trust each size with.

4. **Trace the local endpoint.** With `ollama serve` running, use the `curl` call from this chapter to hit `http://localhost:11434/v1/chat/completions` directly, then run `curl -s http://localhost:11434/api/tags`. Explain in your own words why the existence of an OpenAI-compatible `/v1` endpoint and an Anthropic-compatible endpoint on the same local server is what makes Path B possible at all.

5. **Plan a two-model split.** Assume you have a multi-model host and access to one GPT-style and one Claude-style model. Using Table 2.5, write down which model you would assign to each of the ten stages in Table 1.4 of Chapter 1, and justify the three assignments you are least confident about.

Chapter 3

The Full Pipeline, Walked Through

Chapter 1 showed the ten stages as a table. Chapter 2 got a model running behind them. This chapter walks the whole chain on a project that exists, is small enough to read in an afternoon, and was produced by exactly the process described here.

Everything quoted from here on is copied out of `examples/overwork_alert/` in the Constructor Studio repository. Clone it and follow along. The excerpts are verbatim; where something is cut for length it is marked.

3.1 The Project

Overwork Alert is a small macOS background tool that tracks your active work time and notifies you when you exceed a limit you configured. Its PRD describes it as "a local-first, single-user productivity helper intended for developers and knowledge workers who regularly lose track of time during deep work."

The interesting design point is in the word "active." It measures work time as active time: when you are idle longer than a configurable threshold, the timer pauses automatically and resumes when activity returns. No network, no menubar interface, no cloud sync, single user.

Here is the entire project:

```
examples/overwork_alert/
  architecture/
    PRD.md
    DESIGN.md
    DECOMPOSITION.md
    ADR/general/
      0001-cpt-overwork-alert-adr-cli-daemon-launchagent-no-menubar-v1.md
    features/
      tracker-core.md notifications.md
      cli-control.md launchagent-autostart.md
  src/overwork_alert/
    __init__.py __main__.py cli.py config.py
    daemon.py idle.py ipc.py launchagent.py
    models.py notification_policy.py notify.py
```

Four architecture documents, one ADR, four feature specifications, and eleven Python files totalling about 1,100 lines. That is the whole thing. The ratio of documents to code is high on purpose — the documents exist to show the chain, not because a three-hour timer needs an architecture board.

3.1.1 Registering the system

Before any of the stages, Constructor Fabric has to know that this system exists, where its documents live, and where its code lives. That goes in `artifacts.toml` under the setup directory's `config/`. The real entry for the examples in this repository looks like this:

```
[[systems]]
name = "Examples"
slug = "ex"
kit = "sdlc"
artifacts_dir = "examples"
children = []

[[systems.autodetect]]
kit = "sdlc"
system_root = "{project_root}/examples/$system"
artifacts_root = "{system_root}/architecture"

[systems.autodetect.artifacts.PRD]
pattern = "PRD.md"
traceability = "FULL"

[systems.autodetect.artifacts.FEATURE]
pattern = "features/*.md"
traceability = "FULL"
required = false

[[systems.autodetect.codebase]]
name = "Overwork Alert"
path = "{system_root}/src"
extensions = [".py"]
singleLineComments = ["#"]
multiLineComments = [{start = '"""', end = '"""'}]
```

Three things in that block do real work later.

`slug = "ex"` sets the system prefix that every identifier in this project inherits. That is why every ID you are about to see starts with `cpt-ex-`, with `ovwa` as the sub-slug for Overwork Alert inside the Examples system. The full pattern is `cpt-{system}-{subsystem}-{kind}-{slug}`.

`traceability = "FULL"` means code marker validation is active for that artifact kind. The alternative, `DOCS-ONLY`, prohibits code markers and checks only document-level identifiers and cross-references. You would use `DOCS-ONLY` for ADRs, which do not normally reach code.

The `codebase` block is what makes `cfs validate` and `cfs spec-coverage` able to find markers at all. It says where the source is, which extensions count, and how comments are written in that language. Without it, the chain stops at the last document, and every marker you carefully wrote is invisible.

3.2 Stages 1 and 2: Explore, Then Open the Options

The first two stages leave no artifact behind, which is why they are the two people skip.

```
cf-explore: gather existing product notes, customer constraints,
          and related artifacts
```

For a greenfield tool like this one there is not much to find, and that is itself the useful output: you learn in two minutes that there is no prior art in the repository, no existing idle-detection utility to reuse, and no conventions to inherit. On a brownfield change — adding a billing workflow to an existing product — this stage is the difference between a PRD that respects three existing constraints and a PRD that contradicts them.

```
cf-brainstorm: map the options for a work-time tracker on macOS
```

This is where the three candidate shapes for Overwork Alert came from: a menubar application, a CLI daemon with a LaunchAgent, and a scheduled `launchd` job running periodically. You can see all three preserved in the ADR later in this chapter, which is a good sign — the options that were rejected are still written down.

Use `cf-brainstorm` while scope is open. Once the PRD exists, it is too late for this to change anything.

3.3 Stage 3: The Product Requirements Document

```
cf-sdlc-doc-prd: write the Product Requirements Document
               for the overwork alert tool
```

The skill fills the kit's PRD template. The result has fifteen numbered sections: overview, actors, operational concept, scope, functional requirements, non-functional requirements, public interfaces, use cases, acceptance criteria, dependencies, assumptions, risks, non-goals, open questions, and additional context.

That is a lot of sections for a timer. Most of them are two lines long, and some say "(none)" or "(n/a)." That is intentional and it is one of the reasons the template is worth having: a section that says `(n/a)` tells a reviewer "considered and excluded," whereas a missing section tells a reviewer nothing at all.

3.3.1 Actors get identifiers too

```
#### User

**ID**: ‘cpt-ex-ovwa-actor-user‘

**Role**: Wants to be notified when they have worked too long, adjust
configuration, and control the tracker (status/pause/resume/reset).
```

There are three actors: `cpt-ex-ovwa-actor-user`, `cpt-ex-ovwa-actor-macos`, and `cpt-ex-ovwa-actor-login-runner`. Each requirement then lists which actors it involves, by ID. This is what lets you later ask "what does the login runner actually touch" and get an answer from the repository.

3.3.2 A functional requirement

```
#### FR-002 Configure limit and idle threshold

- [x] ‘p1‘ - **ID**: ‘cpt-ex-ovwa-fr-configurable-limit‘

The system MUST allow the user to configure:

- A daily/session work-time limit (default: 3 hours).
- An idle threshold used to pause active time (default: 5 minutes).
- A repeat reminder interval after the first over-limit alert
  (default: 30 minutes).

Configuration MUST have safe defaults if no configuration is present.

**Actors**:
‘cpt-ex-ovwa-actor-user‘
```

Take that line apart, because its shape repeats a few hundred times through the rest of this chapter.

`[x]` is the task checkbox. It means implemented. `[]` means not yet. `cfs validate` enforces consistency across the chain: a reference marked `[x]` may not point at a definition still marked `[]`.

‘`p1`‘ is the priority, or implementation phase — `p1` through `p9`. It tells you which slice of work this belongs to, and it becomes part of the code marker later.

‘`cpt-ex-ovwa-fr-configurable-limit`‘ is the identifier. `ex` is the system, `ovwa` the subsystem, `fr` the kind (functional requirement), and `configurable-limit` the slug.

Then the requirement text, using MUST in the RFC sense, with the three defaults stated numerically. Remember those three numbers — three hours, five minutes, thirty minutes. They will show up again as `10800`, `300`, and `1800` in `models.py`, four documents from now.

The other five functional requirements follow the same shape:

- `cpt-ex-ovwa-fr-track-active-time` (`p1`) — track active work time, idle-aware, pausing past the threshold and resuming on activity.
- `cpt-ex-ovwa-fr-notify-on-limit` (`p1`) — notify when the limit is exceeded, and repeat at the configured interval until the user goes idle or pauses.
- `cpt-ex-ovwa-fr-manual-reset` (`p2`) — manual reset must exist, and time-of-day automatic reset must not, in v1.
- `cpt-ex-ovwa-fr-autostart` (`p2`) — must run continuously in the background, should support login autostart.
- `cpt-ex-ovwa-fr-cli-controls` (`p2`) — start, status, pause, resume, reset from a CLI.

Note the priority split. The three `p1` requirements are the product; the three `p2` ones are how you drive it. That split is what makes the decomposition in stage 6 possible.

3.3.3 Non-functional requirements, with IDs

Three of them, and they are short:

```
#### Privacy & Data Handling

- [x] `p1` - **ID**: `cpt-ex-ovwa-nfr-privacy-local-only`

- The system MUST be local-first and MUST NOT send tracking data over
  the network by default.
- The system MUST store only minimal local state required to implement
  tracking and alerting.
```

Plus `cpt-ex-ovwa-nfr-reliability` (degrade gracefully if notifications cannot be delivered — tracking continues, CLI status stays available) and `cpt-ex-ovwa-nfr-low-overhead` (suitable for always-on background use, avoid high-frequency polling).

An NFR with an identifier is an NFR that can be traced. You will see all three of these appear again in the DESIGN's NFR allocation section and in the decomposition's coverage lists. An NFR without an ID is a wish.

3.3.4 How you know it worked

```
## 9. Acceptance Criteria

- [ ] Install and first-run setup completed in 10 minutes or less
      on macOS.
- [ ] After exceeding the configured limit while active, the first
      alert appears within 5 seconds.
- [ ] When idle exceeds the configured threshold, active-time
      accumulation pauses within 10 seconds.
- [ ] Users can verify current status via CLI in under 5 seconds.
```

Every one of them has a number in it. That is the entire point of the section. An acceptance criterion without a number is a preference.

Two of these four numbers propagate all the way into a feature specification later: the 5-second first-alert bound and the 10-second accumulation-pause bound.

3.3.5 And the parts that admit what is missing

The PRD has a risks table, a non-goals list, and an open questions section that names two genuinely unresolved decisions:

```
### Open Questions

- Should screen lock be treated as immediate idle regardless of the idle
  threshold? - Owner: User, Target: next iteration
- Should notifications include sound by default, or be notification-only?
  - Owner: User, Target: next iteration
```

Each has an owner and a target. Neither blocks v1. This is the section that stops a document from pretending to more certainty than it has, and it is the first place a reviewer should look.

3.3.6 Review, then check

The document is reviewed against the configured methodology and checklist. You see the findings *before* anything is rewritten, and you approve the fix scope. Then the deterministic part:

```
$ cfs validate
```

At this point in a real run, validation passes with warnings rather than errors, and the warnings are all of one kind: the PRD declares identifiers whose coverage targets are DESIGN, DECOMPOSITION, and FEATURE — documents that do not exist yet. That is expected. The warnings drain away as the chain gets built. Do not try to make them zero on day one; you cannot, and chasing them is how people end up writing all four documents in one pass without reviewing any of them.

3.4 Stage 4: One Decision, Written Down

```
cf-sdlc-doc-adr: record the architecture decision for the execution
                 model of the overwork alert tool
```

The ADR is written before the design, so the design has something to stand on.

```
# ADR-0001: Use CLI daemon + LaunchAgent (no menubar UI)

**ID**: `cpt-ex-ovwa-adr-cli-daemon-launchagent-no-menubar`

## Considered Options

* Menubar application
* CLI daemon + LaunchAgent (chosen)
* Scheduled launchd job (periodic execution)

## Decision Outcome

Chosen option: "CLI daemon + LaunchAgent", because it provides
continuous background tracking and optional autostart while keeping
the example small, local-first, and traceability-friendly.
```

The document has a fixed shape: context and problem statement, decision drivers, considered options, decision outcome, consequences, confirmation, pros and cons of each option, more information, traceability.

Two of those sections are the ones that pay off later.

Consequences states the cost of the choice, not just the benefit:

```
* Good, because the implementation remains small and repository-friendly.
* Good, because it creates a clean separation between user-facing CLI
  controls and the background tracking loop.
* Bad, because the UX is less discoverable than a menubar app.
* Bad, because it requires defining and implementing a local control
  channel between CLI and daemon.
```

That second "Bad" is a prediction, and it came true: `ipc.py` exists solely because of it, 173 lines of Unix socket plumbing that would not have been needed in a menubar app. An ADR that only lists upsides has not made a decision; it has written an advertisement.

Confirmation says how you will know the decision worked:

```
This ADR is confirmed when:

* The system can start the daemon via CLI and keep it running in
  background.
* Autostart can be installed/uninstalled idempotently.
* CLI control flows (status/pause/resume/reset/stop) work through a
  local-only channel.
```

And the traceability section closes the loop back to the PRD:

```
This decision directly addresses the following requirements:

* ‘cpt-ex-ovwa-fr-autostart‘ - Autostart requirement influenced the use
  of LaunchAgent.
* ‘cpt-ex-ovwa-fr-cli-controls‘ - CLI control requirements influenced
  the CLI/daemon approach.
```

Two requirement IDs from the PRD, reused verbatim, each with one line explaining how it pushed the decision. Six months later this is the file that answers "why on earth did we do it this way," and the answer is specific rather than nostalgic.

3.5 Stage 5: The DESIGN Document

```
cf-sdlc-doc-design: write the DESIGN document from the approved
                    Product Requirements Document and
                    Architecture Decision Records
```

Read that request again. The inputs are *named*. This skill transforms approved documents; it does not invent an architecture from a one-line prompt. Give it a one-line prompt and that is exactly what it will have to do.

3.5.1 Drivers: requirement in, solution out

The first substantial section is architecture drivers, and its structure is the heart of the whole method:

```
##### Track active work time (idle-aware)

- [x] ‘p1‘ - ‘cpt-ex-ovwa-fr-track-active-time‘

**Solution**: A daemon runs a periodic loop. Each tick reads current
idle time from macOS (idle detector) and increments ‘active_time‘
only when idle is below the configured threshold and tracking is
not paused.
```

Requirement identifier on the left, engineering answer on the right. The ID from the PRD reappears here unchanged. That repetition *is* the traceability mechanism. It is not clever, and it works.

The same pattern covers all six functional requirements and all three NFRs. Two more, because they show how a design answer differs from a requirement:

```
##### Notify when limit is exceeded and repeat reminders

- [x] ‘p1‘ - ‘cpt-ex-ovwa-fr-notify-on-limit‘

**Solution**: The daemon transitions into an "over limit" state when
‘active_time > limit‘. A notifier component sends a macOS notification
immediately, then repeats at the configured interval while the user
remains active and the session is still over limit.
```

```
##### Reliability

- [x] `p2` - `cpt-ex-ovwa-nfr-reliability`

**Solution**: Notification failures are treated as non-fatal; tracking
continues and status remains queryable via CLI. The daemon loop isolates
OS integration errors so a single failed tick cannot crash the process.
```

The requirement said "degrade gracefully." The design says which errors are isolated where. That is the transformation the stage exists to perform.

The drivers section also carries the ADR forward, by ID:

```
##### Use CLI daemon + LaunchAgent (no menubar UI)

- `cpt-ex-ovwa-adr-cli-daemon-launchagent-no-menubar`

The implementation is split into a long-running daemon and a short-lived
CLI. This keeps v1 small and repository-friendly, enables autostart via
LaunchAgent, and avoids UI lifecycle complexity.
```

3.5.2 Three layers

Table 3.1: Architecture layers, from DESIGN §1.3.

Layer	Responsibility	Technology
CLI / Control	User commands (status/pause/resume/reset), install and uninstall autostart	Python CLI (argparse), local IPC
Tracking Core	Idle-aware accumulation, over-limit state, reminder scheduling	Python daemon loop
OS Integration	Idle time signal, notification delivery, login autostart	`ioreg` idle query, `osascript` notification, launchd LaunchAgent

Three layers for a tool this size. The document is proportionate to the problem, which is worth noticing because the failure mode of template-driven design is a document that describes a nine-layer enterprise architecture around a timer.

3.5.3 Principles and constraints, also with IDs

```
#### Predictable, explicit user control

- [x] `p1` - **ID**: `cpt-ex-ovwa-principle-explicit-control`

The system should not make time-based policy decisions like automatic
midnight resets. Resetting and pausing are explicit user actions via
the CLI.
```

```
#### No automatic reset and no persistence of accumulated time

- [x] `p1` - **ID**: `cpt-ex-ovwa-constraint-no-auto-reset-no-persist`

Accumulated active work time is session-scoped: it resets when the
daemon restarts and only resets during runtime when the user explicitly
invokes manual reset.
```

There are three principles and two constraints, each with an ID: `cpt-ex-ovwa-principle-local-only-minimal-state`, `cpt-ex-ovwa-principle-explicit-control`, `cpt-ex-ovwa-principle-low-overhead`, `cpt-ex-ovwa-constraint-macos-cli-only`, and `cpt-ex-ovwa-constraint-no-auto-reset-no-persist`.

Giving a principle an ID sounds like bureaucracy until the first time somebody proposes a change that quietly violates one. Then you can point at the string and ask which document approved the violation.

3.5.4 Domain model, components, sequences, data

The technical architecture section carries the parts that the code will mirror almost one-to-one.

The domain model is four types and three invariants:

```
- ‘Config‘: ‘limit_seconds‘, ‘idle_threshold_seconds‘,
  ‘repeat_interval_seconds‘, and integration settings (e.g., socket path).
- ‘TrackerState‘: ‘status‘ (running/paused), ‘active_time_seconds‘,
  ‘over_limit_since‘, ‘last_reminder_at‘, ‘last_tick_at‘.
- ‘IdleSample‘: current ‘idle_seconds‘ observed from macOS.
- ‘ControlCommand‘: one of ‘status‘, ‘pause‘, ‘resume‘, ‘reset‘, ‘stop‘.

Invariants:

- ‘active_time_seconds‘ increases only while ‘status=running‘ and
  ‘idle_seconds < idle_threshold_seconds‘.
- When ‘active_time_seconds > limit_seconds‘, the system is considered
  "over limit" until a reset occurs.
- ‘reset‘ sets ‘active_time_seconds‘ to zero and clears over-limit
  reminder scheduling state.
```

Six components follow, each with an ID, a reason to exist, a responsibility scope, explicit boundaries, and dependencies:

- `cpt-ex-ovwa-component-cli` (`p1`) — the sole user-facing entry point.
- `cpt-ex-ovwa-component-daemon` (`p1`) — holds `TrackerState`, runs the tick loop, decides when to notify.
- `cpt-ex-ovwa-component-control-channel` (`p2`) — local-only CLI-to-daemon communication.
- `cpt-ex-ovwa-component-idle-detector` (`p2`) — queries macOS for idle duration.
- `cpt-ex-ovwa-component-notifier` (`p2`) — delivers one macOS notification.
- `cpt-ex-ovwa-component-config-loader` (`p2`) — loads config, applies defaults.

The boundary statements are the useful part. The CLI component's read:

```
##### Responsibility boundaries

- No tracking logic; no accumulation; no background loop
- Does not read idle time or send notifications
```

That is a design decision written as a prohibition, and it is checkable by reading `cli.py`. Which, when you do, holds: `cli.py` parses arguments, sends JSON over a socket, and prints. It never touches `idle.py` or `notify.py`.

Two sequences are specified with Mermaid diagrams and prose: `cpt-ex-ovwa-seq-run-and-alert` (`p1`) for the tick-and-notify loop, and `cpt-ex-ovwa-seq-cli-reset` (`p2`) for a reset arriving over the control channel.

The data section is the one that shows the method adapting rather than insisting. This tool has no database. Instead of skipping the section, the document keeps it and relabels it:

```
No persistent database is used in v1. The following "tables" describe the
conceptual runtime data model that is held in memory only.

#### Table tracker_state (conceptual; in-memory only)

- [x] ‘p2‘ - **ID**: ‘cpt-ex-ovwa-dbtable-tracker-state‘
```

Two conceptual tables, `cpt-ex-ovwa-dbtable-tracker-state` and `cpt-ex-ovwa-dbtable-config`, each with a column schema, constraints, and a worked example row. The config table's example row is:

```
| limit_seconds | idle_threshold_seconds | repeat_interval_seconds |
  control_socket_path |
| 10800 | 300 | 1800 |
  /tmp/overwork-alert.sock |
```

There are the three numbers from FR-002, converted to seconds for the first time.

Finally, the tech stack, marked **Accepted**:

```
Python 3.13+ using the standard library for the daemon loop, CLI, IPC, and
configuration parsing. macOS integration is done via ‘launchd‘ (LaunchAgent
plist + ‘launchctl‘), idle time querying via ‘ioreg‘, and Notification
Center delivery via ‘osascript‘.
```

No third-party dependencies. For a tool that has to run always-on on somebody's laptop, that is a design decision, and putting it in the document means it is one you can be held to.

3.6 Stage 6: Decompose

```
cf-sdlc-decompose: decompose the DESIGN document into
                   reviewable FEATURE work
```

The DECOMPOSITION document turns one design into an ordered list of features with explicit dependencies. It opens by stating its own strategy:

```
**Decomposition Strategy**:
- Group by functional cohesion (each feature implements a coherent
  responsibility)
- Keep dependencies minimal and explicit (tracker core is the foundation)
- Ensure 100% coverage of DESIGN elements (components, sequences, and
  data model items assigned)
- Maintain mutual exclusivity (each component/sequence/data element is
  owned by a single feature)
```

Those last two lines are the checkable ones. Coverage means every component, sequence, and data element in the DESIGN belongs to a feature. Mutual exclusivity means it belongs to exactly one. Together they are what stops the classic decomposition failure where two teams both think the other one owns the idle detector.

Here is one entry in full:

```
### 1. [Tracking Core](features/tracker-core.md) - HIGH

- [x] ‘p1‘ - **ID**: ‘cpt-ex-ovwa-feature-tracker-core‘

- **Purpose**: Implement the daemon tracking loop and the idle-aware
  active-time accumulation model with safe configuration defaults.

- **Depends On**: None

- **Scope**:
  - Load configuration with safe defaults and validation
  - Sample macOS idle time and determine active vs idle
  - Maintain session-scoped in-memory tracker state (no persistence)
  - Accumulate active time only when running and not idle

- **Out of scope**:
  - Persisting accumulated time across restarts
  - Automatic time-of-day resets

- **Requirements Covered**:
  - [x] ‘p1‘ - ‘cpt-ex-ovwa-fr-track-active-time‘
  - [x] ‘p1‘ - ‘cpt-ex-ovwa-fr-configurable-limit‘
  - [x] ‘p1‘ - ‘cpt-ex-ovwa-nfr-privacy-local-only‘
```

```
  - [x] `p2` - `cpt-ex-ovwa-nfr-low-overhead`

- **Design Principles Covered**:
  - [x] `p1` - `cpt-ex-ovwa-principle-local-only-minimal-state`
  - [x] `p2` - `cpt-ex-ovwa-principle-low-overhead`

- **Design Constraints Covered**:
  - [x] `p1` - `cpt-ex-ovwa-constraint-no-auto-reset-no-persist`
  - [x] `p1` - `cpt-ex-ovwa-constraint-macos-cli-only`

- **Domain Model Entities**:
  - Config
  - TrackerState
  - IdleSample

- **Design Components**:
  - [x] `p1` - `cpt-ex-ovwa-component-daemon`
  - [x] `p2` - `cpt-ex-ovwa-component-idle-detector`
  - [x] `p2` - `cpt-ex-ovwa-component-config-loader`

- **API**:
  - `overwork-alert start`

- **Sequences**:
  - [x] `p1` - `cpt-ex-ovwa-seq-run-and-alert`

- **Data**:
  - [x] `p2` - `cpt-ex-ovwa-dbtable-tracker-state`
  - [x] `p2` - `cpt-ex-ovwa-dbtable-config`
```

Notice what that entry is: a contract. It names what it covers from upstream documents, by ID, in seven separate coverage lists. It names what it does not cover. It names the components it owns, so no other feature can claim them. And it names its own dependency — `None`, which is why it is number one.

The other three entries are shorter but structurally identical:

- `cpt-ex-ovwa-feature-notifications` (HIGH, `p1`) — send the alert and the repeats. Depends on tracker core. Owns `cpt-ex-ovwa-component-notifier`. Covers `cpt-ex-ovwa-fr-notify-on-limit` and `cpt-ex-ovwa-nfr-reliability`.
- `cpt-ex-ovwa-feature-cli-control` (MEDIUM, `p2`) — status/pause/resume/reset/stop plus the control channel. Depends on tracker core. Owns `cpt-ex-ovwa-component-cli` and `cpt-ex-ovwa-component-control-channel`.
- `cpt-ex-ovwa-feature-launchagent-autostart` (MEDIUM, `p2`) — the plist and `launchctl`. Depends on *CLI control*, not on tracker core, "because autostart is installed/uninstalled via CLI commands."

And the dependency graph, written out:

```
cpt-ex-ovwa-feature-tracker-core
    |
    +-> cpt-ex-ovwa-feature-notifications
    +-> cpt-ex-ovwa-feature-cli-control
    +-> cpt-ex-ovwa-feature-launchagent-autostart
```

This is where one piece of work becomes four reviewable pieces. Somebody can implement and merge tracker core on its own, and the repository is in a defensible state afterwards: a daemon that accumulates time and does nothing with it. That is a much better place to stop for the weekend than half of all four features.

3.7 Stage 7: The FEATURE Specification

```
cf-sdlc-doc-feature: write the FEATURE spec for the notification
                    policy and delivery
```

This is the last document before code, and the one the implementation is checked against. We will follow `features/notifications.md` in detail because it is the smallest of the four and the most directly traceable into code.

Each feature spec has seven sections: feature context, actor flows, processes and business logic, states, definitions of done, acceptance criteria, and additional context. The behavioural sections — flows, processes, states — are written in CDSL, a plain-English specification language where every step carries its own checkbox, phase, and instruction ID.

3.7.1 Context: pinning the numbers down

```
Configuration parameters (effective defaults in v1):
- limit_seconds: 10800 (3 hours)
- idle_threshold_seconds: 300 (5 minutes)
- repeat_interval_seconds: 1800 (30 minutes)

Acceptance criteria (timing):
- After the tracker first becomes over limit while RUNNING and user is
  active, the first notification MUST be delivered within 5 seconds.
- Repeat reminders MUST NOT occur more frequently than
  repeat_interval_seconds.
```

The 5-second bound came from the PRD's acceptance criteria, four documents ago, and has now been restated in the terms this feature can actually be tested in.

3.7.2 An actor flow in CDSL

```
### Send first over-limit alert

- [x] **ID**: ‘cpt-ex-ovwa-flow-notifications-first-alert‘

**Actors**:
- ‘cpt-ex-ovwa-actor-user‘

1. [x] - ‘p1‘ - Daemon observes active_time_seconds > limit_seconds
   - ‘inst-detect-over-limit‘
2. [x] - ‘p1‘ - **IF** tracker status is not RUNNING **RETURN**
   do_not_notify - ‘inst-skip-on-not-running‘
3. [x] - ‘p1‘ - **IF** current idle_seconds >= idle_threshold_seconds
   **RETURN** do_not_notify - ‘inst-skip-on-idle‘
4. [x] - ‘p1‘ - **IF** over-limit has not been notified yet:
   - ‘inst-check-first-alert‘
5. [x] - ‘p1‘ - Notification Sender delivers macOS notification
   (title + message) - ‘inst-send-notification‘
6. [x] - ‘p1‘ - Daemon records over_limit_since and last_reminder_at
   - ‘inst-record-notify-state‘
```

Six steps, each with a checkbox, a phase, English prose, and an instruction ID. The instruction IDs are the finest-grained thing in the whole system, and they are what code markers attach to.

There is a second flow, `cpt-ex-ovwa-flow-notifications-repeat-reminder`, with the same shape and its own six instruction IDs (`inst-still-over-limit`, `inst-skip-repeat-on-not-running`, `inst-skip-repeat-on-idle`, `inst-check-interval`, `inst-send-reminder`, `inst-update-last-reminder`).

3.7.3 The algorithm

```
### Determine whether to send a notification

- [x] **ID**: ‘cpt-ex-ovwa-algo-notifications-should-notify‘

1. [x] - ‘p1‘ - **IF** tracker status is not RUNNING **RETURN**
   do_not_notify - ‘inst-not-running‘
2. [x] - ‘p1‘ - **IF** current idle_seconds >= idle_threshold_seconds
   **RETURN** do_not_notify - ‘inst-currently-idle‘
3. [x] - ‘p1‘ - **IF** active_time_seconds <= limit_seconds **RETURN**
   do_not_notify - ‘inst-not-over-limit‘
4. [x] - ‘p1‘ - **IF** first alert not sent yet **RETURN** notify_now
   - ‘inst-first-alert‘
5. [x] - ‘p1‘ - **IF** now - last_reminder_at >= repeat_interval_seconds
   **RETURN** notify_now - ‘inst-repeat-alert‘
6. [x] - ‘p1‘ - **RETURN** do_not_notify - ‘inst-default-no‘
```

That is a pure function specified in six lines of English, with a guard-clause structure that will translate directly into Python. Keep it in view — we are about to read the implementation, and the correspondence is line for line.

3.7.4 The state machine

```
### Over-limit notification state

- [x] **ID**: ‘cpt-ex-ovwa-state-notifications-over-limit‘

1. [x] - ‘p1‘ - **FROM** UNDER_LIMIT **TO** OVER_LIMIT_FIRST_SENT
   **WHEN** first alert is delivered - ‘inst-transition-first‘
2. [x] - ‘p1‘ - **FROM** OVER_LIMIT_FIRST_SENT **TO**
   OVER_LIMIT_REMINDING **WHEN** reminder is delivered
   - ‘inst-transition-remind‘
3. [x] - ‘p1‘ - **FROM** OVER_LIMIT_REMINDING **TO** UNDER_LIMIT
   **WHEN** session is reset - ‘inst-transition-reset‘
```

Three transitions. Note the third one: the only way back to `UNDER_LIMIT` is a reset. That is `cpt-ex-ovwa-principle-explicit-control` from the DESIGN, expressed as a missing edge in a state machine.

3.7.5 The definition of done

```
### Over-limit notifications and repeat reminders

- [x] ‘p1‘ - **ID**: ‘cpt-ex-ovwa-dod-notifications-alert-and-repeat‘

When active time exceeds the configured limit while tracking is RUNNING
and the user is active (idle below threshold), the system sends a macOS
notification within 5 seconds. While the user remains active and over
limit, the system repeats notifications at the configured repeat
interval.

**Implementation details**:
- Component: ‘cpt-ex-ovwa-component-notifier‘
- Data: ‘cpt-ex-ovwa-dbtable-tracker-state‘

**Implements**:
- ‘p1‘ - ‘cpt-ex-ovwa-flow-notifications-first-alert‘
- ‘p1‘ - ‘cpt-ex-ovwa-flow-notifications-repeat-reminder‘

- ‘p1‘ - ‘cpt-ex-ovwa-algo-notifications-should-notify‘

**Covers (PRD)**:
- ‘cpt-ex-ovwa-fr-notify-on-limit‘

- ‘cpt-ex-ovwa-nfr-reliability‘

**Covers (DESIGN)**:
- ‘cpt-ex-ovwa-principle-explicit-control‘
```

```
- `cpt-ex-ovwa-constraint-no-auto-reset-no-persist`

- `cpt-ex-ovwa-component-notifier`
- `cpt-ex-ovwa-component-daemon`

- `cpt-ex-ovwa-seq-run-and-alert`

- `cpt-ex-ovwa-dbtable-tracker-state`
```

This block is the join in the middle of the whole system. Read downward and it tells you which flows, algorithms, and states it is made of. Read upward and it tells you which PRD requirement, which NFR, which DESIGN principle, constraint, component, sequence, and data element it discharges.

A definition of done that lists what it implements and what it covers is answerable. Everything after this point is checkable rather than arguable.

3.8 Stage 8: The Code, and Its Markers

```
cf-sdlc-implement: implement the approved FEATURE specification
                   with CPT markers and unit tests
```

3.8.1 The marker vocabulary

Before reading the code, here is the whole notation. Table 3.2 lists the marker kinds that appear in this project, and Table 3.3 shows where they landed.

Table 3.2: CPT marker kinds used in Overwork Alert.

Marker	Meaning
`@cpt-dod:`	This function or block discharges a definition of done.
`@cpt-flow:`	Scope marker: this function implements an actor flow.
`@cpt-algo:`	Scope marker: this function implements a specified algorithm.
`@cpt-state:`	Scope marker: this function implements a state machine.
`@cpt-begin:`	Start of the block implementing one CDSL instruction.
`@cpt-end:`	End of that block.

The form of a scope marker is `@cpt-<kind>:<artifact-id>:<phase>`. The form of a block marker adds the instruction: `@cpt-begin:<artifact-id>:<phase>:<inst-id>`.

They are plain comments. Any language, any editor, no plugin required. That is the entire reason the mechanism works across a polyglot repository.

Table 3.3: Marker density across the nine implementation modules.

Module	Lines	Markers	Role
`daemon.py`	317	99	tick loop, notify policy, command handling
`cli.py`	186	54	argument parsing, control requests, output
`launchagent.py`	146	41	plist build, install, uninstall
`notification_policy.py`	76	19	pure notify decision
`ipc.py`	173	5	Unix socket server and client
`config.py`	73	1	load config, apply defaults
`models.py`	60	0	domain types
`idle.py`	41	0	`ioreg` idle sample
`notify.py`	29	0	`osascript` delivery

Look at the bottom three rows, because they are the most instructive thing in the table. `models.py`, `idle.py`, and `notify.py` carry no markers at all.

That is not an oversight. Markers attach to *specified behaviour* — flows, algorithms, states, and definitions of done written in CDSL. `models.py` declares data types. `idle.py` and `notify.py` are the OS-integration boundary: they shell out to `ioreg` and `osascript` and return best-effort results. There is no CDSL flow that says "parse `HIDIdleTime` out of `ioreg` output with a regex," because that is implementation judgement, which is what Chapter 1 said to leave to the model and the engineer.

Marker density tells you where the specified behaviour is. Its absence tells you where the adapters are. Both are information.

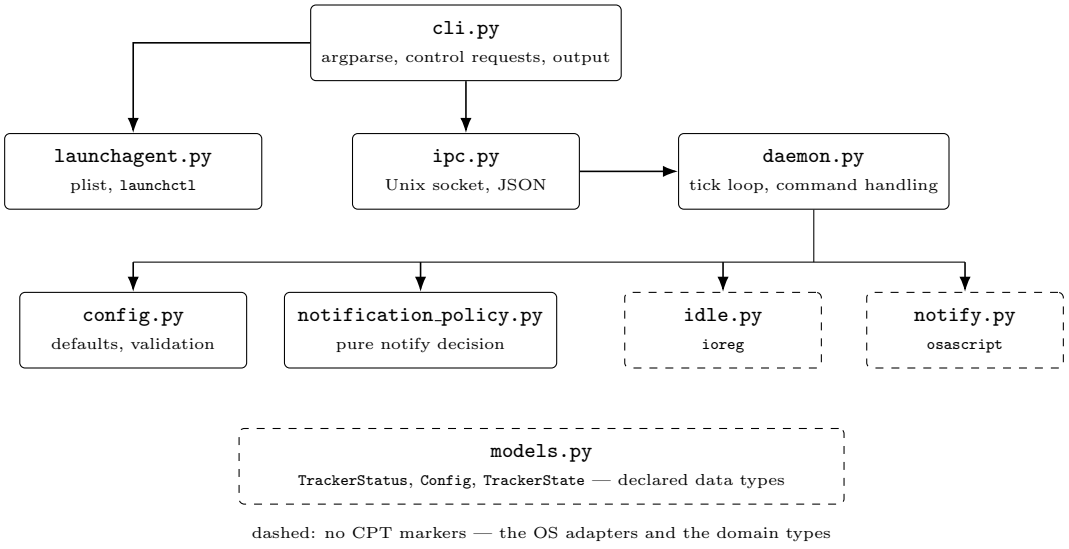


Figure 3.1: The nine modules, and which of them carry no markers.

3.8.2 The domain model

`models.py`, verbatim except for the surrounding imports:

```
class TrackerStatus(str, Enum):
    """Tracker runtime status."""

    RUNNING = "running"
    PAUSED = "paused"


@dataclass(frozen=True)
class Config:
    """Effective configuration used by the daemon and CLI."""

    limit_seconds: int = 10800
    idle_threshold_seconds: int = 300
    repeat_interval_seconds: int = 1800
    tick_interval_seconds: int = 5
    max_tick_delta_seconds: int = 10
    control_socket_path: str = "/tmp/overwork-alert.sock"


@dataclass
class TrackerState:
    """In-memory daemon session state."""

    status: TrackerStatus = TrackerStatus.RUNNING
    active_time_seconds: int = 0
    last_tick_at: float | None = None
    over_limit_since: float | None = None
    last_reminder_at: float | None = None
```

`10800` is three hours. `300` is five minutes. `1800` is thirty minutes. Those are the three defaults from FR-002, five documents earlier, arriving in code as dataclass field defaults.

`Config` is `frozen=True` — immutable, because configuration is loaded once per tick and never mutated. `TrackerState` is not frozen, because the command handler mutates it in place under a lock. The field names match the DESIGN's domain model exactly: `status`, `active_time_seconds`, `over_limit_since`, `last_reminder_at`, `last_tick_at`.

3.8.3 The algorithm, marked line by line

Now `notification_policy.py`. Compare it against the six CDSL steps of `cpt-ex-ovwa-algo-notifications-should-notify` from stage 7:

```
# @cpt-algo:cpt-ex-ovwa-algo-notifications-should-notify:p1
def should_notify(*, state: TrackerState, config: Config,
                  idle_seconds: int | None, now: float) -> bool:
    """Return True if a notification should be delivered now."""
    # @cpt-begin:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-not-running
    if state.status != TrackerStatus.RUNNING:
        return False
    # @cpt-end:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-not-running

    # @cpt-begin:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-currently-idle
    if idle_seconds is None:
        return False

    if idle_seconds >= config.idle_threshold_seconds:
        return False
    # @cpt-end:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-currently-idle

    # @cpt-begin:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-not-over-limit
    if state.active_time_seconds <= config.limit_seconds:
        return False
    # @cpt-end:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-not-over-limit

    # @cpt-begin:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-first-alert
    if state.over_limit_since is None:
        return True
    # @cpt-end:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-first-alert

    if state.last_reminder_at is None:
        return True

    # @cpt-begin:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-repeat-alert
    if (now - state.last_reminder_at) >= config.repeat_interval_seconds:
        return True
    # @cpt-end:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-repeat-alert

    # @cpt-begin:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-default-no
    return False
    # @cpt-end:cpt-ex-ovwa-algo-notifications-should-notify:p1:inst-default-no
```

Read the two side by side and the correspondence is exact. CDSL step 1 becomes the `inst-not-running` block. Step 2 becomes `inst-currently-idle`. Step 3 becomes `inst-not-over-limit`, and note that the code uses `<=` exactly as the specification wrote it, not `<`. Steps 4, 5, and 6 become `inst-first-alert`, `inst-repeat-alert`, and `inst-default-no`.

Three details in that function are worth stopping on.

First, `inst-currently-idle` wraps *two* guards, not one. The CDSL step says "IF current idle_seconds ¿= idle_threshold_seconds RETURN do_not_notify." The code also handles `idle_seconds is None` — the case where the `ioreg` sample failed — inside the same marked block. A marked block is a region of code that discharges an instruction, not necessarily a single statement.

Second, and more interesting, look at these two lines:

```
    if state.last_reminder_at is None:
        return True
```

They sit between the `inst-first-alert` block and the `inst-repeat-alert` block, and they carry no marker at all. That is a defensive guard the implementer added: a state that is over limit, has an `over_limit_since` timestamp, but somehow has no `last_reminder_at`. No CDSL step describes it, so no marker claims it.

This is exactly what markers do and do not guarantee. They guarantee that every `[x]` instruction in the specification has a corresponding region of code. They do not guarantee that every line of code corresponds to an instruction. Unmarked code is legal, and reviewing it is a human job — `cfs validate` will not ask you about those two lines.

Third, `should_notify` is a pure function: no clock reading, no I/O, `now` passed in as a parameter. The module docstring says so out loud: "This module does not perform OS notification delivery; it only decides when to notify and updates in-memory scheduling state." A specified algorithm that has been implemented as a pure function is one you can unit test exhaustively, which is a large part of why the specification bothered to isolate it.

3.8.4 A state machine in code

The second function in the same module implements the three-transition state machine:

```
# @cpt-dod:cpt-ex-ovwa-dod-notifications-alert-and-repeat:p1
def apply_notification_policy(
    *,
    state: TrackerState,
    config: Config,
    idle_seconds: int | None,
    now: float | None = None,
) -> TrackerState:
    """Update notification scheduling state after a notification is delivered."""
    # @cpt-state:cpt-ex-ovwa-state-notifications-over-limit:p1
    n = time.time() if now is None else now

    if not should_notify(state=state, config=config,
                         idle_seconds=idle_seconds, now=n):
        return state

    if state.over_limit_since is None:
        # @cpt-begin:cpt-ex-ovwa-state-notifications-over-limit:p1:inst-transition-first
        return replace(state, over_limit_since=n, last_reminder_at=n)
        # @cpt-end:cpt-ex-ovwa-state-notifications-over-limit:p1:inst-transition-first

    # @cpt-begin:cpt-ex-ovwa-state-notifications-over-limit:p1:inst-transition-remind
    return replace(state, last_reminder_at=n)
    # @cpt-end:cpt-ex-ovwa-state-notifications-over-limit:p1:inst-transition-remind
```

Two markers on one function, doing two different jobs. `@cpt-dod:` says this function discharges `cpt-ex-ovwa-dod-notifications-alert-and-repeat`. `@cpt-state:` says it implements `cpt-ex-ovwa-state-notifications-over-limit`. Then two of the three transitions appear as marked blocks.

The third transition, `inst-transition-reset`, is not here — because a reset does not arrive through the notification policy. It arrives over the control channel. Its marker lives in `daemon.py`, inside the command handler:

```
            if cmd == "reset":
                # @cpt-flow:cpt-ex-ovwa-flow-cli-control-reset:p1
                # @cpt-begin:cpt-ex-ovwa-algo-cli-control-handle-command:p1:inst-handle-reset
                # @cpt-begin:cpt-ex-ovwa-flow-cli-control-reset:p1:inst-clear-state
                self._state.active_time_seconds = 0
                # @cpt-begin:cpt-ex-ovwa-state-notifications-over-limit:p1:inst-transition-reset
                self._state.over_limit_since = None
                self._state.last_reminder_at = None
                # @cpt-end:cpt-ex-ovwa-state-notifications-over-limit:p1:inst-transition-reset

                # @cpt-end:cpt-ex-ovwa-flow-cli-control-reset:p1:inst-clear-state

                # @cpt-end:cpt-ex-ovwa-algo-cli-control-handle-command:p1:inst-handle-reset
```

```
            return {"ok": True}
```

Three nested marked blocks around three statements, from three different artifact identifiers: a flow from `cli-control.md`, an algorithm from the same file, and a state transition from `notifications.md`. One block of code can discharge instructions from several specifications at once, and the nesting shows which is which.

This is also where marker syntax stops being pretty. Two of those five statements have more comment lines than code. It is the honest cost of instruction-level traceability, and it is worth knowing before you decide how much of it your project wants.

3.8.5 The tick loop

`daemon.py` holds the largest flow, `tick_once`, implementing `cpt-ex-ovwa-flow-tracker-core-tick-loop` from `features/tracker-core.md`:

```
# @cpt-flow:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1
def tick_once(*, state: TrackerState, config: Config,
              idle_seconds: int | None, now: float) -> TrackerState:
    """Execute one tracking tick, applying idle/paused guards and delta clamping."""
    # @cpt-begin:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-init-first-tick
    if state.last_tick_at is None:
        return replace(state, last_tick_at=now)
    # @cpt-end:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-init-first-tick

    # @cpt-begin:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-handle-idle-unavailable
    if idle_seconds is None:
        return replace(state, last_tick_at=now)
    # @cpt-end:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-handle-idle-unavailable

    # @cpt-begin:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-skip-on-idle
    if idle_seconds >= config.idle_threshold_seconds:
        return replace(state, last_tick_at=now)
    # @cpt-end:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-skip-on-idle

    # @cpt-begin:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-skip-on-paused
    if state.status == TrackerStatus.PAUSED:
        return replace(state, last_tick_at=now)
    # @cpt-end:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-skip-on-paused

    # @cpt-begin:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-accumulate
    updated = _accumulate_active_time(state=state, config=config, now=now)
    # @cpt-end:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-accumulate

    # @cpt-begin:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-return-state
    return updated
    # @cpt-end:cpt-ex-ovwa-flow-tracker-core-tick-loop:p1:inst-return-state
```

Note that every early return also sets `last_tick_at=now`. That is the CDSL flow's wording, faithfully: "set last_tick_at=now and RETURN (paused by idle)." If it did not, the next successful tick would credit the user with all the idle time as active work — which is the bug the whole feature exists to avoid.

The accumulation step delegates to a marked algorithm:

```
# @cpt-algo:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1
def _accumulate_active_time(*, state: TrackerState, config: Config,
                            now: float) -> TrackerState:
    """Accumulate session active time for one tick."""
    if state.last_tick_at is None:
        return replace(state, last_tick_at=now)

    # @cpt-begin:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-compute-delta
    raw_delta_seconds = now - state.last_tick_at
    # @cpt-end:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-compute-delta

    # @cpt-begin:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-compute-max-delta
    max_tick_delta_seconds = config.max_tick_delta_seconds
    # @cpt-end:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-compute-max-delta
```

```
    delta_seconds = _clamp_delta_seconds(
        delta_seconds=raw_delta_seconds,
        max_tick_delta_seconds=max_tick_delta_seconds,
    )

    # @cpt-begin:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-add-delta
    new_active_time_seconds = state.active_time_seconds + int(delta_seconds)
    # @cpt-end:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-add-delta

    updated = replace(state, active_time_seconds=new_active_time_seconds)

    # @cpt-begin:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-update-last-tick
    updated = replace(updated, last_tick_at=now)
    # @cpt-end:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-update-last-tick

    # @cpt-begin:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-return-updated-state
    return updated
    # @cpt-end:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-return-updated-state
```

Two instructions from that algorithm — steps 2 and 4, negative-delta handling and clamping — are missing from this function. They live in a helper above it:

```
def _clamp_delta_seconds(*, delta_seconds: float,
                         max_tick_delta_seconds: int) -> float:
    # @cpt-begin:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-handle-negative-delta
    if delta_seconds < 0:
        return 0
    # @cpt-end:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-handle-negative-delta

    # @cpt-begin:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-clamp-delta
    if delta_seconds > max_tick_delta_seconds:
        return float(max_tick_delta_seconds)
    # @cpt-end:cpt-ex-ovwa-algo-tracker-core-accumulate-active-time:p1:inst-clamp-delta
    return delta_seconds
```

One specified algorithm, implemented across two functions, with the instruction markers following the code rather than forcing the code into one function. The scope marker `@cpt-algo:` sits on the main function; two of its instruction blocks sit in the helper. Traceability does not require you to write worse code, and this is the concrete proof of that claim.

What the clamp is actually for is stated in the feature spec's validation behaviour: "The daemon MUST clamp large time deltas to max_tick_delta_seconds (e.g., after sleep/wake) to avoid overcounting." Close the laptop lid for four hours, open it, and the first tick would otherwise credit four hours of deep work.

3.8.6 Where the DoD markers land

The daemon's main loop is where the two `p1` definitions of done get discharged:

```
            with self._lock:
                # @cpt-dod:cpt-ex-ovwa-dod-tracker-core-idle-aware-accumulation:p1
                self._state = tick_once(state=self._state, config=config,
                                        idle_seconds=idle_seconds, now=now)

                # @cpt-dod:cpt-ex-ovwa-dod-notifications-alert-and-repeat:p1
                self._state = _maybe_send_overwork_notification(
                    state=self._state,
                    config=config,
                    idle_seconds=idle_seconds,
                    now=now,
                )

            time.sleep(config.tick_interval_seconds)
```

Six `@cpt-dod:` markers exist in the whole project, and they are the coarsest level of the traceability chain. Each one points at a definition of done in a feature spec, and each of those

definitions points back at a PRD requirement. So this ten-line block is, formally, where `cpt-ex-ovwa-fr-track-active-time` and `cpt-ex-ovwa-fr-notify-on-limit` become software.

Also worth noticing: `time.sleep(config.tick_interval_seconds)` with `tick_interval_seconds = 5`. That is `cpt-ex-ovwa-nfr-low-overhead` — "avoid high-frequency polling that would noticeably impact CPU or battery" — as a single line of code. Five seconds also satisfies the PRD's "pauses within 10 seconds" acceptance criterion with a factor of two to spare.

3.8.7 Config defaults, discharged

One more DoD marker, in `config.py`:

```
# @cpt-dod:cpt-ex-ovwa-dod-tracker-core-config-defaults:p1
def load_config(config_path: Path | None = None) -> Config:
    """Load effective config (defaults + validation) from a JSON file."""
    path = config_path or DEFAULT_CONFIG_PATH

    cfg = Config()
    if not path.exists():
        return cfg

    try:
        raw = json.loads(path.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError):
        logger.warning("Config unreadable; using defaults", exc_info=True)
        return cfg

    if not isinstance(raw, dict):
        return cfg
```

Three ways to fail — no file, unreadable or malformed JSON, valid JSON that is not an object — and all three return the defaults. Then each individual field is parsed by `_parse_positive_int`, which returns `None` for anything non-integer or non-positive, and only fields that parsed get applied.

The feature spec required exactly this: "If a configuration value is missing or invalid, the daemon MUST continue using the default." Not the file — the value. A config file with a good `limit_seconds` and a garbage `idle_threshold_seconds` keeps the good one. That granularity is in the specification, so it is in the code, and the marker is what lets you check the claim in ten seconds instead of reading 73 lines.

One nice detail: when `max_tick_delta_seconds` is absent from the file, it is derived rather than defaulted:

```
    if max_tick_delta_seconds is not None:
        cfg = replace(cfg, max_tick_delta_seconds=max_tick_delta_seconds)
    else:
        cfg = replace(cfg, max_tick_delta_seconds=cfg.tick_interval_seconds * 2)
```

The feature spec said `max_tick_delta_seconds: tick_interval_seconds * 2`, and the CDSL algorithm's step 3 says "Compute max_tick_delta_seconds = tick_interval_seconds * 2." If you set a 30-second tick interval and forget to raise the clamp, the clamp follows you.

3.8.8 The unmarked boundary

For contrast, here is `idle.py` in full — one of the three modules with no markers:

```
_IDLE_RE = re.compile(r"\"HIDIdleTime\"\s*=\s*(\d+)")


def get_idle_seconds() -> int | None:
    """Return current idle seconds, or None if the sample is unavailable."""
    try:
        proc = subprocess.run(
            ["ioreg", "-c", "IOHIDSystem"],
            check=False,
```

```
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True,
        )
    except OSError:
        logger.warning("Failed to invoke ioreg", exc_info=True)
        return None

    if proc.returncode != 0:
        logger.warning("ioreg returned non-zero exit code: %s", proc.returncode)
        return None

    m = _IDLE_RE.search(proc.stdout)
    if not m:
        return None

    try:
        idle_ns = int(m.group(1))
    except ValueError:
        return None

    return idle_ns // 1_000_000_000
```

Every failure path returns `None`, which is exactly the contract the DESIGN specified for this component: "Best-effort; failures return an error that the daemon treats as 'unknown idle' and skips accumulation for that tick."

And the daemon honours it, in the block marked `inst-handle-idle-unavailable` above. The specification did not say how to get idle time out of macOS. It said what the boundary must guarantee. The implementation chose `ioreg -c IOHIDSystem` and a regex over `HIDIdleTime`, in nanoseconds, integer-divided down to seconds. That choice is implementation judgement and it has no marker, and the two together are what a good boundary looks like.

3.9 Following One Identifier

Take `cpt-ex-ovwa-fr-notify-on-limit` and walk it.

1. `PRD.md`, as FR-003: "When the tracked active work time exceeds the configured limit, the system MUST notify the user. If the user continues working while over the limit, the system MUST repeat notifications at the configured repeat interval until the user stops working (becomes idle) or manually pauses/resets tracking." Marked `[x]`, priority `p1`, actors `cpt-ex-ovwa-actor-user` and `cpt-ex-ovwa-actor-macos`.

2. `DESIGN.md`, §1.2, as a functional driver with a solution: the daemon enters an over-limit state when `active_time > limit`; a notifier component sends immediately and repeats at the configured interval. Allocated to `cpt-ex-ovwa-component-notifier` and `cpt-ex-ovwa-seq-run-and-alert`.

3. `DECOMPOSITION.md`, §2, entry 2: owned by `cpt-ex-ovwa-feature-notifications`, HIGH priority, depending on `cpt-ex-ovwa-feature-tracker-core`, with "persisting reminder history across restarts" explicitly out of scope.

4. `features/notifications.md`: refined into two actor flows, one six-step algorithm, one three-transition state machine, and one definition of done, `cpt-ex-ovwa-dod-notifications-alert-and-repeat`, whose *Covers (PRD)* list names `cpt-ex-ovwa-fr-notify-on-limit` by ID.

5. `notification_policy.py`: as `@cpt-dod:cpt-ex-ovwa-dod-notifications-alert-and-repeat:p1` on `apply_notification_policy`, and as six `@cpt-begin`/`@cpt-end` pairs around the six branches of `should_notify`.

6. `daemon.py`: as the same DoD marker inside the main loop, plus twelve instruction blocks in `_maybe_send_overwork_notification` covering both actor flows.

Six files, one string, no meeting required. Somebody who has never spoken to the author can start at the requirement and arrive at the exact lines, or start at the lines and arrive at the requirement.

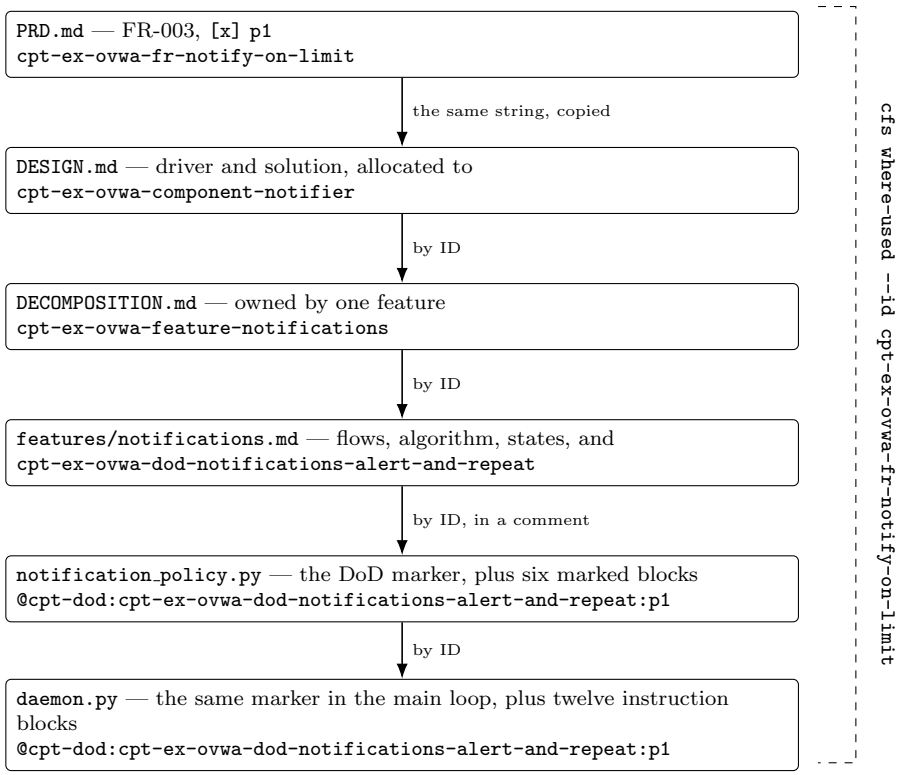


Figure 3.2: One identifier, six files.

3.10 Stages 9 and 10: Review and the Whole Picture

3.10.1 Reviewing the code

```
cf-coding: review the changed code for correctness, regression
          risk, and missing tests
```

Use a fresh chat. Writing context leaks assumptions into review: the session that just wrote `should_notify` is the worst possible reviewer of `should_notify`, because it already believes the design is right. Better still, let the host dispatch a sub-agent, so the reviewer never sees the writer's reasoning at all. Findings first, fixes after you approve the scope.

On this project a good review has concrete places to look. Are the two unmarked lines in `should_notify` correct or leftover? Does `_maybe_send_overwork_notification` call `should_notify` more times than it needs to? Is the lock held across the notification delivery, and does that matter given that `osascript` runs in a subprocess? None of those questions is answerable by validation, and all three are answerable by reading.

3.10.2 Deterministic validation

```
$ cfs validate
$ cfs validate --local-only
```

With all four documents and the code in place, this is where the chain gets checked mechanically. On this project it checks that every ID matches the template for its kind; that the coverage cross-references between PRD, DESIGN, DECOMPOSITION, and FEATURE exist where the constraints require them; that checkbox states are consistent across references; that every `[x]` identifier declared as reaching code has a marker, and every `[]` one does not; and that all 94 `@cpt-begin` markers have matching `@cpt-end` markers, with no empty blocks and no orphans.

That last check is duller than it sounds and catches more than you would expect. Delete a function during a refactor and take its `@cpt-end` with it, and you have an orphan; validation finds it in a second, and a code reviewer scanning a 400-line diff does not.

```
$ cfs spec-coverage
```

This answers the other direction: which identifiers that are supposed to reach code actually have markers. It is the report to run before you claim a feature is done.

3.10.3 The dependency map

```
$ cfs map
```

```
cf-map: find dangling references

cf-map: show which artifacts and code markers connect to the
       notifications FEATURE specification
```

The first is a rendering, built by scanning links, code references, and `@cpt-*` identifiers across the project. The second two are questions about it. Dangling references are how you find the requirement nobody implemented and the document link that broke when a directory was renamed.

Overwork Alert has two such small drifts, and they are worth finding yourself rather than being told about — Exercise 4 points you at the neighbourhood.

3.10.4 Asking about one identifier

```
$ cfs where-defined --id cpt-ex-ovwa-fr-notify-on-limit
$ cfs where-used --id cpt-ex-ovwa-fr-notify-on-limit
$ cfs list-ids --source docs
```

These are the commands for the fifteen minutes before a refactor rather than the fifteen minutes after it. `where-defined` tells you which file owns the ID; `where-used` finds every reference across documents and code; `list-ids` dumps what a source declares.

3.10.5 And the smaller checks

```
$ cfs validate-toc examples/overwork_alert/architecture/PRD.md
$ cfs check-language examples/overwork_alert/architecture/PRD.md
```

Every document in this project carries a generated table of contents between `<!-- toc -->` and `<!-- /toc -->` markers. `validate-toc` confirms it still matches the headings, which it will not after somebody renames a section by hand. `check-language` confirms the document is in an allowed language.

Small, boring checks. They catch the drift nobody notices in review, because nobody rereads a table of contents.

3.11 What This Example Is, and Is Not

Overwork Alert is a small tool, deliberately. Its own PRD says so, in a section called "Example Scope Notes":

```
This PRD is intentionally scoped as a minimal "end-to-end Constructor
Studio SDLC" example within the Constructor Studio repository.
```

It does not prove the approach scales to a large system. Nine implementation modules and 219 code markers is not evidence about a 500-module service mesh, and the honest thing to say is that the marker density in Table 3.3 — 99 markers in a 317-line file — would be intolerable at scale if applied uniformly. Real projects apply it selectively: `FULL` traceability on the artifacts and identifier kinds where somebody will one day ask "which code implements this," and `DOCS-ONLY` elsewhere. That knob exists precisely because uniform application is the wrong default.

What the example does show is what the artifacts look like when the pipeline is followed to the end, without gaps, on a problem small enough to hold in your head. Read it before you start your own project. It is faster than reading the guides, and it is the only place where you can check every claim in this chapter against a file.

Exercises

1. **Run the checks and say what each one does.** Clone the Constructor Studio repository and run `cfs validate`, then `cfs validate --local-only`, then `cfs spec-coverage`, then `cfs map`. For each of the four, write two sentences: what it examined, and what class of mistake it would have caught. Then explain why `--local-only` changes the result in a workspace but not in a single-repository clone.

2. **Trace an identifier you have not seen traced.** Pick `cpt-ex-ovwa-fr-cli-controls` and follow it the way this chapter followed `cpt-ex-ovwa-fr-notify-on-limit`: name the file and section at each of PRD, DESIGN, DECOMPOSITION, and FEATURE, then find every code marker that descends from it. Verify your answer with `cfs where-used --id cpt-ex-ovwa-fr-cli-controls`, and note anything you missed by reading.

3. **Add a marked function and trace it back.** Add a new function to `notification_policy.py` — for example one that reports how long the session has been over limit — give it a CPT scope marker and at least two `@cpt-begin`/`@cpt-end` instruction blocks, and add the corresponding CDSL steps to `features/notifications.md` so the identifiers exist upstream. Then run `cfs validate` and get it green. Finally, write down which definition of done and which PRD requirement your new function sits under, and whether you had to add anything to their coverage lists.

4. **Find the drift.** Open `DESIGN.md` and read section 4, "Additional Context," and section 5, "Traceability." Compare every path they mention against what is actually on disk under `examples/overwork_alert/architecture/`. There are two mismatches. For each one, say what would have to be true for `cfs map` to report it, and whether `cfs validate` would catch it. Then fix them in your clone.

5. **Break a marker on purpose.** Delete a single `@cpt-end` line from `daemon.py` and run `cfs validate`. Record the exact error. Restore it, then instead change one CDSL step in `features/notifications.md` from `[x]` to `[]` without touching the code, and run `cfs validate` again. Explain why both of those are errors, and why that is the behaviour you want rather than a nuisance.

Part II

How Constructor Fabric Is Built

Chapter 4

Studio and PDSL

Part I used Constructor Fabric from the outside. You installed `cfs`, ran `cfs init`, typed `cf-sdlc-doc-prd` into a chat window, and watched a chain of documents appear with identifiers linking them together. Nothing in those three chapters required you to know how any of it works.

Part II is the opposite. From here on the question is not "how do I use this" but "what is actually there, and does it hold up." That change of question matters, because a fair amount of what follows is about places where the code and the words describing the code do not agree. Those are not embarrassments to be skipped past. They are the parts you can learn a method from.

Everything in these four chapters was checked against the source. Where the check failed, it says so.

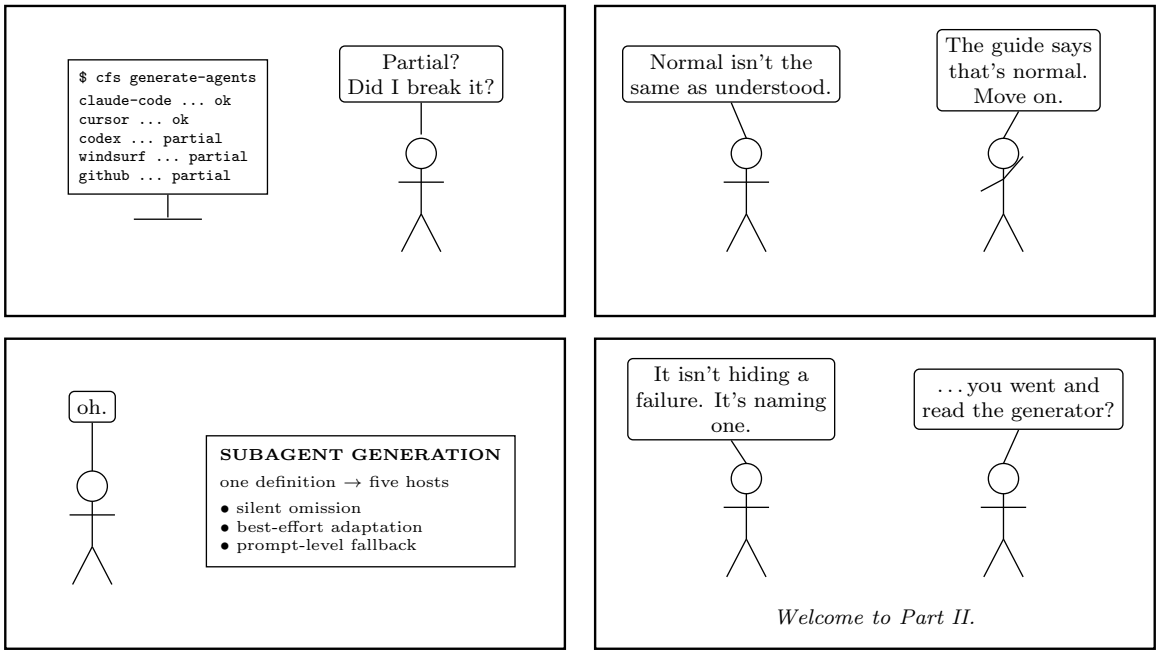


Figure 4.1: Part II, in four panels.

4.1 The Federation

Constructor Fabric is not one repository. It is roughly forty, in five languages, under Apache License 2.0. There are three flagship subsystems:

Studio The orchestration and authoring layer. This chapter.

Gears The plugin-composable service runtime, with sub-repositories `gears-rust` and `gears-frontx`. Chapter 5.

Insight The data platform, with `insight` and `insight-lite`. Chapter 6.

Before looking at any code, notice what is *not* in that list. There is no single build system. There is no monorepo. There is no shared parent directory that everything hangs off. Each flagship subsystem is independently versioned and carries its own sub-repositories.

What holds it together at development time is a workspace configuration that resolves source references across repository boundaries — the same mechanism you met in Chapter 1 as `cfs workspace-init` and `cfs workspace-add`. That is not a convenience feature bolted onto a single-repository tool. It is how the platform's own developers work, because there is no other way to work on forty repositories at once.

So the word to hold onto is **federation**, and it will keep coming back in each of the next three chapters with a different meaning each time. Studio federates repositories. Gears federates processes and network boundaries. Insight federates data sources. Same word, three genuinely different problems.

One more name is worth knowing because it appears in the platform's own architecture material: Kitsoki. It sits outside the federation deliberately. It is a comparator, not a member.

4.2 What Studio Is

Studio is the orchestration and authoring layer. Its job is to decide what an AI agent working on this codebase is allowed to do, and in what order. It is written mostly in Python, plus a very large amount of structured markdown.

That last clause is the reason Studio deserves a chapter of its own rather than a paragraph. Part of Studio's control flow is not compiled code at all. It is text, read at run time by a large language model. We will come back to exactly what that means, and what it costs, at the end of this chapter.

4.2.1 Two responsibilities, two philosophies

Keep these separate in your head, because Studio does:

Governance Which capabilities exist, at what version, installed how.

Execution What the agent actually does with them.

Governance is structured, versioned, reviewed, and mechanically checked. It lives in TOML manifests, has a resolution order, has a provenance record, and fails a validator when it is malformed. Execution is prose read by a model.

Same system, two completely different engineering philosophies. That split is more interesting than either half on its own, and Chapter 7 will show that it is also, accidentally, a position in a category-wide argument.

4.2.2 Exactly two workflows

Studio constrains agent-facing control flow to **two** universal workflows:

- `Generate`, for write-intent tasks.
- `Analyze`, for read-intent tasks.

Both share one execution protocol. There is an explicit design rule that domain-specific behaviour is injected as *content* inside that shared protocol, rather than as a new workflow type. Nobody gets to add a third workflow because their domain feels special.

You have already seen the user-facing end of this. Chapter 1 noted that `cf-analyze` and `cf-generate` are stable router entrypoints that resolve the best matching skill rather than doing the work themselves. Those two names are not a convenience alias layer. They are the two workflows, exposed directly.

Compare this with any plugin system where every plugin invents its own lifecycle. Two workflows and one protocol means there is exactly one place to look when you want to know what an agent will do next.

4.2.3 One canonical source of control flow

`SKILL.md` is the single version-controlled source of truth for how an agent operates the platform. Five different external AI coding tools are supported — the five from Chapter 2 — and each of the five gets a *generated shim* that references `SKILL.md`.

Not a forked copy. A reference.

This is what `cfs generate-agents` is doing when it writes `.claude/`, `.cursor/`, `.codex/`, `.windsurf/` and `.github/`. It is not translating the rules five times. It is emitting five pointers at one rule set.

Why it matters: five forked copies drift. Drift in control-flow rules is a security problem, not a tidiness problem. If the Cursor copy of a gate has fallen a release behind the Claude Code copy, then the same repository enforces different rules depending on which editor somebody happened to open it in, and nobody will notice until it matters.

Remember the phrase **compile once, adapt per host**. It appears three times in Part II, in mechanisms with no relationship to each other, and the third sighting is in software written by strangers.

4.2.4 Handoff as an explicit envelope

When one skill hands off to another, the result is a five-status envelope. A skill that cannot proceed returns a visible `blocked` result naming three things:

- exactly what is missing,
- why it matters,
- whether an override exists.

The alternative — which you will see in a great many prompt-based systems — is a gate buried inside narrative instructions. Somewhere in nine hundred words of guidance there is a sentence saying the workflow should not continue without an approved requirements document, and whether it does or not depends on how much attention the model paid to paragraph six.

A gate a reader has to *infer* is not a gate. This is the same argument as "fail loudly" in any other language, applied to a medium where failing quietly is the default.

4.3 The Kit System

Studio's core knows about workflows, manifests, versions, and files. It knows nothing about software architecture reviews, or code ranking, or project management. Those live in kits: installable domain packages.

The real ones in the repository are `studio-kit-sdlc` — the kit Part I used for everything — plus `studio-kit-code-ranker` and `studio-kits-pm`.

The core's boundary is stated in its own documentation, and the statement is worth quoting because it is a design commitment rather than a description:

> "core does not interpret kit-specific semantics — it only knows that a kit is registered and where its files live"

Semantics-blind. The core can tell you that `studio-kit-sdlc` is installed and that its ADR template is at a particular path. It cannot tell you what an ADR is.

4.3.1 The kit model that got deleted

The first kit design was a *blueprint* model. Kits generated files. SHA-256 hashes detected whether a user had customised a file since installation. Marker-level three-way merging reconciled an upstream change with a local edit. Roughly fifteen hundred lines of merge logic.

It worked, in the sense that it ran. It was replaced.

The reasoning is recorded in the design documents, and it is one sentence:

> "the kit model becomes trivially simple: kit = directory of files, update = file-level diff"

Fifteen hundred lines of three-way merge, replaced by `diff`.

This is the single most useful engineering habit in this part of the book, so it is worth being explicit about what happened. Somebody looked at a mechanism they had already built — built, tested, shipped — and asked whether the problem it solved was a real problem. Did anyone actually need marker-level reconciliation of an upstream template change against a local edit inside the same paragraph? Or was file-level granularity enough, given that a user who has customised a file can be told so and asked?

It was enough. It usually is. The expensive mechanism was answering a question nobody had asked.

4.3.2 Four levels of extensibility

Kit extensibility today operates at four independently addressable levels:

kit-level Install an entirely new domain kit.

artifact-level Add a new artifact kind inside an existing kit.

resource-level A per-resource override that survives kit updates.

placement-level Relocate individual resources on disk, resolved through template variables at install time.

The third one is the interesting one, and the phrase that carries the weight is "survives kit updates." Any extension mechanism that loses your customisations on upgrade is not an extension mechanism. It is a fork with extra steps, and the people using it will stop upgrading.

The fourth level is why the `artifacts.toml` entries in Chapter 3 contain strings like `"{project_root}/examples/$system"`. Those template variables resolve at install time, which is what lets one kit place its resources differently in two repositories with different layouts.

4.3.3 Manifest formats: three plus one

A kit declares itself through a manifest, and there are four ways in:

1. the current canonical `.cf-studio-kit.toml`;
2. two superseded legacy formats;
3. an implicit directory-layout convention, with *no* manifest file at all.

One dispatch function tries these in a fixed order of precedence. Whichever branch matches gets converted into one shared immutable output type. Downstream code cannot tell which branch produced it.

That last property is the whole point. Supporting four input formats is a migration cost you pay once, in one function. Supporting four input formats *everywhere* — by letting the difference leak into the rest of the codebase as conditionals — is a cost you pay forever.

Here is a real manifest, from the SDLC kit:

```
# Generated by cfs kit normalize. Review before publishing.

manifest_version = "1.0"

[[kits]]
slug = "sdlc"
name = "sdlc"
version = "1.0"

[[kits.resources]]
id = "adr_template"
kind = "template"
source = "artifacts/ADR/template.md"
install_path = "artifacts/ADR/template.md"
type = "file"
user_modifiable = true
description = "ADR artifact template"
```

Read that carefully, because several decisions are visible in it.

Every resource declares an identifier, a source path, a destination, and a type. And it declares `user_modifiable`. That single boolean is what makes the resource-level override of the previous section safe: the kit author states, per resource, whether a local edit is a supported thing to do or a mistake waiting to be overwritten.

Public resource and agent names are auto-prefixed, so two kits cannot collide over a name like `review` — with an explicit per-resource opt-out, because somebody will always need one.

Then look at the first line. This file is generated by a normalizer, `cfs kit normalize`. So the canonical format is also the *output* format of the migration path off the legacy formats. The four-way dispatch function and the normalizer are two halves of one strategy: accept anything, emit one thing.

4.3.4 Resolving a kit: version authority

Kits can come from GitHub, as Chapter 1 showed with `cfs kit install constructorfabric/studio-kit-sdlc`. So: which version do you get?

There is a strict, explicitly ordered precedence:

1. an explicit user selector;
2. the latest published Release;
3. a semver-tag fallback;
4. offline last-known metadata.

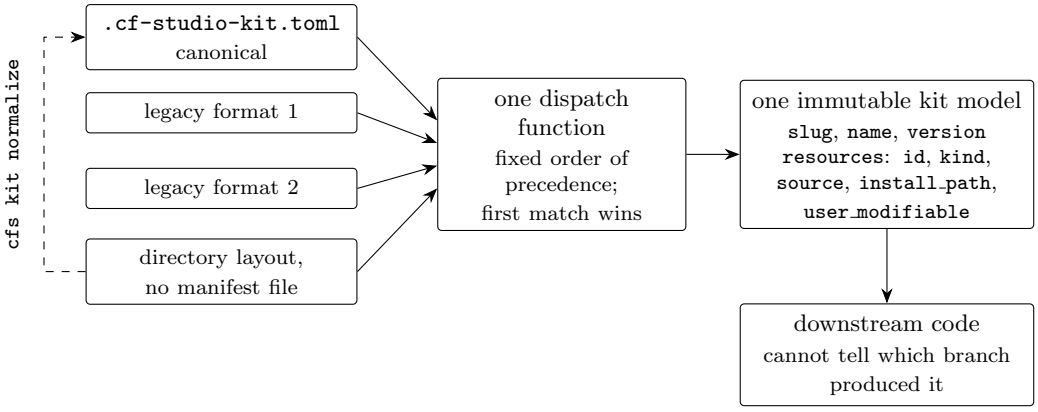


Figure 4.2: Four ways in, one kit model out.

And one categorical rule: a locally copied version file is **never** authoritative for GitHub-backed state.

That rule is worth more than it looks. A local file is a cache. A cache that claims authority is a bug generator, and specifically it is the kind of bug that produces two engineers on the same team confidently disagreeing about which version is installed, both correct about their own machine.

4.3.5 Display version versus content identity

Every install persists a structured provenance record, and it separates two things people habitually confuse:

Display version The release or tag shown to a user.

Content identity A commit SHA, an asset digest, or a tree identity.

So "what version do I think I have" and "what bytes are actually installed" are tracked as independent facts, not as one fact with two names.

Anyone who has chased a bug through a re-tagged release will appreciate this immediately. Anyone who has not, will.

4.3.6 The mirror override layer

In front of the whole resolution chain sits a global mirror-override layer. It redirects any GitHub-derived URL through a longest-prefix-matched override table. It was added specifically to support internal mirrors and forks, which is a requirement that arrives the first time a large organisation adopts the platform and discovers its build network cannot reach `github.com`.

The design property worth naming is where it sits. It is *in front* of resolution and it changes nothing downstream. The version-authority logic of the previous two sections never learns that mirrors exist.

That is a clean seam. Compare it with the alternative, which is an `if mirror:` branch in nine places, eight of which get updated when the requirement changes.

4.4 The Four Layers That Are Not Layers

Studio's multi-repository federation model is built around what its own documentation calls a **four-layer manifest walk-up hierarchy**. The four layers are core, kit, orchestrator repository, and individual repository. They are discovered by walking up the filesystem, with zero explicit configuration.

The documentation states the design goal directly:

> "walk-up discovery requires zero explicit configuration for multi-repo setups — filesystem structure IS the configuration"

That part is true, and it is a nice idea. Now read the code.

```
def discover_layers(repo_root: Path, studio_root: Path) -> List[ManifestLayer]:
    """Discover manifest layers in resolution order.

    1. Kit layers - from ``core.toml`` kit registrations
    2. Master Repo layer - discovered via walk-up (if found)
    3. Repo layer - from ``{studio_root}/config/manifest.toml``

    Missing layers (no ``manifest.toml``) are silently omitted.
    Parse errors result in ``ManifestLayerState.PARSE_ERROR`` layers.
    """
```

Source: `studio/skills/studio/scripts/studio/utils/layer_discovery.py`

Read the return type again: `List[ManifestLayer]`. It returns a list, in resolution order.

4.4.1 What "Layered Systems" actually requires

Garlan and Shaw's Layered Systems style is not "things with the word layer in them." It requires two properties:

- a strict one-way dependency direction between adjacent layers;
- a defined service interface at each boundary.

And there is disqualifying evidence, stated in advance so the judgement is not retrofitted to the answer:

- central-kernel dispatch to registered components — that is closer to Microkernel;
- a merge or overlay cascade with no enforced call direction.

Hold that second bullet next to `discover_layers`.

4.4.2 The verdict

What the code does is collect manifests from several places, order them, and merge them by precedence.

Layer n does not call layer $n-1$. Layer n does not offer a service interface to layer $n+1$. Nothing enforces any call direction, because there are no calls between "layers" at all. There is one function that produces a list.

This is a config-precedence merge cascade. It is a perfectly good config-precedence merge cascade. It is not Layered Systems, and the name says it is.

4.4.3 Why this is worth four pages

Nobody lied. Somebody reached for a familiar architectural word for a structure that has levels in it. That happens constantly, including in your own code, and the naming instinct behind it is not even a bad instinct — "layer" really is the closest common word for what `discover_layers` returns.

The cost is not aesthetic. It is two specific things:

- The next engineer reads "layered" and reasons about isolation guarantees that do not exist. They assume the kit layer cannot reach into repository configuration, because in a layered system it could not, and they write code on that assumption.

- A reviewer has no grounds to block a change that violates a rule nobody wrote down. If the rule were real there would be something to point at. There is not, so the violation lands.

A name is a claim. Claims get checked. The method, in one line: read the definition, then read the code, then decide.

4.4.4 To be fair to Studio

The mechanism has real properties worth keeping, and dismissing it because the label is wrong would be its own kind of sloppiness.

An `includes` directive lets component packages join generation without flattening into one root manifest. Each composed component's originating layer stays individually inspectable through a provenance report — so when a resolved value surprises you, you can find out which manifest contributed it.

And look at the last two lines of that docstring again. Missing manifests are silently omitted, which is the right default for an optional layer. Parse errors are *preserved as a layer state*, `ManifestLayerState.PARSE_ERROR`, rather than thrown away or fatally raised.

That last one is genuinely careful engineering. A broken manifest does not vanish and it does not crash discovery. It becomes visible data that a diagnostic can report. The design is fine. The label is wrong.

4.4.5 A promise for the next chapter

You may reasonably now suspect that nobody ever builds real layers.

Chapter 5 looks at `gears-frontx`, which documents an L1/L2/L3 layered SDK. That one passes: strict one-way dependencies, and a continuous-integration rule that fails the build on a violation.

Same organisation, same word, opposite result. The difference is not vocabulary. It is enforcement.

4.5 PDSL

PDSL is the Prompt-level Domain-Specific Language: a compact notation for writing agent instructions in `skills/`, `workflows/`, and `requirements/`.

Its own specification is unusually blunt about scope:

> "The goal is not to create a programming language. The goal is to make mandatory behavior clear, reviewable, and hard to misread."

Source: `studio/architecture/specs/PDSL.md`

The division of labour is stated as a rule: keep prose for context and rationale, use PDSL for behaviour. A skill file is not written entirely in PDSL. It is written in English, with PDSL blocks where the English would have to be precise anyway.

4.5.1 What it is for

The specification lists the intended uses:

- workflow phases,
- state gates,
- approval menus,
- UX prompts,
- error handling,
- required and forbidden actions,
- handoff rules.

Every one of those is a place where being vague costs money. “Ask the user before making changes” is a sentence a model can satisfy in six different ways, one of which is asking a question and then proceeding regardless of the answer.

4.5.2 Design goals, as written

PDSL is meant to be:

- compact enough to replace repetitive prose,
- readable by humans without a parser,
- explicit about state, branches, and stop points,
- easy for an LLM to follow as an execution contract,
- stable under copy, review, and partial editing.

Plus one rule: PDSL does not depend on hidden semantics.

“Readable by humans without a parser” and “execution contract” sitting in the same list is the whole design tension of this notation, and everything interesting about PDSL follows from trying to have both.

4.5.3 The core shape

```
UNIT <name>
PURPOSE:
  <one sentence>
STATE:
  - SET <name>: <allowed values>
WHEN:
  - REQUIRE <condition>
DO:
  - RUN <ordered actions>
RULES:
  - ALWAYS <rule>
  - NEVER <rule>
ON_ERROR:
  <error> -> <actions>
```

Source: `studio/architecture/specs/PDSL.md`

Menus are first-class, with their own block type:

```
MENU <name>:
  TITLE: <menu title>
  OPTIONS:
    1 <choice> -> <actions>
  INVALID:
    EMIT <retry instruction>
    WAIT user.reply
    STOP_TURN
```

A whole block type exists for "ask the human and handle a bad answer." The `INVALID` branch is not optional decoration; it is where most conversational agents fall apart. The user types something that is not one of the three offered options, and the agent either guesses which one they meant or abandons the menu and starts doing work.

Blocks can be omitted. `PURPOSE`, `WHEN` and `DO` are the minimum for executable behaviour.

4.5.4 The keyword set

About twenty keywords, listed in Table 4.1.

Table 4.1: The PDSL keyword set, from the specification.

Keyword	Meaning
`UNIT`	Named instruction unit, phase, gate, or reusable rule.
`PURPOSE`	Why this unit exists.
`INPUT` / `OUTPUT`	Required inputs; expected result or handoff.
`STATE`	State variables and their allowed values.
`WHEN`	Entry condition.
`DO`	Ordered required actions.
`SET`	Assign state.
`LOAD`	Load a referenced prompt asset or context slice.
`RUN`	Execute a named local unit, probe, check, or workflow step.
`EMIT` / `EMIT_MENU`	Show user-facing text; show a named menu.
`WAIT`	Stop for user input.
`STOP_TURN`	End the assistant turn immediately.
`CONTINUE`	Move to a named unit or phase.
`DISPATCH`	Invoke a named sub-agent or worker contract.
`RETURN`	Return a manifest, report, checkpoint, or handoff.
`REQUIRE`	Required precondition.
`RULES` with `ALWAYS` / `NEVER`	Mandatory constraints.

Read that list as a language designer rather than as a prompt author.

`STATE`, `WHEN`, `WAIT`, `STOP_TURN`, `CONTINUE`: that is a finite state machine with its transitions written in English. `DISPATCH` is a call into another agent, so there is a call stack of sorts. `RULES` with `ALWAYS` and `NEVER` is an assertion mechanism.

Twenty keywords, no formal grammar for any of them. Which raises an obvious question, and we will get to it.

4.5.5 A real unit

Here is the first unit of `studio/workflows/analyze.md` — the `Analyze` workflow itself, one of the two that exist.

```
UNIT AnalyzeBootstrap
PURPOSE: Load the runtime routing rules needed before any
        analyze routing work.
DO:
  LOAD {cf-studio-path}/.core/skills/studio/modules/runtime/
```

```
      workflow-bootstrap.md
  RUN WorkflowBootstrapRouterPrelude
  RUN WorkflowBootstrapSimpleModeGate
  RUN WorkflowBootstrapCommandWorkflowResolution
  CONTINUE AnalyzeRoute
RULES:
  ALWAYS load command-resolution and workflow-resolution
        before AnalyzeRoute
  NEVER require cf or CFS_INIT before routing; this workflow
       owns its prerequisite loads
```

Source: `studio/workflows/analyze.md`

Read it as code, because it is trying to be code.

`DO` is an ordered block: one `LOAD`, then three `RUN`s, then a `CONTINUE`. `CONTINUE AnalyzeRoute` is a jump to a named unit. The `LOAD` path is written against a template variable, `{cf-studio-path}`, rather than a literal directory — so the unit does not hard-code the setup directory you chose during `cfs init`.

`RULES` is an invariant section, and the `NEVER` is the interesting half. It says: do not require `cf` or `CFS_INIT` before routing, because *this* workflow owns its prerequisite loads. That is an ownership contract between two pieces of prose. Some other unit might reasonably assume it must check initialisation first; this rule tells it not to, and says who is responsible instead.

The next unit in the same file is bigger:

```
UNIT AnalyzeRoute
PURPOSE: Capture the analyze intent, resolve cf-* skills via
        WorkflowResolution, and route to the chosen skill.
STATE:
  SET ORIGINAL_INTENT: string (default unset, scope workflow_run)
  SET AVAILABLE_SKILLS: list (default unset, scope workflow_run)
WHEN:
  REQUIRE WorkflowResolution is loaded
DO:
  RUN WorkflowResolution to resolve the available cf-* skills
  SET AVAILABLE_SKILLS = the resolved cf-* skills
  CONTINUE AnalyzeNoMatch WHEN AVAILABLE_SKILLS is empty
  CONTINUE AnalyzeRouteIntentFlow WHEN ORIGINAL_INTENT != unset
INVARIANTS:
  NEVER let a companion route bypass any selected skill's WAIT,
       STOP_TURN, approval, plan, or sub-agent gate
```

Source: `studio/workflows/analyze.md` (abridged)

Four things to notice.

`STATE` declares variables with a *scope*: `workflow_run`. So there is a lifetime model. `ORIGINAL_INTENT` does not persist past this run.

The jumps are guarded: `CONTINUE X WHEN condition`. Two of them, in order, which means the first matching one wins and the ordering is significant.

There is comparison against a sentinel: `!= unset`. Not empty string, not null. A distinguished value meaning "never assigned," which is exactly the distinction you want when the difference between "the user gave no intent" and "the user gave an empty intent" changes the route.

And there is an `INVARIANTS` block that talks about *other* units' gates. Read it: never let a companion route bypass any selected skill's `WAIT`, `STOP_TURN`, approval, plan, or sub-agent gate. Those are the gates you approved in Chapter 1. This one line is what stops a routing shortcut from skipping them.

None of this is enforced by a type system, a parser, or a test. It is enforced by a model reading English and choosing to comply. Sit with how strange that is for a moment, because the rest of this chapter is about the consequences.

4.5.6 Compile once, adapt per host: the first sighting

Subagent definitions for external AI coding tools are authored once, in one semantic model. One host's format is treated as the canonical, full-fidelity representation. Every other host's output is a documented, *lossy* adaptation.

There are three named degradation strategies for a property a host has no equivalent for:

silent omission The property is dropped. The host has no concept of it and nothing sensible can be substituted.

best-effort adaptation The property is mapped onto the nearest thing the host does have, accepting that the semantics are not identical.

prompt-level fallback The property is re-expressed as instruction text, so a capability the host lacks structurally is recovered as prose the model reads.

The engineering content here is not the generation. Generating five config formats from one model is a morning's work. The engineering content is *naming the losses* — deciding, per property per host, which of those three things happens, and writing it down so that when a Cursor user reports behaviour a Claude Code user does not see, there is a documented answer.

This is also the mechanism behind the partial-success message Chapter 1 told you not to treat as an error. When `cfs generate-agents` reports that some host-specific capabilities were skipped, that is silent omission being reported out loud.

4.5.7 The same motif, somewhere unrelated

Studio's plan-decomposition mechanism has nothing structurally to do with subagent generation. Different code, different purpose, different authors.

It compiles the same underlying work-decomposition decision into a different target grammar again. When delegating to `ralphex`, an external autonomous-execution CLI, Studio decides what the decomposition is and what rules apply, then exports a bounded, self-contained plan for a completely different tool to run.

Twice in one subsystem, in mechanisms with no relationship to each other. Chapter 7 finds it a third time, in platforms built by strangers.

4.5.8 When is an LLM allowed to decide anything?

Studio has a design principle governing this, stated plainly:

> "if a task can be expressed as a regex, a schema check, or a graph traversal — it MUST NOT require an LLM"

That is a good rule well beyond AI systems. It is the same instinct as "do not use reflection where a function call works." A deterministic mechanism is cheaper, faster, testable, and gives the same answer twice.

It is also exactly the rule Chapter 1 described from the user's side: `cfs validate` is deterministic because identifier format checking is a regex problem, and no model is involved in it.

The question is whether they actually follow the rule. The evidence turns out to be negative evidence, which is the good kind.

4.5.9 One hundred and fifty routers

More than 150 PDSL modules follow a `*-router.md`, `*-routing.md`, or `*-dispatch.md` naming convention.

Names like that suggest something rich: a chain of candidate handlers, a coordinator holding state across attempts, real decision logic about which handler is most appropriate. Reading a representative sample told a much simpler story. Each one is:

- a flat, priority-ordered match against a fixed menu of named targets,
- exactly one handoff,
- and an explicit instruction not to get further involved.

That is a dispatch table. Nothing more.

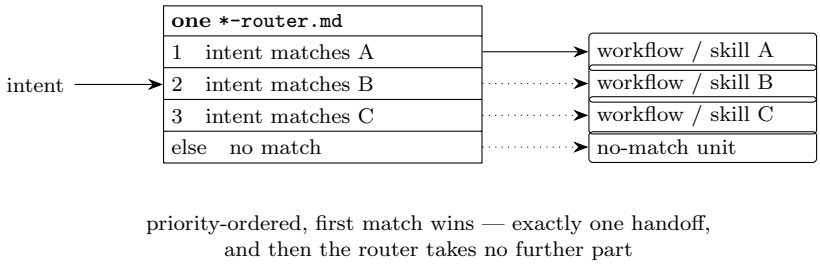


Figure 4.3: What the 150 routers actually do.

The punchline is what happens when you look for the same shape elsewhere. The identical structure was found independently in Studio's own Python dispatcher: same idiom, in a general-purpose language, with an actual implementation behind it.

So the same decision structure appears in ordinary Python code, and in markdown read by a model with nothing implementing it at all. Whatever "dispatch table" names is a fact about how people decompose decisions, not a fact about any language or execution model.

It also means those 150 files honour the rule from the previous section. A priority-ordered match against a fixed list is a graph traversal. It does not need a model to reason about it, and these files do not ask one to. They ask it to read a table.

4.6 The Missing Interpreter

PDSL has a keyword set, block structure, state variables with scopes, and guarded jumps. Something is reading it and executing it.

So: where is the interpreter?

Go looking in the codebase. Grep for a parser. Grep for an expression class hierarchy. Grep for an evaluator, a stack, an environment, a step function.

There is none.

4.6.1 What PDSL-aware code exists

Exactly one module in the platform understands PDSL: a validator, about 730 lines. What it does:

- extract fenced `pdsl` blocks with source coordinates,
- check block structure, section headers, and menu option numbering,

- emit findings with a rule identifier, severity, line, and column.

What it does not do is run anything. Its own docstring describes it as "PDSL block extraction and deterministic validation helpers."

Here is its central data type:

```
@dataclass(frozen=True)
class PdslFinding:
    """Deterministic validation finding for a PDSL block."""

    rule_id: str
    severity: str
    message: str
    source_path: str
    block_index: int
    line: int
    column: int
    ...
```

Source: `studio/skills/studio/scripts/studio/utils/pdsl.py`

Rule identifier, severity, line, column. That is a linter's data model, not a virtual machine's. A virtual machine's central type has a program counter and an environment. This one has coordinates in a text file.

4.6.2 So where does execution happen?

Inside the model.

The markdown goes into a large language model's context, and the model behaves as the text describes. No parse tree. No environment. No stack. The evaluation happens in weights that neither the authors nor any reader can inspect.

The interpreter has been fully externalized out of the codebase.

4.6.3 Judging it against the criteria

The Interpreter/Virtual Machine architectural style requires two separable things:

1. a notation preserved *as data* at run time, not pre-compiled into the host language's control flow;

2. an interpreter or virtual machine that maintains state and steps through that notation's instructions.

PDSL satisfies (1) without qualification. The markdown is genuinely the run-time artifact — it is shipped as text, read as text, and changing it changes behaviour without recompiling anything.

PDSL has no code-visible artifact satisfying (2).

Result: split. Half of a style.

At the finer-grained level of design patterns, the same artifact fails outright. Gang-of-Four Interpreter is a claim about *implementation structure*: a grammar reified as a hierarchy of expression classes, with composite members recursively evaluating their own constituents. Nothing of that shape exists anywhere in the platform.

So the two levels of description disagree about one artifact. The finer level says nothing is there. The coarser level says half of something is. Which is exactly what you would expect if the mechanism is real but its realizing component is not source code.

4.6.4 A useful contrast

OpenHands is an unrelated open-source agent platform. It has an embedded terminal tool, and the shape is superficially the same: the model writes shell script text, and the text is never compiled into the SDK's control flow.

But that text goes to a persistent, code-visible interpreter. A tmux- or subprocess-backed shell session, with real interpreter state — working directory, environment variables — surviving across turns, and fully inspectable.

Same style. Completely different amount of engineering you can actually audit.

Put the cases side by side and you get a spectrum rather than a binary:

fully externalized PDSL. Interpretation entirely inside the model.

registry-mediated dispatch A JSON tool call, parsed and dispatched against a live tool registry. Some interpreter state is code-resident.

fully code-resident A shell session with persistent, inspectable state.

Where a control-flow notation is executed matters as much as whether it is. We do not currently have good vocabulary for saying which component plays the interpreter role, which is a gap in how we describe this class of system rather than a defect in any one system.

4.6.5 What the absence actually costs

Be precise here, because "no interpreter" sounds like an insult and is not meant as one. PDSL works. Part I is evidence that it works.

The cost is specific. With no reified grammar, PDSL has no implementation artifact in which its semantics could be located, audited, or tested independently of the model that reads it.

You cannot write a unit test for what `STOP_TURN` means. There is no function to call, no state to assert on afterwards. You can write a test for what the linter says about `STOP_TURN` — that a malformed block produces finding `X` at line `Y` — and that is a genuinely useful test. It is also a different test. It checks the notation's shape, not its behaviour.

4.6.6 The organisation's own self-assessment

An internal research memo benchmarks PDSL against the academic structured-prompting literature. It rates the explicit `STATE`, `WHEN`, `WAIT`, `STOP_TURN` and `CONTINUE` constructs as showing "strong alignment" with finite-state-machine-style prompting.

And then it names two gaps, without flinching:

- There is no first-class transition object. Transition logic is spread across several constructs instead of being one current-state, condition, next-state representation. You saw this in `AnalyzeRoute`: the transitions are two `CONTINUE ... WHEN` lines inside `DO`, mixed in with a `RUN` and a `SET`.
- There is no schema-bound validation of a state decision's return value. Nothing checks that what comes back from a decision point is one of the allowed values.

Very few organisations publish this kind of self-critique. Give them credit for it, and then notice how much easier it made the analysis in this chapter: two of the sharpest criticisms of PDSL are the platform's own.

4.6.7 Open questions

These are not rhetorical. Nobody has good answers yet.

- What is the correctness criterion for a language whose only implementation is a model?
- If you change model vendor, did your program change?
- Is a linter a partial specification, and can you grow it into a full one?
- Should PDSL grow an interpreter, and would that defeat its purpose?

There is a good thesis in there for somebody.

4.7 Where That Leaves Us

Five things from this chapter:

- Studio splits into a structured governance layer and an unstructured execution layer, on purpose.
- The kit system got radically simpler when somebody questioned the problem instead of the solution.
- The "four-layer hierarchy" is a config-precedence merge cascade. Check names against code, always.
- PDSL is a real notation with real structure and no interpreter you can read.
- Compile once, adapt per host appeared twice already, in unrelated mechanisms.

Chapter 5 goes to the other end of the same organisation, where things are written in Rust and most of the rules are enforced by a compiler. The culture change is immediate.

Exercises

1. **Find the layer discovery code and describe its role.** Locate `studio/skills/studio/scripts/studio/utils/layer_discovery.py` in the Studio repository. Read the full implementation of `discover_layers`, not just the docstring quoted in this chapter. Then answer in a short paragraph each: (a) what order the returned list is in and what consumes that order; (b) what happens to a `manifest.toml` that exists but does not parse, and where in the codebase that state is later read; (c) whether anything in the module would have to change if the mechanism were renamed from "layers" to "precedence sources."

2. **Trace one PDSL routing decision.** Open `studio/workflows/analyze.md`. Starting from `AnalyzeBootstrap`, follow every `RUN` and `CONTINUE` until you reach a unit that performs actual work rather than routing. Draw the result as a graph, marking each edge as unconditional or guarded, and each node with the state variables it reads or writes. Then state how many units the graph contains and where the first `WAIT` or `STOP_TURN` appears.

3. **Write a minimal PDSL unit.** Using only the keywords in Table 4.1, write a unit that gates on the presence of an approved requirements document before allowing implementation to begin. It must declare its state with an explicit scope, use a guarded `CONTINUE`, present a `MENU` with a populated `INVALID` branch when the document is missing, and include a `RULES` block with at least one `NEVER`. Keep it under twenty-five lines. Then run the PDSL validator over it and fix whatever it reports.

4. **Audit a name in your own code.** Pick one architectural or pattern word used in a codebase you work on — "layer", "pipeline", "factory", "registry", "middleware", anything. Write down the pattern's *defining* property from a reference source, in one sentence, before looking at the code. Then find the code that would have to exist for that property to hold. Report one of three verdicts: confirmed, split, or rejected — and if it is not confirmed, name what the structure actually is.

5. **The interpreter you cannot test.** Write a unit test for a PDSL construct. You will not succeed; that is the exercise. Document precisely where you get stuck: what would you assert on, what would you have to instantiate, and what would a passing test actually prove? Then write the test you *can* write — a validator test over a malformed block — and explain in three or four sentences what the difference between the two tests tells you about where PDSL's semantics live.

Chapter 5

Gears, the Plugin Runtime

Chapter 4 looked at Studio: an orchestration layer, half of it written in prose. We found a mechanism called a four-layer hierarchy that is a merge cascade, and a language with no interpreter.

This chapter goes to the other end of the same organisation. Gears is written in Rust and most of its rules are enforced by a compiler. The culture change is immediate and it is worth paying attention to, because the two subsystems are solving structurally similar problems — optional, independently addressable extension points — with completely different tools.

5.1 What Gears Is

Gears is a plugin-composable service runtime. The unit of composition is called a **gear**: a Rust type that implements a small core trait, plus whatever optional capability traits it wants. The runtime discovers gears, orders them, and drives them through a fixed lifecycle.

Real gears in the repository: `api-gateway`, `authn-resolver`, `authz-resolver`, `tenant-resolver`, `grpc-hub`, `quota-enforcement`, `event-broker`. Those last few will matter in Chapter 7, which is entirely about what three of them do.

`gears-rust` is the core and takes up most of this chapter. `gears-frontx` is the TypeScript frontend counterpart, with its own plugin system, its own event bus, and its own layering. There are also `gears-csharp`, `gears-mobile` and `gears-web`.

The name is doing real work. A gear only meshes if its teeth match, and the whole design is about making "do the teeth match?" a compile-time question.

5.1.1 The problem being solved

You have thirty services. Some need a database. Some serve HTTP. Some run background work. Some need to be started and stopped; some are pure request handlers.

The naive answer is one big `main.rs` that constructs all of them in the right order. That file becomes the thing everyone edits and nobody understands. Every new service touches it. Every reordering is a merge conflict. And it makes "build a deployment with only these six services" a manual exercise in commenting things out and hoping the remaining initialisation order is still valid.

Gears' answer is categorical: *no central file enumerates which gears exist.* Composition happens at link time, and ordering is computed.

Those two properties are the whole design. Everything else in this chapter is mechanism in service of them.

5.2 Capabilities and the Registry

Every gear implements one required trait, `Gear`, which has a single `init`. Everything else is optional and separately addressable.

The runtime asks each gear: do you have a database capability? A REST capability? Do you need starting and stopping? Gears that answer no are simply skipped for that phase — there is no registration step to forget and no abstract method to stub out with a body that throws.

This is the same idea as Studio's four kit extension levels, realised through a completely different mechanism. Chapter 7 will point out that all four subsystems in the federation needed this idea and none of them built it the same way.

5.2.1 The capability set

`DatabaseCapability` Provides migrations.

`RestApiCapability` Registers routes and an optional healthcheck.

`ApiGatewayCapability` The `rest_host`: owns the actual HTTP server.

`RunnableCapability` Stateful background work, with start and stop.

`GrpcServiceCapability` / `GrpcHubCapability` The same split for gRPC.

`SystemCapability` Internal, for gears that need runtime internals before init.

Source: `gears-rust/libs/toolkit/src/contracts.rs`

Note the `rest_host` distinction, because it is the sharpest thing in that list. **One** gear owns the server. **Many** register routes onto it. Those are different capabilities with different cardinalities, and conflating them is how you end up with two services both trying to bind port 8080.

The same split appears for gRPC: one hub, many services.

5.2.2 A capability trait, for real

```
/// REST API capability: Pure wiring; must be sync.
/// Runs AFTER DB migrations.
pub trait RestApiCapability: Send + Sync {
    fn register_rest(
        &self,
        ctx: &crate::context::GearCtx,
        router: Router,
        openapi: &dyn OpenApiRegistry,
    ) -> anyhow::Result<Router>;

    /// Readiness healthcheck, run on every /readyz and /health.
    /// `None` (default) opts out.
    fn healthcheck(&self, _ctx: &crate::context::GearCtx)
        -> Option<Arc<dyn crate::healthcheck::Healthcheck>> { None }
}
```

Source: `gears-rust/libs/toolkit/src/contracts.rs`

Read that trait carefully. Four things are decided in it.

`register_rest` takes a `Router` and returns a `Router`. So route registration is a fold over gears, not mutation of a global. Each gear receives the router as it stands, adds to it, and hands it back. That makes the composition order explicit and makes it impossible for a gear to register routes outside its turn.

"Pure wiring; must be sync" is a stated constraint on implementors. No `async`, no I/O, no waiting on another service. Route registration happens in one synchronous pass and cannot deadlock on a peer that has not started yet.

"Runs AFTER DB migrations" is a phase-ordering promise made in a doc comment. It is not enforced by a type, and the honest reading is that it is documentation rather than a guarantee — but it is documentation in the place an implementor will actually be looking, which is worth more than the same sentence in an architecture document.

`healthcheck` has a default returning `None`, so opting out is free. Defaults that mean "not interested" are what makes a capability optional without a separate registration step. This is the small mechanical trick that lets the runtime ask twenty questions of thirty gears without anyone writing six hundred answers.

5.2.3 Advice buried in a doc comment

The `healthcheck` doc comment is worth reading in full, because it is operational wisdom rather than API documentation.

It says to return *one* composite check per gear, not many fine-grained probes, because "the aggregate is what decides whether the pod stays in traffic." If your gear reports six independent probes and the orchestrator ORs them, you have not gained resolution; you have gained six ways to be evicted.

Then it says that external SaaS and LLM dependencies should usually **not** gate readiness, because "a provider outage would evict every pod from rotation and take the whole service down." Report those as `degraded`. Reserve `unhealthy` for what the gear itself cannot serve through.

That is a distinction with real consequences. A readiness probe answers "should traffic come to me," and the answer when your model provider is down is generally still yes — most requests do not touch it, and the ones that do should fail individually rather than taking the fleet out of rotation.

Somebody learned that the hard way and then wrote it next to the function. That is the correct place for it.

5.2.4 The registry's view of a capability

```
/// A single capability variant that a gear can provide.
pub enum Capability {
    Database(Arc<dyn contracts::DatabaseCapability>),
    RestApi(Arc<dyn contracts::RestApiCapability>),
    ApiGateway(Arc<dyn contracts::ApiGatewayCapability>),
    Runnable(Arc<dyn contracts::RunnableCapability>),
    System(Arc<dyn contracts::SystemCapability>),
    GrpcHub(Arc<dyn contracts::GrpcHubCapability>),
    GrpcService(Arc<dyn contracts::GrpcServiceCapability>),
}
```

Source: `gears-rust/libs/toolkit/src/registry.rs`

A closed enum, one variant per capability, each holding a trait object behind an `Arc`. Closed is a deliberate choice: adding a capability is a change to this enum and therefore a change every exhaustive match has to acknowledge.

5.2.5 Type-safe querying, without downcasting

Now the part worth studying:

```
/// Trait for capability tags that allow type-safe querying.
pub trait CapTag {
    type Out: ?Sized + 'static;
    fn try_get(cap: &Capability) -> Option<&Arc<Self::Out>>;
}

pub struct RestApiCap;
impl CapTag for RestApiCap {
    type Out = dyn contracts::RestApiCapability;
    fn try_get(cap: &Capability) -> Option<&Arc<Self::Out>> {
        match cap {
            Capability::RestApi(v) => Some(v),
            _ => None,
        }
    }
}
```

Source: `gears-rust/libs/toolkit/src/registry.rs`

`RestApiCap` is a zero-sized marker type whose only job is to carry the mapping from "the capability I want" to "the enum variant that holds it" and "the trait object type that comes out."

Why bother? The obvious alternative is to store `Box<dyn Any>` and downcast at every use site. That compiles, and it is wrong in a specific way: the mapping from what-I-want to which-variant lives at each call site. Twenty call sites, twenty copies of the same match, and nineteen of them get updated when the enum changes.

Here it lives once, in the tag's `try_get`. Call sites become `set.query::<RestApiCap>()` and cannot ask for the wrong thing — the associated type `Out` determines the return type, so a mismatch is a compile error rather than a runtime `None`.

The pattern name is not the point. The point is that a mapping written once cannot drift.

5.2.6 The capability set type

```
pub struct CapabilitySet { /* ... */ }

impl CapabilitySet {
    pub fn has<T: CapTag>(&self) -> bool { /* ... */ }
    pub fn query<T: CapTag>(&self) -> Option<Arc<T::Out>> { /* ... */ }
    pub fn labels(&self) -> Vec<&'static str> { /* ... */ }
}
```

Three methods, three distinct jobs.

`has` answers "should this gear take part in this phase." The runtime calls it while walking a phase and skips gears that answer false.

`query` answers "give me the handle so I can call it."

`labels` exists for diagnostics — it gives you the human-readable list of what a gear actually provides. That is the sort of method you only add after debugging a wiring problem at two in the morning, and its presence is a decent signal that this runtime has been operated rather than only written.

5.2.7 How a gear declares itself

```
/// Main API Gateway gear - owns the HTTP server (`rest_host`) and
/// collects typed operation specs to emit a single OpenAPI document.
#[toolkit::gear(
    name = "api-gateway",
    capabilities = [rest_host, rest, stateful],
    deps = [grpc_hub, authn_resolver],
    lifecycle(entry = "serve", stop_timeout = "30s", await_ready)
)]
```

```
pub struct ApiGateway {
    config: ArcSwap<ApiGatewayConfig>,
    openapi_registry: Arc<OpenApiRegistryImpl>,
    // ...
}
```

Source: `gears-rust/gears/system/api-gateway/src/gear.rs`

Everything is in that attribute:

`name` The runtime identity. This is the string the directory service resolves against later in this chapter.

`capabilities` Which phases this gear takes part in. Here: `rest_host` because it owns the server, `rest` because it also registers its own routes, `stateful` because it needs starting and stopping.

`deps` Which other gears must be initialised first — `grpc_hub` and `authn_resolver`.

`lifecycle` Entry point `serve`, a 30-second stop timeout, and `await_ready` so the runtime waits for readiness before proceeding.

This is a declaration, not a registration call. Nobody writes `registry.add(ApiGateway::new())` anywhere. That is the point of the next section.

5.2.8 Link-time discovery

```
/// The function type submitted by the macro via `inventory::submit!`.
pub struct Registrator(pub fn(&mut RegistryBuilder));
inventory::collect!(Registrator);

/// Discover via inventory, have registrators fill the builder,
/// then build & topo-sort.
pub fn discover_and_build() -> Result<Self, RegistryError> {
    let mut b = RegistryBuilder::default();
    for r in ::inventory::iter::<Registrator> {
        (r.0)(&mut b);
    }
    b.build_topo_sorted()
}
```

Source: `gears-rust/libs/toolkit/src/registry.rs`

Here is what happens. The `#[toolkit::gear]` attribute macro emits, for each gear, a function that knows how to add that gear to a `RegistryBuilder`, and submits it via `inventory::submit!`. The `inventory` crate collects those registrator functions from across the whole workspace at compile time.

At startup, `discover_and_build` iterates over them and lets each fill the builder. Then `build_topo_sorted` topologically sorts the dependency graph declared by all those `deps` lists.

So initialisation order is *derived*, not written down. Nobody maintains a list. If you add a dependency that creates a cycle, you get a `RegistryError` at build-and-sort time — a named error — rather than a mysterious runtime hang where two gears each wait for the other.

And "build a deployment with only these six services" becomes a Cargo feature or a binary target that links only six gear crates. Nothing else changes, because nothing else knew the other twenty-four existed.

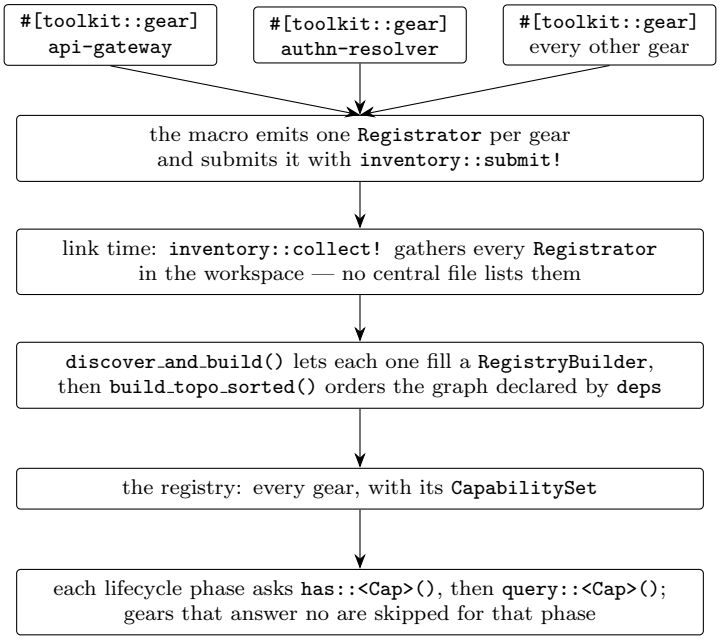


Figure 5.1: From an attribute to a runtime capability query.

5.2.9 The price of cleverness

Every link-time-magic system has one of these, so it is worth naming this one.

The macro generates hidden `pub use` re-exports, so the linker keeps each dependency's registration alive rather than stripping a crate nothing appears to reference. Those re-exports only exist in macro-expanded code.

Consequence: `cargo-shear`, the unused-dependency checker, cannot see them, and flags the crates as unused. The documented workaround is to add the crate name to an ignore list in the workspace manifest, or continuous integration fails.

That is an ongoing tax, paid per gear, forever. The mature move is to write the workaround next to the magic, which they did. The immature move — and the common one — is to disable the checker entirely, which trades one real false positive for the loss of every true positive it would ever have found.

5.2.10 Keeping the domain layer clean

Swapping capability implementations only works if business logic does not quietly depend on infrastructure types. The moment a domain function takes a database row type as a parameter, the ability to change database is gone whatever the capability trait says.

The usual approach to this is a code-review convention, plus hope.

Gears uses a custom lint instead. It statically rejects database or HTTP types appearing anywhere inside a type annotated as a domain model — including inside generic parameters, which is exactly where these things actually hide. A field of type `Vec<DbUser>` is the same violation as a field of type `DbUser`, and only the second one gets caught by reading carefully.

Convention becomes compile failure. Hold onto that sentence; it recurs in `gears-frontx` later in this chapter and five more times in Chapter 7.

5.3 The Host Runtime Lifecycle

```
//! High-level phase order:
//! - pre_init (system gears only)
```

```
//! - DB migrations (gears with DB capability)
//! - init (all gears)
//! - proxy-wiring (#[toolkit::consumes] clients; feature-gated)
//! - post_init (system gears only; after *all* init complete)
//! - REST wiring (requires a single REST host)
//! - gRPC registration (requires a single gRPC hub)
//! - start/stop (stateful gears)
//! - OoP spawn / wait / stop (host-only orchestration)
```

Source: `gears-rust/libs/toolkit/src/runtime/host_runtime.rs`

That is the whole runtime, in nine lines of module documentation. Walk it.

5.3.1 pre_init and migrations

pre_init runs only for gears with the system capability, before *any* gear's init has completed. This is where runtime internals get wired into the gears that need them — the small number of gears that are part of the machinery rather than users of it. The default implementation is a no-op, so most gears ignore it entirely.

DB migrations run for every gear with a database capability. And here is a design rule worth quoting exactly:

> "Gears MUST NOT receive raw database connections. They only return migration definitions."

Read what that buys. A gear declares the schema changes it needs; the runtime owns the connection and applies them. No gear can run an ad-hoc query during migration, hold a connection open, or discover it has a pool of its own. It also means the runtime can order migrations, wrap them, or run them against a different engine without touching a single gear.

5.3.2 init, proxy-wiring, post_init

init runs for all gears, in the topological order computed by `build_topo_sorted`.

proxy-wiring resolves the clients a gear declared it consumes, via `#[toolkit::consumes]`. It explicitly does *not* wait for provider availability: the client handle is wired, but the thing behind it may not be serving yet. That is eventual readiness, adopted by an accepted decision record rather than by accident.

post_init runs for system gears again, but now after every gear's init is done.

Why two system hooks? Because "before anything" and "after everything" are genuinely different needs, and cramming both into `init` produces ordering bugs. A gear that must observe the fully assembled set of gears cannot do it from `init`, because during `init` the set is half-assembled by construction.

5.3.3 rest, grpc, start, stop

rest composes the final router against the single REST host — the fold from `register_rest`, executed once.

grpc collects services from gRPC gears, and is skipped entirely if there is no hub and no services. Skipping a phase when nothing needs it is cheaper than making every deployment carry a gRPC stack.

start starts all stateful gears.

stop stops them, in reverse order. Reverse order on shutdown is not decoration. If A depends on B, then A must stop first, or A spends its last moments making calls into a B that is already tearing down.

5.3.4 One ordering constraint, spelled out

From the module documentation:

> "Proxy-wiring in particular must stay before `start`: a gear resolving a consumed contract during its own `start` has to find the client in the `ClientHub` regardless of profile."

That is a constraint with a reason attached, which is the only kind worth writing down. "Regardless of profile" is the operative phrase: whether the provider is in-process or out-of-process, the consumer's `start` must find a handle. So wiring has to be complete before anything starts, even though the thing behind the handle may not be ready.

Two lifecycle paths exist: in-process and out-of-process. Both run the phases in the same *relative* order. The out-of-process path has no REST-wiring, no directory-register, and no spawn phases, because those are host-only concerns.

Same skeleton, different subset. That is the only sane way to keep two paths honest — the alternative is two orderings that agree today and diverge in eight months.

5.3.5 Stopping properly

The stop phase implements a two-phase shutdown per gear, and it is wrapped in a watchdog that force-exits the process if stop hangs on a blocking syscall.

The detail worth noticing: the watchdog is disarmed whether stop succeeds *or* fails. A failing stop phase does not leave an armed timer behind to kill the process thirty seconds later during whatever cleanup follows.

Small detail, and exactly the kind that separates code that has been in production from code that has not.

5.4 Is This Template Method?

Look at the shape of the lifecycle: a fixed sequence of steps, with named extension points that default to no-ops. That is close to a textbook description of Template Method's hook operations, and a detector built to find Template Method flagged it immediately.

The detector was wrong. Why it was wrong is more useful than the fact that it was.

Template Method is fundamentally an **inheritance** mechanism. One class defines the algorithm and calls its own overridable methods; a subclass overrides the hooks. The abstract parent and the concrete child are two points in one hierarchy, and the dispatch is virtual dispatch.

In Gears, the lifecycle sequence lives on a single, *never-subclassed* runtime type. The hook implementations belong to dozens of unrelated service types — `ApiGateway`, `TenantResolver`, and so on — which share no ancestor. They are reached through a capability query over a heterogeneous registry.

That is composition across independent types, not virtual dispatch inside one hierarchy. Structurally it looks the same from a distance. Mechanically it is the opposite: in Template Method the parent knows the hooks exist because it declared them; here the runtime knows nothing about who implements what until it asks the registry.

5.4.1 The same mistake, twice more

Once you know what to look for, the same wrong flag appears elsewhere in the same runtime.

A worker-lifecycle notification mechanism: its own documentation calls it an Observer, and it is one. A pluggable storage-backend abstraction: that is Strategy. Both got flagged as Template Method, for the same reason — the no-op-hook shape.

A detector's structural shape is at best a necessary condition, never a sufficient one. Trust a purely mechanical detector and you get a confidently wrong list, which is worse than no list, because somebody will act on it.

5.5 ClientHub: Telling Patterns Apart

Gear A needs to call gear B. B might be in the same process, or behind a network hop, and A should not care. So A asks for "the thing that implements this trait" and gets a handle.

The thing that answers is called `ClientHub`. Now: what pattern is that? Three candidates are plausible, and working through them properly is a transferable exercise.

5.5.1 ClientHub, in its own words

```
//! Minimalistic, type-safe ClientHub.
//!
//! Design goals:
//! - Providers register an implementation once (local or remote).
//! - Consumers fetch by *interface type* (trait object):
//! `get::<dyn my::Api>()`.
//! - For plugin-like scenarios, multiple implementations of the
//! same interface can coexist under different scopes.
//!
//! Implementation details:
//! - Key = type name. We use `type_name::<T>()`, which works
//! for `T = dyn Trait`.
//! - Value = `Arc<T>` stored as `Box<dyn Any + Send + Sync>`.
```

Source: `gears-rust/libs/toolkit/src/client_hub.rs`

And the interface is genuinely this small:

```
pub fn register<T>(&self, client: Arc<T>)
where T: ?Sized + Send + Sync + 'static
{
    let type_key = TypeKey::of::<T>();
    let mut w = self.map.write();
    w.insert(type_key, Box::new(client));
}

pub fn get<T>(&self) -> Result<Arc<T>, ClientHubError>
where T: ?Sized + Send + Sync + 'static
{
    let type_key = TypeKey::of::<T>();
    let r = self.map.read();
    // ... look up, downcast, return
}
```

Source: `gears-rust/libs/toolkit/src/client_hub.rs`

Two functions and a read-write-locked map. The `?Sized` bound is what makes `T = dyn Trait` legal, and that bound is the entire design.

5.5.2 Candidate one: Singleton

There is one hub per runtime, so the word "singleton" comes to mind.

Singleton's defining property is not "there is one." It is that the class *enforces* that there can only be one. A private constructor, a static accessor, and no way to get a second instance.

`ClientHub` has a public, freely callable constructor with no such enforcement. And its own tests exploit exactly that freedom, building independent hubs so that one test's registrations do not leak into another's.

Rejected. One instance by convention is not Singleton.

5.5.3 Candidate two: Registry

A keyed store of things you look up by name. That is Registry, from Fowler's enterprise pattern set, and Gears has real ones elsewhere: `OpenApiRegistry`, `GearRegistry`, the healthcheck registry.

Registry's intent is a well-known object that other objects use to *find* common objects and services, keyed however you like.

The distinction from Service Locator is genuinely subtle and mostly about intent. Registry stores things. Service Locator abstracts *how you obtain an implementation of an interface.*

5.5.4 Candidate three: Service Locator

Look at what the key actually is: `type_name::<T>()` where `T` is a **trait object**.

You do not look up "the billing client." You look up "whatever implements `dyn BillingApi`." The caller states an interface and receives an implementation, without knowing which one, or where it lives. Local or remote, decided at registration time, invisible at use time.

That is Service Locator in the precise sense. Confirmed.

5.5.5 How the claim got confirmed

The verification is worth describing because the method generalises.

It started from two files that a first-stage analysis had cited. Read alone, those two files gave a weak-ish case — the code is compatible with all three candidates, and a doc comment saying "fetch by interface type" is suggestive rather than decisive.

Widening to a repository-wide sweep for the terminology turned up something better: an independent, non-draft architecture document, from an unrelated team in the same organisation, describing this mechanism unprompted as "the typed service locator."

The methodological lesson runs in the opposite direction from the usual warning. Verification scoped to a lead's cited location can *understate* the evidence. Grep wider than the claim.

5.5.6 Scopes

```
/// A scope for resolving multiple implementations of the same
/// interface type. This is intentionally opaque: the scope
/// semantics are defined by the caller.
pub struct ClientScope(Arc<str>);

pub fn get_scoped<T>(&self, scope: &ClientScope)
    -> Result<Arc<T>, ClientHubError>
where T: ?Sized + Send + Sync + 'static { /* ... */ }
```

Source: `gears-rust/libs/toolkit/src/client_hub.rs`

"Intentionally opaque" is a design decision, stated as one. The hub refuses to know what a scope means. A caller can use it for tenants, for API versions, for deployment regions, for test fixtures — and the hub's code never grows a branch per interpretation.

5.5.7 Why this exercise is worth your time

You will be asked, in interviews and in code review, "what pattern is this?" The useless answer is the one that matches on shape, which is what the Template Method detector did.

The useful method is three steps:

1. Write down the pattern's *defining* property, not its picture.

2. Find the code that would have to exist for that property to hold.

3. If it is not there, say so, and name what is actually there.

Applied here it took about ten minutes and rejected two of three candidates. Applied in Chapter 4 it produced the merge cascade verdict. It is the same three steps every time.

5.6 Gears-frontx

`gears-frontx` is the TypeScript frontend counterpart to `gears-rust`: nine independently versioned packages — `api`, `state`, `screensets`, `i18n`, `framework`, `react`, `cli`, `studio`, `docs`.

Different language, different runtime, same organisation, same problems. Reading it next to `gears-rust` is the point.

5.6.1 Plugin composition, frontend edition

```
/**
 * Create a HAI3 app builder for custom plugin composition.
 *
 * @example
 * const app = createHAI3()
 * .use(screensets())
 * .use(themes())
 * .build();
 */
export function createHAI3(config?: HAI3Config): HAI3AppBuilder {
  return new HAI3AppBuilderImpl(config);
}
```

Source: `gears-frontx/packages/framework/src/createHAI3.ts`

And `.use()` itself:

```
use(plugin: HAI3Plugin | PluginFactory | HAI3Plugin[]): HAI3AppBuilder {
  if (Array.isArray(plugin)) { // presets return arrays
    plugin.forEach((p) => this.use(p));
    return this;
  }
  const resolved = resolvePlugin(plugin);
  if (this.plugins.some((p) => p.name === resolved.name)) {
    if (this.config.devMode) {
      console.warn(`Plugin "${resolved.name}" is already
        registered. Skipping duplicate.`);
    }
    return this;
  }
  this.plugins.push(resolved);
  return this;
}
```

Source: `gears-frontx/packages/framework/src/createHAI3.ts`

Two things to notice.

Duplicates are silently skipped, keyed by plugin *name*, with a warning only in development mode. Quietly deduplicating is a choice, and the reasoning behind it is sound: a preset that includes a plugin you also added by hand should not explode. The dev-mode warning keeps the information available to the person who can act on it.

And `.use()` does almost nothing. It appends to an array. All the real work is deferred to `.build()`. This is a Builder in the ordinary sense: accumulate, then construct once.

5.6.2 What .build() does, in order

1. Resolve dependencies: a depth-first topological sort over each plugin's declared dependencies.
2. Invoke per-plugin registration callbacks, in that resolved order.
3. Aggregate every plugin's registries, state slices, effect initialisers and actions into one set.
4. Create the shared application state store.
5. Initialise effects.
6. Assemble the running application object.
7. Run a final per-plugin initialisation callback.

Step 1 is the same topological sort as `build_topo_sorted` in the Rust registry, written independently in a different language by different people. Step 7 is the same idea as `post_init`: a hook that runs after the whole set exists.

Two teams, two languages, no shared code, and the same two structural requirements fell out. Chapter 7 argues that when unrelated teams reinvent the same shape, the problem is real. This is a small instance of that.

5.6.3 A second composition axis

Plugins also extend framework-level TypeScript types without touching framework source. The mechanism is declaration merging.

So there are two composition systems running side by side: one at run time over plugin objects, and one at compile time over types. Neither knows about the other, and both are required for a plugin to feel first-class — a plugin that adds behaviour but cannot add the type describing its own configuration is a plugin you use with casts.

If you have never used declaration merging, this is a good reason to go read about it.

5.6.4 The layered SDK, and whether it is real

`gears-frontx` documents its packages as a layered architecture:

L1 (SDK) `state`, `screensets`, `api`, `i18n`.

L2 (Framework) `framework`.

L3 (React) `react`.

Plus the application layer in `src/`, which sits on top of all of it.

Chapter 4 rejected Studio's "four-layer hierarchy." So: is this one real? Here are the rules, from the dependency-cruiser configuration:

```
{
  name: 'sdk-no-framework-import', severity: 'error',
  from: { path: '^packages/(state|screensets|api|i18n)/' },
  to: { path: '^packages/(framework|react)/' },
  comment: 'SDK VIOLATION: L1 cannot import Framework (L2) or React (L3).'
}, {
  name: 'framework-no-react', severity: 'error',
  from: { path: '^packages/framework/' },
  to: { path: '^packages/react/' },
  comment: 'LAYER VIOLATION: Framework (L2) cannot import React (L3).'
}, {
  name: 'react-no-sdk', severity: 'error',
```

```
  from: { path: '^packages/react/' },
  to: { path: '^packages/(state|screensets|api|i18n)/' },
  comment: 'LAYER VIOLATION: React (L3) cannot import SDK (L1) directly.
           Use @gears-frontx/framework re-exports.'
}
```

Source: `gears-frontx/.dependency-cruiser.cjs`

Check it against the two criteria from Chapter 4.

Strict one-way dependency direction between adjacent layers: yes, and it is a list of rules rather than a diagram. L1 may not import L2 or L3. L2 may not import L3.

A defined service interface at each boundary: yes, and `react-no-sdk` is where you can see it. L3 may not reach past L2 into L1. It must go through `framework`'s re-exports — and that re-export surface *is* the boundary interface. The comment even tells the offending developer which one to use.

Severity `error`, run by dependency-cruiser in continuous integration.

Confirmed as Layered Systems.

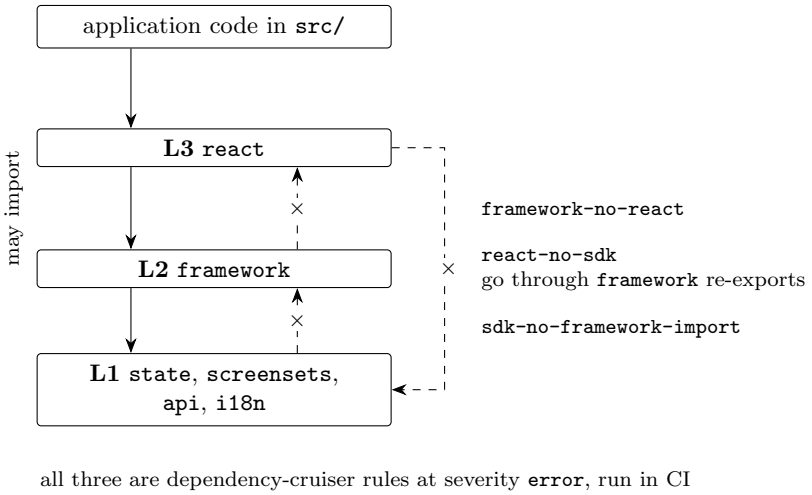


Figure 5.2: Three layers, and the imports that fail the build.

5.6.5 The callback

Same organisation. Same word, "layer." Opposite result.

Studio's four layers are a precedence-ordered list of manifests, merged. Nothing calls anything, so nothing can be enforced, so the word carries no guarantee.

Frontx's three layers are real import edges, with the forbidden ones failing a build.

The difference between an architecture and a diagram is a CI job. That is probably the single most transferable sentence in Part II, and it is worth being precise about why. Both projects wrote the word down. One of them also wrote down what happens when somebody violates it, and made that happen automatically. The second act is the entire difference.

5.6.6 The EventBus

```
class EventBusImpl implements IEventBus<EventPayloadMap> {
  private handlers: Map<string, Set<EventHandler<unknown>>> = new Map();

  emit<K extends keyof EventPayloadMap>(
    eventType: K,
    ...args: EventPayloadMap[K] extends void ? [] : [EventPayloadMap[K]]
```

```
  ): void {
    const handlers = this.handlers.get(eventType as string);
    if (handlers) {
      const payload = args[0];
      handlers.forEach((handler) => handler(payload));
    }
  }
  // on(...) returns a Subscription with unsubscribe()
}
```

Source: `gears-frontx/packages/state/src/EventBus.ts`

Event-Based / Implicit Invocation requires three things:

- real many-to-many publish and subscribe,
- dynamic handler registration *and* deregistration,
- emitting code holding no reference to specific handlers.

`Map<string, Set<Handler>>` gives many-to-many: many event types, many handlers each. `on()` returns a subscription object with `unsubscribe()`, so deregistration is first-class rather than an afterthought — which matters in a frontend, where a component that subscribes on mount and never unsubscribes is a leak. And `emit` touches only the set, never a named handler.

Confirmed without qualification. And it is called `EventBus`, which is what it is.

5.6.7 Observer, not Mediator

An early informal note hedged between the two, and the distinction matters.

Mediator's defining property is an object that *actively coordinates* communication between colleagues that would otherwise reference each other. The mediator contains logic about what should happen when A changes: it decides that B must be updated and C must be disabled.

An event bus that merely lets publishers and subscribers not know about each other has Observer's property instead. Decoupling on its own implies no coordination logic. `emit` forwards a payload to a set; it does not decide anything.

So: Observer, or more precisely publish-subscribe. Not Mediator.

5.6.8 A doc comment that is not true

A separate component in the same codebase claims, in its own doc comment, to provide "the core effect coordination infrastructure." That looked like the real Mediator, so it was checked for exactly that reason.

Its initialisation routine consists of three explanatory comments and zero executable statements. No cross-plugin coordination is registered anywhere in it.

The claim has no corresponding code at the location it describes.

One caveat, stated precisely, because the honest version of this finding is narrower than the exciting version: a second, substantive coordinator with the same generic name lives elsewhere in `gears-frontx`, managing microfrontend runtime connections. What is rejected is this one component's claim about itself, not coordination in the framework.

5.7 Federation at a Different Granularity

Studio federates repositories. Gears federates *processes and network boundaries*. Same word, genuinely different problem.

A gear can run in your process, or in another one, or on another machine. And the calling code should not change.

5.7.1 A default that got reversed

Earlier architecture documentation describes gRPC as the out-of-process transport. A later, much more detailed design document explicitly reverses this to REST. gRPC stays available, opt-in, for performance-critical internal paths.

There is also a hard non-regression constraint attached: existing single-process deployments must be *provably* unaffected.

Reporting this honestly matters more than the fact itself. Read only the older document and you would describe the system wrongly, with confidence, citing a real source. Architecture documents have dates, and you have to check them. Chapter 6 contains a much larger instance of the same lesson.

5.7.2 The directory service

```
pub trait DirectoryClient: Send + Sync {
    /// Resolve a gRPC service by its logical name to an endpoint
    async fn resolve_grpc_service(&self, service_name: &str)
        -> Result<ServiceEndpoint>;

    /// Resolve a REST endpoint (HTTP base URL) for a gear by name.
    async fn resolve_rest_service(&self, gear_name: &str)
        -> Result<ServiceEndpoint>;

    /// Retrieve the OpenAPI spec (JSON) published by a gear.
    async fn get_openapi_spec(&self, gear_name: &str) -> Result<String>;

    async fn list_instances(&self, gear: &str)
        -> Result<Vec<ServiceInstanceInfo>>;
}
```

Source: `gears-rust/libs/system-sdks/sdks/directory/src/api.rs`

Out-of-process gears self-register their REST endpoint and OpenAPI specification with the directory. Callers resolve peers *by name* — the `name` from the `#[toolkit::gear]` attribute. There is no static configuration file listing where everything lives.

There are two implementations of this one trait:

- a local one that delegates to the in-process gear manager;
- a gRPC client for out-of-process gears.

Same trait, so calling code cannot tell which it got. That is the whole trick, and it is the same trick as `ClientHub`: state an interface, receive an implementation, do not learn where it lives.

5.7.3 Two careful details in that API

`list_instances` entries are spec-free: only the OpenAPI spec *hash* travels, never the document. So a multi-instance response stays bounded regardless of spec size — and a consumer can still detect a spec change from the hash. That is the useful half of the information at a fraction of the bytes.

An endpoint-less instance is still returned, with `None`, never an empty-string sentinel. "No endpoint" is not an error and is not an empty string that some caller will eventually try to parse as a URL.

Both of these are the difference between an API that survives contact with production and one that does not.

5.7.4 Where the routes get served

A minimal gateway-provider abstraction lets the same route declarations be exposed through either a built-in reverse proxy, on-premise, or an external gateway such as Kong, in Kubernetes.

Gear code does not depend on which one is active. This is the same move as the directory's two implementations, one level up.

Notice the recurring habit, because it is the house style: one trait, two realisations, the choice made at wiring time. Chapter 7 finds it again in the identity layer.

5.7.5 API versioning at the type level

How do you serve v1 and v2 of an API at once?

Gears' answer: two differently-versioned traits are, to the dependency-injection mechanism, simply two different types. `ClientHub` keys on `type_name::<T>()`, and `dyn ApiV1` and `dyn ApiV2` are different names, so both can be registered and both resolve unambiguously.

So a consumer's choice of API version is a **compile-time property**, not a configuration value. You cannot misconfigure your way into calling the wrong version. You also cannot switch version without recompiling, which is the honest cost and is written down as one.

5.7.6 Trait inheritance, considered and rejected

The obvious alternative was to version by trait inheritance: `ApiV2: ApiV1`. It was rejected, with a one-line argument recorded in the decision:

> "it can only express additive changes, which need no new version"

Read that again. The mechanism handles exactly the case that does not need it. If v2 only adds methods, v1 consumers are unaffected and there was no reason to cut a version. The case you actually need versioning for — a changed signature, a changed contract, a removed method — is the case trait inheritance cannot express.

Breaking changes instead ship as an entirely new, parallel trait pair, served alongside the old one for a defined deprecation window.

This decision record is worth studying because it names why the tempting answer fails. Most rejected-alternatives sections say "we did not choose X." This one says what X cannot do.

5.7.7 Backward compatibility, enforced by CI

Code generation keeps the client and server in step. But code generation only guarantees that *today's* client matches today's server.

So there is an OpenAPI-diff gate in continuous integration. It fails the build if a deployed service would stop accepting what an *older* client version generates.

Again: the rule exists because a job runs it, not because a document states it.

5.8 Two More Frontend Details Worth Keeping

5.8.1 The blob URL trick

Frontx has nine independently versioned packages and a microfrontend runtime, and that combination produces a specific, non-obvious problem.

A naive Module Federation setup lets sibling microfrontends unintentionally share module-level state — including each other's event bus and application store. Why? Because browsers cache ES modules by URL identity. Two microfrontends loading the same URL get the same

module instance, whether or not you wanted that. Module Federation's sharing guarantee, which exists to deduplicate a React copy, also deduplicates your singleton state.

The fix deliberately defeats that guarantee. Each microfrontend load fetches its module source as *text*, then evaluates it behind a freshly minted, UUID-unique blob URL. Unique URL, therefore unique module instance, therefore no accidental sharing.

And there is a documented, permanent cost: those blob URLs can never be revoked for the lifetime of the page, because a browser's module loading may resolve and cache the URL reference before the body finishes executing. Revoke it and you risk breaking a load that has not finished.

A leak, accepted knowingly, and written down. That is what a real trade-off looks like — not a decision with no downside, but a decision whose downside is named and bounded.

5.8.2 Publishing nine packages

The nine packages must publish in strict dependency order. Not Changesets, not Lerna: a custom pipeline.

It walks the packages in layer order, detects version changes via `git diff`, and is safe to re-run after a partial failure.

That last property is the one that matters. Any release script that cannot be re-run after failing halfway will, eventually, fail halfway — and then somebody publishes package four by hand at eleven at night and gets the version wrong.

5.9 Where That Leaves Us

- Capabilities are optional traits the runtime queries for. Composition happens at link time and ordering is computed, so no central file enumerates the gears.
- A fixed phase sequence with no-op hooks looks like Template Method and is not, because there is no inheritance anywhere in it.
- `ClientHub` is a Service Locator, keyed by trait-object type name. Not Registry, and definitely not Singleton.
- Frontx's L1/L2/L3 layering is real, because dependency-cruiser fails the build. Studio's was not, because nothing did.
- The `EventBus` is a clean publish-subscribe, correctly named — and a neighbouring doc comment in the same codebase is simply false.

Chapter 6 moves to Insight, where the invariants are about data instead of types, and where the sharpest lesson in Part II is waiting.

Exercises

1. **Find the contracts file and explain the cardinality split.** Locate `gears-rust/libs/toolkit/src/contracts.rs`. List every capability trait it declares, and for each one state whether the runtime expects exactly one gear to provide it or many. Then explain, in a short paragraph, what would break at which lifecycle phase if two gears both declared the `rest_host` capability, and find the code that detects it.
2. **Trace a request from attribute to running route.** Start from the `#[toolkit::gear]` attribute on `ApiGateway` in `gears-rust/gears/system/api-gateway/src/gear.rs`.

Follow the chain: macro expansion, `inventory::submit!`, `discover_and_build`, `build_topo_sorted`, then each lifecycle phase in `host_runtime.rs` that this gear participates in, given its declared capabilities. Produce an ordered list of the phases it takes part in and the phases it is skipped for, and say why for each.

3. **Write a minimal gear.** Write a gear that satisfies the pattern shown in this chapter: it implements the core `Gear` trait and exactly one capability, declares one dependency and no lifecycle entry point, and opts out of the healthcheck by relying on the trait default. Then answer without running it: at which phase does your gear's code execute, and what happens if the gear named in your `deps` is not linked into the binary?

4. **Settle a pattern question the hard way.** Pick `OpenApiRegistry` or the healthcheck registry in `gears-rust`. Apply the three-step method from this chapter: write the defining property of Registry from a reference source before reading any code; identify what code must exist for it to hold; then read the implementation and deliver a verdict of confirmed, split, or rejected. If you reject it, name what the structure actually is.

5. **Compare two layer claims.** Read `gears-frontx/.dependency-cruiser.cjs` in full, then re-read the `discover_layers` docstring from Chapter 4. Write half a page comparing them against the two Layered Systems criteria. Then propose, concretely, what Studio would have to add for its four-layer claim to become defensible — name the check, name where it would run, and name the violation it would catch.

Chapter 6

Insight, Data and Lineage

Chapter 4 looked at an orchestration layer with half its control flow written in prose. Chapter 5 looked at a plugin runtime where most of the rules are enforced by a compiler.

Insight is the third flagship subsystem, and it is a different discipline again. The invariants here are about *data*, not types. A compiler cannot help you. There is no equivalent of a capability trait to make a mistake unrepresentable. What you have instead is table structure, one hard rule, and a willingness to enforce it.

There is also one story at the end of this chapter that is worth remembering longer than anything else in Part II, and it is not a story about Insight's engineering. It is a story about believing a design document.

6.1 What Insight Is

Insight is a polyglot data platform. It ingests data from third-party developer tools and normalises it into something you can ask questions of.

The sources it actually connects to: Jira, Slack, Zulip, Zoom, Microsoft 365, Confluence, Outline, Zendesk, HubSpot, BambooHR, Microsoft Entra, GitHub Copilot, Cursor, ChatGPT Team, Claude Team.

The languages in play: Python, SQL, Rust, TypeScript, plus a great deal of YAML.

The output is metrics about how software actually gets built in an organisation — which is a politically loaded product, and that turns out to have shaped the architecture in at least one place we will come to.

6.1.1 Why a data platform belongs in an architecture study

Data platforms have a reputation for being architecturally shapeless: a pile of SQL, a scheduler, and hope. Every transformation is a file, every file knows about three other files, and the only person who understands the dependency graph left last year.

Insight is not that, and the reason is one hard invariant that this chapter spends a section on. It is also the clearest instance in the whole federation of a **named architectural style** that survives verification — and, in the same subsystem, the clearest instance of documentation that does not.

The shape is three stages, one direction, and one edge that is not allowed to exist:

$$\text{sources} \rightarrow \text{Bronze (raw)} \rightarrow \text{Silver (normalised)} \rightarrow \text{Gold (analysis-ready)}$$

and Bronze $\rightarrow$ Gold is forbidden.

6.2 Connectors

Studio extends through kits. Gears extends through capabilities. Insight extends through **connectors**: new third-party data sources.

Same underlying design problem — optional, independently addressable extension points — and a third completely different mechanism.

A connector is a directory with a `connector.yaml` manifest. Two types are supported, and the distinction matters more than it looks:

`nocode` A declarative specification, executed entirely by a third-party runtime: the Airbyte Connector Development Kit. No Python of the connector's own, at all.

`cdk` A thin Python extractor built against that same third-party framework.

So the extraction machinery is inherited, not designed here. Insight's own documentation is candid about this: CDK connectors are "thin extractors" that "transform nothing downstream."

6.2.1 A real connector manifest

```
# Airbyte Declarative Source for zendesk - INSIGHT-459.
#
# Streams:
# 1. support_tickets - incremental, DatetimeBasedCursor
# 2. support_agents - full refresh
# 4. support_ticket_events - per-ticket audit log
# (SubstreamPartitionRouter over
# support_ticket_ids)
#
# Auth: HTTP Basic Auth, username = "{email}/token"

version: "7.0.4"
type: DeclarativeSource
```

Source: `insight/src/ingestion/connectors/support/zendesk/connector.yaml`

Read the header comment rather than the two lines of YAML. It carries a ticket identifier, a per-stream inventory with each stream's sync mode, and the authentication scheme. `support_tickets` is incremental with a `DatetimeBasedCursor`; `support_agents` is full refresh; `support_ticket_events` is a substream partitioned over ticket ids.

That is the whole operational character of the connector in eight lines of comment, which is what you want when the thing you are debugging is why one stream is six hours behind.

6.2.2 Four fields stamped on every record

```
standard_fields: &standard_fields
  - type: AddedFieldDefinition
    path: [tenant_id]
    value: "{{ config['insight_tenant_id'] }}"
  - type: AddedFieldDefinition
    path: [source_id]
    value: "{{ config['insight_source_id'] }}"
  - type: AddedFieldDefinition
    path: [collected_at]
    value: "{{ now_utc().strftime('%Y-%m-%dT%H:%M:%SZ') }}"
  - type: AddedFieldDefinition
    path: [data_source]
    value: insight_zendesk
```

Source: `insight/src/ingestion/connectors/support/zendesk/connector.yaml`

Tenant, source, collection time, source name. On every row, at the point of extraction.

Two things follow from "at the point of extraction." First, there is no later stage where a record could arrive without provenance, because the only way in stamps it. Second, the YAML anchor `&standard_fields` means every stream in this connector shares one definition of what provenance means — so a connector author cannot accidentally give one stream three of the four fields.

6.2.3 Where the engineering actually is

It is worth being honest about attribution in an architecture study. If extraction is inherited from Airbyte, what did this organisation build?

Everything downstream of extraction:

- normalisation,
- identity resolution across tools,
- the lineage tracking we are about to look at.

Their documentation says this plainly, which is unusual and worth crediting. Most vendor architecture documents draw the inherited part in the same colour as the built part, and you find out which is which when you file a bug against the wrong repository.

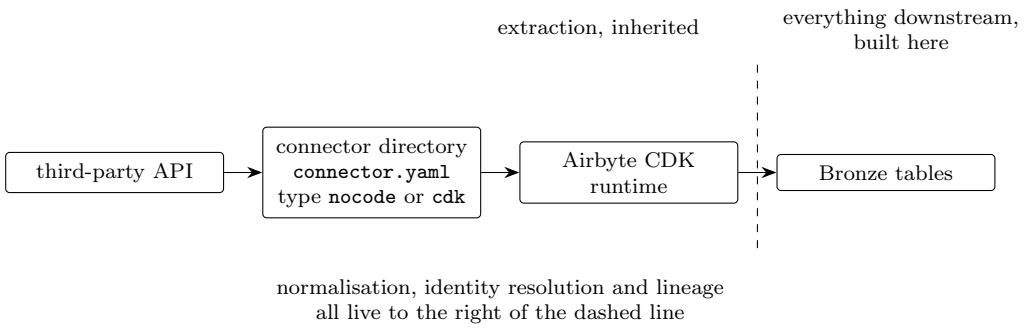


Figure 6.1: Where the borrowed part stops.

6.2.4 Three tiers of connector authorship

first-party Built and fully maintained by the platform team.

community Open-source contributors, reviewed by the platform team.

self-service Customers writing their own; the platform provides the SDK and the validation.

The stated reason for this structure is a scaling argument, not a technical one. There are thousands of potential customers, each with a different tooling stack. Requiring the platform team to build every connector "creates a permanent bottleneck."

That is the honest reason most extension mechanisms exist. Extensibility is an answer to an organisational problem: the team cannot be in the critical path of every integration, so the integration point has to be something a stranger can use without asking permission. Chapter 7 makes the same observation across all four subsystems at once.

6.3 Bronze, Silver, Gold

Three named stages, each a set of tables:

Bronze Raw data, exactly as the source gave it, plus the stamped fields.

Silver Cleaned, normalised, identity-resolved.

Gold Aggregated, analysis-ready tables.

The naming convention is not original to this platform; the medallion architecture is common in data engineering. What is worth studying is what they did with it.

6.3.1 What actually lives in Gold

Roughly twenty SQL models, and the names tell you the shape of the product: `git_metric_observations`, `task_metric_observations`, `collab_metric_observations`, `ai_metric_observations`, `wiki_metric_observations`.

And next to each one, a matching `*_metric_evidence`.

Every observation has an evidence table beside it. So a number in a dashboard can be traced back to the rows that produced it.

That pairing is a design decision about trust, not about SQL. Recall what this product measures: how software gets built, by named people. Metrics about people's work that cannot be audited will not survive contact with the people. The first time somebody's manager says "this says your review latency doubled" and the only available answer is "the pipeline says so," the metric is dead. An evidence table makes the answer "here are the fourteen reviews it counted," which is a conversation that can go somewhere.

6.3.2 Identity resolution is the hard part

The same human being appears as a Jira account id, a Slack user id, a Git commit email, and a Microsoft Entra object id. None of those match. Some of them change.

Insight resolves this in exactly one place: the Bronze-to-Silver step.

There is a whole `identity/` tree of alias rules, plus macros such as `resolve_person_id` and `create_identity_persons`. And there is a Gold model named `identity_resolution_coverage`, because the honest answer to "did we resolve everybody?" is a percentage, not a yes.

Shipping the coverage number as a first-class model rather than a log line is the same instinct as the evidence tables. The system's own uncertainty is part of the product.

6.3.3 The macro library

Shared transformation logic lives in dbt macros, and the list reads like a domain model:

`promote_bronze_to_rmt` The Bronze-to-Silver promotion.

`resolve_person_id`, `create_identity_persons` Identity.

`silver_incremental_watermark` Incremental loading.

`git_file_category`, `git_file_extension` Classification.

`iso8601_duration_seconds` The tedious one somebody had to write once.

Twenty-odd macros, and the whole normalisation story of this platform is in them. Not in a schema registry. Not in JSON Schema. In macros.

Remember that when we get to the last section of this chapter, because it is the single fact that resolves the puzzle there.

6.4 The Invariant

> **Principle: Bronze is never queried at Gold level.** All data must pass through the Silver layer before reaching Gold. *This is a hard constraint, not a recommendation.*
>
> Source: `insight/inbox/architecture/CONNECTORS_ARCHITECTURE.md`

That is a direct quote, under its own heading, in the connector architecture document. The phrase "not a recommendation" is doing real work in that sentence, and the rest of this section is why.

6.4.1 Why the invariant exists

The document gives four reasons, and none of them is aesthetic:

1. Bronze holds raw source-native identifiers. `person_id` has not been assigned yet.
2. Identity resolution runs *only* in the Bronze-to-Silver step.
3. Workspace isolation via `workspace_id` is guaranteed only at Silver and above.
4. Gold queries read exclusively from Silver `class_*` tables.

Reasons 1 and 2 are correctness: a Gold query against Bronze would be aggregating over unresolved identities, so one person would appear as four people and every per-person metric would be wrong in a way that looks plausible.

Reason 3 is different in kind. It is a **security** property. A Gold query reaching into Bronze would be reading across tenant boundaries.

That reframes the whole invariant. It is not a style rule about pipeline hygiene. It is the thing standing between one customer's dashboard and another customer's data, and the mechanism enforcing it is that Gold models only ever name Silver tables. Which is exactly the kind of enforcement Chapter 7 argues you should not rely on — and this chapter's final section is about what happens when a document's claims are not checked.

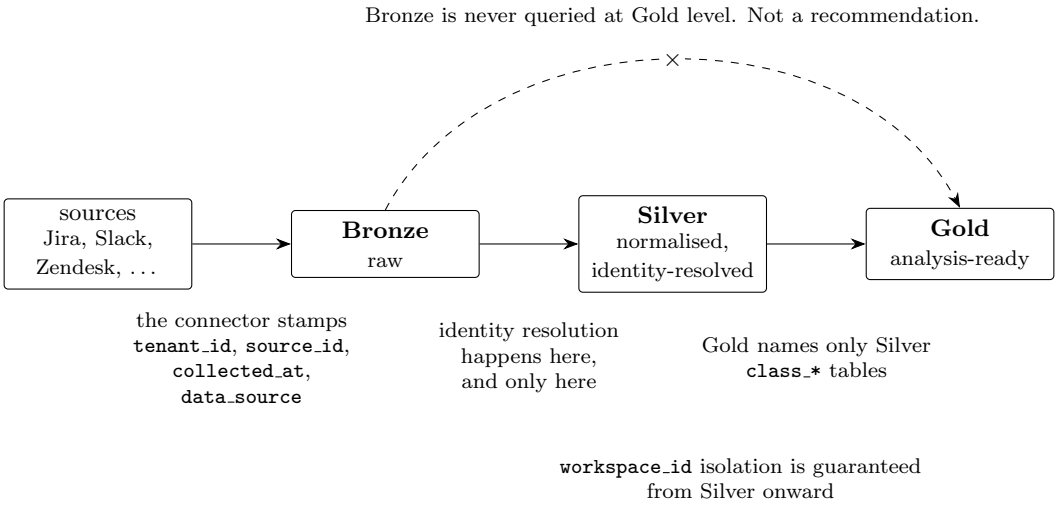


Figure 6.2: Three stages, one direction, one forbidden edge.

6.4.2 The documented exception

Some data cannot be attributed to an individual person: organisation-level aggregates, anonymous usage counters. GitHub Copilot's organisation usage figures are the canonical case.

The tempting shortcut is obvious. There is no person to resolve, so let Gold read those straight out of Bronze. The correctness reasons do not apply.

What they did instead: create a Silver stream anyway, keyed by `(workspace_id, date)` without `person_id`. The canonical example is `class_ai_org_usage`.

The reason given is reason 3 from the previous section. Gold needs to apply the standard workspace isolation and query through the standard interface, and the only way to have that is for the data to be in Silver, person or no person.

The exception was engineered to preserve the invariant rather than to escape it. That is the distinguishing mark of an invariant somebody actually believes in. An invariant with escape hatches is a default.

6.4.3 Tenant isolation, chosen deliberately

An accepted decision record requires `workspace_id` to appear as a partition or primary-key component on *every* Bronze, Silver and Gold table.

The alternatives were considered and rejected by name:

- schema-per-tenant,
- database-per-tenant.

Both were rejected as incompatible with the target operational scale on the chosen storage engine.

Note the form of that argument. It is not "this is better." It is "this does not work *for us*, at *our* scale, on *this* engine."

Decision records that name their constraints age much better than ones that name their preferences. If the storage engine changes, this record tells the next reader exactly which premise to re-examine. A record saying "schema-per-tenant is worse" tells them nothing and will be re-litigated from scratch.

6.5 Pipes and Filters, For Real

Now the verification. Criteria first, declared before looking at the code, so the judgement is not fitted to the answer.

Pipe-and-Filter, in Garlan and Shaw's sense, requires filters that:

- transform data communicated *exclusively* through ports,
- hold no shared mutable state,
- are unaware of their neighbours' identity.

Disqualifying evidence:

- fan-out/fan-in topology that breaks linear composition,
- in-place mutation of a shared context object across stages,
- a side channel bypassing the nominal chain.

Hold Bronze/Silver/Gold against that list.

6.5.1 Why the medallion pipeline passes

Communication is exclusively through tables. There is no other channel — no shared cache, no message the stages pass each other, no coordinator holding state across them.

No shared mutable context object is threaded through the stages. Each stage reads tables and writes tables.

A Silver model does not know which Gold model will read it. This is the neighbour-ignorance criterion, and dbt's dependency model makes it structural: the Gold model names the Silver model, not the other way round.

Composition is linear: Bronze, then Silver, then Gold. And the one edge that would break it, Bronze to Gold, is explicitly forbidden.

Confirmed — and it is the strongest-evidenced claim in the entire platform.

6.5.2 Why the evidence is unusually strong

Three independent kinds of evidence land on this one claim at once:

1. The layering is named explicitly and repeatedly, across several non-draft documents.
2. The documentation commits to a hard invariant, stated as a constraint rather than a suggestion.
3. SQL transformation models carry a machine-checkable marker tying code back to the named principle.

Author declaration, structural confirmation, and a code-level link, all holding simultaneously. That combination occurs essentially nowhere else in this codebase, and the third item is the subject of the next section.

6.5.3 Three pipelines that did not pass

Pipe-and-Filter was tested four times across two platforms. One confirmation, three rejections:

- Studio's map command — informally called a "pipeline" in its own codebase. Rejected.
- Gears' API-gateway middleware stack. Rejected, because stages mutate a shared context.
- OpenHands' agent run loop. Rejected.

The middleware stack is instructive in the other direction. Nobody ever called it a pipeline. So it correctly avoided a name its structure would not have earned — and it is Chain of Responsibility, which it does satisfy.

That is the outcome you want from a naming discipline. Not every mechanism has to be a famous style. It has to be called what it is.

6.5.4 A style that is simply absent

Repository/Blackboard requires several independent components interacting *only* through a shared passive store, with no awareness of one another. Ordinary CRUD access does not qualify, and neither does an object-relational mapper.

`insight-lite` was checked for it, on the reasonable guess that a data platform might have one. It does not. Rejected, on both platforms it was tested against.

Reporting an absence is worth doing, for two reasons. A silent omission is indistinguishable from a style nobody examined. And leaving it out understates how wide the search actually was — which matters when the headline finding is about names that do not hold up.

6.5.5 Naming, scored

Across twenty architectural-style judgements, five carried an explicit style-suggestive name. Table 6.1 is the scoreboard.

Table 6.1: Style-suggestive names, checked against their structures.

Name	Where	Verdict
`EventBus`	`gears-frontx`	correct
`PubSub`	OpenHands	correct
"L1/L2/L3" layering	`gears-frontx` packages	correct
"pipeline"	Studio's map command	wrong
"four-layer hierarchy"	Studio's manifests	wrong

Three out of five. Not a disaster, and not a record you should trust blindly either. The two failures both used a structural word for something that was not that structure.

6.6 Traceability Markers

Insight uses a convention called `@cpt-principle`: a comment in a SQL model naming, by identifier, the architectural principle that model implements.

Twenty-six such markers exist, across five distinct principle identifiers, and every one of them is scoped to the same Bronze-to-Silver dataflow.

Code-to-documentation linkage this specific is not something the pattern-detection literature describes at all. It is also, structurally, the same idea you used in Chapter 3: a stable identifier repeated verbatim in a document and in the code that implements it, so a mechanical tool can join them. Same mechanism, applied to architectural principles instead of functional requirements.

6.6.1 A marker in the wild

```
{# Bootstrap model for Slack bronze -> RMT promotion.
   Counterpart of `jira__bronze_promoted` for Slack. See ADR-0002.
   The `promote_bronze_to_rmt` macro is idempotent - already-RMT
   tables are detected and skipped on subsequent runs. #}

-- @cpt-principle:cpt-dataflow-principle-promote-bronze:p1
{{ config(materialized='view', schema='staging', tags=['slack']) }}

{% do promote_bronze_to_rmt(table='bronze_slack.channels',
                            order_by='unique_key') %}
{% do promote_bronze_to_rmt(table='bronze_slack.messages',
                            order_by='unique_key') %}
{% do promote_bronze_to_rmt(table='bronze_slack.users',
                            order_by='unique_key') %}

SELECT 1 AS promoted
```

Source: `insight/src/ingestion/connectors/collaboration/slack/dbt/slack__bronze_promoted.sql`

Read the whole file, because almost every line is doing something.

The marker sits directly above the `config` call, so it travels with the code. Move the model, and the marker moves; delete the model, and the marker goes with it. That is the property that makes this kind of link survive, and it is the same reason Chapter 3's CPT markers live in comments next to the function rather than in a separate mapping file.

The comment names its counterpart for another source (`jira__bronze_promoted`) and the decision record behind both (ADR-0002). So a reader who arrives here from the Slack side can find the Jira side and the reasoning without a search.

`promote_bronze_to_rmt` is the shared macro from the library above. The file is a list of calls to it, one per Bronze table — which is why "normalisation lives in macros" is the operative fact about this platform.

"The macro is idempotent" is stated explicitly, and that is exactly what you need to know before re-running a failed job at midnight. It also tells you how: already-promoted tables are detected and skipped.

And the final `SELECT 1 AS promoted` exists because the tool needs the model to return *something*. It is a small honest artifact of using dbt for a step that is a side effect rather than a query.

6.6.2 The marker family

`@cpt-principle` is not alone. The same convention runs through the whole platform, and you have already met most of it in Chapter 3:

`@cpt-algo` This code implements a named algorithm.

`@cpt-dod` This code satisfies a named definition-of-done item.

`@cpt-state` State handling for a named requirement.

`@cpt-begin` / `@cpt-end` Delimiting an instance region.

These appear in `gears-frontx`'s `EventBus` and in the Rust layer-discovery code from Chapter 4 — both quoted earlier in Part II without comment. Go back and look.

So the traceability idea crosses subsystem and language boundaries: Python, Rust, TypeScript, SQL, and the Markdown documents that define the identifiers. It is the one convention shared by all three flagship subsystems.

6.6.3 One awkward fact

Six principle identifiers are referenced by these markers.

Not one of them is defined in prose, in any markdown document, anywhere in the repository.

So the convention is entirely self-documenting at its point of use. It is not backed by a separate specification artifact. If you want to know what `cpt-dataflow-principle-promote-bronze` means, you read the models that claim it and infer.

Is that bad? Genuinely arguable, and both sides have a real case:

- A link to a document that does not exist is a broken link. The marker promises a definition and there is none.
- A document that exists and drifts from the code is arguably worse, because it looks authoritative while being wrong.

What is not arguable is that you should know which situation you are in. And in this codebase, the way to find out is the subject of the rest of this chapter.

6.7 The Unifier

Everything so far in this chapter has held up. This section is about something that did not, and about how long it took to notice.

It is worth telling in full because the mistake was made *by the study*, not only by the platform.

6.7.1 What the documentation describes

Insight's architecture documentation describes a mechanism called the **Unifier**:

- JSON-Schema-defined canonical entity types;
- which normalise the heterogeneous schemas that different third-party connectors produce;
- and propagate semantic metadata downstream into a shared dictionary.

Read on its own, this is an elegant, well-specified design. In the enterprise-integration literature it is a textbook Canonical Data Model, and it is the right answer to Insight's actual problem: fifteen source systems with fifteen notions of what a "user" is.

It even has a checklist:

```
New Connector Checklist:
+------------------------------------------------------------+
| [ ] connector.yaml - manifest with endpoints |
| [ ] unifier_mapping.yaml - field mappings + semantic data |
| [ ] src/client.py - API client |
| [ ] src/extractor.py - data extraction |
| [ ] src/transformer.py - apply unifier mapping |
| [ ] dbt/staging/stg_{source}.sql - staging model |
| [ ] identity/aliases_{source}.yaml - identity resolution |
| [ ] tests/ - unit and integration tests |
+------------------------------------------------------------+
```

Source: `insight/inbox/architecture/CONNECTORS_ARCHITECTURE.md`

Eight items, with filenames. This is not a vague aspiration. It names the artifacts a connector author must produce, including `unifier_mapping.yaml` and a `transformer.py` that applies it.

6.7.2 The first verification pass

The Unifier was recorded as **confirmed**.

The reasoning at the time was not stupid, which is exactly the problem:

- the claim comes from the same document as the Bronze/Silver/Gold invariant;
- that invariant *is* correct, and was independently corroborated three separate ways;
- so the document had earned some credibility.

Credibility got transferred from one claim to another claim in the same file. Nobody separately checked whether the Unifier had an implementation.

6.7.3 The second pass

A later, dedicated deep read of Insight went looking specifically.

A repository-wide search for the term, across every line of Insight's source: zero results. Thirty-two mentions in markdown. None in code.

No `unifier_mapping.yaml` exists anywhere. No `transformer.py` applying one.

And the document itself carries a subtitle, right under the title:

> *"Conceptual architecture for data ingestion subsystem"*

It said so on line three. Nobody read line three carefully enough.

6.7.4 What actually does the normalising

The work described by the Unifier does happen. Something has to normalise Jira and Slack into one shape, and Silver demonstrably contains normalised data.

It happens through an entirely different, **unnamed** mechanism: a family of dbt staging models, unified by a shared macro — the `promote_bronze_to_rmt` from earlier in this chapter.

It accomplishes something like the same practical goal. Without the schema registry, without the JSON Schemas, and without the semantic dictionary.

So the design is not implemented, and the problem is solved anyway, differently. That is the pattern to expect. Unimplemented designs are usually not unmet needs; they are needs that got met by whatever was to hand while the document stayed as written.

6.7.5 Three lessons, in order of importance

1. **A system's own design documentation is a claim about that system.** It is evaluable with the same skepticism as any other claim. It does not get a free pass past verification because the organisation being studied wrote it.

2. **Credibility does not transfer between claims in the same document.** One correct paragraph earns that paragraph credit, and nothing else. The Bronze/Silver/Gold invariant being triple-corroborated says nothing whatsoever about the Unifier three sections later.

3. **Check your own conclusions on a second pass.** The first pass here was careful and still wrong.

6.7.6 Why report it rather than quietly fix it

The easy option was to correct the draft and say nothing.

That would have hidden the one thing worth knowing: this class of error is easy to make. An architecture study willing to disclose its own documentation-as-authority lapse is showing you that the method works, because the method is what caught it.

It also generalises far past this one finding. You will be handed design documents for the rest of your career. Some fraction of every one of them describes something that was never built — not through dishonesty, but because documents are written before code and are not always updated after.

6.7.7 How to tell, quickly

A practical checklist, from this exercise:

1. Read the document's own subtitle and status line first.

2. Grep the codebase for the mechanism's name, then for the artifact filenames it promises.

3. Look for a test. Unimplemented designs have no tests.

4. Ask what would break if the mechanism vanished. If nothing, it may already have.

All four take under ten minutes. The Unifier failed all four.

6.7.8 Names that turned out to be empty

The Unifier is not alone. Across the platform, eight named claims failed verification, and Part II has covered most of them:

- Studio's map command, called a "pipeline";
- Studio's manifest utility, called a "four-layer hierarchy";
- 150-plus PDSL modules named `*-router.md`, implementing a dispatch table;
- a `repository.rs` in a `domain` directory, embedding raw SQL and implementing no interface;
- a `gears-frontx` doc comment claiming "core effect coordination" and registering none;
- the Unifier.

One caveat, and it cuts against the headline: that list is selection-biased upward. These are the claims that looked suspicious enough to check. Nobody checked the boring names that turned out to be accurate, so the failure rate in this list is not the failure rate in the codebase.

Which is a limitation of the sample, not a defence of the names.

6.8 Where That Leaves Us

- Insight extends along a third axis again — data sources, through YAML connector manifests wrapping a third-party extraction runtime.
- Bronze, Silver, Gold is a real Pipes-and-Filters instance, with a security-motivated invariant rather than a hygiene rule.
- The one documented exception, `class_ai_org_usage`, was engineered to preserve the invariant rather than to escape it.
- `@cpt-principle` markers link SQL back to named principles, and the principles are not written down anywhere.
- The Unifier is a well-specified design with no implementation, and the first verification pass believed it.

Chapter 7 goes to the sharpest engineering in the platform — authorization and identity — and then zooms out to the ten other platforms in this category, and to what four chapters of reading somebody else's architecture actually taught us.

Exercises

1. **Find the invariant and its enforcement.** Locate `insight/inbox/architecture/-CONNECTORS_ARCHITECTURE.md` and read the Bronze-is-never-queried-at-Gold-level section in full. Then go looking for what enforces it: is there a test, a lint, a dbt selector, a CI job, or nothing but the sentence? Report what you find. If the answer is nothing, design the check — name the artifact it would inspect, the condition it would assert, and where in the build it would run.

2. **Trace one row from source to metric.** Pick one Gold model, for example `collab_metric_observations`. Walk backwards: which Silver `class_*` tables does it read, which Bronze tables promoted into those, and which connector stamped those Bronze rows. Then walk forward to its `*_metric_evidence` counterpart and say precisely what an auditor could reconstruct from it. Note every point in the chain where `workspace_id` appears.

3. **Apply the ten-minute test.** Take any architecture document from a project you have access to — your own, your employer's, or another open-source project — and run the four-step checklist from this chapter against one mechanism it describes: read the status line, grep for the name, grep for the promised filenames, look for a test. Report the verdict and, more importantly, which of the four steps was decisive.

4. **Write a minimal promotion model.** Using the Slack promotion model in this chapter as the pattern, write a dbt model that promotes two Bronze tables from a source of your choosing. It must carry an `@cpt-principle` marker above its `config` call, call a shared promotion macro rather than inlining SQL, state in its header comment whether the macro is idempotent, and return a value. Then explain in two sentences why the final `SELECT` exists.

5. **Score the marker convention.** Pick one of the five `@cpt-principle` identifiers used in Insight's SQL models. Find every model that claims it. Then write the definition that identifier ought to have — one paragraph, inferred from the models alone — and state whether writing it down would have made the codebase better or merely added something else to keep current. Argue for one answer.

Chapter 7

Security and the Bigger Picture

Three chapters on how Constructor Fabric is built. This one does two things.

First, authorization and identity, which is where the sharpest engineering in the platform lives. It is also the part of the platform where the design philosophy that has been showing up in fragments throughout Part II appears in its clearest form, five separate times in one subsystem.

Then a step back: how this platform compares to the ten others in the same category, and what four chapters of reading somebody else's architecture actually taught us.

7.1 The Problem, Stated Properly

Gears is a modular platform for building *multi-tenant vendor platforms*. That phrase is worth unpacking, because the whole design follows from it.

Each vendor building on Gears brings their own identity provider, their own authorization model, their own tenant service, and their own policy manager. Gears has to integrate with all of that without assuming a policy model. Not role-based. Not attribute-based. Not relationship-based.

So the design cannot contain an opinion about how policies are written. Every tempting shortcut — "we will just have roles," "we will just have a permissions table" — is closed off by the requirement before the design starts.

7.1.1 Three requirements from the decision record

Performance at scale Bulk updates, cross-tenant queries, hierarchical access, all without a bottleneck.

Simplicity for gear developers The record states that "authorization enforcement must be hard to get wrong" — ideally just "ask the decision point, apply the response." And it notes the operational reason this matters: "even shared libraries are costly to update across deployments."

Seamless vendor integration No resource sync, no policy format requirements. Vendors do not replicate millions of resources into their authorization service.

Source: `gears-rust/docs/arch/authorization/ADR/0001-pdp-pep-authorization-model.md`

The second requirement is the one that shapes the code. "Hard to get wrong" is not a wish; the following sections are five mechanisms that implement it. And the parenthetical about shared libraries is the reason it has to be hard to get wrong *structurally* rather than by a helper function — if the fix for a class of mistake is "upgrade the library," the mistake stays in production for a year.

7.1.2 The four roles

The record adopts NIST SP 800-162's decomposition, and it names all four roles:

PDP Policy decision point. Evaluates policy, returns a decision.

PEP Policy enforcement point. Enforces the decision at the resource.

PAP Policy administration point. Where policies are authored.

PIP Policy information point. Supplies extra attributes for the decision.

Adopting an external standard here is the right call for a boring reason: everybody who has to integrate already knows these four letters. A bespoke vocabulary would have to be taught to every vendor.

And the assignment of roles is concrete:

PEP Domain gears, with the toolkit providing shared enforcement infrastructure.

PDP The *vendor's* authorization service, reached through the AuthZ Resolver gear.

PAP Entirely vendor-controlled. The record is blunt about this: Gears never sees or stores policies.

PIP The Tenant Resolver and Resource Group Resolver gears.

All four have concrete realisations. This is not a diagram with two boxes implemented and two aspirational — which, after Chapter 6, is a thing you should check for rather than assume.

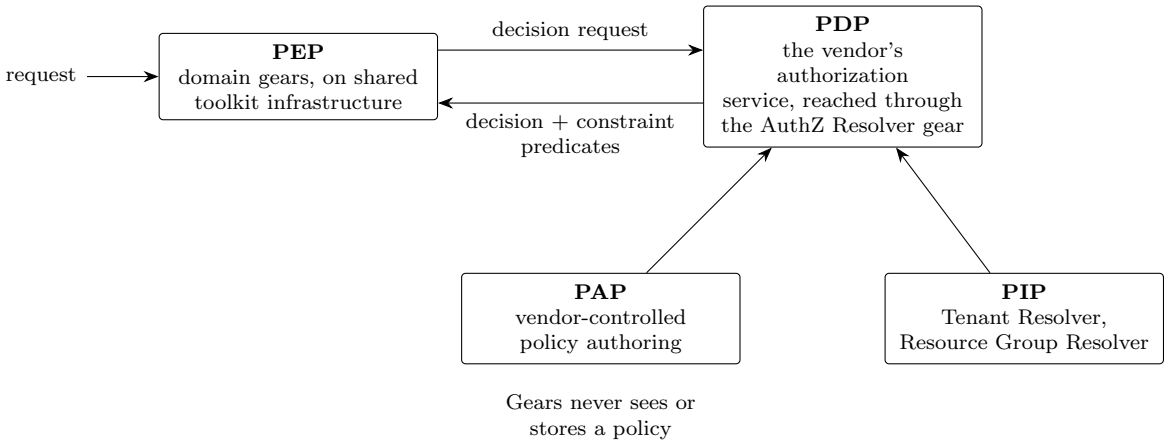


Figure 7.1: The four NIST roles, and who supplies each one.

7.2 The Problem a Yes/No Answer Cannot Solve

Here is the actual design problem, and it is a good one.

A user asks for page three of a list of resources. Classic authorization says: for each candidate row, ask the decision point, keep it if allowed.

That is one network round trip *per row*.

It also breaks pagination outright. You cannot ask the database for twenty rows if you only find out afterwards that eleven of them are forbidden. Now you have nine rows on a page that should have twenty, and no way to compute what "page four" means.

Point-in-time boolean authorization does not compose with queries. That sentence is the whole problem.

7.2.1 Five options, one chosen

The record lists what was considered:

A Gear-level authorization, where the enforcement point is also the decision point.

B Google Zanzibar / relationship-based access control.

C OpenID AuthZEN 1.0, as-is.

D Open Policy Agent partial evaluation.

E AuthZEN 1.0 plus constraint extensions.

E was chosen.

Note that C — the unmodified standard — was on the list and lost. That is worth pausing on, because "use the standard as written" is usually the safe answer and the record had to argue against it. Which means the argument is recorded, and you can check it.

7.2.2 The idea: return a predicate, not a boolean

The authorization layer returns **SQL-friendly constraint predicates**.

So one decision-point call authorises an entire query, instead of every row the query might return. The enforcement point compiles those filters directly to SQL, with no intermediate translation layer. Constraints carry typed filters with known operators: `eq`, `in`, `in_closure`.

The stated design driver: "O(1) authorization overhead per query, not O(N) per resource."

The mental shift is from post-fetch filtering to query-time enforcement. You do not fetch rows and then discard the forbidden ones. You ask the database a question that cannot return them. Pagination works again, because the count the database reports is already the authorised count.

7.2.3 The consequences, as written

Positive, in the record's own words: standards-based, single endpoint for all operations, vendor-neutral, SQL-first, fail-closed.

Negative, also in the record's own words:

- "non-standard extension requires documentation and tooling";
- "PDP complexity: must understand both AuthZEN and constraint extensions";
- "may not be compatible with off-the-shelf AuthZEN PDPs".

And a named risk: AuthZEN may add constraint-like features later, which is either an alignment opportunity or a divergence problem depending on whether the shapes match.

A decision record that lists what its choice costs is worth ten that list what it gains. The third negative is the expensive one — it means a vendor cannot point a stock AuthZEN implementation at this and be done — and stating it up front is the difference between a known limitation and a support ticket.

7.2.4 Failing closed, eleven ways

The design document enumerates what the enforcement point does when something is off. All of it denies:

- decision false or missing → deny all;
- constraints required but absent → deny all;
- decision point unreachable, network failure, timeout → deny all;
- unknown predicate type → treat that constraint as false;
- empty `predicates: []` → malformed → deny all;
- empty `constraints: []` → deny all, since an OR over an empty set is false.

There are eleven of these rules in total. Every branch lands on deny.

Enumerating them is the real work. Any single one of these is obvious in isolation; what is hard is being sure you have covered every way the input can be wrong, and a numbered list is how you find out you have not.

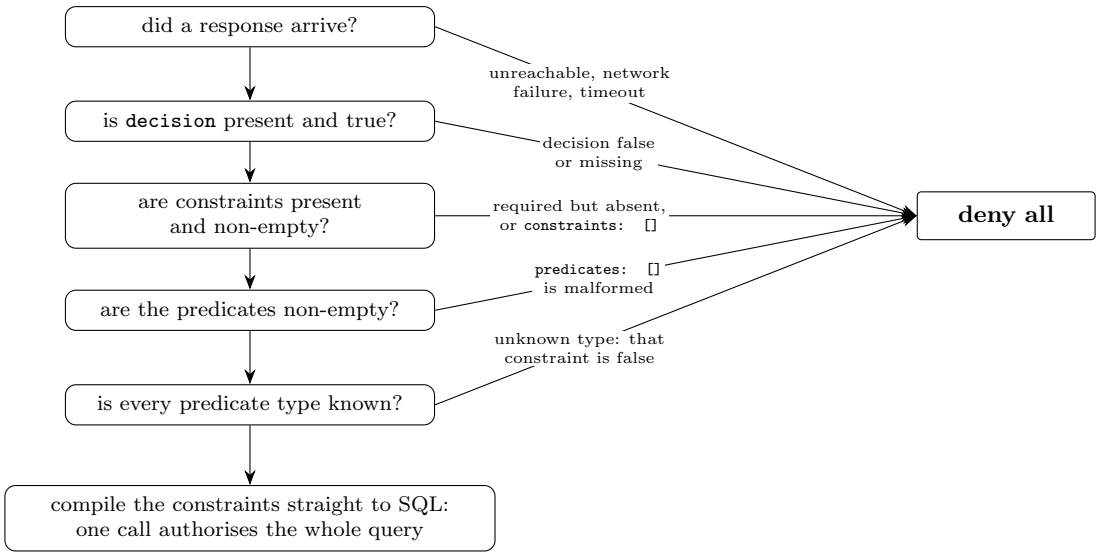


Figure 7.2: Every way out except one ends in a denial.

7.2.5 One of those rules is worth a closer look

An `In` predicate arriving with an empty value list is treated as false.

Two reasons are given, and they are different kinds of reason:

semantic An empty set means "match nothing," which is a deny.

mechanical Passing it to the ORM would generate `WHERE col IN ()`, a SQL error on some engines.

And it is labelled a decision-point contract violation, which the enforcement point must log as an error before failing closed.

Deny, log, and name whose fault it is. Silently denying would be safe and undebuggable — the vendor's PDP would be emitting malformed responses, users would be seeing empty lists, and nothing anywhere would say why.

7.3 Tenant Scoping

The constraints come back and have to become part of a real SQL statement.

A compile-time-checked derive macro maps a type's fields to the authorization-relevant properties it carries. So the mapping from "property name in a policy" to "column in a table" is checked by the compiler. Rename the field, and the mapping fails to build rather than silently authorising against a column that no longer exists.

Then there is the interesting part: making it impossible to skip.

7.3.1 Bypassing the scope is a build failure

```
# Disallow direct execution methods that bypass secure wrappers
disallowed-methods = [
    { path = "sea_orm::Select::all",
      reason = "Use SecureSelect::all() after .secure().scope_with()" },
    { path = "sea_orm::Select::one",
      reason = "Use SecureSelect::one() after .secure().scope_with()" },
    { path = "sea_orm::UpdateMany::exec",
      reason = "Use SecureUpdateMany::exec() after .secure().scope_with()" },
    { path = "sea_orm::DeleteMany::exec",
      reason = "Use SecureDeleteMany::exec() after .secure().scope_with()" },
]
```

Source: `gears-rust/clippy.toml`

Read that configuration again, because it is more aggressive than it first looks.

These are the ORM's own ordinary query-execution methods. `Select::all()` is how you run a select in SeaORM. Calling it directly is banned, project-wide, by a lint.

Every entry carries a `reason` that tells you what to do instead. So a developer who reaches for the unscoped method does not get a mysterious refusal; they get a compiler message naming the scoped replacement and the two calls that have to precede it.

The fastest path is the safe path, because the unsafe path does not build. That is the same move as the domain-model lint in Chapter 5: convention becomes compile failure.

7.3.2 The empty scope

What happens if a request arrives with no tenant identifiers and no resource identifiers?

An empty access scope compiles down to a query predicate that is **unsatisfiable by construction**. Zero rows. Not because a check ran and failed, but because the SQL cannot match anything.

Compare that with the usual arrangement, where an empty filter list means "no filtering," which means everything. Same code path, opposite default, and the difference between the two is a data breach.

This is the single cheapest security decision in the platform. It costs nothing at run time. It is a choice about what the identity element of your filter composition is, made once, at the point where somebody could equally have written `if filters.is_empty() { return query }`.

7.3.3 Hierarchies without recursion

Tenants form a tree, and a common query is "every resource visible from this tenant or any of its descendants."

The naive answer is a recursive walk in application code, which is N queries and gets slower as the customer grows.

Gears maintains a materialised **closure table** over the tenant hierarchy. So an `in_tenant_subtree` predicate compiles to one efficient SQL subquery. The same trick exists for resource groups, with `in_group_subtree`.

Denormalisation, chosen on purpose, for a query shape that would otherwise be pathological. It has a maintenance cost — the closure table has to be updated when the hierarchy changes — and that cost is paid on writes to a structure that changes rarely, in exchange for reads that happen constantly.

7.3.4 The sentence worth framing

> "A tenant-scope bypass that ships is rarely one someone declined to test; it is usually one that no existing control was obliged to catch, so no reviewer had grounds to block it."

That is from the platform's own internal documentation.

It is an argument for structural, compiler-enforced controls over review discipline, and the argument is subtler than "reviewers miss things." It says the reviewer did not miss anything. There was nothing to point at. The change violated no stated rule, tripped no check, and looked like every other query in the file. Blocking it would have required the reviewer to invent the rule on the spot and defend it.

That generalises far past authorization. It is the same failure mode as Chapter 4's four layers: a rule nobody wrote down is a rule nobody can enforce, and a reviewer with no artifact to cite loses the argument.

7.4 Identity Across Boundaries

Chapter 5 showed gears resolving each other by name across process boundaries. That story has a security half we skipped.

How does a request's identity survive a hop from one process to another?

The tempting answers are a shared session token, or a fast-path trust relationship between processes — process B trusts what process A tells it, because they are both ours. Neither is what happens here.

7.4.1 Re-validate at every hop

Identity federates via a JSON Web Token, independently re-validated at **every** hop, against one identity-provider trust root.

And there is an explicit design statement that the platform's own inter-process peer authentication *never* substitutes for this per-hop tenant-plane validation.

Two separate planes, and keeping them separate is the point:

- Is this peer who it says it is? That is peer authentication.
- Is this *user* allowed to do this? That is tenant-plane validation.

Collapsing those two is one of the classic ways multi-tenant systems leak. Process A is authenticated, so B accepts A's claim about which tenant the request belongs to, and now a bug in A is a cross-tenant read in B.

7.4.2 A vendor-neutral identity type

One internal type is the only identity shape any Gears component may depend on.

Authentication-provider-specific claims are translated away at an identity-broker boundary, before they can reach application code. So no gear ever learns which identity provider is in use.

Same move as the directory client and the gateway provider in Chapter 5: one type, several realisations, chosen at the edge. By now this should read as the house style rather than as a clever trick.

7.4.3 The token is not in the type

The bearer token is explicitly excluded from that identity type's own serialisation.

Work through the consequence. A receiving process cannot reconstruct identity by trusting deserialised state from a peer, because the material it would need to shortcut is not in the payload. It has to re-validate independently. Not because a rule says so — because there is nothing else it can do.

The design makes the wrong thing impossible instead of forbidden.

That is the same idea as the empty access scope, and the same idea as the disallowed ORM methods. Three unrelated mechanisms, one philosophy.

7.4.4 Cryptography that fails closed

A parallel compliance posture: FIPS-140-3 cryptography that fails closed, at build time or at process bootstrap, on every supported operating system. If the validated primitives are not available, the process does not start.

And a disclosure worth noticing. The platform's own documentation states that it holds **no** cryptographic-module validation of its own. It depends entirely on already-validated operating-system and library primitives.

Vendors routinely blur this, and the blurring is easy: "FIPS-validated cryptography" is true of a system that calls a validated library, and it reads as a claim about the system. Saying "we inherit our validation" out loud is unusual, and it is the same instinct as Insight's honesty about inheriting extraction from Airbyte in Chapter 6.

7.4.5 The security section, in one idea

Prefer a structural guarantee that fails loudly and early over a policy that depends on every future engineer remembering to follow it.

Realised five separate ways in this one subsystem:

1. a lint that bans the unsafe query methods;
2. a derive macro that checks the property mapping at compile time;
3. an empty scope that compiles to an unsatisfiable predicate;
4. eleven deny-by-default branches in the enforcement point;
5. an identity type that omits the token so a shortcut cannot be taken.

None of these is clever. All of them are cheap. That is the point — this is not a list of sophisticated defences, it is a list of decisions somebody made in the ten minutes when the alternative was equally easy.

7.5 Ten Other Platforms

Constructor Fabric is one platform in a category that barely existed two years ago: agentic software-lifecycle orchestration — systems that coordinate LLM agents across planning, implementation and review. Not "assist a developer with one task inside an editor," which is a different and older product category.

So: is anything in the last three chapters actually distinctive? Ten comparators, four axes, and one important caveat about evidence.

7.5.1 How the comparators were chosen

Two parallel channels, cross-checked against each other rather than either trusted alone:

- independent web research against each candidate's own documentation, repository and licence file;
- the organisation's own first-party competitive analysis, naming fourteen candidates.

Two names were **excluded entirely**. One resolved to materially different products under the same name in the two channels. One returned nothing verifiable under any search variant.

Verify independently, or do not cite by name. A comparative table's authority comes from its exclusions as much as its rows.

7.5.2 One rebrand, mid-study

Windsurf and Devin are treated as *one* entry, not two data points.

Windsurf was acquired by Cognition, Devin's developer, and rebranded "Devin Desktop" in June 2026. That happened *during* the research window for this comparison.

Which is itself the finding. A six-month-old comparative table in this category risks naming companies that no longer exist under those names. If you ever rely on a table like this for a real decision, re-verify the rows that matter.

7.5.3 The four axes

Derived independently of any vendor's own competitive framing:

Control-flow model What actually drives task execution: a compiled specification, natural language read by a model, or ordinary code with an LLM assisting.

Extensibility What a team would use to customise behaviour.

Federation Explicit support for work across multiple repositories, or one project at a time.

Openness Is the source available for inspection at all.

The fourth axis decides whether the other three can be verified or only characterised, which is why it is on the list at all.

7.5.4 The comparison

For reference, Constructor Fabric on the same four axes:

Control flow Prompt-interpreted, via PDSL (Chapter 4).

Extensibility Plugin- and capability-composed (Chapters 4 and 5).

Table 7.1: Ten comparators across the four axes.

Platform	Control flow	Extensibility	Federation	Open?
Kiro	Structured spec (requirements, design, tasks)	Steering files, hooks, MCP	Multi-root workspace, no deep cross-repo	Closed
Planu	Structured spec, approval-gated	MCP tool suite	Not documented	Closed
cc-sdd	Structured spec, TDD loop	Portable skills (8 hosts)	Single-repo by design	Open (MIT)
Claude Code	Natural-language agent loop	MCP, skills, sub-agents, hooks	Single-repo by default	Closed
Cursor	Natural-language agent loop	Rules, skills, MCP	Multi-root workspace	Closed
GitHub Copilot	Natural-language agent loop	MCP, custom agent files	Single-repo scoped token	Closed
Devin	Natural-language, editable plan	MCP, playbooks, custom API	Multi-repo workspace	Closed
Factory.ai	Hybrid: spec-driven (Missions) or natural language	Custom droids, MCP registry	Not documented	Closed
OpenHands	Natural-language event loop	Skills (MCP-integrated)	Single-repo, explicitly	Open (MIT)
Augment Cosmos	Hybrid: natural-language advisor, YAML config	Custom experts, MCP	Explicit, core capability	Closed

Federation Federation-native, spanning an organisation-level federation of independently versioned repositories.

Openness Apache License 2.0.

The federation row is the one nobody else really matches. Look back at Table 7.1: "single-repo by default," "single-repo by design," "single-repo, explicitly," "not documented." The two entries that do claim multi-repository support claim it at the level of an editor workspace or a hosted workspace, not at the level of resolving artifact references across forty independently versioned repositories.

That is not a marketing point. It is why `cfs workspace-add` exists in Chapter 1 and why the manifest walk-up of Chapter 4 exists at all — the platform was built by people whose own codebase has this shape.

7.5.5 The evidence caveat, in bold

Of the eleven platforms including the subject, exactly **two** others are open source: cc-sdd and OpenHands, both MIT.

Constructor Fabric itself is Apache-2.0 and was inspected at source level — which is what the last three chapters were.

Every other characterisation rests on public documentation, blog posts and vendor material. For eight of the ten rows in Table 7.1, this is a study of public self-presentation, not of architecture.

Read those rows with correspondingly less confidence. And notice how easy it would have been to print the table without saying so. After Chapter 6, you should have a specific reason to distrust a table built from vendor documentation: the Unifier was in a vendor document too, and it was written by people with source access.

7.5.6 The split running through the category

Read the control-flow column and a real philosophical divide appears.

Structured, spec-driven, approval-gated: Kiro, Planu, cc-sdd. The commitment is that a human-legible artifact exists as a checkpoint *before* changes reach the codebase. The cost is more upfront authoring effort per task.

Autonomous, natural-language-driven: Claude Code, Cursor, Devin, OpenHands. The commitment is minimal upfront specification and maximal agent autonomy. Review happens after changes exist, not before they are attempted.

Factory.ai and Augment Cosmos hedge, and label themselves hybrid.

If Part I felt like the first camp — and it did, since Chapter 3 is nine artifacts before a line of code — that is because the SDLC kit is squarely a spec-driven workflow. But the platform underneath it is not.

7.5.7 Which side is Constructor Fabric on?

Neither, and not by accident.

PDSL is unambiguously in the natural-language camp. No formal grammar gates what an agent does before it acts, as Chapter 4 established at some length.

Studio's kit and manifest layer is thoroughly structured, versioned, and reviewed before any agent touches it.

So the *governance* layer is spec-driven and the *execution* layer is not. We noticed this split at the start of Chapter 4 without knowing it was a category-level position.

7.5.8 Why straddling might be right

There is a real tension underneath the split, and neither pure answer scales:

- Full upfront specification does not scale to open-ended orchestration work. You cannot write a specification for "figure out why this pipeline is slow."
- Fully unconstrained autonomy does not scale to forty repositories with production consequences. Somebody has to be able to say what an agent is allowed to do, and have that be true.

Splitting between a structured governance layer and an unstructured execution layer is one coherent way to have both. Governance answers "what capabilities exist and who approved them," and it can be reviewed like any other code. Execution answers "what to do next," and it stays flexible.

That is not finished work. It is a reasonable response to a problem nobody has solved, and you will be making this exact trade-off in your own systems, repeatedly.

7.6 Recurring Motifs

7.6.1 Compile once, adapt per host: the third sighting

Chapter 4 found it twice in Studio: subagent generation, with three named degradation strategies, and plan decomposition exported to `ralphex`, an unrelated external CLI.

Now, in platforms with no relationship to Constructor Fabric at all:

- Factory.ai's droid configuration: authored once per project, adapted per invoking context.
- cc-sdd's entire value proposition: one skill set, portable across **eight** coding-agent hosts.

Three organisations. No shared code, no shared style guide.

Why did the convergence happen? Not coincidence, and not three teams copying each other. The tooling landscape is fragmented across enough incompatible host formats — Claude Code,

Cursor, Copilot, Codex, and more, each with its own configuration schema — that any platform wanting to reach more than one of them faces the identical choice.

And the identical answer keeps winning: author the canonical behaviour once, in whichever host format is richest; generate the rest as *disclosed, degraded* adaptations; rather than maintaining N copies by hand.

Convergent reinvention is evidence of a real problem.

7.6.2 A second convergence

Two unrelated teams independently built a real many-to-many publish-subscribe mechanism: `gears-frontx`'s `EventBus` from Chapter 5, and OpenHands' `PubSub`.

Both satisfy the criteria without qualification. Both are correctly named.

Why publish-subscribe specifically, in this category? Because any system driven by an external, only partly predictable process needs to decouple state changes from the many things that react to them. When you cannot know in advance what a turn will produce, you cannot wire the producers to the consumers by hand.

It is also the one place in the whole corpus where convergent reinvention and accurate naming coincide.

7.7 What We Learned

7.7.1 The scoreboard

Across seventy-eight verified claims about design patterns and architectural styles:

- thirty-eight survived verification *exactly* as first named;
- eighteen more survived in part.

On the Constructor Fabric pattern side specifically, fifty-four percent needed correction from the name they were first proposed under.

Read that number carefully, because the obvious conclusion is the wrong one. Both patterns and styles are demonstrably present in this codebase, and they are load-bearing — the capability registry, the medallion pipeline, the layered SDK, the service locator, the publish-subscribe bus all do real work.

The gap is not between the names and the code. It is between the code and what its authors say about it.

7.7.2 Four subsystems, one design problem

Every subsystem needed the same thing: optional, independently addressable extension points.

Studio Kits, four extension levels, TOML manifests.

gears-rust Capability traits, queried by a registry, collected at link time.

gears-frontx A fluent plugin chain plus TypeScript declaration merging.

Insight Connector manifests in YAML, wrapping a third-party extraction runtime.

Four mechanisms with almost nothing structurally in common, solving one problem. One is markdown and TOML. One is a Rust trait system with linker tricks. One is a builder plus a compiler feature. One is YAML delegating to somebody else's runtime.

That recurrence is the strongest evidence that the problem is real. And each mechanism fits its material: you could not have built the Rust one in YAML, and the YAML one would have been absurd in Rust.

7.7.3 The habit that mattered most

Author-declaration-first, then check:

1. Start from whatever the engineers wrote down about a mechanism.
2. Treat that statement as *the claim to be tested*, not as the answer.
3. Look for independent corroboration in the implementation before reporting it.

Applied consistently, this found the layer that is not a layer, the interpreter that does not exist, the coordinator that coordinates nothing, and the Unifier.

It also *strengthened* the `ClientHub` claim, by finding corroboration nobody had cited — the independent architecture document calling it a typed service locator.

The method is not adversarial. It just does not stop early.

7.7.4 Five things to keep

1. **A name is a claim.** Read the definition, read the code, then decide.
2. **The difference between an architecture and a diagram is a CI job.**
3. **Design documents describe systems that were sometimes never built.** Check.
4. **Prefer a structural guarantee that fails early over a rule people must remember.**
5. **When unrelated teams reinvent the same shape, the problem is real.**

7.7.5 What is still open

Threads nobody has pulled yet, and any of them would make a thesis:

- What is the correctness criterion for a language whose only implementation is a model?
- Can a linter grow into a specification?
- How do you keep a comparative table current in a category that rebrands mid-study?
- Does the governance/execution split generalise, or is it one organisation's accident?

Part II ends here. You now know more about how one of these platforms is actually built than almost anyone writing about the category, which is a slightly alarming thing to be able to say.

Part III turns that around. Having read how the extension mechanisms work from the inside, you are going to use one: authoring a kit of your own, against the mechanism Chapter 4 described and the real kits that ship with the platform.

Exercises

1. **Find the clippy configuration and extend it.** Locate `gears-rust/clippy.toml`. For each of the four banned methods, find one real call site of the corresponding `Secure*` wrapper in the codebase and write down the full call chain it required. Then propose one additional entry for `disallowed-methods` that would close a bypass the current four do not cover, with a `reason` string in the same style. Justify why your entry is needed and what it would break.

2. **Trace an authorised query end to end.** Take a paginated list request against a tenant-scoped resource. Trace it through: the PEP's call to the AuthZ Resolver gear, the constraint predicates that come back, the derive macro's property-to-column mapping, and the final SQL. State at which step each of the following would be caught: a missing decision field, an unknown predicate type, an `In` predicate with an empty list, and a scope with no tenant identifiers at all.

3. **Write a minimal fail-closed rule set.** Pick a decision your own code makes on the basis of an external response — a feature flag, a quota check, a licence check, anything. Enumerate every way that response can be wrong: absent, malformed, empty, unreachable, timed out, of an unknown shape. For each, write the branch and the resulting behaviour, in the style of the eleven rules in this chapter. Then count how many of those branches your current code actually has.

4. **Re-verify two rows.** Take any two closed-source rows from Table 7.1 and re-verify them against the vendor's current public documentation. For each of the four axes, record what you can confirm, what you can only characterise from marketing copy, and what has changed since this table was compiled. Then write one paragraph on what your findings say about the shelf life of comparative tables in this category.

5. **Audit one claim in your own system.** Apply the author-declaration-first method to a mechanism in a codebase you own. Find what the documentation, doc comments, or README say it is. Write down the defining property of that claim. Then look for the code that must exist for the property to hold, and deliver one of three verdicts: confirmed, split, or rejected. If it is not confirmed, propose either a rename or the smallest check — lint, test, or CI rule — that would make the claim true.

Part III

Extending Constructor Fabric

Chapter 8

The Kit-Authoring Mechanism

Parts I and II gave you the two views that most books stop at. Part I used the system. Part II opened it up and checked whether the description matched the code.

Part III does the third thing, which is the one that decides whether a tool survives contact with a real organisation: it adds something. Not by patching the core, and not by writing a process document nobody reads, but through the mechanism the platform provides for exactly this purpose. The unit of extension is called a **kit**.

This chapter is the mechanism. Chapter 9 walks a real one, file by file. Chapter 10 puts you in the author's chair.

8.1 Why Anything Needs Extending

Every tool ships with somebody's opinion baked in. Constructor Fabric ships with the SDLC kit — PRD, ADR, DESIGN, DECOMPOSITION, FEATURE — and you spent all of Chapter 3 inside it. Those are good defaults. They are still defaults.

The specific ways they stop fitting are boringly predictable:

- your company numbers its architecture decisions differently;
- a regulator wants a section the template does not have;
- your reviewers reject documents for reasons nobody ever wrote down;
- your process has a stage — a security sign-off, a data-protection assessment, a customer-visible change notice — that the shipped kit has no concept of.

What happens next is the interesting part, and it happens the same way in most organisations. The team adopts the tool, the tool almost fits, and the team quietly stops using the half that does not. The unused half is replaced by knowledge in people's heads and by comments in pull requests. Which means the process is now undocumented, unenforced, and mortal: it leaves when those people leave.

A kit is where you put the process so that it stops living in heads.

8.1.1 Two bad options and one good one

There are three ways to make a tool do something it does not do.

Fork it and patch the core. This works on Monday. By Friday your fork is unmergeable, and from then on every upstream release is your problem rather than the maintainer's. Anyone who has maintained a patched fork of anything knows the shape of this: the patch is small, the rebase is not, and the cost arrives later than the decision.

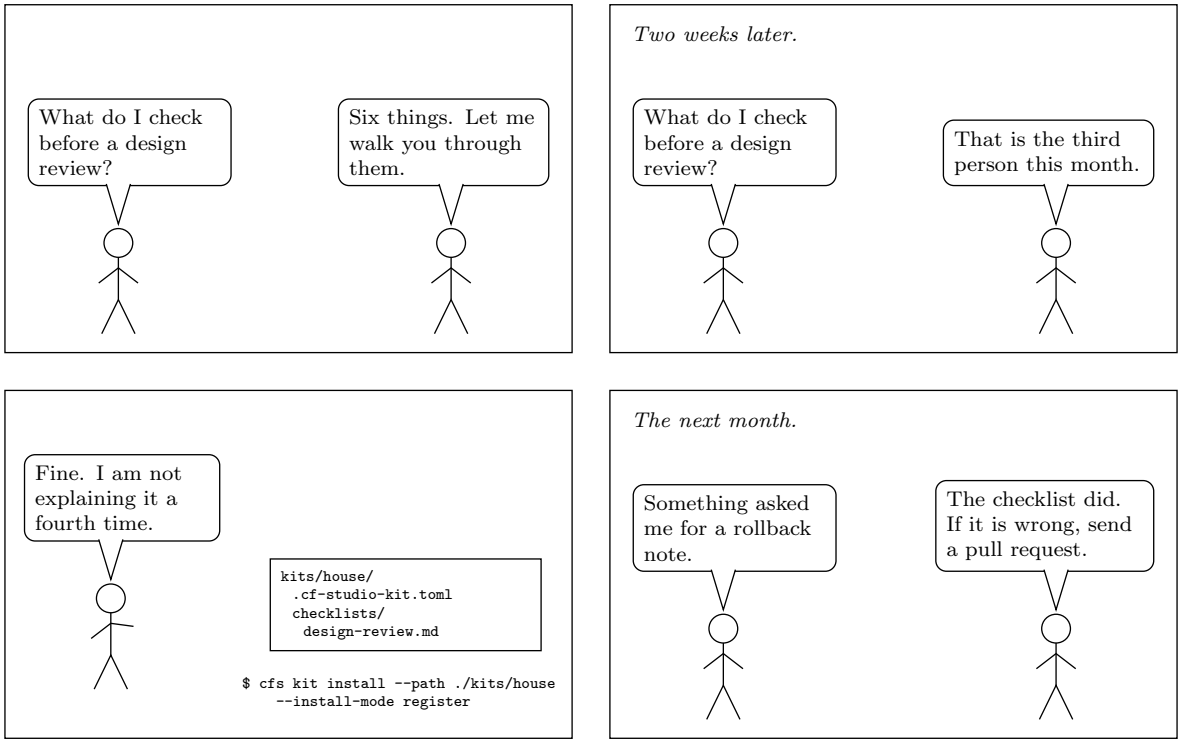


Figure 8.1: Why anybody writes a kit.

Write the rules on a wiki page and hope. Nobody reads it and nothing enforces it. This is the option most teams take by accident, because it requires no decision.

Package the rules as a kit and register it. The core stays untouched and upgradeable. Your rules become files in your repository, reviewed like code. A pull request can now change the company process, and someone has to approve it. The agent reads your templates and rules exactly the way it reads the built-in ones, because the mechanism does not distinguish between them.

That last property is worth pausing on. There is no second-class extension path here. The SDLC kit you have been using for seven chapters is not privileged core code with a plugin API bolted alongside it. It is a kit, declared the same way yours will be, resolved by the same loader. Chapter 4 quoted the design commitment behind this: core does not interpret kit-specific semantics, it only knows that a kit is registered and where its files live. The core cannot tell an ADR from a threat model from a research request. That semantic blindness is what makes your kit a peer rather than a guest.

8.2 A Kit, Concretely

A kit is a directory. It has one manifest file at its root, named `.cf-studio-kit.toml`, plus whatever files that manifest declares.

That is the entire definition. The recommended layout for a small project kit looks like this:

```
kits/company-sdlc/
|-- .cf-studio-kit.toml
|-- skills/
| `-- company-review/SKILL.md
|-- rules/
| |-- architecture.md
| |-- security.md
| `-- testing.md
|-- templates/
| |-- PRD.md
```

```
| |-- ADR.md
| |-- DESIGN.md
| `-- FEATURE.md
|-- checklists/
| |-- design-review.md
| |-- code-review.md
| `-- release-readiness.md
`-- constraints/
    `-- FEATURE/constraints.toml
```

Markdown and TOML. That is the whole technology stack.

No compilation step. No plugin API. No version-matched binary. No generated bindings, no interface to implement, no lifecycle hook to register. Which is the point rather than a limitation: a product manager can edit a checklist, and a compliance officer can review a rules file, without either of them being able to build the project.

The manifest is the only file in a kit with a schema you must respect. Everything else is prose that a model will read at the moment it needs it. If you have read Chapter 4, you already know the consequence of that: the execution half of this system is text, and its correctness is a matter of how carefully the text was written, not of whether it compiles.

8.2.1 Directory names are conventions, not law

The layout above is a *recommendation*. Nothing in the loader requires a directory called `templates/`. What the loader requires is that the manifest declare where things are. The shipped SDLC kit uses a different shape entirely — one folder per document kind under `artifacts/`, each containing four files — and it is just as legal.

Use names that match your organisation's vocabulary. A security-heavy company will naturally end up with `threat-model.md`, `data-handling.md`, and `security-review.md`. A platform team ends up with service templates and deployment checklists. The mechanism does not care, and that is deliberate: the vocabulary is the part your organisation actually owns.

8.2.2 Where kits come from

Three sources, in increasing order of ceremony:

Local A folder inside your project, conventionally `kits/<name>/`.

GitHub `cfs kit install <owner/repo[@ref]>`.

Generic git `cfs kit install git/<url>[//<subdir>][@<kit>] --version <ref>`.

For a company process, start local. The file lives next to the code it governs, the review happens in the same pull request as the code change that motivated it, and there is no publishing step to get wrong. Move the kit to a git source the moment a second repository needs it, and not before.

8.2.3 What belongs in a kit

The guide is specific about this, and the list is worth reading as a prompt rather than a rule:

- document templates for requirements, decisions, design, features, and releases;
- rules for architecture, security, testing, performance, observability, operations, and code style;
- review checklists per discipline;
- constraints for required headings, identifiers, cross-references, and CPT ID structure;

- skills for project-specific workflows or domain knowledge;
- workflow resources that change how a skill gathers context, asks questions, writes files, or validates output;
- scripts for deterministic checks and repeatable tasks.

And one prohibition: *do not put generated runtime files in a kit.* Generated Studio files and generated AI-tool integration files must stay repairable outputs. The moment a generated file becomes kit content, you have two sources of truth for the same thing and no way to tell which one is stale.

There is a second, subtler prohibition about workflow resources specifically. When a kit provides workflows, the shared Studio gates are part of the contract. Context discovery flows through the workflow-prep gate. Brainstorming stays an explicit choice. Plan-first runs before substantive multi-step work. Review fixes go through the findings browser and fix-scope approval. A project kit may add domain-specific prompts and checks *around* those gates. It must not silently bypass them. A kit that quietly removes a gate is not an extension; it is a downgrade wearing an extension's clothes.

8.3 Register Mode Versus Copy Mode

Installing a local kit means choosing between two modes, and the choice has real consequences.

Register mode leaves the files where they are. Studio records the registered kit root in `core.toml` and re-derives effective resource paths from the manifest at run time. Per-resource bindings are *not* persisted. So `cfs generate-agents`, `cfs validate`, `cfs info`, and skill routing can all use the resources without anything having been copied anywhere.

```
$ cfs kit install --path ./kits/company-sdlc --install-mode register
```

Copy mode copies the kit resources into the Constructor Studio setup directory, and the effective bindings are persisted in `core.toml`.

```
$ cfs kit install --path ./kits/company-sdlc --install-mode copy
```

For project-level extensibility, prefer register mode unless you have a clear reason to copy. The reasons to prefer it stack up quickly:

- you edit the kit in place and the change is live, with no reinstall step between writing a rule and testing it;
- the kit source stays visible in git, so reviewers inspect kit changes like any other project change;
- there is exactly one copy of every file, so there is nothing to drift.

Register mode also enforces a boundary that is easy to miss and important. The local kit path must stay inside the project root *after symlink resolution.* A project cannot silently come to depend on arbitrary files elsewhere on the machine. Think about what the alternative would allow: a repository that validates on your laptop because of a symlink into your home directory, and fails in continuous integration for reasons nobody can reproduce.

Two syntaxes exist for the same thing. The `--path` flag is supported for compatibility; the positional `path/...` form mirrors the generic `git/...` source style used by the install and update commands:

```
$ cfs kit install path/./kits/company-sdlc --install-mode register
```

8.3.1 Who owns what

One division of labour, stated once, so that the rest of the chapter makes sense:

The core owns the engine. Routing, gates, the authoring loop, validation.

Your kit owns the content. What a document looks like, what a reviewer checks, what the author must not do.

Your kit does not reimplement the engine. A kit workflow is usually a thin preset that binds a document kind and delegates to the shared machinery. The shipped SDLC workflows do exactly that, which is why `doc-prd.md` is a short file rather than a long one, and why its description says "thin preset" out loud. Chapter 9 will show a kit whose workflows are considerably thicker, and it will be worth asking, when you get there, whether they earn it.

8.4 `.cf-studio-kit.toml` Is the Whole Contract

There is one canonical authoring format for new kits, and there is no second one. Chapter 4 explained why the loader nevertheless accepts four input shapes — the canonical format, two superseded legacy formats, and an implicit directory convention with no manifest at all — and why all four collapse into one immutable output type before anything downstream sees them. That is a migration path. It is not a menu. When you author a new kit, you author `.cf-studio-kit.toml`.

Three rules govern the top level:

- the file must start with `manifest_version = "1.0"`;
- it must use `[[kits]]` with nested `[[kits.resources]]`;
- kit slugs must be unique within the file.

The required top-level shape:

```
manifest_version = "1.0"

[[kits]]
slug = "..."
name = "..."
version = "..."
```

Notice that `[[kits]]` is an array of tables. One file may declare more than one kit, with repeated `[[kits]]` blocks. That is not a curiosity; Chapter 9 walks a repository built on exactly that property.

8.4.1 The smallest legal kit

Four lines of header and one resource. That is a working kit:

```
manifest_version = "1.0"

[[kits]]
slug = "company-sdlc"
name = "Company SDLC"
version = "0.1"

[[kits.resources]]
id = "prd_template"
kind = "template"
source = "templates/PRD.md"
```

Everything else in this chapter is refinement of those two blocks. If you remember nothing else about the manifest format, remember that a kit is a header and a list of resources, and that a resource is a name, a kind, and a path.

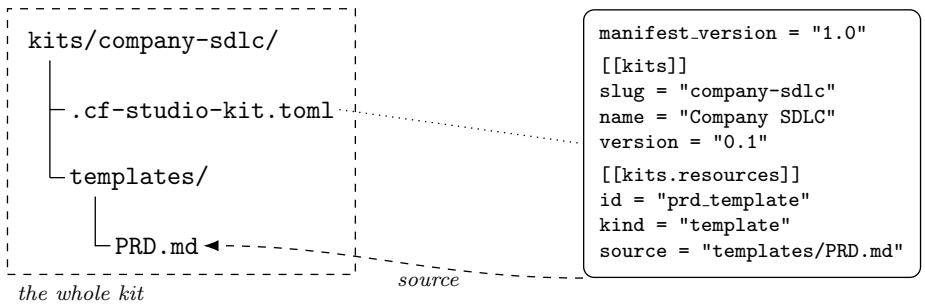


Figure 8.2: The smallest legal kit: a manifest, and the one file it declares.

8.4.2 The kit header, field by field

`manifest_version` The schema version, currently `"1.0"`, and it goes first.

`slug` The short machine identifier. It also feeds the public name prefix, so choose it as though you will be typing it for years, because you will.

`name` The human-readable kit name.

`version` Your kit's own version, which you bump when you change the kit.

8.4.3 A resource entry, in full

Here is a resource entry with every field you are likely to write, taken from the shipped SDLC kit with the identifier shortened:

```
[[kits.resources]]
id = "prd_template"
kind = "template"
source = "artifacts/PRD/template.md"
install_path = "artifacts/PRD/template.md"
type = "file"
user_modifiable = true
description = "PRD artifact template"
```

Three of those fields are required. The rest earn their place.

Required: `id`, `kind`, `source`.

`id` Unique within the kit. This is what other files refer to, and it is the source of template variables such as {`prd_template`}.

`kind` What sort of thing this is. Nine values, listed in the next section.

`source` A path relative to the kit root. Always author-facing.

Optional, and worth using: `install_path`, `type`, `public`, `description`, `user_modifiable`. Plus `aliases`, `generated_targets`, and `origin`, which matter for published and migrated kits. Plus three nested config blocks: `[kits.resources.agent]`, `[kits.resources.permissions]`, and `[kits.resources.targets.<target>]`.

Two of the optional fields deserve more than a line.

`type` takes `file` or `directory`, and nothing else. A validator enforces that.

`user_modifiable` is the boolean that makes safe local customisation possible. Chapter 4 described resource-level overrides that survive kit updates; `user_modifiable` is the kit author's declaration, per resource, of whether editing the installed copy is a supported thing to do or a mistake waiting to be overwritten. Templates and checklists are usually `true`. Generated entry

points and the constraints contract are usually `false`. Setting it thoughtlessly is how you end up either blocking legitimate customisation or promising stability you do not intend to honour.

And write the `description`. Every resource. It costs one line and it is what makes a sixty-resource manifest readable by somebody who did not write it, including you in a year.

8.4.4 `source` against `install_path`

This pair confuses people, so state it precisely.

`source` is where the file lives in your kit. `install_path` is the default effective destination for a *copy-mode* install. In register mode, the runtime derives effective resource paths from manifest `source` plus the registered manifest root — so in register mode `install_path` is inert.

In the shipped SDLC kit the two are usually identical, which makes both install modes behave the same way and hides the distinction. They differ when a file is renamed on install. The real case, and it appears in more than one kit, is a file authored as `agents-res.md` and installed as `AGENTS.md`:

```
[[kits.resources]]
id = "agents"
kind = "rule"
source = "agents-res.md"
install_path = "AGENTS.md"
type = "file"
user_modifiable = false
public = true
description = "Kit AGENTS.md for agent integration entry points"
generated_targets = ["installed"]
```

The rename exists because `AGENTS.md` at a kit root would be picked up as an agent entry point in the kit's own repository, which is not what the kit author wants while developing it. Small problem, small solution, and it explains why the two fields exist as separate concepts rather than one.

8.4.5 What the manifest must not contain

Some fields are machine-owned. Writing them by hand puts your kit into a state the tooling did not produce, which is the worst kind of state to debug because nothing in the system expects it.

The manifest must not carry:

- an author-facing `binding_path`;
- `[kits.<slug>.content_identity]`;
- `[kits.<slug>.resource_hashes]`;
- an author-facing `generated_name`.

And one rule that generalises the four: `core.toml` is installed state only. It is never source truth for authored manifest content. If you find yourself editing `core.toml` to make a kit work, stop — you are editing the cache.

8.4.6 Validation rules worth memorising

Table 8.1 lists the constraints that `cfs validate-kits` enforces. Break one and it names the offending line, which is the best thing a validator can do for you.

The last row is the one that catches people, and it is the row that carries the most design intent. Artifact bindings — the mapping that tells a document kind which template and which example belong to it — are allowed only on a constraints resource. There is therefore exactly one place in a kit where artifact wiring lives, and a reader looking for it does not have to search.

Rule	Applies to
`type` must be `file` or `directory`	every resource
`public = true`	only `skill`, `agent`, `rule`
`prefix_generated_name = false`	only public resources or nested subagents
`generated_targets`	only public resources or nested subagents
`artifacts.<KIND>` bindings	only `kind = "constraints"`

Table 8.1: Manifest validation rules enforced by `cfs validate-kits`.

Kind	Class	What it is
`skill`	public	a workflow the user can invoke
`agent`	public	a configured worker
`rule`	public	an instruction the agent must obey
`template`	supporting	the shape of a document
`checklist`	supporting	what a reviewer looks for
`constraints`	supporting	machine-checkable structure and bindings
`script`	supporting	a deterministic check or its configuration
`directory`	supporting	a whole folder, declared as one resource
`other`	supporting	a declared file that fits no named kind

Table 8.2: The nine resource kinds, by class.

8.5 The Nine Resource Kinds

Nine kinds, split into three public and six supporting, and listed in Table 8.2. Public kinds get a generated public name and become visible to the user and the agent. Supporting kinds are referenced by identifier from something else.

There is one legacy alias: `workflow`, which may be normalized to `skill`. New manifests are authored as `kind = "skill"`. If you see `kind = "workflow"` in an older kit, it is not wrong, it is old.

8.5.1 `template`

A markdown file with headings and prompts in braces. The braces are instructions to whoever — or whatever — fills the document in:

```
# PRD - {Module/Feature Name}

## 1. Overview

### 1.1 Purpose

{1-2 paragraphs: What is this system/module and
what problem does it solve?}

### 1.2 Background / Problem Statement

{2-3 paragraphs: Context, current pain points,
why this capability is needed now.}
```

Note what the prompts contain: not just the topic but the expected length. "1–2 paragraphs" is doing real work. Without it, the same heading produces a sentence from one author and two pages from another, and a reviewer has no grounds to object to either.

8.5.2 `checklist`

Semantic quality criteria, kept deliberately separate from the template. The shipped PRD checklist is organised by area of expertise:

```
# PRD Expert Checklist

**Artifact**: Product Requirements Document (PRD)
**Version**: 1.2
**Purpose**: Comprehensive quality checklist

## MUST HAVE
   - BUSINESS Expertise (BIZ)
   - ARCHITECTURE Expertise (ARCH)
   - SECURITY Expertise (SEC)
   - PERFORMANCE Expertise (PERF)
```

The separation of template from checklist is one of the two or three most transferable ideas in the kit model, and it is worth spelling out why. The template answers "what sections must exist." The checklist answers "is what is in them any good." Those two questions change at different rates, are owned by different people, and are checked at different times. Merged into one file they contaminate each other: reviewers start treating structure as quality, and authors start treating a filled-in heading as a satisfied requirement.

8.5.3 `rule`

A public kind. The SDLC kit uses it for its `AGENTS.md` entry point — that is the manifest entry shown earlier. But most rule content in practice is prose, and in the shipped kits it is prose written in PDSL, the notation Chapter 4 took apart:

```
UNIT AdrAuthoring

PURPOSE:
  Author or revise one ADR (one decision per record) that
  follows the template, has a unique well-formed ID, valid
  status, and complete rationale.

WHEN:
  - REQUIRE authoring or revising an ADR
```

Declarative, imperative-mood, and blunt. Chapter 9 shows five PDSL units from one real rules file, including the part where the notation earns its keep.

8.5.4 `constraints`

Everything above is prose that an agent interprets. Constraints are the part a machine checks: required headings, identifier kinds, cross-references.

```
[[artifacts.ADR.headings]]
id = "adr-context"
level = 2
required = true
multiple = false
numbered = false
pattern = "Context and Problem Statement"
description = "Problem context and motivating forces."
```

That is a heading contract. `cfs validate` reads it and either finds an H2 matching the pattern or does not. No interpretation, no judgement, same answer every time.

A constraints resource is also the only place that may carry `artifacts.<KIND>` bindings, which is how a document kind learns which template and which example belong to it:

```
[[kits.resources]]
id = "constraints"
kind = "constraints"
```

```
source = "constraints.toml"
type = "file"
user_modifiable = false

[kits.resources.artifacts]
[kits.resources.artifacts.ADR]
template = "adr_template"
examples = "adr_example"

[kits.resources.artifacts.PRD]
template = "prd_template"
examples = "prd_example"
```

Read the values in those bindings carefully. They are resource identifiers, not paths. This is the mechanism that makes a kit relocatable, and the next section explains why that matters more than it looks.

8.5.5 `skill`

Public, and the kind that gives your kit a front door. Here is a real entry:

```
[[kits.resources]]
id = "workflow_doc_prd"
kind = "skill"
source = "workflows/doc-prd.md"
install_path = "workflows/doc-prd.md"
type = "file"
user_modifiable = false
public = true
description = "PRD authoring preset workflow (wraps cf-write-docs)"
generated_targets = ["installed"]
origin = "legacy-workflow"
```

Now notice what is absent: there is no `name` field. The public name comes from the source file's own frontmatter:

```
---
cf-studio: true
type: workflow
name: cf-sdlc-doc-prd
description: Invoke when the user asks to author, write,
  revise, or generate a PRD ...
version: 1.0
purpose: Thin preset that binds the PRD artifact KIND ...
---
```

So `workflow_doc_prd` stays internal, and `cf-sdlc-doc-prd` is what a user types. `resource.id` is never the public name of a top-level public resource.

This is a good place for a note on reading the platform's own documentation carefully, in the spirit of Part II. The canonical manifest contract states that public kinds "also require `name`"; the naming rules in the same document state that public `skill`, `agent`, and `rule` resources derive their public generated name *only* from source frontmatter `name`. The shipped kits follow the second statement: their public skill entries carry no `name` field at all. Both sentences describe a real rule — a public resource must have a resolvable name — but only one of them tells you where to put it. When the two readings disagree, the installed kits are the tiebreaker, and they say frontmatter.

8.5.6 `agent`

Also public, and the kind with the most knobs. The agent-capable resource fields are `mode`, `isolation`, `model`, `tools`, `disallowed_tools`, `skills`, `color`, `memory_dir`, `role`, `target`, `provider`, `reasoning_effort`, `context_window`, and `subagents`.

Nested subagents live under `[[kits.resources.agent.subagents]]`, and their naming semantics mirror `agents.toml`. A subagent has no separate `name` field: its public name comes from `subagents[].id`, and its prompt file comes from `subagents[].source` or `prompt_file`.

Two different fields for two different situations — frontmatter `name` for top-level public resources, `subagents[].id` for nested ones — and mixing them up is a common first mistake.

8.5.7 `script` and `directory`

`script` is for a deterministic check, a repeatable task, or the configuration of one. `directory` is for when you want a whole folder along for the ride:

```
[[kits.resources]]
id = "change_impact_config"
kind = "script"
source = "scripts/change-impact-analysis.toml"
type = "file"
user_modifiable = true
description = "Kit-owned thresholds for the change-impact-analysis workflow"

[[kits.resources]]
id = "scripts"
kind = "directory"
source = "scripts"
type = "directory"
user_modifiable = true
description = "Kit scripts and PR review prompts"
```

The `directory` kind is the pragmatic choice when a folder's contents change often and enumerating every file would make the manifest a maintenance burden. The trade-off is real: you lose per-file `description` and per-file `user_modifiable`, and you gain not having to touch the manifest every time you add a checker.

8.5.8 `other`

The honest escape hatch: a declared file that fits none of the named kinds.

The shipped SDLC kit uses `other` for every `rules.md` and every `examples/example.md`. Read that twice, because it tells you something about the taxonomy. Even the reference kit does not force every file into a neat box — and the two file types it declines to categorise are among the most important in the kit.

Use `other` when the honest answer is "it is a file my workflow reads." Do not use it to dodge a validation rule. If you find yourself reaching for `other` to escape the constraint that `public = true` is only valid on three kinds, you have a design problem, not a taxonomy problem.

8.6 Names and Collisions

Install three kits and you may end up with three workflows that all want to be called `doc-prd`. The core also publishes skills, and it got there first.

A collision here is not a compile error. It is an agent picking the wrong workflow, which is a considerably worse failure than a crash, because it looks like it worked. The document gets written. It uses somebody else's template. You find out at review, or later.

So there is a prefix rule, and it is not optional.

8.6.1 The prefix rule

Public names carry `cf-` or `cf-<kit-slug>-`. Kit slug `sdlc` plus workflow `doc-prd` gives the published name `cf-sdlc-doc-prd`. Kit slug `compete` plus workflow `request` gives `cf-compete-request`, which is the name you will be typing in Chapter 9.

There is no double-prefixing. If the frontmatter name already starts with the prefix, it is left alone — so `cf-sdlc-doc-prd` does not become `cf-sdlc-cf-sdlc-doc-prd`.

8.6.2 Opting out

`prefix_generated_name = false` publishes the name as it stands:

```
[[kits.resources]]
id = "house_review"
kind = "skill"
name = "acme-review"
source = "skills/house-review/SKILL.md"
type = "file"
public = true
prefix_generated_name = false
```

Valid only on public resources or nested subagents. Use it when your organisation has its own naming standard and you mean it. Do not use it to shorten a name you were too lazy to type — the prefix is the only thing standing between you and the silent-wrong-workflow failure described above.

8.6.3 A naming convention worth adopting

Pick a slug you can live with, because it ends up in every public name. Short, lowercase, hyphenated: `sdlc`, `compete`, `company-sdlc`.

Then give internal identifiers underscores: `prd_template`, `adr_checklist`, `research_request_rules`. The shipped kits follow exactly this split, and it reads well for a reason that has nothing to do with aesthetics: the visual difference between `cf-compete-request` and `research_request_rules` is a free reminder of which namespace you are in. Hyphens are public. Underscores are internal. You will never accidentally type one where the other belongs.

8.6.4 Why identifiers and not paths

Resource identifiers are the source of template variables such as {`prd_template`}. A rules file that says `LOAD` {`prd_template`} is not naming a path; it is naming a resource that resolves through the manifest, per kit.

Three consequences follow, and all three matter:

- Move the file, update `source` in one place, and every reference keeps working.
- In register mode, variable resolution comes from the canonical manifest plus the registered manifest root — so the same rules file works whether the kit is registered in place or copied.
- `cfs info` and `cfs resolve-vars` reason *per kit*, not as one merged variable blob. So two kits can each declare a `prd_template` without fighting.

That third point is the one that makes multi-kit installations tractable. If variables were global, adding a kit could break an unrelated one, and the failure would be invisible until a document came out wrong.

8.7 The Flow, End to End

Five steps, six commands. Learn them.

```
$ cfs kit normalize ./kits/company-sdlc
$ less ./kits/company-sdlc/.cf-studio-kit.toml
$ cfs kit install --path ./kits/company-sdlc --install-mode register
$ cfs generate-agents
$ cfs validate-kits
$ cfs validate
```

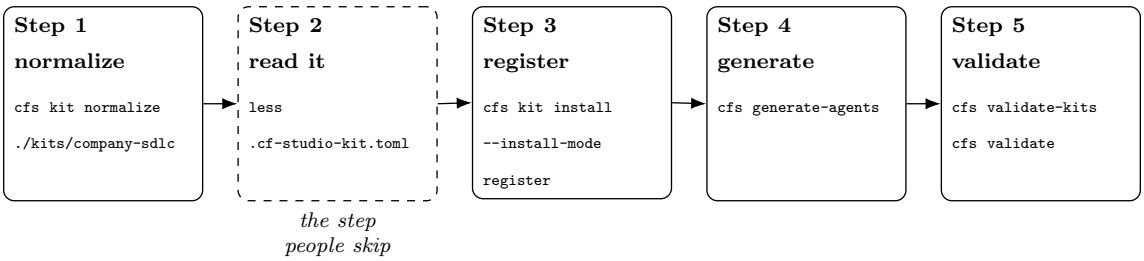


Figure 8.3: Registering a local kit: five steps, six commands.

The second line is the one people skip, and it is the one that matters.

8.7.1 Step 1: normalize

```
$ cfs kit normalize ./kits/company-sdlc
```

This points the kit workflow at a folder and asks it to produce a canonical manifest. It discovers candidate resources by reading the layout, classifies them into kinds, and synthesizes a manifest preview.

Discovery is read-only. Nothing is written until you approve.

One rule about how the normalizer resolves disagreements is worth knowing before you run it, because it is a statement of priorities: if the observed kit layout conflicts with the canonical manifest rules, the workflow normalizes the *layout* into the manifest rather than weakening the manifest contract. The contract does not bend to accommodate your folder names.

8.7.2 Step 2: read what it wrote

The generated file tells you what it is, in a comment on the first line:

```
# Generated by cfs kit normalize. Review before publishing.

manifest_version = "1.0"

[[kits]]
slug = "sdlc"
name = "sdlc"
version = "1.0"
```

"Review before publishing" means you.

Automatic classification is a guess informed by your folder names. It is a good guess — a file under `templates/` is probably a template — but a guess is exactly what it is. Check every `kind`. Check every `public`. Check every `user_modifiable`, because that one is a promise to your users about what they may edit, and a normalizer cannot know what you intend to promise.

8.7.3 Step 3: install in register mode

```
$ cfs kit install --path ./kits/company-sdlc --install-mode register
```

The kit root is recorded in `core.toml`. The files do not move. Effective resource paths are derived at run time from the manifest. The kit path must stay inside the project root, symlinks resolved.

8.7.4 Step 4: generate the integration files

```
$ cfs generate-agents
```

Your kit is registered, but your AI coding tool has not been told. This command writes the generated integration files for the tools in use — the same mechanism Chapter 4 described as *compile once, adapt per host*, now carrying your kit's skills alongside the built-in ones.

Those generated files are repairable outputs, ignored by git by default. In a multi-repository workspace, run this in each repository.

Forget this step and your new workflow appears not to exist. This is the single most common "my kit does not work" report, and the fix is one command.

8.7.5 Step 5: validate

```
$ cfs validate-kits
$ cfs validate
$ cfs map
```

`validate-kits` checks the manifest contract. `validate` checks project artifacts and traceability. `map` renders the dependency map, which is how you see what you actually built.

Put the first two in continuous integration on day one, not on the day you first need them. Registered local kits are normal project files: review them in pull requests, run the deterministic checks in CI, and keep changes small enough that product, architecture, quality, security, operations, and development reviewers can each understand what changed.

8.7.6 One deterministic pass is not enough

For a workflow or review change, add one manual chat dry run in addition to the deterministic checks. Confirm four things:

- the route captures the target before discovery;
- optional explore or brainstorm is offered where appropriate;
- a plan is requested before substantive multi-step work;
- review findings are shown before any fix is approved.

These are user-facing behaviour checks, not file-format checks. A validator will confirm that your workflow file parses and that its frontmatter is well formed. It will never notice that the workflow asks its clarifying question after it has already started writing. Only a human running the workflow once will notice that.

8.7.7 Updating a registered kit

```
$ cfs kit update --path ./kits/company-sdlc
$ cfs generate-agents
$ cfs validate-kits
```

Both syntaxes work here too — `cfs kit update path/./kits/company-sdlc` is equivalent.

One property of the update path is worth understanding, because it is the kind of detail that only shows up as a confusing diff. Manifest-backed updates compare resources by *manifest identity*, not by raw filename alone. If a resource is sourced from one filename but installed to another path, Studio updates the bound target in place and does not report that installed target as "deleted upstream."

Which is exactly the `agents-res.md`-to-`AGENTS.md` case from earlier. Without identity-based comparison, every update would claim your `AGENTS.md` had vanished from the kit and that a mysterious `agents-res.md` had appeared.

8.8 More Than One Repository

For multi-repository work, keep the extension point explicit. The common pattern is a division by role:

- one documentation or architecture repository owns requirements documents, decision records, design documents, and the shared kits;
- one user interface repository owns frontend code;
- one backend repository, or several service repositories, own implementation code;
- a workspace connects those repositories for validation, maps, and cross-repository traceability.

The workspace commands are the ones from Chapter 1:

```
$ cfs workspace-init
$ cfs workspace-add --name docs --path ../docs-repo --role artifacts
$ cfs workspace-add --name services --path ../services-repo --role codebase
```

When the kit source lives inside the main project or workspace repository, register it in place as usual:

```
$ cfs kit install --path ./kits/company-sdlc --install-mode register
```

And if each repository needs its own generated AI-tool files, run `cfs generate-agents` in each repository after the kit or workspace configuration changes. This is the multi-repository version of the forgotten-step problem, and it presents identically: the kit works in one repository and appears not to exist in the next one over.

8.9 Migrating Off the Old Model

If you inherit a project that extends Constructor Studio through a project-level `manifest.toml`, include chains, and orchestrator-layer discovery, you are looking at the superseded mechanism. It still works. It should not be your first choice for new work.

The migration is five steps:

1. Move project-specific skills, rules, templates, checklists, constraints, scripts, and workflow resources into a local kit directory.
2. Add or normalize the kit manifest.
3. Register the kit in place.
4. Regenerate the AI coding tool files.
5. Validate the kit and the repository.

```
$ cfs kit install --path ./kits/company-sdlc --install-mode register
$ cfs generate-agents
```

You want	You need
a new document type	a kit
a new review standard	a kit
a new house rule	a kit
a new runtime service	a gear
a new endpoint	a gear
new persistence	a gear

Table 8.3: Choosing between the two extension mechanisms.

The stated reason for preferring kits is worth quoting in substance, because it is an argument about review rather than about capability: kits provide a clearer review boundary and match the installation, update, validation, and distribution model that Constructor Studio already uses for everything else. A manifest hierarchy with include chains can express the same rules. What it cannot do is present them to a reviewer as one reviewable unit.

8.10 A Gear Is Not a Kit

One boundary before the exercises, because people reach for the wrong mechanism about a third of the time.

Kits are markdown and TOML, read at run time by an agent. A gear — Chapter 5's subject — is compiled Rust, linked into a server binary. The shapes are not comparable:

```
examples/toolkit/users-info/
|-- users-info-sdk/ # public contract: trait, models, errors
|-- users-info/ # the gear: domain, infra, api, wiring
`-- users-info-server/ # registers gears + boots the runtime
```

Two crates plus a thin server crate. The SDK crate is the boundary between gears: consumers depend on it and on nothing else.

Everything a kit does through declaration, a gear does through code. Instead of declaring resources, a gear declares capabilities and dependencies in `gear.rs`. Instead of a name prefix, it registers its SDK implementation in `ClientHub` at initialisation. Instead of `cfs validate-kits`, the check is `cargo build`.

Table 8.3 is the decision, compressed.

The one-line version: kits change what the agent knows; gears change what the system does. The organisational version is more useful. A kit is edited by whoever owns the process. A gear is edited by whoever owns the code. If the person who needs the change cannot compile the project, they need a kit.

8.11 Where That Leaves Us

- A kit is a directory with `.cf-studio-kit.toml` at its root.
- The manifest declares resources; each needs `id`, `kind`, and `source`.
- Nine kinds. Three are public and get prefixed names; six are supporting and get referenced by identifier.
- Register mode leaves your files in place and is what you want.
- Normalize, read, register, generate, validate.
- References go through identifiers, never paths.

- The core never gets patched.

Chapter 9 takes a kit that somebody actually shipped — eleven artifact kinds, fourteen workflows, eight checker scripts, sixty-three declared resources — and reads it the way you would read it on your first day of maintaining it.

Exercises

1. **Read a real manifest and account for every resource.** Open `studio-kit-sdlc/.cf-studio-kit.toml`. Count the `[[kits.resources]]` entries and tabulate them by `kind`. Then answer: how many resources are public, and which files are they? How many carry `user_modifiable = false`, and can you state in one sentence per file why the author decided a user should not edit it? Finally, find every resource where `source` and `install_path` differ, and explain each difference.

2. **Write the smallest legal kit and prove it is legal.** Create `kits/scratch/` containing exactly two files: a `.cf-studio-kit.toml` following the minimal shape in this chapter, and the one template it declares. Register it with `cfs kit install --path ./kits/scratch --install-mode register`, then run `cfs generate-agents` and `cfs validate-kits`. Record the exact output. Then break it four times, one at a time — remove `manifest_version`; set `type = "folder"`; set `public = true` on the template; add an `artifacts.PRD` binding to it — and for each, record the validator's message verbatim. Which of the four gave you the message that would have been hardest to diagnose without this chapter?

3. **Predict the public names.** Given a kit with `slug = "acme"` and four public resources whose source frontmatter `name` fields are `review`, `cf-review`, `acme-review`, and `cf-acme-review`, write down the published name of each under the prefix rule. Then state which of the four you would actually ship and why, and which one is a trap. Verify your predictions against a real kit by finding a frontmatter `name` that already carries the prefix and checking that the installed name is not double-prefixed.

4. **Register mode against copy mode, empirically.** Take the scratch kit from Exercise 2. Install it in register mode, then inspect `core.toml` and write down exactly what was recorded. Reinstall it in copy mode with `--install-mode copy` and inspect `core.toml` again. Describe the difference in one paragraph. Then edit the template's text in the kit directory under each mode in turn, and determine in each case whether the change is live without a reinstall. State which mode you would use for a company kit and give a reason that is not simply "the guide says so."

5. **Find something the validator cannot catch.** Write a kit workflow file — a `kind = "skill"` resource — that passes `cfs validate-kits` cleanly but violates one of the four behaviour checks from the manual dry-run list in this chapter. For example, make it write a file before it asks its clarifying question. Then explain in a short paragraph why no file-format check could detect the problem, and propose the smallest addition to the kit — a checklist item, a constraints entry, a script — that would make the violation visible to a reviewer.

6. **Choose the mechanism.** For each of the following, decide kit or gear, and justify in two sentences: (a) your organisation requires a data-protection impact assessment before any feature touching personal data reaches implementation; (b) you need requirements documents to expose a read-only HTTP endpoint so an internal dashboard can display their approval status; (c) your reviewers want architecture decision records to carry a

mandatory “options rejected” section with at least two entries; (d) you want CPT identifiers stored in a database so they can be queried across forty repositories. For any case where you find yourself wanting both, say what belongs on each side of the boundary.

Chapter 9

A Real Kit, Walked Through

Chapter 8 gave you the mechanism. Mechanisms are easy to describe and hard to judge. What a manifest format cannot tell you is whether the format holds up when somebody uses it for something the designers did not have in mind, or how much prose a working kit actually contains, or which of the nine resource kinds get used in practice and which ones sit unused.

So this chapter reads one shipped kit — `compete`, the competitive analysis kit in the `studio-kits-pm` repository — the way you would read it on your first day of maintaining it. It is a good choice for three reasons. It is not the SDLC kit, so nothing about it is privileged. It is substantial: eleven artifact kinds, fourteen workflows, eight checker scripts, sixty-three declared resources. And its domain is not software delivery at all, which means every mechanism it uses is being used generically, exactly as Chapter 8 claimed it would be.

We will go through the whole kit's shape first, then take one artifact kind — `RESEARCH-REQUEST` — apart completely, then look at the deterministic checking layer that sits underneath all of it.

9.1 The Kit's Shape

9.1.1 First surprise: there is no `.cf-studio-kit.toml` here

Open `kits/compete/` and you will not find the manifest Chapter 8 told you to expect. What you find instead is a short file called `conf.toml`, and it is short:

```
# Competitive Analysis Kit Configuration

slug = "compete"
name = "Constructor Studio Competitive Analysis Kit"
version = "1.15.0"
```

Five lines. No `manifest_version`, no `[[kits]]` block, no `[[kits.resources]]` entries. By the contract in the previous chapter this is not a manifest at all.

It is not meant to be. `compete` is one kit inside a *multi-kit repository*, and the repository states the arrangement in its own conventions:

> "One kit per directory under `kits/<slug>/`; the repository-root `.cf-studio-kit.toml` is the single canonical manifest — one `[[kits]]` entry per kit, resource `source` paths repo-relative (`kits/<slug>/...`), `install_path` project-relative."

So the canonical manifest is one directory up, at `studio-kits-pm/.cf-studio-kit.toml`, and it is the thing Chapter 8 described. It opens exactly as specified:

```
# Canonical multi-kit manifest for this repository.
# Each [[kits]] entry is one installable kit; resource ‘source‘ paths are
# repo-relative (kits/<slug>/...), ‘install_path‘ stays project-relative.
```

```
manifest_version = "1.0"

[[kits]]
slug = "compete"
name = "compete"
version = "1.15.0"
```

This is that `[[kits]]` array-of-tables property from the previous chapter earning its keep. One file, one `[[kits]]` entry per kit, and `cfs` reads it to offer the kit selector on install — which is why `cfs kit install` takes a `--kit` flag that accepts one slug, a comma-separated list, or `all`.

The per-kit `conf.toml` is therefore not a competing manifest format. It is the kit's own record of its identity, sitting next to the kit's files, where a maintainer working inside `kits/compete/` will see it. Do not read it as an alternative to `.cf-studio-kit.toml`; read it as the kit's name tag.

There is a small lesson in the version numbers, incidentally. `conf.toml` and the root manifest both say `1.15.0`. The kit's own `README.md` says "Version 1.14.0" in its Status section. Two of the three are machine-read and agree; the one that drifted is the prose. That is the normal failure mode for duplicated facts, and it is an argument for keeping the authoritative copy in exactly one place — which the repository does, and then undermines slightly by restating it in a README.

9.1.2 The root, and three counts worth checking

The kit root, with the interesting entries:

```
kits/compete/
|-- conf.toml # slug, name, version
|-- SKILL.md # kit entry skill (frontmatter name: cf)
|-- agents-res.md # installed as AGENTS.md
|-- constraints.toml # 37 kB deterministic contract
|-- whatsnew.toml # release notes
|-- README.md
|-- artifacts/ # 11 artifact kinds
|-- workflows/ # 14 flow files
|-- scripts/ # 8 stdlib-only Python checkers
|-- guides/ # ARCHITECTURE, CAPTURE, DEMO, EVIDENCE, QUICKSTART
`-- examples/ # sample-evidence/, hermetic, used by CI
```

Note `agents-res.md` and its installed name. That is the source-versus-`install_path` rename from Chapter 8, in a second kit, doing the same job:

```
[[kits.resources]]
id = "agents"
kind = "rule"
source = "kits/compete/agents-res.md"
install_path = "AGENTS.md"
type = "file"
user_modifiable = false
public = true
description = "Kit AGENTS.md for agent integration entry points"
generated_targets = ["installed"]
```

The three counts are worth verifying yourself, because they are the kind of number that gets stated once in a README and then quietly goes out of date. Counting directories under `artifacts/` gives eleven; counting files under `workflows/` gives fourteen; counting Python files under `scripts/` gives eight. The kit's own `SKILL.md` frontmatter enumerates all eleven artifact kinds and all fourteen workflows by name, so there is a second, independent source that agrees.

Table 9.1 lists the eleven artifact kinds against the nine research flows they implement.

Nine flows, eleven kinds. The two extras are not flows: `RESEARCH-REVIEW` audits a completed run, and `FEATURE-TAXONOMY` is a normalisation contract that the comparison matrix binds to. Fourteen workflows against eleven kinds, similarly, because three workflows produce no artifact

Flow	Artifact kind	Skill
1	RESEARCH-REQUEST	cf-compete-request
2	CANDIDATE-REGISTER	cf-compete-discover
3	ORIENTATION-CARD	cf-compete-orient
4	COMPANY-PROFILE	cf-compete-profile
5	COMPANY-DOSSIER	cf-compete-dossier
6	COMPARISON-MATRIX	cf-compete-compare
7	OSS-REVIEW	cf-compete-oss-review
8	MECHANISM-STUDY	cf-compete-mechanism
9	WATCH-REPORT	cf-compete-watch
—	RESEARCH-REVIEW	cf-compete-review
—	FEATURE-TAXONOMY	cf-compete-taxonomy

Table 9.1: The eleven artifact kinds in the `compete` kit.

of their own — `capture` and `intake` feed the evidence layer, and `run` is an express orchestrator that drives the whole pipeline from one plan approval.

That asymmetry is the first genuinely instructive thing about this kit. A naive reading of Chapter 8 would suggest one workflow per artifact kind, the way the SDLC kit's four document presets suggest. This kit has supporting workflows that produce infrastructure rather than documents, and artifact kinds that exist to constrain other artifact kinds. The mechanism permits both because it does not know what an artifact is.

9.1.3 The layout the whole kit is built on

From the kit's own README:

```
artifacts/<KIND>/{template.md, rules.md, checklist.md, examples/example.md}
workflows/<flow>.md # one file per flow, generated as cf-compete-* skills
constraints.toml # deterministic heading/ID contract for cfs validate
scripts/ # stdlib-only evidence tooling and checkers
examples/sample-evidence/ # hermetic OpenSpec evidence set used by examples and CI
guides/ # QUICKSTART, ARCHITECTURE, EVIDENCE
```

Four files per artifact kind, eleven times. That is the same regularity Chapter 8 pointed at in the SDLC kit — four files, five times — reproduced by a different author in a different domain.

Notice that this is not enforced by anything. Nothing in the manifest contract requires a folder per artifact kind, and nothing requires those four filenames. Both kits arrived at the pattern because the four files answer four different questions:

`template.md` What sections does the document have?

`rules.md` What must the author do?

`checklist.md` What must the reviewer verify?

`examples/example.md` How deep is deep enough?

Split that way, each file has one job and one reason to change. Merge any two of them and you get the contamination described in the previous chapter: structure mistaken for quality, or a filled-in heading mistaken for a satisfied requirement.

Kind	Count	What they are
other	22	11 rules.md + 11 examples/example.md
skill	15	14 workflows + SKILL.md
template	11	one per artifact kind
checklist	11	one per artifact kind
directory	2	scripts/ and guides/
constraints	1	constraints.toml
rule	1	agents-res.md

Table 9.2: Resource kinds actually used by the compete kit.

9.1.4 What the manifest declares

Sixty-three [[kits.resources]] entries, distributed as shown in Table 9.2.

Two of the nine kinds do not appear at all. There is no agent resource — the kit configures no workers of its own — and no script resource, despite the kit shipping eight Python checkers, because scripts/ is declared once as a directory:

```
[[kits.resources]]
id = "scripts"
kind = "directory"
source = "kits/compete/scripts"
install_path = "scripts"
type = "directory"
user_modifiable = true
description = "Deterministic evidence tooling (build_evidence_index.py, check_evidence.py)"
```

That is the trade-off from Chapter 8, taken deliberately. Eight scripts declared individually would be eight entries to maintain and eight descriptions to keep current; declared as a directory, adding a ninth checker requires no manifest change at all. The cost is visible in that very description, which names two of the eight scripts and has not been updated to mention the other six.

The guides/ entry shows the other side of the trade, and its description is doing something a per-file declaration would not:

```
[[kits.resources]]
id = "guides"
kind = "directory"
source = "kits/compete/guides"
install_path = "guides"
type = "directory"
user_modifiable = true
description = "Kit method guides (QUICKSTART, ARCHITECTURE, EVIDENCE, CAPTURE, DEMO) - the capture
    workflow requires guides/CAPTURE.md and guides/EVIDENCE.md at runtime"
```

Read the second clause. Two files inside that directory are runtime dependencies of a workflow, and because the directory is declared as a unit there is nowhere structural to record that fact. So the author recorded it in the description. That is a workaround, and an honest one: the information is at least in the manifest, where somebody deleting guides/CAPTURE.md might see it.

9.1.5 The design principles, stated by the kit itself

Before taking an artifact apart, it is worth reading what the kit says it is for, because every rule in the next section is downstream of one of these four commitments:

The request is the root. Every research artifact cites the RESEARCH-REQUEST that commissioned it; an omitted request field is visibly "unspecified", never silently defaulted.

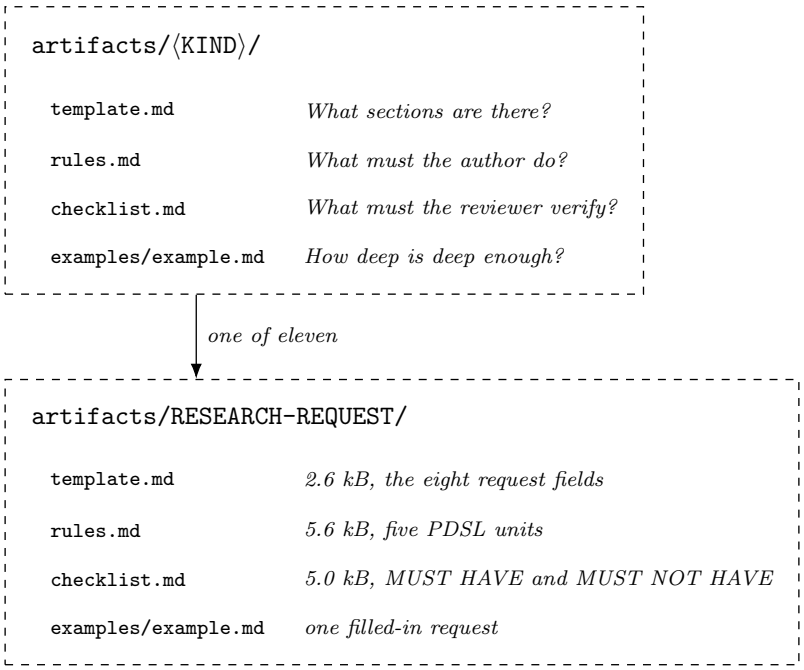


Figure 9.1: Four files per artifact kind, eleven times over.

Evidence or it did not happen. Facts, vendor claims, third-party estimates, and unknowns are distinct statuses. Every factual statement resolves to an `EVD-###` evidence record with an access date.

Absence of evidence is not evidence of absence. Missing public documentation is recorded as `not_observed` or `not_assessed`, never as "lacks the feature".

No rankings. The kit never produces a universal market ranking, a winner/loser score, or an unsupported feature-parity claim, and a research result never auto-creates a product decision.

Those are epistemic commitments, not formatting preferences. They are also exactly the kind of thing that lives in people's heads and dies when they leave — which is the argument for kits from the opening of Chapter 8, arriving here as a concrete case. A competitive analyst who knows not to say "competitor X lacks feature Y" when the documentation is merely silent has a professional discipline. Encoded as a kit rule, that discipline is enforceable by a machine and reviewable in a pull request.

9.2 `RESEARCH-REQUEST`, Top to Bottom

`RESEARCH-REQUEST` is Flow 1 and the root of the kit's traceability chain. Four files, and we will read all four.

```
artifacts/RESEARCH-REQUEST/
|-- template.md # 2.6 kB
|-- rules.md # 5.6 kB
|-- checklist.md # 5.0 kB
‘-- examples/
    ‘-- example.md
```

9.2.1 The template

It opens with frontmatter, which is the first thing to notice: the artifact has machine-readable metadata before any prose.

```
---
date: {YYYY-MM-DD}
status: {draft | active}
---

# Research Request - {Concise target name}

**ID**: `cpt-{system}-req-{slug}`

## Table of Contents

<!-- generated by `cfs toc` -->
```

Three things in eleven lines.

The frontmatter carries `date` and `status`, with the permitted status values written into the placeholder as an enumeration. A model filling this in is told the vocabulary at the point of use rather than in a separate document it may or may not have loaded.

The `ID` line is a CPT identifier in the pattern from Chapter 3 — `cpt-{system}-req-{slug}` — with `req` as the kind segment. The identifier is on its own line, in backticks, immediately under the H1. That placement is not decorative: a regular expression in one of the kit's scripts finds it there, and we will see that regular expression later in this chapter.

The Table of Contents section is a heading with a comment where content will go. `cfs toc` fills it, and `cfs validate-toc` checks it. The template declares the slot; the tooling owns the contents.

Then the substance. Section 1 is a table:

```
## 1. Request Fields

| Field | Selected value |
| --- | --- |
| Target | {A named company, a product, a required capability, or a Studio product/vision/features input -
     or "unspecified"} |
| Decision question | {The question this research must support, e.g. "which named products should be
    examined beside {target}?" - or "unspecified"} |
| Scope | {Direct / adjacent / emerging boundary, plus any product, geography, role, stage/job, or
    capability boundary - or "unspecified"} |
| Comparison frame | {One frame: role x stage/job taxonomy, product outcome frame, or a named mechanism
    lens - or "unspecified"} |
| Depth | {discovery scan | orientation | standard profile | deep dossier | comparison | watch - or "
    unspecified"} |
| Requested output | {Register, one-page card/slide, dossier, matrix, open-source review, mechanism brief,
     or monitoring report - or "unspecified"} |
| Evidence cutoff | {YYYY-MM-DD - or "unspecified"} |
| Access boundary | {e.g. "retained public sources only" - or "unspecified"} |
```

Eight fields, and every single placeholder ends with the same four words: "or `"unspecified"`".

Eight repetitions of the same clause in one table is not sloppy authoring. It is the kit's first design principle, expressed at the only place where it can actually be obeyed. A model filling in the Target row does not have to remember a rule from a different file about what to do when the requester did not state a target. The instruction is in the cell.

This is worth generalising, because it is the single most portable technique in this chapter. If a rule has one place where it can be violated, put the rule there. Prose in a rules file is read once at the start of a workflow and competes for attention with everything else in the context window. A placeholder is read at the exact moment the decision is made.

Section 2 is the boundary, and it prescribes its own content:

```
## 2. Boundary
```

```
{What this request can establish, and what it cannot. State explicitly that the
requested depth bounds the claims: a discovery scan cannot establish feature
coverage, product quality, market rank, commercial viability, or source-code
reuse.}
```

A required section whose job is to state the limits of the document it appears in. Very few document templates in any discipline contain a mandatory section for what the document does not establish, and the omission is the reason research findings get quoted out of scope.

Section 3 keeps the original input:

```
## 3. Evidence and Original Files

- Original request: {verbatim quote or path of the user-supplied request text}
- {Any named input sources, with evidence locators (`EVD-###`) when already indexed, or plain paths when
    not}
```

The verbatim original is what makes the eight-field extraction auditable later. Somebody reading this request in six months can check whether "Depth: standard profile" is what the requester actually asked for or an interpretation somebody made.

And Section 4 is the most interesting part of the template, because most of it is commented out:

```
## 4. Traceability

No artifacts have been derived from this request yet.

<!--
Once downstream artifacts are created, their authoring workflows replace the
line above with one back-link per derived artifact (this section is required
and holds the mandatory request -> artifact back-references):

- `cpt-{system}-reg-{slug}` - {Candidate register title}
- `cpt-{system}-ori-{slug}` - {Orientation card title}
- `cpt-{system}-prof-{slug}` - {Company profile title}
- `cpt-{system}-dos-{slug}` - {Company dossier title}
- `cpt-{system}-cmp-{slug}` - {Comparison matrix title}
- `cpt-{system}-oss-{slug}` - {OSS review title}
- `cpt-{system}-mech-{slug}` - {Mechanism study title}
- `cpt-{system}-wat-{slug}` - {Watch report title}
- `cpt-{system}-rev-{slug}` - {Research review title}
- `cpt-{system}-tax-{slug}` - {Feature taxonomy title}
-->
```

The section is required, so it must exist in every request. But a fresh request has no derived artifacts, and writing speculative back-links would create dangling references that fail validation. So the template ships a default line — "No artifacts have been derived from this request yet." — and puts the ten possible back-link forms in an HTML comment, where a human or a model can read the expected shape without the shapes themselves being content.

The comment is documentation delivered at the point of use, and it survives into the authored artifact, so the next workflow that appends a back-link finds the format waiting for it. Ten ID kind segments in one place: `reg`, `ori`, `prof`, `dos`, `cmp`, `oss`, `mech`, `wat`, `rev`, `tax`. Together with `req`, that is the kit's whole identifier vocabulary, and it is legible from a single template.

9.2.2 The rules

`rules.md` is where the PDSL from Chapter 4 appears in production. Five units, in fenced `pdsl` blocks, each with a single responsibility:

`RequestAuthoring` write the artifact

`RequestOmissions` what must not be in it

`RequestValidate` check it

`RequestErrorHandling` recover from bad input

`RequestNextSteps` what to offer afterwards

The authoring unit starts with a purpose and a state declaration:

```
UNIT RequestAuthoring

PURPOSE:
  Author one RESEARCH-REQUEST artifact: the structured record of what question
  must be answered, at which depth, within which evidence boundary. The request
  is the root of the traceability chain - every later register, profile,
  dossier, or comparison must be attributable to a request.

STATE:
  - SET request_source: file | inline
    default: inline when the user typed the request in chat

WHEN:
  - REQUIRE recording a new competitive-research request, or revising a draft one
```

Then the `DO` block, which is the workflow's actual procedure. Three lines from it carry most of the weight:

```
DO:
  - LOAD {cf-studio-path}/config/artifacts.toml to resolve the RESEARCH-REQUEST directory
  - SET default path {artifacts_dir}/RESEARCH-REQUEST/{yyyy-mm-dd}-{slug}.md and register in artifacts.
      toml
  - LOAD {research_request_template} for structure
  - LOAD {research_request_example} for content depth reference
  - RUN capture the raw request text verbatim (from the supplied file or the chat message)
  - RUN extract each of the eight request fields from the raw text: target, decision question, scope,
      comparison frame, depth, requested output, evidence cutoff, access boundary
  - RUN set every field the requester did not state to the literal value "unspecified" - never infer,
      default, or borrow a value
  - RUN author Section 2 (Boundary): what this request can establish at the selected depth, and what it
      cannot
  - RUN author Section 3 (Evidence and Original Files): the verbatim request text or its path, plus any
      named input sources
  - RUN author Section 4 (Traceability) with the default line "No artifacts have been derived from this
      request yet." - a fresh request MUST NOT list downstream back-links until derived artifacts exist
  - SET ID cpt-{system}-req-{slug} for the request
  - RUN verify ID uniqueness with cfs list-ids
  - LOAD {research_request_checklist} for self-review of semantic quality
```

Look at the `LOAD` arguments: {`research_request_template`}, {`research_request_example`}, {`research_request_checklist`}. Those are resource identifiers from the manifest, exactly as Chapter 8 described. No paths appear anywhere in this file. The kit could be reorganised on disk tomorrow, and as long as the manifest's `source` values were updated, this rules file would not need a single edit.

Then the `RULES` block, which is where the constraints live:

```
RULES:
  - ALWAYS follow {research_request_template} structure with required frontmatter
  - ALWAYS render an omitted field as the literal "unspecified" so the omission stays visible in every
      later artifact
  - ALWAYS keep the field values faithful to the requester's words; normalization (e.g. mapping "quick
      look" to depth "orientation") must be shown to the requester for confirmation
  - ALWAYS state the depth-appropriate claim boundary in Section 2
  - ALWAYS keep Section 4 (Traceability) present but free of downstream references in a fresh request;
      downstream workflows add the mandatory request -> artifact back-links later
  - ALWAYS treat {research_request_checklist} as the semantic source of truth
  - NEVER include placeholder content (TODO, TBD, FIXME)
  - NEVER reference an artifact ID that is not yet defined (dangling references fail validation)
  - NEVER create duplicate IDs
  - NEVER duplicate {research_request_checklist} semantic criteria in this file
```

Two of those are worth separating out.

The third `ALWAYS` handles the case that a naive implementation gets wrong. The requester said "quick look"; the kit's depth vocabulary says "orientation". Something has to bridge those, and the rule does not forbid the mapping — it requires that the mapping be *shown to the requester for confirmation.* That is a much better rule than either forbidding normalisation or permitting it silently, and it is the sort of rule that only gets written by somebody who has been burned.

The last `NEVER` is a rule about the kit's own file layout rather than about the artifact: do not duplicate the checklist's semantic criteria here. Two files, two jobs, and an explicit prohibition against the drift that comes from copying content between them. The shipped SDLC kit's rules files carry the same prohibition. Both authors evidently learned that the four-file split only works if something actively prevents the files from growing into each other.

9.2.3 The omissions unit, and the value of an identifier

The second PDSL unit is short and does one thing:

```
UNIT RequestOmissions

PURPOSE:
  Enforce MUST NOT HAVE content; report any occurrence as a violation.

RULES:
  - NEVER CPT-REQ-NO-001 (CRITICAL): silently defaulting or inferring an omitted field instead of
      recording the literal "unspecified"
  - NEVER CPT-REQ-NO-002 (CRITICAL): research findings, competitor claims, feature scores, or source
      extracts embedded in the request - they belong in downstream artifacts
  - NEVER CPT-REQ-NO-003 (HIGH): presenting the request as a product decision, requirement, or delivery
      commitment
  - NEVER CPT-REQ-NO-004 (HIGH): more than one comparison frame in a single request (frames may be mapped,
       never silently mixed)
  - NEVER CPT-REQ-NO-005 (HIGH): placeholder content (TODO, TBD, FIXME)
```

Five prohibitions, each with a stable identifier and a severity.

The identifiers are the part to steal. `CPT-REQ-NO-001` is not decoration: it appears again, verbatim, in `checklist.md`'s MUST NOT HAVE section, which means a reviewer's finding and the authoring rule that should have prevented it share a name. When a review report says `CPT-REQ-NO-001`, you can grep for it and find both the rule that was broken and the check that caught it. Compare that with the alternative, which is a review comment saying "you defaulted a field" and no way to connect it to anything.

`CPT-REQ-NO-001` is also the first design principle, stated for the third time in three files — placeholder text in the template, an `ALWAYS` in the authoring unit, and a `NEVER` with a severity here. Three statements of one rule, at three different moments: when the field is filled in, when the document is authored, and when it is reviewed. That is not redundancy. Each one catches a failure the other two cannot.

And `CPT-REQ-NO-004` is the sharpest of the five. More than one comparison frame in a single request is forbidden — and the parenthesis explains why the rule is not simply "one frame": frames *may* be mapped, never silently mixed. Again, the prohibition is on the silence, not on the operation.

9.2.4 The validation unit

```
UNIT RequestValidate

PURPOSE:
  Validate the RESEARCH-REQUEST deterministically and semantically, then report.

DO:
  - RUN cfs validate --artifact <path> for template structure, ID format, no placeholders
```

```
  - RUN cfs toc <artifact-file> to generate/update Table of Contents
  - RUN cfs validate-toc <artifact-file> - must report PASS
  - LOAD {research_request_checklist} for semantic validation and report format
  - RETURN validation report (Structural PASS/FAIL, Semantic PASS/FAIL with issues)

RULES:
  - ALWAYS use {research_request_checklist} as the source for semantic criteria and the review report
      format
  - NEVER mark validation done while any check reports FAIL or error
  - NEVER duplicate checklist semantic criteria or report templates here
```

Two kinds of validation, named separately and reported separately. Structural validation is three `cfs` commands with deterministic answers. Semantic validation is a model reading a checklist. The unit returns both, labelled, so a reader of the report knows which findings came from a machine and which came from a judgement.

That distinction is the whole architecture of this kit in miniature, and its `guides/ARCHITECTURE.md` states the division explicitly:

> "`cfs` enforces heading outlines, ID formats, and cross-artifact coverage; it cannot see table rows. The epistemically critical checks — every claim row carries a resolvable evidence locator, enum vocabularies, ISO dates — are the scripts' job, and the scripts are mandatory members of every workflow's deterministic gate."

"It cannot see table rows" is an unusually candid statement of a tool's limit, and it is the reason the kit ships eight scripts.

9.2.5 Error handling, and asking instead of guessing

```
UNIT RequestErrorHandling

PURPOSE:
  Recover from input failures and escalate ambiguous situations.

ON_ERROR:
  request_text_missing ->
    EMIT "No request text supplied - provide the request inline or as a file path"
    RUN ask the user for the request text

  ambiguous_field_mapping ->
    EMIT "The request wording maps to more than one value for {field}"
    RUN present the candidate readings and WAIT for the user's choice

  duplicate_request ->
    EMIT "A registered RESEARCH-REQUEST with a matching target and decision question already exists: {id}"
    RUN ask whether to revise the existing request or record a new one

RULES:
  - ALWAYS ask the user rather than guess when a field mapping is ambiguous
```

Three named error conditions, each with a message and a recovery action. Note the `WAIT` in the second one: this is the PDSL keyword that stops the turn and hands control back to the user, and it is what turns "present the candidate readings" from a suggestion into a gate.

The single `RULES` line is the general form of all three: ask rather than guess. Chapter 4 made the argument that a gate a reader has to infer is not a gate. This unit is the positive version — the gate is named, the condition that triggers it is named, and the recovery is a `WAIT`, not a hope.

9.2.6 The workflow

`workflows/request.md` is the public face. Its frontmatter is what `cfs generate-agents` turns into an invocable skill:

```
---
cf-studio: true
type: workflow
name: cf-compete-request
description: Invoke when the user asks to record a structured competitive-research request - e.g. "find
     competitors to <target>", "set up a research request for <company>", "record this competitive
     question", "cf-compete request <text or path>". Captures the eight request fields (target, decision
     question, scope, comparison frame, depth, requested output, evidence cutoff, access boundary),
     renders every omitted field as "unspecified", and produces the RESEARCH-REQUEST artifact that anchors
      all later competitive-research work.
version: 1.0
purpose: Create a structured competitive-research request (Flow 1) - the root of the traceability chain
---
```

The `name` is `cf-compete-request`: prefix, slug, workflow name, exactly as the prefix rule specifies. And the corresponding manifest entry carries no `name` field at all, confirming what Chapter 8 concluded about where public names come from:

```
[[kits.resources]]
id = "workflow_request"
kind = "skill"
source = "kits/compete/workflows/request.md"
install_path = "workflows/request.md"
type = "file"
user_modifiable = false
public = true
description = "Create a structured competitive-research request (Flow 1)"
generated_targets = ["installed"]
```

The `description` in the frontmatter deserves separate attention. It is long, and it is long on purpose: it contains four example phrasings a user might type. That description is what the router matches against when deciding which skill a request belongs to. A terse description — "creates a research request" — would route worse. This is the one place in a kit where verbosity is a feature, because the reader is a matching mechanism rather than a person.

The file's first instruction, before any content, is a delegation:

```
ALWAYS open and follow `{cf-studio-path}/.core/skills/cf-studio/SKILL.md` FIRST WHEN {cf-studio-mode} is `
     off`
```

That is the "your kit does not reimplement the engine" boundary from the previous chapter, written as the first line of the workflow.

Then the header block, the routing table, and something unusual:

```
**Type**: Intake + Structuring
**Role**: Research intake analyst
**Output**: `{artifacts_dir}/RESEARCH-REQUEST/{yyyy-mm-dd}-{slug}.md`

## Route

6 steps -> RESEARCH-REQUEST. Questions: 1 confirmation (Step 1: the restated
ask), 1 decision (Step 2: the eight request fields), 1 closing confirmation
(Step 6). Steps 3-5 run without questions. Announce the route before Step 1 and
prefix every question per the SKILL.md progress protocol.
```

The workflow declares, up front, how many steps it has and how many times it will interrupt you — and it commits to announcing that to the user before it starts. Three questions: one confirmation, one decision, one closing confirmation. Steps 3 to 5 run silently.

That is a contract about interaction, not about output. It exists because the common failure of an agentic workflow is not producing the wrong document; it is producing the right document after eleven unannounced questions, or after none when one was needed. A declared question budget makes both failures visible.

9.2.7 The six steps

Step 1 captures the raw text and checks for duplicates:

```
## Step 1: Capture the raw request

Take the request text verbatim from the user's message or the supplied file.
STOP and ask for input WHEN no request text resolves.
Check `cfs list-ids` for an existing RESEARCH-REQUEST with the same target and
decision question; when one exists, ask whether to revise it or record a new
request (**user gate - confirmation**).
```

Step 2 is the extraction, and it is the step the whole kit is built around:

```
## Step 2: Extract and confirm the eight fields

Map the raw text onto the eight request fields: target, decision question,
scope, comparison frame, depth, requested output, evidence cutoff, access
boundary. Set every field the requester did not state to the literal
"unspecified". Present the field table - including every "unspecified" - and
any wording normalizations for confirmation (**user gate - decision**).
```

"Including every `unspecified`" is the fourth appearance of the same rule. The user is shown the gaps, not just the content. A workflow that presented only the filled-in fields would let an omission pass unnoticed at the one moment when the person who could fill it in is actually present.

Step 3 authors the artifact, delegating structure to the template and procedure to the rules file. Then Step 4:

```
## Step 4: Deterministic gate

// turbo
Run: `cfs toc <artifact-file>` then `cfs validate --artifact <artifact-file>`
and `cfs validate-toc <artifact-file>`.
Fix and re-run until all report PASS.
```

Five lines, and the most important five in the file.

"Fix and re-run until all report PASS" is a loop with a machine-checkable exit condition. Nothing about it depends on the model's judgement. Either `cfs validate` reports PASS or the loop continues. Chapter 3 showed you the user-facing side of these commands; here they are as a gate inside a workflow, which is where they do the most good.

The `// turbo` marker above the commands is the annotation permitting the host tool to run them without asking each time. It is on this step and not on others, which is the right distribution: read-only validators are safe to run unattended; the steps that write files and ask questions are not.

Step 5 is the semantic pass — load the checklist, review against it, apply approved fixes, *then re-run Step 4*. That ordering matters. A semantic fix can break structural validity, so the deterministic gate runs again after it, not before.

Step 6 presents the result and waits.

9.2.8 The validation criteria, restated as a checklist

The workflow closes with its own acceptance criteria:

```
## Validation Criteria

- [ ] All eight fields present; every omitted field carries the literal "unspecified"
- [ ] Stated values trace to the original request text; normalizations were confirmed
- [ ] Section 2 states what the request cannot establish at its depth
- [ ] Section 3 preserves the verbatim request text or its path
- [ ] Section 4 carries only the default line for a fresh request
- [ ] `cfs validate --artifact` and `cfs validate-toc` report PASS
- [ ] Artifact registered in `artifacts.toml`
```

Seven checkboxes, each traceable to a rule you have now read in either the template, the rules file, or the checklist. Nothing new is introduced here; the list is a summary with a machine-checkable last two entries.

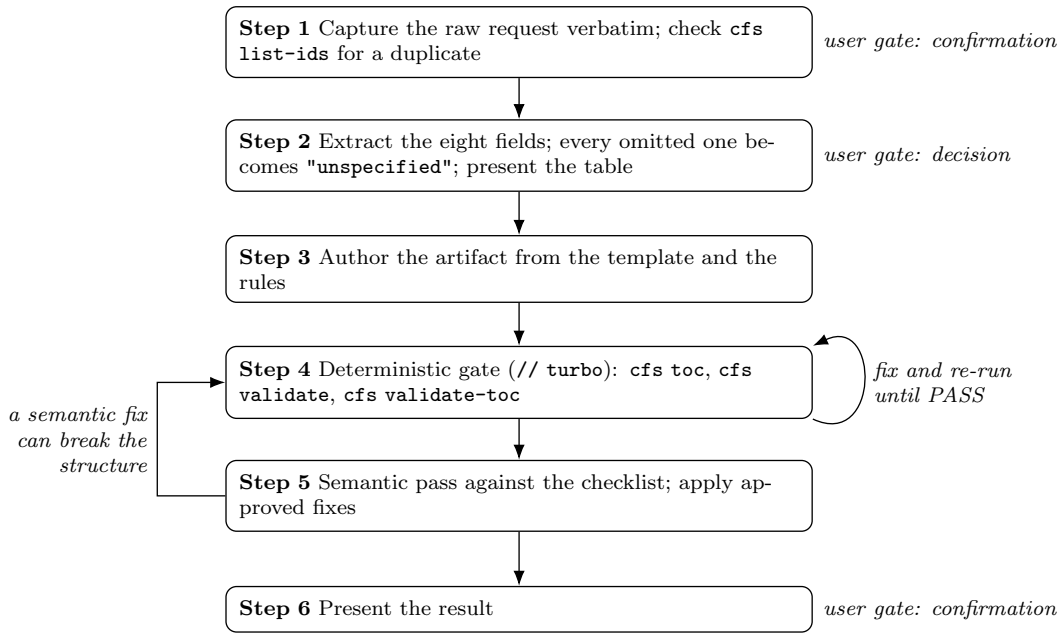


Figure 9.2: The six steps of `workflows/request.md`, and its three user gates.

9.2.9 The deterministic contract

One more file, and it is the one that makes all of the above enforceable rather than aspirational. `constraints.toml` is 37 kB, and its `RESEARCH-REQUEST` section declares five required headings. Two of them:

```
[[artifacts."RESEARCH-REQUEST".headings]]
id = "req-h1-title"
level = 1
required = true
multiple = false
numbered = false
pattern = 'Research Request\s*[...]\s*.+' # dash class: em, en, hyphen
description = "Research request title (H1): 'Research Request - {target}'."

[[artifacts."RESEARCH-REQUEST".headings]]
id = "req-fields"
level = 2
required = true
multiple = false
numbered = true
pattern = "Request Fields"
description = "The eight request fields (target, decision question, scope, comparison frame, depth,
     requested output, evidence cutoff, access boundary); omitted fields carry the literal value '
     unspecified'."
```

The first `pattern` is a regular expression, and in the source file its character class contains the three real dash characters — em dash, en dash, hyphen — which are elided above for typesetting. What it requires is that the H1 read “Research Request”, then one of those three dashes, then at least one more character. So a document titled “Research Request” with no target fails, deterministically, with no model involved.

Then the identifier declaration:

```
[artifacts."RESEARCH-REQUEST".identifiers.req]
name = "Competitive-Research Request"
description = "One structured competitive-research request: what question must be answered, at which depth
     , within which evidence boundary."
required = true
task = false
priority = false
to_code = false
```

```
template = "cpt-{system}-req-{slug}"
headings = ["req-h1-title"]
```

`to_code = false` is the field that tells you what kind of artifact this is. A FEATURE specification in the SDLC kit produces identifiers that appear in source code as CPT markers — the mechanism from Chapter 3. A research request never will. So the flag is false, and `cfs validate` does not go looking for code markers that will never exist.

And every downstream kind declares a reference back:

```
[artifacts."CANDIDATE-REGISTER".identifiers.reg.references."RESEARCH-REQUEST"]
```

Ten of those, one per downstream kind. That is "the request is the root" as a machine-checkable relation rather than a design principle. The kit's architecture guide notes the deliberate asymmetry: the request itself carries no reverse-coverage obligation, because a discovery-scan request never yields a profile, and requiring one would produce permanent validation noise.

Which is a small, sharp piece of engineering judgement. A symmetric rule would have been easier to implement and would have made the validator useless, because a validator that always reports warnings gets ignored.

9.3 Installing It

The commands, verbatim from the kit's README. From GitHub:

```
cfs kit install constructorfabric/studio-kits-pm --kit compete
cfs generate-agents
```

For local development, from a clone of the repository:

```
cfs kit install --path <path-to-this-clone> --kit compete --install-mode copy
```

Then register a system in `.cf-studio/config/artifacts.toml`:

```
[[systems]]
name = "MyResearch"
slug = "myresearch"
kit = "compete"
codebase = []
children = []
```

Three things to notice.

The `--kit compete` flag is the multi-kit selector: the repository's manifest declares kits, and this picks one. Per the repository README, it accepts one slug, a comma-separated list, or `all`, and `--version <tag-or-sha>` pins a release.

The `[[systems]]` entry is the project-side registration from Chapter 3 — the same `artifacts.toml` that told the SDLC pipeline where to put documents. Note `codebase = []`: this system has no code. A competitive-research system produces documents and nothing else, and the configuration format accommodates that without a special case.

And note that the local-development command uses `--install-mode copy`, not `register`. That looks like it contradicts Chapter 8's advice, and it does not: register mode requires the kit path to stay inside the project root after symlink resolution, and a clone of `studio-kits-pm` sitting somewhere else on your machine does not satisfy that. Copy is the correct mode for developing a kit that lives in its own repository. Register is the correct mode for a kit that lives inside the project it governs. The two pieces of advice apply to two different situations, and knowing which situation you are in is the whole skill.

9.3.1 The development gate

The kit's own checks, from its README's Development section, run from the repository root:

Script	Scope
`build_evidence_index.py`	builds the `EVD-###` index
`check_evidence.py`	checks the evidence layer
`check_artifact_refs.py`	one artifact at a time
`check_coverage.py`	a whole research run
`mine_terms.py`	term extraction
`render_matrix_deck.py`	matrix deck export
`render_pricing.py`	pricing benchmark view
`render_views.py`	csv/html exports

Table 9.3: The eight deterministic scripts shipped with the `compete` kit.

```
cfs validate-kits .
cfs kit normalize --dry-run . --json
cfs pdsl validate kits/compete/workflows/*.md kits/compete/artifacts/*/rules.md
```

Three commands, three different things checked. `validate-kits` checks the kit structure and runs the example self-checks. `kit normalize --dry-run` confirms the manifest is what the normalizer would have produced — a drift check between the hand-maintained manifest and the canonical form. `pdsl validate` lints the prompt DSL across all fourteen workflows and all eleven rules files.

That third command is the PDSL validator from Chapter 4, being used the way that chapter said it could be: not to execute the notation, but to check that it is well formed. It is the only automated check that touches the kit's prose, and it verifies syntax rather than meaning.

CI runs all three on every pull request, plus TOML parsing, script compilation, and the kit's own deterministic script checks.

9.4 The Deterministic Half

Everything so far has been text a model reads. The eight scripts under `scripts/` are the other half, and they are stdlib-only Python by policy — the repository's conventions require it, so a kit can be installed without dragging in a dependency tree.

Table 9.3 lists them.

`check_coverage.py` is the interesting one, at 496 lines, because it occupies a scope that neither `cfs validate` nor a single-artifact checker can reach. Its module docstring says so directly, and it doubles as the script's `--help` text:

```
"""Report a research run's cross-artifact coverage.

Where ``check_artifact_refs.py`` validates one artifact at a time, this
script reads the whole artifact set produced for a research request and
reports what no single-artifact check can see:
...
The output is a plain-text report for the cf-compete-review workflow; the
summary line is stable for scripting:
``check_coverage: N/9 flow(s) covered, M open next action(s), K warning(s)``

This is a report, not a gate: an open next action is research routing,
not a defect. Exit codes: 0 = report produced, 1 = usage error.
"""
```

Two sentences in there are worth more than the rest of the script.

"The summary line is stable for scripting" is a public interface declaration in a docstring. Some other program is going to grep that line, and the author is promising not to change its shape.

"This is a report, not a gate" is the more valuable one. Everything else in this kit is a gate: validators pass or fail, rules say `NEVER`, the deterministic step loops until PASS. This script deliberately is not. An open next action — a candidate in the register with no profile written yet — is research routing, not a defect. Exit code 0 either way.

Getting that distinction wrong is one of the most common ways a quality tool becomes shelfware. A checker that fails the build for something that is not wrong gets disabled within a month, and then the checks that *were* real go with it.

9.4.1 The CLI

Standard `argparse`, with the docstring as the description:

```
def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("artifacts_dir", type=Path,
                        help="directory holding the <KIND>/ artifact folders")
    parser.add_argument("--evidence-dir", type=Path, required=True)
    parser.add_argument("--companies-dir", type=Path, default=None)
    parser.add_argument("--as-of", type=str, default=None,
                        help="reference date for watch freshness (ISO; default: today)")
    args = parser.parse_args()

    if not args.artifacts_dir.is_dir():
        print(f"error: not a directory: {args.artifacts_dir}", file=sys.stderr)
        return 1
    if args.as_of and not ISO_DATE_RE.fullmatch(args.as_of):
        print(f"error: --as-of must be an ISO date: {args.as_of}", file=sys.stderr)
        return 1
    as_of = (dt.date.fromisoformat(args.as_of) if args.as_of else dt.date.today())
```

The `--as-of` option is a detail worth copying. Watch-report freshness depends on today's date, which would make the script's output non-reproducible — and therefore untestable. An injectable reference date makes the whole thing deterministic under test, and the kit's hermetic `examples/sample-evidence/` set is what it gets pointed at in CI.

9.4.2 One coverage check, in full

Here is the evidence-density section of the report, which is a good example of the script's whole style:

```
    # -- Evidence density ------------------------------------------
    print("EVIDENCE DENSITY")
    floors = {"ORIENTATION-CARD": 1, "COMPANY-PROFILE": 2, "COMPANY-DOSSIER": 4}
    for kind, floor in floors.items():
        for artifact in artifacts.get(kind, []):
            evds = {e for e in evd_tokens(artifact.text) if e in evd_index}
            classes = source_classes(evds, evd_index)
            line = (f" {artifact.label}: {len(evds)} distinct EVD, "
                    f"classes: {', '.join(classes) if classes else 'none'}")
            if len(evds) < floor:
                line += f" - WARNING: below the soft floor of {floor} for {kind}"
                warnings += 1
            print(line)
```

Read the `floors` dictionary. An orientation card needs one distinct evidence record, a standard profile two, a deep dossier four. Those numbers encode a piece of professional judgement about how much sourcing each depth of research requires, and they encode it as data rather than as prose in a checklist.

They are also called *soft* floors, and the code means it: below the floor produces a `WARNING` and increments a counter, and the script still returns 0. A dossier with three evidence records is thin, not invalid.

Two other details. `evds` is filtered by `if e in evd_index` — an `EVD-###` token mentioned in an artifact but absent from the evidence index does not count toward density. Citing a

record that does not exist earns you nothing. And the report prints the *source classes* the records cover, not just the count, so a profile citing four extracts from the same pricing page is visibly different from one citing four different classes of source. That distinction is the point of the whole evidence-depth contract, and it is one line of output.

9.4.3 Where a checker beats a rule

One more excerpt, because it demonstrates something no amount of prose in a rules file could achieve. The script has to decide whether a register's recommended next action — "Orientation card; verify open-source posture" — has actually been carried out. So it needs to know whether some artifact is "about" a named company. And it draws a distinction:

```
def matches(self, tokens: set[str]) -> bool:
    """Whether this artifact is about the named subject.

    Single-subject kinds (card, profile, dossier) match on their title
    and frontmatter company only - a passing mention in the body must
    not count as fulfilment. Multi-subject kinds (matrix, OSS review,
    mechanism study, watch) cover several subjects in their tables, so
    the whole text is searched with word boundaries.
    """
    if self.kind in SUBJECT_KINDS:
        haystack = (self.title + "\n"
                    + self.front.get("company", "")).lower()
    else:
        haystack = self.text.lower()
    return any(re.search(rf"\b{re.escape(t)}\b", haystack)
               for t in tokens)
```

"A passing mention in the body must not count as fulfilment." A company profile that happens to name a competitor in one sentence is not a profile *of* that competitor, so single-subject kinds are matched on title and frontmatter only. Multi-subject kinds legitimately cover several companies in their tables, so they are searched full-text.

You could write that rule in prose and ask a model to apply it. It would mostly work. Written as eight lines of Python with a stopword list guarding against generic tokens like "platform" and "framework", it works the same way every time, and its failure modes are inspectable.

That is the honest division of labour in this kit, and it is the answer to the question Part II kept circling. Prose for judgement — is this decision question specific enough, is this boundary statement appropriate to the depth. Code for counting, matching, and comparing. The eight scripts are not a concession that the prose layer failed; they are the parts of the job that were never a language problem.

9.5 What to Take From This

The pattern to imitate is smaller than the kit. Strip `compete` down and what remains is:

- one directory per artifact kind, holding four files with four distinct jobs;
- a rules file in PDSL that loads everything by resource identifier and never by path;
- prohibitions with stable identifiers and severities, shared between the rules file and the checklist;
- a deterministic gate that loops until `cfs validate` passes;
- a constraints entry declaring the headings and the identifier pattern;
- scripts for the checks that are counting problems rather than language problems, reporting rather than gating when the finding is not a defect.

The three files worth reading again before you write anything of your own are `artifacts/RESEARCH-REQUEST/template.md`, `artifacts/RESEARCH-REQUEST/rules.md`, and `workflows/request.md`. Between them they are under 14 kB of markdown, and they contain a complete, working, shipped answer to the question "what does one artifact kind consist of."

That is the set Chapter 10 asks you to imitate. Not to copy — your domain is not competitive research, and a template full of somebody else's field names would be worse than nothing. To imitate: the same four files, the same separation of jobs, the same discipline of putting each rule at the one place it can be violated.

Exercises

1. **Verify the counts and find the drift.** Independently count the directories under `kits/compete/artifacts/`, the files under `kits/compete/workflows/`, and the Python files under `kits/compete/scripts/`. Cross-check each count against two other sources: the artifact and workflow lists in `kits/compete/SKILL.md`'s frontmatter `description`, and the flow table in `kits/compete/README.md`. Report every disagreement you find, and for each one state which source you would trust and why. Then propose the smallest change that would make one of the disagreeing sources impossible to get wrong again.

2. **Trace one rule through four files.** Take the rule "an omitted request field is recorded as the literal `unspecified`, never defaulted." Find every place it is expressed: in `artifacts/RESEARCH-REQUEST/template.md`, in `artifacts/RESEARCH-REQUEST/rules.md`, in `artifacts/RESEARCH-REQUEST/checklist.md`, and in `workflows/request.md`. Quote each occurrence. Then, for each, name the specific moment in the process at which that expression of the rule would catch a violation the others would miss. Finally, identify which of the four occurrences a purely deterministic check could enforce and which could not, and say what that tells you about where the kit's real guarantees come from.

3. **Read the second half of a script.** Open `kits/compete/scripts/check_coverage.py` and read the register next-action section of `main` — the block that classifies each candidate row as `done`, `partial`, `open`, or `MANUAL`. Write down, in your own words, exactly what makes a row `partial` rather than `open`. Then find the `ACTION_KINDS` table and explain why it is checked in order with every match collected rather than stopping at the first hit. Construct one register row that would be classified `MANUAL` and one that would be classified `partial`, and say which of the two is a better outcome for a reviewer.

4. **Install it and produce one artifact.** Install the `compete` kit into a scratch project using the commands from its README, register a system in `artifacts.toml`, and run `cf-compete-request` with a deliberately sparse ask — a target and nothing else. Record which of the eight fields came back as `unspecified`, whether the workflow announced its route before Step 1, and how many times it asked you a question. Compare the question count against the three declared in the workflow's `Route` section. Then run `cfs validate --artifact` on the result and report the output.

5. **Break the deterministic contract on purpose.** Take the request artifact from Exercise 4 and make four separate edits, reverting each before the next: rename the H1 to just "Research Request"; delete the `## 4. Traceability` heading; change the ID to `cpt-myresearch-request-something`; add the word `TODO` to Section 2. Run `cfs validate --artifact` after each and record the exact message. Then map each failure

back to the specific lines in `kits/compete/constraints.toml` or `artifacts/RESEARCH-REQUEST/rules.md` that caused it. Which of the four was *not* caught deterministically, and what would have caught it?

6. **Judge the directory declaration.** The kit declares `scripts/` as a single `kind = "directory"` resource whose description names two of its eight files. Write the eight individual `kind = "script"` entries that would replace it, with an accurate `description` and a considered `user_modifiable` for each. Then argue the case both ways in one paragraph each: what the kit gains from the per-file version, and what it gains from the current one. State which you would ship and what would change your mind.

7. **Find the boundary between rules and scripts.** The kit's architecture guide says `cfs` "cannot see table rows," which is why the scripts exist. Pick three checks the kit currently performs — one by `cfs validate` via `constraints.toml`, one by a PDSL `NEVER` rule, one by a Python script — and for each, argue whether it is in the right place. Then find one check you believe is in the wrong place and say where it should move and what it would cost to move it.

Chapter 10

Building Your Own Kit

You now have the mechanism from Chapter 8 and one worked example from Chapter 9. What is left is the part no chapter can do for you: deciding what your kit should contain.

This chapter is therefore mostly exercises. The text between them is a decision procedure — eight steps, in the order you will actually face them — and the exercises at the end are the chapter's real content. Work them on a kit you would genuinely use. A kit built for a process you do not have is an exercise in TOML syntax, and you already did that one.

10.1 Step 0: Decide Whether You Need a Kit

The first decision is whether to build anything.

Chapter 1 listed the cases where Constructor Fabric itself is overhead. The same honesty applies here, and the questions are narrower:

- **Is the gap in structure or in content?** If your PRDs are fine but people write them badly, you need a better checklist, not a kit — and `user_modifiable = true` on the shipped checklist means you can edit it in place.
- **Is it one heading or one document kind?** One extra required heading in an existing artifact is a constraints change. A document kind the SDLC kit has no concept of is a kit.
- **Does more than one repository need it?** If not, and if the change is small, a local edit may genuinely be the right answer for another six months.
- **Is anyone going to maintain it?** A kit is a process encoded in files. Files rot. If nobody owns the kit, you have moved the problem rather than solved it.

Two signals mean you have crossed the line and should build a kit. The first is a document kind that does not exist in any shipped kit — a threat model, a data-protection impact assessment, a customer-visible change notice, a research request. The second is a rule your reviewers enforce that nobody has written down. Chapter 8 called that the process that lives in heads, and it is the single best reason to start.

In practice the single most common trigger is narrower than either signal on its own: you already have a specific, bounded set of requirements — a conference's submission rules, a client's compliance checklist, a house style — and you want them enforced the way the SDLC kit enforces its own. That is a legitimate reason by itself, and it is the case "A paper-writing kit," later in this chapter, works through in full.

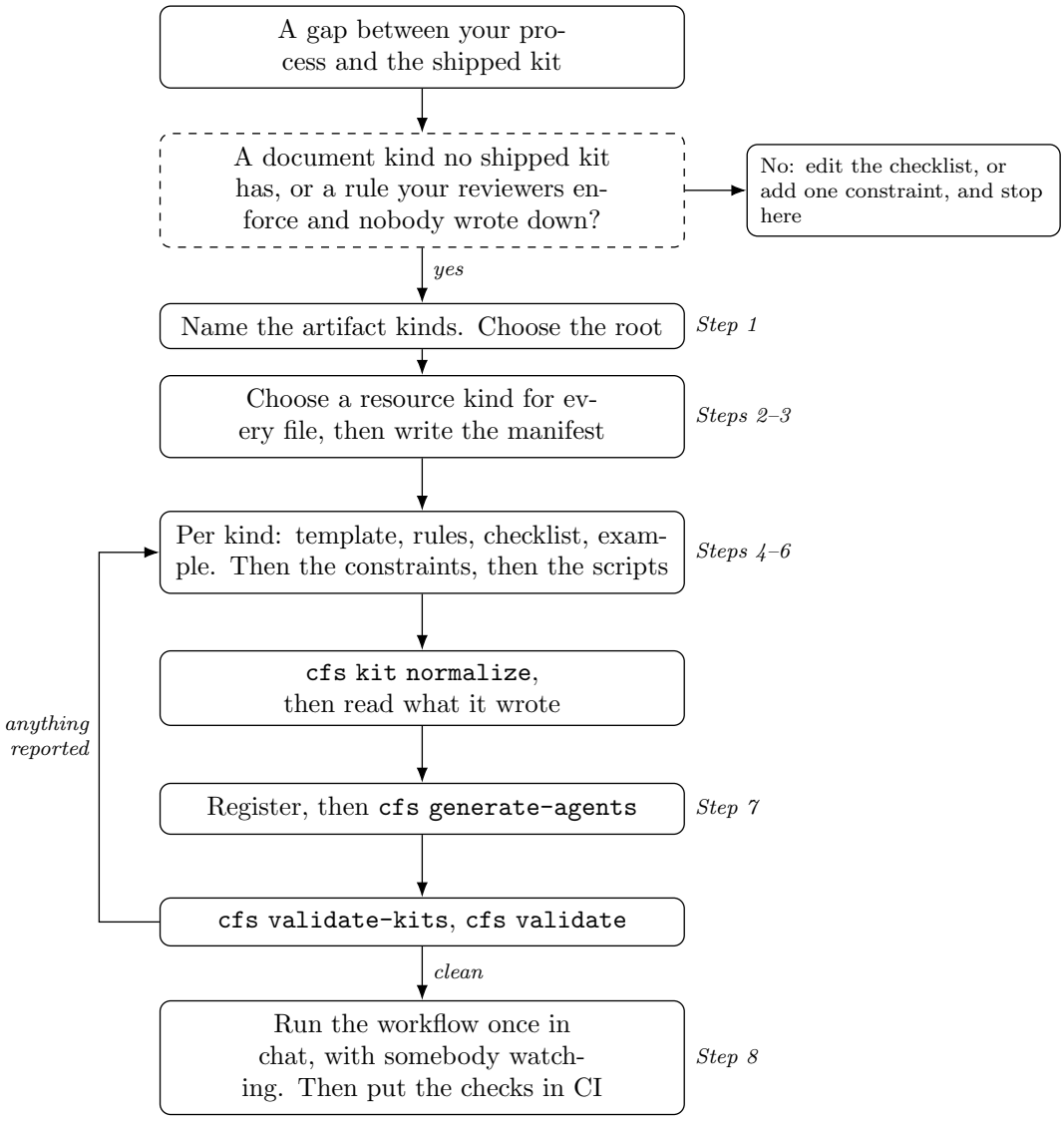


Figure 10.1: Planning a kit, from the gap to the checks in CI.

You are declaring	Use
the shape of a document	`template`
what a reviewer verifies	`checklist`
how the author must proceed	`other`
a filled-in reference document	`other`
a workflow the user invokes	`skill`
the kit's entry point	`skill`
required headings and ID patterns	`constraints`
your `AGENTS.md` source	`rule`
a folder of checkers or guides	`directory`
one checker or its config	`script`
a configured worker	`agent`

Table 10.1: Choosing a resource kind for a file you want to declare.

10.2 Step 1: Name the Artifact Kinds

An artifact kind is a document type with its own structure, its own review criteria, and its own identifier segment. Naming them is the design decision that everything else follows from, and it is worth being slow about.

Three tests, in order.

Can you name the identifier segment? The `compete` kit has eleven, and every one of them is a short lowercase token in a CPT pattern: `req`, `reg`, `ori`, `prof`, `dos`, `cmp`, `oss`, `mech`, `wat`, `rev`, `tax`. If you cannot pick a segment for your kind without ambiguity, the kind is probably two kinds, or half of one.

Does it have a distinct reviewer? If the same person, with the same expertise, reviews two of your proposed kinds against the same criteria, they are one kind with two templates at most.

Does it have a distinct claim boundary? This is the test Chapter 9 taught, and it is the strongest of the three. A `RESEARCH-REQUEST` establishes a question; an `ORIENTATION-CARD` establishes a first read on one company; a `COMPANY-DOSSIER` establishes a sourced account. Those are genuinely different in what they license a reader to conclude, which is why they are three kinds rather than three depths of one. If two of your proposed kinds would carry the same boundary statement, merge them.

Then write down, before touching a manifest, which kind is the *root*. The `compete` kit's answer is stated as a design principle — the request is the root — and it determined the shape of ten `references` entries in `constraints.toml`. The SDLC kit's root is the PRD. Your kit needs one too, and choosing it late is expensive.

10.3 Step 2: Choose the Resource Kinds

With the artifact kinds named, the resource kinds mostly follow. Table 10.1 is the mapping, derived from what both shipped kits actually do.

Two rows in that table surprise people, so justify them.

Rules files are `other`. The `rule` kind is a public kind, and a per-artifact `rules.md` is not public — it is loaded by identifier from a workflow. Both shipped kits declare every `rules.md` as `other`, and reserve `rule` for the `AGENTS.md` source. That is not a workaround; it is the taxonomy working as designed, and it is why Chapter 8 called `other` an honest escape hatch rather than a dumping ground.

Examples are `other` too. A filled-in reference document has no schema and no named kind. It is a file your workflow reads. Say so.

You will probably not need `agent` on a first kit. Neither shipped kit in this book's examples declares one, and getting a kit's document flow right is enough work for a first version.

10.4 Step 3: Write the Manifest

Start from the smallest legal kit and grow it. Here is the shape again, because this is the moment you need it:

```
manifest_version = "1.0"

[[kits]]
slug = "..."
name = "..."
version = "0.1"

[[kits.resources]]
id = "..."
kind = "..."
source = "..."
```

Four decisions before you type anything.

The slug. Short, lowercase, hyphenated. It goes into every public name your kit ever publishes, so `cf-<slug>-<workflow>` is the string you are committing to. Say it out loud. If it is annoying to say, it will be annoying to type for the next three years.

One kit or several. If you have one domain, one `[[kits]]` block and a manifest at the kit root. If you are building a family — the `studio-kits-pm` case from Chapter 9 — one canonical manifest at the repository root, one `[[kits]]` block per kit, with `source` paths repository-relative and `install_path` values project-relative. Do not start with the multi-kit shape unless you already have the second kit.

Identifier convention. Underscores for resource identifiers, hyphens for public names. `research_request_template` against `cf-compete-request`. Adopt this on line one and you will never wonder which namespace a string belongs to.

`user_modifiable`, per resource. Templates and checklists `true`; generated entry points, the constraints contract, and workflow files `false`. Both shipped kits follow that split. Decide it deliberately, because it is a promise about what survives an update.

Then let the normalizer do the tedious part:

```
$ cfs kit normalize ./kits/<your-kit>
```

It reads your layout, classifies candidate resources, and synthesizes a preview. Discovery is read-only; nothing is written until you approve. And then read what it wrote — the file's own first line tells you to. Automatic classification is a good guess informed by your folder names, and a guess is what it is.

10.5 Step 4: The Four Files, Per Artifact Kind

For each artifact kind, four files with four distinct jobs. This is the pattern Chapter 9 recommended imitating, and here is what each file has to accomplish.

10.5.1 `template.md`

Frontmatter first, with enumerated values written into the placeholders. Then the H1 carrying the identifier line, then the sections.

Four things to copy from `RESEARCH-REQUEST`'s template:

- **Put length expectations in the prompts.** "1–2 paragraphs" is doing real work; without it, the same heading produces a sentence from one author and two pages from another.

- **Put the rule where it can be violated.** The eight-field table repeats "or `unspecified`" eight times, once per cell, because the cell is where the decision happens. A rule stated only in a rules file is read once, early, competing with everything else in the context window.

- **Include a boundary section if your artifact makes claims.** A required section stating what the document does *not* establish is rare in document templates and is the reason findings get quoted out of scope.

- **Start from a document you already have, not a blank page.** If the gap you are filling is a specific set of requirements — a conference's format, a client's checklist — you almost certainly already have that requirement written down somewhere: an email, a call for papers, a contract. Draft the template around that document's actual structure, then generalise it. A template invented from nothing tends to reproduce what the author remembers rather than what the requirement actually says.

And one structural trick: use HTML comments for format documentation that should survive into the authored artifact. The `RESEARCH-REQUEST` template ships its ten back-link forms in a comment, so the next workflow to append a back-link finds the expected shape waiting for it, without the shapes being content.

10.5.2 `rules.md`

PDSL, in fenced `pdsl` blocks, one `UNIT` per responsibility. The five from Chapter 9 are a good default set: authoring, omissions, validation, error handling, next steps.

Load everything by resource identifier. Never write a path. That single discipline is what lets you reorganise the kit on disk without editing prose.

Give your prohibitions stable identifiers and severities — `CPT-REQ-NO-001 (CRITICAL)` — and use the same identifiers in the checklist. A review finding and the authoring rule that should have prevented it then share a name, and both are greppable.

And include the self-denying rule that both shipped kits include: never duplicate the checklist's semantic criteria here. The four-file split only works if something actively prevents the files from growing into each other.

10.5.3 `checklist.md`

Semantic criteria only, organised into MUST HAVE and MUST NOT HAVE, with a severity dictionary at the top so that "CRITICAL" means one thing throughout.

Two features of the `RESEARCH-REQUEST` checklist are worth stealing outright.

The first is an *applicability context*, which states when sparseness is not a defect and instructs the reviewer to mark a non-applicable check `N/A` with a one-line justification rather than skipping it silently. A checklist without this gets one of two failure modes: reviewers flagging legitimate omissions, or reviewers quietly ignoring half the items.

The second is a reporting contract: report *only* problems, never a list of what is fine, and give each issue five parts — why applicable, the issue, the evidence quoted from the artifact, why it matters, and a concrete proposal with acceptance criteria. Then a fixed report format. Reviews become comparable across artifacts and across months, which is the entire point of writing the checklist down.

10.5.4 `examples/example.md`

One filled-in artifact, at the depth you actually expect. Its job is to answer "how deep is deep enough," which no amount of prose in a template can settle.

Make it hermetic if anything checks it. The `compete` kit's `examples/sample-evidence/` exists so that CI can run the kit's own scripts against a fixed input and get the same answer every time. An example that depends on live data is not an example, it is a flaky test.

10.6 Step 5: Constraints — What a Machine Can Check

`constraints.toml` is where prose becomes enforceable. For each artifact kind you declare the required headings and the identifier pattern.

A heading entry, with the fields that matter:

```
[[artifacts.<KIND>.headings]]
id = "..."
level = 2
required = true
multiple = false
numbered = true
pattern = "..."
description = "..."
```

The `pattern` may be a literal string or a regular expression, so you can require not just that a heading exists but that it carries content — the `RESEARCH-REQUEST` H1 requires a target after the dash, which is how a generically titled document fails without a model being consulted.

An identifier entry declares the CPT pattern and the flags:

```
[artifacts.<KIND>.identifiers.<seg>]
name = "..."
description = "..."
required = true
task = false
priority = false
to_code = false
template = "cpt-{system}-<seg>-{slug}"
headings = ["..."]
```

`to_code` is the flag to think hardest about. Set it true and `cfs validate` will expect the identifier to reach source code as a CPT marker — the mechanism from Chapter 3. Set it false for artifacts that never touch code, as `compete` does throughout. Getting this wrong produces either missing traceability or permanent false warnings.

Then the cross-references. If your kit has a root artifact, every downstream kind should declare a reference back to it:

```
[artifacts.<DOWNSTREAM>.identifiers.<seg>.references."<ROOT>"]
```

And — this is the judgement call — do *not* make it symmetric. The `compete` kit deliberately gives the root no reverse-coverage obligation, because a discovery-scan request never yields a profile and requiring one would produce permanent validation noise. A validator that always warns is a validator that gets ignored, and then the real findings go with it.

Finally, wire the artifact bindings, which live on the constraints resource and nowhere else:

```
[kits.resources.artifacts]
[kits.resources.artifacts.<KIND>]
template = "<kind>_template"
examples = "<kind>_example"
```

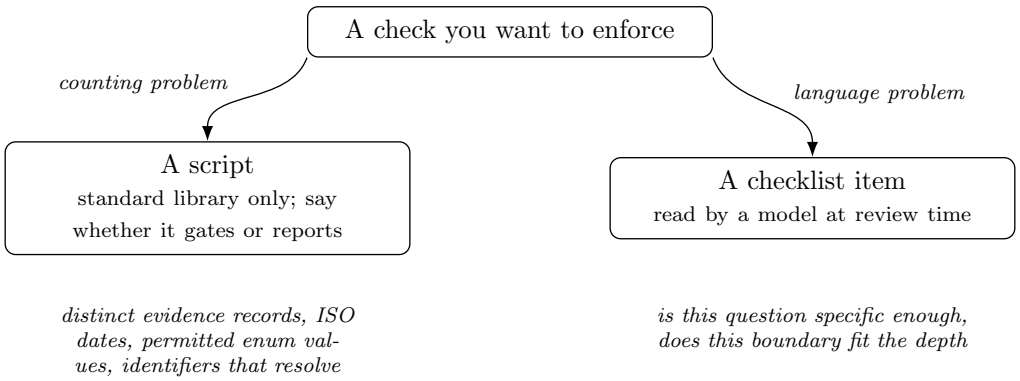


Figure 10.2: Counting problems go in a script; language problems go in the checklist.

10.7 Step 6: Scripts — What Only a Script Can Check

The test for whether something belongs in a script is not difficulty. It is whether the check is a *counting problem* or a *language problem*.

Counting problems: how many distinct evidence records does this artifact cite, do all the identifiers in this table exist, is every date an ISO date, does every enum value come from the permitted vocabulary, how many rows does this matrix have. Chapter 9's evidence-density check is the canonical shape — a dictionary of thresholds and a loop.

Language problems: is this decision question specific enough, is this boundary statement appropriate to the depth, does this rationale actually justify the decision. Those go in the checklist, and a model reads them.

Three rules for the scripts themselves, all visible in the `compete` kit's practice.

Standard library only. The repository requires it, and the reason is that a kit must be installable without dragging in a dependency tree. A kit that needs `pip install` to validate is a kit some team will not adopt.

Injectable reference date. `check_coverage.py` takes `--as-of` precisely so that any check depending on today's date is reproducible under test. Anything reading the clock is untestable until you make the clock an argument.

Decide gate or report, and say which. The distinction is worth more than any individual check. `check_coverage.py` declares itself in its own docstring — "This is a report, not a gate" — and returns exit code 0 even with warnings, because an unexecuted research step is routing rather than a defect. A checker that fails the build for something that is not wrong gets disabled within a month.

And give your report a stable summary line, as that script does, so something else can grep it.

10.8 Step 7: Register, Generate, Validate

The flow from Chapter 8, applied to your kit:

```
$ cfs kit normalize ./kits/<your-kit>
$ less ./kits/<your-kit>/.cf-studio-kit.toml
$ cfs kit install --path ./kits/<your-kit> --install-mode register
$ cfs generate-agents
$ cfs validate-kits
$ cfs validate
```

Register mode, because the kit lives inside the project it governs. If instead you are developing a kit in its own repository — the `studio-kits-pm` situation — copy mode is correct, because register mode requires the kit path to stay inside the project root after symlink resolution.

Expect to run `cfs generate-agents` and forget it at least once. Your workflow will appear not to exist, and the fix will be one command.

Then the part the deterministic checks cannot do. Run your workflow once, in chat, and confirm four things:

- the route captures the target before discovery;
- optional explore or brainstorm is offered where appropriate;
- a plan is requested before substantive multi-step work;
- review findings are shown before any fix is approved.

A validator will confirm that your workflow file parses. It will never notice that the workflow starts writing before it asks its clarifying question. Only running it will.

While you are there, borrow the `compete` kit's route declaration and check it against reality. That workflow announces "6 steps, three questions" before Step 1. Count how many questions yours actually asks. If the number differs from the number you declared, one of the two is wrong, and finding out which is a five-minute job that saves a user's afternoon.

10.9 Step 8: Ship It, Then Add the Checks

Once it works, put the checks in continuous integration on day one rather than on the day you first need them. The `studio-kits-pm` gate is a reasonable minimum:

```
$ cfs validate-kits .
$ cfs kit normalize --dry-run . --json
$ cfs pdsl validate kits/<slug>/workflows/*.md kits/<slug>/artifacts/*/rules.md
```

The middle command is the one people do not think of, and it is a drift check: it confirms your hand-maintained manifest is what the normalizer would have produced. The third lints your PDSL — syntax, not meaning, which is what Chapter 4 established is all a PDSL validator can do. Add TOML parsing, script compilation, and your own deterministic checks alongside them.

Bump `version` in the manifest when the kit changes. And keep exactly one authoritative copy of that version number. Chapter 9 showed a kit whose manifest and `conf.toml` agreed on `1.15.0` while its README still said `1.14.0` — the two machine-read copies stayed in step and the prose copy drifted, which is what always happens.

If the kit you end up with turns out to be general enough that it is not just yours — a house style for a common venue, a compliance pattern more than one team needs — it is worth publishing as its own repository, the way `studio-kit-sdlc` and `studio-kits-pm` are, rather than leaving it buried inside one project. Chapter 8 showed how a kit installs straight from a Git remote for exactly this reason.

10.10 Two Design Briefs Worth Trying

Neither of these exists. They are thought experiments, offered because they exercise different parts of the mechanism, and either would make a reasonable subject for the exercises below.

10.10.1 A paper-writing kit

Academic writing is an obvious thing to want a kit for. The process is document-heavy, the review criteria are real and mostly unwritten, and the failure modes are consistent enough that a checklist would catch them.

The design questions are the interesting part, and they are all Step 1 questions. Is a paper one artifact kind or several — outline, draft, submission, response to reviewers? What is the root: the research question, or the venue's call for papers? Which of those has a claim boundary distinct from the others? Does a related-work section need its own evidence-locator discipline, the way the `compete` kit's claims do, and if so is that a constraints problem or a script problem? What does `to_code` mean for an artifact that never reaches source code, and what replaces the code-marker end of traceability — a figure, a dataset, an analysis script?

And the sharpest one: what would your kit's equivalent of "never silently default" be? Every kit that is worth building has one rule of that shape, and finding it is the design work.

10.10.2 A house overlay on the SDLC kit

The other common case is not a new domain but a stricter version of an existing one. Your organisation uses PRDs, ADRs, DESIGN, DECOMPOSITION, and FEATURE exactly as Chapter 3 showed — and additionally requires a security sign-off section in every DESIGN, forbids ADRs without at least two rejected options, and mandates a data-classification field in every FEATURE.

This brief is harder than it looks, and the difficulty is the lesson. None of those three requirements is a new artifact kind. All three are constraints and checklist items on kinds somebody else owns. So: do you add a kit alongside the SDLC kit, edit the SDLC kit's `user_modifiable` resources in place, or fork its templates into your own kit and accept that you now maintain them? Chapter 4's four levels of extensibility — kit, artifact, resource, placement — are the vocabulary for answering that, and the answer is not the same for all three requirements.

10.11 Failure Modes to Expect

Six, roughly in the order people hit them.

1. **The workflow does not exist.** You forgot `cfs generate-agents`. In a workspace, you forgot it in one repository.

2. **The normalizer's guess shipped.** You ran `cfs kit normalize` and did not read the output, so a checklist is declared as `other` and a template is `user_modifiable = false`.

3. **A name collision you cannot see.** Your workflow and a shipped one both answer to something similar, and the agent picks the wrong one. It looks like it worked. Check the published names, and do not opt out of the prefix.

4. **The checklist and the rules file diverge.** You put a semantic criterion in `rules.md` because it was convenient, and six months later the two files disagree. This is why both shipped kits carry an explicit `NEVER` against it.

5. **The validator always warns.** You made a cross-reference obligation symmetric, or set `to_code = true` on an artifact that never reaches code. Within a month nobody reads the output.

6. **The kit works and nobody uses it.** The workflow asks eleven unannounced questions, or the template's prompts do not say how long a section should be, so authors guess and reviewers argue. Both are caught by running it once with somebody watching.

10.12 Where That Leaves Us

Three chapters ago, extending this platform meant reading a manifest format. What it actually means is this: naming the document kinds your organisation really has, writing down the rules your reviewers already enforce, deciding which of those rules a machine can check and which need judgement, and putting each rule at the one place it can be violated.

The mechanism is small. `.cf-studio-kit.toml`, nine resource kinds, five commands. That is deliberate — Chapter 4 showed fifteen hundred lines of three-way merge logic being replaced by `diff` because somebody asked whether the expensive mechanism was answering a question anyone had. The kit model is what survived that question.

What is not small is the content. Eleven artifact kinds, forty-four files, and eight scripts is what one team's professional discipline looks like when it is written down instead of remembered. That is the work, and it is worth doing for the same reason the opening of Chapter 8 gave: a process that lives in heads dies when those heads leave.

Exercises

These are the chapter. Work them on a real process — your team's, your research group's, your own — and the result will be a kit you keep.

1. **Write the design brief before any code.** Pick a domain where you have a real, currently-undocumented process. On one page, answer: what are the artifact kinds, what is the identifier segment for each, which one is the root, and what is the claim boundary for each kind stated in one sentence. Then apply the three tests from Step 1 to your own list and delete or merge anything that fails. Report how many kinds you started with, how many survived, and which test did the killing. If nothing was merged or deleted, you were not strict enough — run the claim-boundary test again.

2. **Find your "never silently default" rule.** Every kit worth building has one rule of that shape: a discipline that a careful practitioner in your domain follows and that a careless one violates invisibly. Write yours down in one sentence. Then write it three more times: as placeholder text inside a template cell, as an `ALWAYS` line in a PDSL `RULES` block, and as a MUST NOT HAVE item with a stable identifier and a severity. Explain, for each of the three, which failure it catches that the other two would miss.

3. **Write the manifest.** For the kit you designed in Exercise 1, write a real `.cf-studio-kit.toml` — or, if you are building a family, one `[[kits]]` block in a repository-root manifest plus a per-kit `conf.toml`, as Chapter 9 showed. Declare every resource with `id`, `kind`, `source`, and a real `description`. Make a deliberate decision on `user_modifiable` for each and justify the `false` ones in one line each. Then tabulate your resources by kind, as Chapter 9 did for `compete`, and say which of the nine kinds you did not use and why.

4. **Draft the template and the rules for one kind.** Take your root artifact kind and write `template.md` and `rules.md` in full, imitating `artifacts/RESEARCH-REQUEST/template.md` and `artifacts/RESEARCH-REQUEST/rules.md`. The template must have frontmatter with enumerated values in its placeholders, an identifier line, and at least one section that states what the artifact does not establish. The rules file must have at least three PDSL units, must load every resource by identifier and never by path, and must carry at least three `NEVER` rules with stable identifiers and severities. Keep the template under 100 lines.

5. **Normalize, register, and fix what breaks.** Put your kit at `kits/<slug>/` in a scratch project. Run `cfs kit normalize ./kits/<slug>` and diff its output against the manifest you hand-wrote in Exercise 3 — record every disagreement and decide, one at a time, which of you is right. Then run the full flow: `cfs kit install --path ./kits/<slug> --install-mode register`, `cfs generate-agents`, `cfs validate-kits`, `cfs validate`. Fix every reported problem and record the message and the fix for each. Report how many iterations it took to get a clean run.

6. **Write the constraints and prove they bite.** Add a `constraints.toml` declaring, for your root kind, every required heading and the identifier pattern — including one heading whose `pattern` is a regular expression requiring content, not just a title. Set `to_code` deliberately and justify the value. Then author one artifact of your kind and break it four ways, reverting each time: remove a required heading, malform the identifier, satisfy the regex heading with a bare title, insert a `TODO`. Record which breaks `cfs validate` catches and which it does not, and for each miss, say what would have caught it.

7. **Write one deterministic checker.** Identify one check in your domain that is a counting problem rather than a language problem. Write it as a standard-library-only Python script with an `argparse` CLI, a module docstring that doubles as `--help`, a stable one-line summary at the end of its output, and an explicit decision — stated in the docstring — about whether it is a gate or a report. If any part of it depends on the current date, make the date an argument. Then run it against your example artifact and paste the output. Finally, argue in one paragraph why this check could not have been a checklist item.

8. **Run it with somebody watching.** Write the workflow file for your root kind, including a `Route` section that declares the number of steps and the number and type of questions, as `workflows/request.md` does. Register the kit and run the workflow in chat, with a colleague reading over your shoulder. Record: whether the route was announced before the first step, how many questions were actually asked against how many you declared, whether the deterministic gate looped, and every point at which your colleague was confused. Then make the smallest change to the kit that fixes the worst of those — and say why a validator could not have found it.

9. **Decide the extension level for three real requirements.** Take the house-overlay brief from this chapter, or three equivalent requirements from your own organisation. For each, decide which of Chapter 4's four extensibility levels applies — kit, artifact, resource, or placement — and what you would actually do. Then state, for each, what you would be committing to maintain for the next two years. If any of the three leads you to fork a shipped template, say what you will do when that template changes upstream.

Chapter 11

Mental Models and the Shape of a Kit

Everything up to here has been about software. The pipeline in Chapter 3 produces a PRD, an ADR, a DESIGN, a DECOMPOSITION, and a FEATURE specification, and it ends in code with markers in it. The kit mechanism in Chapter 8 exists so that a team can change that process without forking the tool.

This chapter asks a question the earlier chapters never needed to ask: why does that structure work, and is the reason specific to software?

The short answer is that it is not. The problem the kit structure solves is not a documentation problem. It is a problem about what is in people's heads, and that problem has a name, a literature, and a history going back to 1943. Once you can see the structure that way, it becomes reasonable to ask whether the same machinery would help with other things people build together out of thinking — a paper, a grant proposal — and what exactly would have to change.

That is the whole content of this chapter: the theory, the argument, and two worked adaptations. It is the one part of this book that describes something nobody has built yet, and the last section says so plainly.

11.1 Three Books About What Is In Your Head

Three works set up everything that follows. None of them is about software.

11.1.1 Craik: the mind runs a model

Kenneth Craik published *The Nature of Explanation* in 1943.[1] It is a short book, and its central idea is now so widely absorbed that it is easy to miss how strange it was at the time.

Craik's proposal was this. When you think about the world, you are not operating on the world. You are operating on a small-scale model of it that you carry around inside your head. The model is built from experience, it is made of whatever material brains are made of, and its purpose is prediction. You run the model forward, see what happens in the model, and act on the result. The alternative — try the action in reality and find out — is slower and sometimes fatal.

His own comparison was to engineering. An engineer who wants to know whether a bridge design will stand up does not build the bridge and watch. They build something that behaves like the bridge in the respects that matter, and they load that instead. Craik's claim was that thought is that, done internally: the nervous system builds a working model of a bit of the world and uses it to anticipate.

[1] Kenneth Craik, *The Nature of Explanation*. Cambridge: Cambridge University Press, 1943.

Two consequences matter for us, and both are consequences Craik himself drew.

First, the model is not the world. It is a reduced stand-in, kept because it is cheap and useful, not because it is complete. A model is built to be adequate for some purpose and it is silent about everything outside that purpose.

Second, and this is the part that engineers find intuitive: a model can be wrong, and the wrongness is invisible from the inside. Running a wrong model feels exactly like running a right one. You get an answer either way. Reality is the only thing that tells the difference, and it usually tells you late.

11.1.2 Norman: your model and my model are not the same model

Forty years later, Dedre Gentner and Albert Stevens edited a collected volume called *Mental Models*, and Donald Norman's chapter in it is the one that turned Craik's idea into a design problem.[2]

Norman's chapter separates two things that ordinary usage runs together. There is the system itself — the calculator, the thermostat, the codebase. There is the *conceptual model* of that system: an explicit, deliberately constructed account of how it works, invented by somebody in order to be taught, written down, and argued about. And there is the *mental model* that an individual person actually holds: the private thing in one head, assembled out of use, guesswork, half-remembered explanations, and analogy to other systems.

The conceptual model is public and can be inspected. The mental model is private and cannot.

Norman then says what the private ones are actually like, based on watching people use things. They are incomplete: people know the parts they need and nothing about the rest. They are unstable: details are forgotten, especially details that have not been used for a while. Their boundaries are fuzzy, so mechanisms from one system get borrowed into another that merely looks similar. They tolerate superstition: if a redundant extra step is cheap, people keep doing it rather than find out whether it is necessary. And they are parsimonious in a way that looks lazy but is rational — people would rather perform an extra physical action than work out in advance whether that action is needed.

None of that is a complaint about users. It is a description of what understanding is like when it is left to form on its own. And it is the reason the distinction between the two kinds of model is a practical one rather than a philosophical one. If a system has no conceptual model that anyone has actually written down, then everybody working on it is running a private model, each one incomplete in a different place, and nothing in the situation will surface the disagreement until the models are made to touch reality or each other.

The design conclusion follows directly. Good tools do not hope that private models converge. They provide an explicit conceptual model, put it where people can see it, and give the system a shape that keeps each person's private understanding tied both to the way the system really works and to what the other people on the project believe.

11.1.3 Johnson-Laird: reasoning is running models, not applying rules

The third work is Philip Johnson-Laird's *Mental Models*, also 1983.[3] Its scope is wider and its claim is stronger.

The received view of reasoning at the time was that people who reason correctly are, in effect, applying rules of logic to sentences. Johnson-Laird argued that this is not what happens. What people do instead is build a model of the situation the sentences describe — a specific,

[2] Donald A. Norman, "Some Observations on Mental Models," in Dedre Gentner and Albert L. Stevens (eds.), *Mental Models*. Hillsdale, NJ: Lawrence Erlbaum Associates, 1983.

[3] Philip N. Johnson-Laird, *Mental Models: Towards a Cognitive Science of Language, Inference, and Consciousness*. Cambridge: Cambridge University Press, 1983.

concrete arrangement of the things involved — read a conclusion off that model, and then, if they are careful, look for an alternative arrangement of the same premises that would break the conclusion.

That account explains something that a rules-based account struggles with: which reasoning problems are hard for people, and why. Problems that need several distinct models to be held and compared are hard. Problems settled by a single obvious model are easy, even when they are logically equivalent to the hard ones. Most reasoning errors turn out not to be broken rules but neglected models — the person built one arrangement consistent with the premises, found it sufficient, and never looked for the second one.

For our purposes the useful consequence is this: if reasoning proceeds by building and comparing models of a situation, then how a situation is presented is not cosmetic. A presentation that makes one arrangement easy to build and another arrangement hard will systematically produce the first conclusion. This is why the layout of a document has consequences beyond tidiness, and it is worth keeping in mind when we get to what a template does.

11.2 What a Kit Is, Read Through Norman

Now the argument.

Chapter 8 described a kit as the place you put your process so that it stops living in heads. That is the operational description, and it is correct. Read alongside Norman, it is also something more specific.

The five SDLC artifacts are answers to five questions that every software project has to answer whether or not anyone writes them down:

PRD What are we building, and why does it deserve to exist?

ADR Which decision did we take, and what did we reject?

DESIGN What structure discharges those requirements?

DECOMPOSITION Which pieces is it made of?

FEATURE What does "done" mean for one piece?

On a project with no artifacts, every one of those questions still has an answer in every participant's head. They are private mental models in Norman's sense, and they have all his properties. They are incomplete — each person knows their own area. They are unstable — what "done" meant in March is not what it means in June, and nobody noticed the shift. Their boundaries are fuzzy — assumptions get imported from the last project, which looked similar. And they diverge, silently, because there is nothing in the situation that forces two people to discover that they disagree.

What the kit does is convert those five private models into one external structure that everybody reads. That much is what any documentation convention claims to do. Two things make this one different.

The first is that the structure is linked rather than merely written. Every identifier follows the pattern `cpt-{system}-{subsystem}-{kind}-{slug}`, and a FEATURE specification's definition of done lists which PRD requirements, which DESIGN principles, which components, and which sequences it discharges. Chapter 3 made the point about that coverage block directly: read downward and it tells you what the feature is made of, read upward and it tells you what it is for. A prose document tells you what one person believed. A linked document tells you what that belief was connected to, and lets you go and look.

The second is that the links are checked by something that cannot be talked round. `cfs validate` confirms that every identifier matches the template for its kind, that the coverage

cross-references between PRD, DESIGN, DECOMPOSITION, and FEATURE exist where the constraints require them, that checkbox states agree across references, and that every `@cpt-begin` marker has a matching `@cpt-end` with no orphans. `cfs map` finds dangling references. `cfs where-used` answers, for one identifier, where it is referenced across documents and code.

That combination is what makes the kit a Norman-style conceptual model rather than a wiki page. A wiki page is an assertion about the system. A validated artifact chain is an assertion that has been mechanically compared against the rest of the assertions, and that fails loudly when it stops agreeing with them. The private models still exist — they always will — but they now have something outside themselves to be corrected against, and so does everybody else's.

11.2.1 Why this generalises

Here is the step that this chapter rests on. The failure mode being addressed — private, incomplete, divergent models of a shared object of work — is not a property of software. It is a property of several people working on one thing made of thinking.

Consider writing a paper with three co-authors. Each of them has a private model of what the paper claims. Nobody has written that claim down as a single sentence, because everyone believes it is obvious. Three drafts in, the disagreement surfaces: one author has been writing a methods paper, another has been writing an empirical-result paper, and the third has been writing a position paper, out of the same experiments. Or a subtler version, which is more common and more damaging: all three agree on the sentence and disagree about what would count as supporting it. One thinks the experiment settles the question. One thinks it is suggestive and the claim should be hedged. Neither of them knows that the other thinks differently, because the disagreement is about a relationship between a claim and some evidence, and there is nowhere in a manuscript where that relationship is written down.

Consider writing a grant proposal. The same shape appears with money attached. Everyone on the consortium agrees the area is important. Nobody has stated the specific outcome being promised, so each partner privately holds a different one, usually the one closest to what they already wanted to do. Everyone has a private theory of what the funder rewards, assembled from rumour and from the last proposal they had rejected, and those theories are never compared because nobody writes them down.

That is the same failure Norman describes, in domains that have never had a compiler. Which makes it worth asking what CF's mechanisms would look like there.

The precedent for asking is already in this book. Chapter 9 walked the `compete` kit, whose domain is competitive research and whose `artifacts.toml` entry reads `codebase = []`, because the system produces documents and nothing else. Its `RESEARCH-REQUEST` identifiers carry `to_code = false`, so `cfs validate` does not go looking for markers that will never exist. Everything else the kit does — the request as the root, the mandatory back-references, the deterministic gate, the stable prohibition identifiers — works unchanged. The mechanism has already been taken out of software once. What it kept was traceability, not compilation, and that is the piece worth carrying further.

11.3 Writing a Paper

So: what would a paper kit be? Not a template pack. A kit in the sense of Chapter 8 — artifact kinds with identifiers, a root, a constraints contract, and a gate.

Work it through mechanism by mechanism.

11.3.1 The PRD role: the claim, written before the prose

A PRD states what is being built and why it deserves to exist, before any code. The paper equivalent is a single artifact that states the paper's claim and its contribution, before any

drafting: what is being asserted, why it is not already known, and what would have to be true for the paper to be worth publishing.

The important design decision is that this artifact is not the abstract. An abstract is written to persuade, it is written last, and it is allowed to be vague in the places where vagueness reads well. This artifact is written first and is allowed to be ugly. Its job is to be specific enough that a co-author can disagree with it.

Two features are worth taking directly from the `compete` kit's `RESEARCH-REQUEST` template. The first is the required boundary section: a section whose only job is to state what the paper does *not* establish. As Chapter 9 noted, very few document templates in any discipline contain a mandatory section for their own limits, and the omission is the reason findings get quoted out of scope. In a paper the boundary section has a second function — it is where the hedge/no-hedge disagreement between co-authors becomes visible in week one instead of week nine.

The second is the `"unspecified"` discipline. An omitted field is rendered as the literal value `unspecified` and never inferred, so the gap stays visible in every artifact downstream. Applied to a paper: if nobody has decided yet whether the contribution is the method or the result, the field says so, in writing, where all three authors can see it.

The failure this prevents is the three-drafts-in discovery. Not because writing one page makes people agree, but because writing one page makes the disagreement happen while it is still cheap.

11.3.2 The DESIGN role: the argument as a structure

A DESIGN document does not restate the requirements. It states the structure that discharges them — principles, constraints, components, sequences — and each element carries an identifier that a FEATURE specification can later claim to cover.

The paper equivalent is the argument, written as a structure rather than as prose. Which claims are being made, in what order, and what supports each one. Crucially it also records *why* that order: a claim that depends on an earlier claim has to come after it, and a claim that only makes sense once a definition is in place has to come after the definition.

This is where Johnson-Laird earns his place in this chapter rather than just his footnote. If a reader reasons by building a model of the situation the text describes, then the order in which the text supplies material determines which model gets built first, and the first model is the one that gets kept. An argument structure that is written down and reviewed is the point at which the authors decide which model they are asking the reader to build, deliberately, rather than discovering months later that the section order they inherited from a previous paper builds a different one.

The failure this prevents is the paper whose sections do not build the argument it claims to build. That failure is nearly invisible from inside the manuscript, because each section reads well on its own. It is highly visible in a two-column table of claim against support, which is what this artifact is.

11.3.3 The CPT ID role: a claim and its evidence, connected

This is the sharpest analogy in the chapter, because traceability is the most concrete thing CF does.

In the SDLC kit, an identifier such as `cpt-ex-ovwa-fr-notify-on-limit` is declared once, in the PRD, and referenced from the DESIGN, the DECOMPOSITION, the FEATURE specification, and the code. `cfs where-used` finds every reference. The value is not that the string is short; it is that a claim about implementation — "we built the notification requirement" — stops being something the author asserts and becomes something a reader can check in one command.

The paper equivalent is an identifier on every claim and every piece of evidence, and a declared relation between them.

```
cpt-{paper}-clm-{slug} one claim the paper makes
cpt-{paper}-evd-{slug} one piece of evidence: a table, a figure,
                       a run, a dataset, a proof, a cited result
```

Every claim declares the evidence identifiers that support it. Every evidence record declares where it came from — which script, which dataset version, which run, which page of which cited paper. The `compete` kit already does the evidence half of this with its `EVD-###` records and access dates, and its second design principle states the rule the whole thing rests on: every factual statement resolves to an evidence record.

What this buys is worth stating precisely, because it is not "better bookkeeping."

For a co-author, it converts a question that is socially expensive into one that is free. Asking "are you sure Table 3 actually shows that?" is an accusation. Following `cpt-paper-clm-latency-improves` to the two evidence records it cites, and finding that one of them is a pilot run on a different dataset, is just reading. Norman's problem was that private models cannot be inspected. This makes the part that matters — what each author thinks supports what — inspectable.

For a reviewer, it collapses the most tedious part of reviewing. A reviewer's real work on a claim is finding the evidence for it, which currently means searching the manuscript for the figure that the sentence in Section 5 is probably about. Given identifiers, that step is free, and the reviewer's attention goes to the question only they can answer: whether the evidence actually supports the claim.

For the authors six months later, it answers the question Chapter 1's comic strip asks about code — which thing implements this? — in its research form: which run produced this number, and is the script that produced it the one still in the repository?

11.3.4 What a validate step would check

Now the part that has to be specified carefully, because it is where the analogy is strongest and where it is most tempting to overclaim.

A paper kit's `cfs validate` pass could check the following deterministically, with no model involved and the same answer every time:

- every claim identifier is well formed and unique;
- every claim cites at least one evidence identifier;
- every cited evidence identifier exists — no dangling references, which is exactly what `cfs map` already reports for artifacts;
- every evidence record is cited by at least one claim — orphaned evidence, which in a paper usually means a figure that survived a rewrite and now supports nothing;
- every section of the manuscript maps to at least one claim in the argument structure, and every claim appears in at least one section;
- every claim in the abstract appears in the claim register — the check that catches the abstract that grew a promise the body does not make;
- the required headings exist, including the boundary section.

The heading part is not hypothetical machinery. It is the same contract Chapter 9 showed, with different strings:

```
[[artifacts."PAPER-CLAIM".headings]]
id = "claim-boundary"
level = 2
required = true
multiple = false
numbered = true
pattern = "What This Paper Does Not Establish"
description = "The limits of the claim: what a reader may not conclude."

[artifacts."PAPER-CLAIM".identifiers.clm]
name = "Paper Claim"
required = true
to_code = false
template = "cpt-{paper}-clm-{slug}"
headings = ["claim-h1-title"]
```

And the checks that count things rather than parse things belong in scripts, for the reason the `compete` kit's architecture guide gives in one sentence: `cfs` enforces heading outlines, identifier formats, and cross-artifact coverage, and it cannot see table rows. Evidence density per claim is a counting problem. So is "how many claims in this paper rest on exactly one run." The `compete` kit's soft floors — one distinct evidence record for an orientation card, two for a profile, four for a dossier — are the pattern to copy, including the part where falling below the floor prints a warning and returns zero. A claim supported by one run is thin, not invalid.

11.3.5 Where this analogy strains, and it does

Software has a compiler. A paper does not, and no amount of structure will supply one.

That is not a small caveat, so state it exactly. In the SDLC pipeline, the artifact chain is checked by `cfs validate`, but the artifact chain is not the only thing being checked. Underneath it there are unit tests and a build, and they answer a different kind of question: does the thing actually work. That signal is unambiguous, it is repeatable, and it does not care who wrote the code. Chapter 1's comic strip makes the distinction its punch line — a green validate run means the shape is right, not that it works — and in software you have a second mechanism for the second half.

A paper has no second mechanism. The question "does this argument work" — does this evidence actually support this claim, is this the right comparison, is the effect real — is a judgement. It is made by people with domain expertise, it is contested in good faith by other people with domain expertise, and it is sometimes settled only years later by somebody failing to replicate the result. Nothing deterministic touches it.

So be clear about what the paper kit would and would not deliver. It would make the *structure* of the argument explicit and checkable: that a link exists, that it points somewhere real, that nothing is orphaned, that the abstract does not promise more than the body. It would not tell you whether the link is any good. A paper can pass every check listed above and still be wrong in the way that matters most, and a paper full of validation failures can be a good paper badly assembled.

The honest framing is the one the book has already used for the division between templates and checklists. A template answers what sections must exist; a checklist answers whether what is in them is any good, and the second question needs a reader. In the paper case that asymmetry is much more severe than in software, because the deterministic half is a smaller fraction of the total job. The value is real, and it is bounded: the mechanism removes the failures caused by nobody having written the connection down, and leaves untouched every failure caused by the connection being weak. Those are different failure classes, and it is worth being precise about which one is being addressed, because a tool that is oversold as addressing the second will be abandoned when it does not.

11.4 Writing a Grant Proposal

The same pattern, more briefly, because the shape is now familiar and the differences are what matter.

The PRD role is a promise artifact: the specific outcome the project undertakes to produce, stated separately from the area it works in. Those two get conflated constantly, and the conflation is the whole problem. "Advancing the state of the art in federated learning for clinical data" is an area. "A validated protocol, evaluated on three hospital datasets, that reduces inter-site variance by a stated margin" is an outcome. The first cannot be failed and therefore cannot be assessed. The second can be, which is exactly why proposals drift toward the first. A required field that must contain an outcome, with the boundary section stating what the project will not deliver, is the mechanism that stops the drift — and it also surfaces the consortium disagreement, in month one, about which outcome everyone thought they had signed up to.

The DESIGN role is the method chain: why this approach produces that outcome. Not that the topic is important, and not a list of activities. A declared path from what the team will do to the outcome that was promised, with the step that is most likely to fail named as such. Proposals that fail on review very often fail here, and the reviewer's comment is some version of "the significance is clear but the approach is not convincing." That comment is a report that the method chain was never written down, because writing it down is what makes its weak link visible — to the reviewer, and first of all to the authors.

The identifier role connects one promised deliverable to the one work package that produces it:

```
cpt-{proposal}-del-{slug} one promised deliverable
cpt-{proposal}-wp-{slug} one work package, with effort and a date
```

Every deliverable names its work package. Every work package names the deliverables it produces. The two deterministic checks that fall out of this are both worth having, and the second one is the useful one. A deliverable with no work package is a promise nobody is doing — the abstract has grown a commitment that the plan does not deliver. A work package with no deliverable is effort and budget attached to no promise, which reviewers find reliably and comment on unkindly. Both are structural, both are invisible in a seventy-page document read linearly, and both are trivial to find once the identifiers exist.

And the honest caveat, which is different from the paper case and sharper. A funding decision is not only an assessment of the proposal. Calls have priorities that are not written in the call text. Panels have compositions, and the same proposal scores differently in front of different panels. Consortium balance across countries and institution types matters in ways no methods section can influence. Programme managers have portfolios to fill and gaps to avoid. All of that is real, it is strategic and political rather than technical, and no template captures any of it.

There is a second asymmetry that makes this worse than the paper case. A paper that is rejected comes back with reviews, and the authors learn something. A proposal that is rejected usually comes back with a score and a paragraph, once, and the thing that actually decided it is not in the paragraph. So the feedback loop that would let a proposal kit's rules improve over time is much weaker.

What a kit can honestly claim here is narrow: it can make the promise legible and check that the document delivers what its own abstract claims. It cannot make a proposal fundable, and a kit that implied otherwise would be selling the one thing it does not have.

11.5 What This Chapter Is, and Is Not

Everything else in this book describes something that exists. Commands were run, files were read, counts were verified against the repositories, and where a source contradicted itself the

contradiction was reported rather than smoothed over.

This chapter is not that. Nothing described here has been built. There is no paper kit and no proposal kit. No template in these sections has been filled in by a real author under deadline, no constraints file has been run against a real manuscript, and no researcher has yet reached the point in a real project where they decide the structure is not worth the effort and quietly stop using half of it — which, as Chapter 8 pointed out, is what actually happens to processes that almost fit.

So read this as a design argument with a stated basis, and hold it to the standard Chapter 10 sets for any new kit. The basis is that CF's artifact chain is a Norman-style conceptual model with a machine check attached, that the failure it addresses is divergent private understanding among collaborators, and that this failure is not specific to software. The argument is that the two lifecycles sketched here have that failure badly, and that the transferable part of the mechanism is traceability rather than compilation.

The open questions are the ones only use can settle. Whether the claim register survives contact with a real revision cycle, where a paper is rewritten twice and half the evidence identifiers change meaning. Whether the deterministic checks catch enough to be worth the friction, or whether they mostly report things the authors already knew. Whether a proposal kit would improve proposals or merely make them longer. Whether the correct unit is a kit at all, or whether some of this belongs in a script, or in a checklist, or nowhere.

Those are answerable, and the way to answer them is the one this book has recommended throughout: build the smallest version, register it in place, run it on something real, and pay attention to the first thing the author works around.

Exercises

The skill this chapter is about is not reading three books. It is looking at a piece of shared work, finding the place where two people are running different private models of it, and building something small and external that makes them collide. These exercises are that, on your own work. Write the answers down — the whole point of the chapter is that an answer held in your head does not count.

1. **Diagnose one recurring disagreement.** Pick a disagreement from your own team that has come back at least twice in the last six months — the same argument, in different words, about the same object of work. Write two columns, one per party, and in each column put the proposition that party holds and assumes the other party also holds. Then write the single sentence they actually differ on. Finally, name which of Norman's properties produced the divergence: incompleteness, instability, fuzzy boundaries imported from a previous project, superstition, or parsimony. If you cannot get the difference down to one sentence, you have found two disagreements and should do this twice.

   Then list every occasion in your current process at which both parties looked at the same artifact — a ticket, a review, a standup, a document — and for each, say what that artifact required them to write down and why writing it down did not force the difference into the open. Finish with one sentence of the form: "nothing in our process ever asks anyone to state X." That sentence is the specification for the next exercise.

2. **Design exactly one externalizing mechanism, in a new domain.** Choose a domain this chapter did not work through: a code review, a hiring decision, a product roadmap, an incident postmortem, an on-call handover, a customer commitment. Design *one* mechanism — one template field, one stable identifier scheme, or one checklist item. Not a kit, not a document suite: one. State (a) the private model failure it targets, in Norman's terms, (b) the literal text of the field, the identifier pattern, or the checklist line, (c) who

fills it in and at what moment, and (d) what a reader can now check that they could not check before. Then argue in two sentences why this mechanism and not the other obvious one.

3. **Sort five checks into the three columns.** For the domain you chose in Exercise 2, write down five checks you would want somebody or something to perform. Classify each one as: enforceable by a heading and identifier contract, a counting problem for a script, or a judgement that needs a reader. For each counting problem, decide whether it is a gate or a warning that returns zero, and justify the call. For each judgement, write the one sentence you would put in a checklist. If all five land in one column, you have not looked hard enough — in particular, if all five are judgements, you have designed a culture change rather than a mechanism.

4. **Stress-test the claim register on a real paper.** Take a paper you have written, co-authored, or reviewed in which a disagreement about what the evidence supported surfaced late. Reconstruct the register as it would have stood at the moment the disagreement existed but was invisible: the claim identifiers, the evidence identifiers, and the declared relation between them. Then answer, honestly, whether any of the seven deterministic checks in *What a validate step would check* would have fired. If one would have, say which and what the author would have seen. If none would have, state what the disagreement was actually about, and say whether *any* deterministic check could reach it. A negative answer here is a correct answer and is worth more than a strained positive one.

5. **Name one failure a template cannot fix.** For the domain you chose in Exercise 2, name one failure mode that no template, however well designed, will prevent — and prove it, by describing a filled-in artifact that satisfies every field correctly, would pass every check you wrote in Exercise 3, and still exhibits the failure. Then say what does address that failure: a specific person, a test, an enforced delay, a change to who gets rewarded. State its cost. If you cannot name anything that addresses it, say that too, and say what you will do instead of pretending the template covers it.

6. **Break your own mechanism.** Take the mechanism from Exercise 2 and try to kill it. Answer three questions in writing. First: what is the cheapest string a tired person can put in that field at six o'clock on the day of the deadline that satisfies it and carries no information? Second: if that string appears in a third of all instances after three months, does the mechanism still pay for its friction? Third: what would you have to change so that the cheap string is not available — an enumerated set of values, a cross-reference that has to resolve, a second reader? Then either make that change, or write down the occurrence rate of the cheap string at which you commit to removing the mechanism. Choosing in advance is the point; a mechanism nobody will ever retire becomes a tax.

7. **Find the one you already route around.** Look at your current process for a mechanism that already exists to externalize a shared model and is now filled in without thought: a required ticket field, a sign-off, a form, a section everybody copies from the last document. Work out which private model it was originally built to expose. Then decide which of two things happened: the models converged and the mechanism is now redundant, or the models diverged anyway and the mechanism is worse than nothing because it certifies agreement that does not exist. Deliver a verdict — delete, or rewrite — and, if rewrite, apply Exercise 6 to your rewrite before you ship it.

Solutions to Exercises

Every exercise in this book that has a determinate answer is answered here against the real files: the Constructor Studio repository, the `overwork_alert` example, `gears-rust`, `gears-frontx`, `insight`, `studio-kit-sdlc`, and `studio-kits-pm`. Paths, identifiers, error messages, and counts are quoted from those repositories at the time of writing; if a number has moved, the method for re-deriving it is stated so you can check rather than trust.

The open-ended exercises — design your own kit, audit a name in your own code, measure your own laptop — get one fully worked example each. A worked example is not a marking scheme. Where the exercise is genuinely about your own context, the example is there to show the shape of a good answer, not to be copied.

A.1 Chapter 1: Getting Started

1. **Install and verify.**

   `.cf-studio/.core/` is generated runtime: the engine's own skills, workflows, and modules, unpacked into your repository so the host tool can read them. It is gitignored by default and you should never edit it by hand — the next `cfs init` or `cfs update` rewrites it from the pinned engine version, and your edit disappears without a diff to explain it.

   `.cf-studio/config/` is the opposite in every respect. It is tracked, it is yours, and it is the part your team reviews: `AGENTS.md`, `SKILL.md`, `rules/`, and — once you start using the SDLC kit for real — `artifacts.toml`. Edit it deliberately and review changes to it in pull requests, because a change here changes what every agent in the repository is told to do.

   `.cf-studio/version.toml` is tracked and records the pinned Studio engine request. Editing it by hand is legal but is a version-pin decision, not a routine edit; it is what makes a second `cfs init` a repair rather than an upgrade. Treat it the way you treat a lockfile.

   `.claude/` (and its four siblings) is generated host integration, gitignored by default, and never hand-edited. It is the output of `cfs generate-agents`, which is the compile-once-adapt-per-host mechanism of Chapter 4 seen from the outside. If you want different content there, change `config/` or a kit and regenerate.

   The pattern to take away: two directories are inputs, two are outputs, and telling them apart on day one saves you from the most common wasted afternoon in this system — editing an output and wondering why the change keeps reverting.

2. **Terminal or chat?**

   Terminal: `cfs validate`, `cfs kit check-updates`, `cfs map`, `cfs generate-agents --agent claude`.

   Chat: `cf-help`, `cf-explore: ...`, `cf-map: find dangling references`, `cf-sdlc-doc-prd: ...`.

The rule is mechanical and admits no exceptions: anything starting with `cfs` is a terminal binary, anything starting with `cf-` is a skill name typed into a chat window. The one pair designed to catch you is `cfs map` and `cf-map`, which are genuinely different things. `cfs map` builds the dependency graph and writes a file. `cf-map: ...` is a workflow that asks questions *about* that graph and needs it to exist first.

The failure that costs real time is typing `cfs validate` into chat. You will not get an error. You will get a fluent, plausible description of what validation might have reported, which is worse than an error because it looks like a result and you will act on it. If you are ever unsure whether a number in your chat window came from a command or from a model, scroll up and find the command output. If there is none, there was no run.

3. **Rewrite four bad requests.**

   Worked against a small Python service with an existing test suite. Yours will name your own files; the shape is what matters.

```
cf-plan: break the move from the synchronous billing client to the
        queued one into phases that each leave main deployable

cf-coding-review: review src/api/auth.py for missing input validation
                 and list findings by severity before any fix

cf-coding-tests: add tests for the three uncovered branches in
                payments/retry.py reported by the last coverage run

cf-explain: walk me through what tokens/refresh.py does and where its
           expiry assumption is stated in the design document
```

   Each rewrite does the same three things the originals did not. It names a target, so the workflow has something to load instead of guessing. It bounds the scope, so the run ends. And it makes the success criterion visible, so you and the model agree in advance what "done" looks like. Notice that "tell me if this code is good" became "walk me through X and find where assumption Y is written down" — the original was not a small request, it was an unanswerable one, and the fix was to decide what you actually wanted to know.

4. **Clone and orient.**

   The four architecture documents are `architecture/PRD.md`, `architecture/DESIGN.md`, `architecture/DECOMPOSITION.md`, and the single ADR under `architecture/ADR/general/`:

```
0001-cpt-overwork-alert-adr-cli-daemon-launchagent-no-menubar-v1.md
```

   Four more feature specifications live under `architecture/features/`: `tracker-core.md`, `notifications.md`, `cli-control.md`, and `launchagent-autostart.md`.

   The nine implementation modules under `src/overwork_alert/` are `cli.py`, `config.py`, `daemon.py`, `idle.py`, `ipc.py`, `launchagent.py`, `models.py`, `notification_policy.py`, and `notify.py`. (`__init__.py` and `__main__.py` bring the file count to eleven and carry no logic.)

   The correct mapping, which you can verify in Chapter 3 or by grepping for `@cpt-`:

   - `daemon.py` — three feature specs at once: `tracker-core.md` (the tick loop and accumulation), `notifications.md` (the send-and-repeat flows), and `cli-control.md` (the command handler).
   - `config.py`, `idle.py`, `models.py` — `tracker-core.md`.
   - `notification_policy.py`, `notify.py` — `notifications.md`.

- `cli.py`, `ipc.py` — `cli-control.md` (`cli.py` also carries the install and uninstall flows from `launchagent-autostart.md`).
- `launchagent.py` — `launchagent-autostart.md`.

If you guessed one file per document you were wrong in the interesting direction, and the reason is worth keeping: features are units of *reviewable work*, not units of compilation. `daemon.py` is where three features meet, which is exactly why it is the file with 99 markers.

5. **Predict the check.**

   The mistake it catches is a document marked done on the strength of an upstream document that is not. A feature specification says `[x]` against `cpt-ex-ovwa-fr-notify-on-limit` — meaning "this is implemented" — while the PRD still has that requirement at `[]`. In real projects that happens when somebody implements from a conversation instead of from the approved requirement, or when a requirement is reverted upstream and the downstream artifact is never revisited. The check turns a silent inconsistency into a validation error at the exact moment somebody claims completion.

   The reverse case is allowed because it is the normal state of work in progress. A definition marked `[x]` with references still at `[]` means the requirement is settled and the work that depends on it has not been done yet. That is a plan, not a defect — and forbidding it would make the validator warn continuously from the moment the PRD is approved until the last line of code lands, which is the fastest way to teach a team to ignore a validator.

   The asymmetry is the point. The rule fires on "claimed more than is true" and stays quiet on "not finished yet."

A.2 Chapter 2: Choosing and Setting Up a Free or Cheap LLM

1. **Get one host working end to end.**

   This is a report on your own setup, so there is no single right answer. What a complete answer contains: the path you chose, the wall-clock time, and one honest failure.

   For reference, the two failures that account for most first attempts:

   On Path A, the workflow does not appear in chat. Ninety per cent of the time `cfs generate-agents` was never run, or was run in a different repository than the one you have open. The tell is that `cf` produces a normal chatty answer rather than a routing menu.

   On Path B, `claude` still asks you to log in. That is always the two environment variables not reaching the process that launched Claude Code — typically because you exported them in one terminal and started the editor from another. Verify with `echo $ANTHROPIC_BASE_URL` *in the shell that will run the command*, not in the one where you typed the export.

   If nothing went wrong, the step you were least sure about is almost certainly the one you should write down, because you will repeat it on the next machine.

2. **Re-verify three numbers.**

   The answer is whatever the provider pages say on the day you check, and this book cannot supply it: the numbers in Chapter 2 were checked in August 2026 and the chapter says out loud that they rot. What the exercise is really training is a habit, so grade yourself on the habit rather than the result.

   A complete answer records, for each claim: the URL, the date you checked, the current value, and whether it matches. If it has changed, one line with the corrected value. That

last part matters more than it looks — a note saying "changed" without the new number is a note you will have to re-do.

Two structural expectations that are safe to state in advance. First, the country eligibility claims are the most volatile of the three kinds, because they change without announcements and without changelogs. Second, if a free tier has disappeared entirely, check whether it was replaced by a credit grant with an expiry date, because vendors substitute one for the other and the substitution is easy to miss.

3. **Measure your own laptop's ceiling.**

   This depends on hardware this book cannot see, and inventing a number for you would defeat the exercise. What you should record: total memory, whether it is unified, the quantisation you pulled, the wall-clock time to first token and to completion for the same prompt at both sizes, and — the decisive observation — whether the larger model stayed resident or spilled to CPU.

   How to tell that it spilled without any special tooling: throughput collapses by roughly an order of magnitude rather than degrading gently, and the fans go on and stay on. LM Studio shows you memory allocation directly if you want to see it rather than infer it.

   The judgement to write down afterwards, phrased against Chapter 1's operations rather than against a benchmark: a 7–8B model that runs comfortably is worth trusting with `cf-explain` on one file, small marker fixes, and first-pass documentation. Reserve `cf-plan`, decomposition, and any multi-file implementation for whatever your largest resident model is — and if nothing in the 27–32B range stays resident on your machine, that is a real answer, and the correct conclusion is Path A or Path C rather than a slower Path B.

4. **Trace the local endpoint.**

   `/v1/chat/completions` answers with an OpenAI-shaped response body; `/api/tags` answers with Ollama's own model inventory. The same process on port 11434 is serving at least three different wire protocols: its native API, an OpenAI-compatible surface, and — since Ollama 0.14.0 — an Anthropic-Messages-compatible surface.

   That last one is the whole of Path B, and it is worth being precise about why. Claude Code has no plugin interface for third-party models. What it has is a base URL and an auth token, and a client that speaks the Anthropic Messages API. It cannot be taught a new protocol; it can only be pointed at a different address that already speaks the one it knows. So the adaptation has to happen on the server side, and Ollama did it: it presents the protocol the client expects, in front of whatever open-weight model you pulled.

   This is the same shape as `ClientHub` in Chapter 5 and the directory client's two implementations: state an interface, receive an implementation, do not learn where it lives. Here the interface is an HTTP contract instead of a Rust trait, and the consumer is a closed-source binary that was never designed to be redirected. Protocol compatibility is what makes a component substitutable when you do not control either end.

5. **Plan a two-model split.**

   One reasonable assignment, using Table 2.5's tiers with a GPT-style model (G) and a Claude-style model (C):

   (a) Gather what exists — G. Wide, shallow, artifact-heavy reading.

   (b) Open the options — either; brainstorming rewards breadth over precision.

   (c) PRD — G. Artifact transformation with many small correctness details.

   (d) ADR — G. Same reason, plus the value is in the rejected options being argued rather than listed.

(e) DESIGN — G. The chapter's clearest "artifact transformation" row.

(f) Decompose — G, and this is the one with the highest cost of a small mistake, since a coverage gap here is invisible until implementation.

(g) FEATURE spec — G.

(h) Implement — C. Turning an approved specification into code is the row where Claude-style models are noted as often stronger.

(i) Review the code — G, in a fresh chat or a sub-agent, and ideally not the family that wrote it.

(j) Map and dangling references — fast tier or inherit; this is a lookup over a generated graph and barely uses the model at all.

The three assignments to be least confident about, and why:

Stage 2 (brainstorm). The tier table has no row for it, because brainstorming has no correctness criterion to optimise for. Either family produces usable options; the differentiator is whether it produces *different* options rather than three phrasings of one. If you have both, running the same brainstorm twice and diffing the two option sets is more valuable than picking a winner.

Stage 6 (decompose). Assigned to G on the strength of the artifact-transformation row, but decomposition is also a judgement call about what constitutes a coherent unit of work, which is closer to design than to transformation. This is the assignment most likely to be wrong.

Stage 9 (review). The tier table says fast-tier for a first pass and a GPT-style review model if available. The reason for hesitating is that the strongest constraint on review quality is not model family at all — it is whether the reviewer saw the writing conversation. A weaker model in a clean context beats a stronger one carrying the author's assumptions, and no row in the table captures that.

A.3 Chapter 3: The Full Pipeline, Walked Through

1. **Run the checks and say what each one does.**

   `cfs validate` examines every artifact registered in `artifacts.toml` plus the code registered in each system's `codebase` block: required headings against the kit's `constraints.toml`, identifier format against each kind's template, checkbox consistency between definitions and references, cross-artifact coverage where the constraints require it, and — for kinds with `traceability = "FULL"` — whether `[x]` identifiers have code markers and `[]` ones do not. The class of mistake it catches: a chain that has stopped agreeing with itself. A DESIGN driver referencing a requirement identifier that the PRD no longer defines is caught here and nowhere else.

   `cfs validate --local-only` examines exactly the same things, minus cross-repository resolution. What it catches is the same; what it stops doing is failing on identifiers whose definitions live in a sibling repository it can reach but you did not want it to walk today.

   `cfs spec-coverage` runs the check in the other direction: of the identifiers declared to reach code, which actually have markers. Validation asks "is every marker legitimate"; coverage asks "is every promise kept." It catches the feature declared done whose implementation never happened, which validation will not report if the artifact side is internally consistent.

   `cfs map` scans markdown links, source files, and `@cpt-*` identifiers across every configured source and writes the graph. Its summary reports the mode (`single-repo` or `federated`

with reachable and unreachable source counts), how many markdown and source files were scanned, edge counts by type (`file-link`, `cpt-doc`, `cpt-impl`), and a `Phantom IDs` line counting dangling `@cpt` uses — markers in code referring to identifiers no artifact defines. That last number is what it catches that nothing else does.

Why `--local-only` changes nothing in a single-repository clone: there is nothing cross-repository to skip. The flag suppresses resolution against workspace members registered by `cfs workspace-add`. With no workspace, the set of artifacts collected is identical either way, so both runs examine the same files and report the same result. In a workspace the flag is the difference between validating your repository and validating your repository against everyone else's current state — which is exactly what you do not want while iterating, and exactly what you do want in CI.

2. **Trace an identifier you have not seen traced.**

   `cpt-ex-ovwa-fr-cli-controls`, end to end:

   `PRD.md`, section 5, as FR-006 "Provide CLI controls (status/pause/resume/reset)", marked `[x]`, priority `p2`, with `cpt-ex-ovwa-actor-user` as its only actor. The requirement text lists five capabilities: start, view status, pause, resume, reset.

   `DESIGN.md`, §1.2, in the architecture drivers, restated verbatim with a **Solution**: "A CLI provides user-facing commands and communicates with the daemon via a local-only control interface to query status and issue commands." The ADR also cites it, in its own traceability section, as one of the two requirements that pushed the CLI-daemon decision.

   `DECOMPOSITION.md`, §2, entry 3, "CLI Control + Local IPC", MEDIUM, `cpt-ex-ovwa-feature-cli-control`, depending on `cpt-ex-ovwa-feature-tracker-core`. It covers `cpt-ex-ovwa-fr-cli-controls` and `cpt-ex-ovwa-fr-manual-reset`, owns `cpt-ex-ovwa-component-cli` and `cpt-ex-ovwa-component-control-channel`, and puts a network-accessible API and multi-user support explicitly out of scope.

   `features/cli-control.md` refines it into five actor flows (`cpt-ex-ovwa-flow-cli-control-status`, `-reset`, `-pause`, `-resume`, `-stop`), one algorithm (`cpt-ex-ovwa-algo-cli-control-handle-command`), one state machine (`cpt-ex-ovwa-state-cli-control-request-lifecycle`), and one definition of done, `cpt-ex-ovwa-dod-cli-control-reset-and-controls`, whose *Covers (PRD)* list names `cpt-ex-ovwa-fr-cli-controls`.

   In code, the descendants land in three files and nowhere else:

   - `cli.py` — 48 marker lines: the five flow scope markers with their instruction blocks (11 lines for `status`, 9 each for `reset`, `pause`, `resume`, `stop`) plus the single `@cpt-dod:cpt-ex-ovwa-dod-cli-control-reset-and-controls:p1`.
   - `daemon.py` — 30 marker lines: the same five flows appearing again on the daemon side of each command (3 lines each) plus 15 lines for `cpt-ex-ovwa-algo-cli-control-handle-command`.
   - `ipc.py` — 5 marker lines, all `cpt-ex-ovwa-state-cli-control-request-lifecycle`: the scope marker and one instruction block.

   What reading tends to miss, and `cfs where-used` does not: that each flow appears *twice*, once in `cli.py` and once in `daemon.py`. The flow is a round trip across a socket, so it has two implementation sites, and a reader tracing from the CLI stops at the socket write. `ipc.py` is also easy to miss entirely — five markers in a 173-line file is a thin trail, and it is thin for the reason Chapter 3 gives about `idle.py`: socket plumbing is implementation judgement, and only the request lifecycle was specified.

3. **Add a marked function and trace it back.**

   One worked answer. Add to `features/notifications.md`, under section 3, a fourth CDSL algorithm:

```
### Report time spent over limit

- [x] **ID**: ‘cpt-ex-ovwa-algo-notifications-over-limit-duration‘

1. [x] - ‘p1‘ - **IF** over_limit_since is unset **RETURN** 0
   - ‘inst-not-over-limit-yet‘
2. [x] - ‘p1‘ - **RETURN** now - over_limit_since as whole seconds
   - ‘inst-compute-duration‘
```

   Then in `notification_policy.py`:

```
# @cpt-algo:cpt-ex-ovwa-algo-notifications-over-limit-duration:p1
def over_limit_seconds(*, state: TrackerState, now: float) -> int:
    """Return whole seconds since the session first went over limit."""
    # @cpt-begin:cpt-ex-ovwa-algo-notifications-over-limit-duration:p1:inst-not-over-limit-yet
    if state.over_limit_since is None:
        return 0
    # @cpt-end:cpt-ex-ovwa-algo-notifications-over-limit-duration:p1:inst-not-over-limit-yet

    # @cpt-begin:cpt-ex-ovwa-algo-notifications-over-limit-duration:p1:inst-compute-duration
    return int(now - state.over_limit_since)
    # @cpt-end:cpt-ex-ovwa-algo-notifications-over-limit-duration:p1:inst-compute-duration
```

   Getting to green takes three things, in this order. The identifier must exist upstream before the marker exists in code, or the marker is an orphan (`code-orphan-ref`). The CDSL steps must be `[x]`, or the code marker points at unchecked work (`code-task-unchecked`). And every `@cpt-begin` needs its `@cpt-end` with something between them, or you get `marker-begin-no-end` or `marker-empty-block`.

   The definition of done it sits under is `cpt-ex-ovwa-dod-notifications-alert-and-repeat`, and the PRD requirement above that is `cpt-ex-ovwa-fr-notify-on-limit`. And yes: you have to add the new algorithm identifier to that definition of done's **Implements** list. This is the part people skip, and it is the part that carries the information. A marked function that no definition of done claims is code with a label on it, not traceable code — you can walk from the code to the algorithm and then the trail stops, because nothing upstream says the algorithm is part of anything. Nothing forces you to add it. That is precisely why it is worth noticing.

4. **Find the drift.**

   Both mismatches are real and both are in the two closing sections of `DESIGN.md`.

   First, §4 “Additional Context” cites the ADR as

```
examples/overwork_alert/architecture/ADR/general/
  0001-cpt-ex-ovwa-adr-cli-daemon-launchagent-no-menubar-v1.md
```

   The file actually on disk is `0001-cpt-overwork-alert-adr-cli-daemon-launchagent-no-menubar-v1.md`. The slug segment in the filename is `cpt-overwork-alert-`, not `cpt-ex-ovwa-`. The document was written after the system slug scheme settled and the filename before it, or the reverse; either way the name in the document is a name that never existed.

   Second, §5 “Traceability” links `**Features**: [specs/](./specs/)`. There is no `specs/` directory. The feature specifications are in `features/`, which is what `DECOMPOSITION.md` links to correctly, entry by entry.

   Would `cfs map` report them? Not as written, and the reason is specific enough to be worth knowing. Map's link extractor (`commands/map/links.py`) recognises two forms:

markdown links `[label](target)`, and prose paths of the shape `{var}/....md`. The §4 citations are neither — they are paths inside backticks, invisible to both regular expressions. The §5 entry *is* a markdown link, so it is extracted, but then `_append_file_link_edge` drops it: an edge is only appended when the target resolves to a markdown node that was actually scanned, and `./specs/` resolves to nothing. Unresolvable link targets are silently discarded, not recorded. The `dangling` flag that map does report exists only for `@cpt-*` identifier uses — the `Phantom IDs` line in the summary — and neither of these is an identifier reference.

So for map to report the §5 drift, it would have to emit an edge (or a finding) for a link target it could not resolve, which today it deliberately does not. For it to report the §4 drift you would additionally need backticked paths treated as references.

Would `cfs validate` catch them? No. Validation works over headings, identifiers, checkbox states, and cross-artifact identifier coverage. It never resolves a filesystem path mentioned in prose. Both drifts are the kind of error this system is structurally blind to, which is the useful thing about the exercise: traceability by identifier is strong, and traceability by path is not traceability at all.

The fix is two lines: correct the ADR filename in §4, and change `./specs/` to `./features/` in §5.

5. **Break a marker on purpose.**

   Delete one `@cpt-end` line from `daemon.py` and validation reports error code `marker-begin-no-end`, with the message shape:

```
@cpt-begin for ‘cpt-ex-ovwa-flow-tracker-core-tick-loop‘ inst
‘inst-skip-on-idle‘ at line 214 in ‘daemon.py‘ was never closed
with @cpt-end
```

   The identifier, the instruction, the line, and the file. Note that the scanner keys open blocks on identifier plus phase plus instruction, so deleting one `@cpt-end` orphans exactly one block — the surrounding nested blocks in `daemon.py`'s command handler close normally.

   Restore it, then flip one CDSL step in `features/notifications.md` from `[x]` to `[]` and validation reports `cdsl-step-unchecked`:

```
CDSL step ‘cpt-ex-ovwa-algo-notifications-should-notify‘ inst
inst-first-alert is unchecked but parent ID is already checked
in FEATURE artifact
```

   Why both are errors, and why that is what you want. The first is an error about the code alone: an unclosed block has no end, so the region it claims to mark is undefined, and every downstream consumer — coverage reports, the map's `cpt-impl` edges, a future refactor tool — would have to guess. It appears in real life not as vandalism but as a deleted function that took its closing comment with it, in the middle of a 400-line diff nobody reads to the end.

   The second is an error about the relationship: an unchecked step underneath a checked parent claims, simultaneously, that the algorithm is done and that one of its steps is not. That is the same asymmetry as Chapter 1's exercise 5, applied one level down. It happens when somebody reverts a behaviour in the specification and forgets that the code implementing it is still there.

   Both look like nuisances the first time you hit them and are the reason the chain is worth anything the tenth time. The check has no opinion about your code's quality; it only refuses to let the documents and the code disagree about what exists.

A.4 Chapter 4: Studio and PDSL

1. **Find the layer discovery code and describe its role.**

   (a) Order, and what consumes it. The returned list is built in three appends: kit layers first (from `core.toml` kit registrations, in `core.toml` iteration order), then the master repo layer if walk-up found one, then the repo layer from `{studio_root}/config/manifest.toml`. It is outermost-first, and the code comments label the three blocks "Step 1", "Step 3-6", and "Step 2" in that physical order — a small confusion worth noticing, since the step numbers are not the execution order.

   Two things consume it. `cfs generate-agents` (`commands/agents.py`) calls it and immediately filters to `layer.scope == "kit"`, with a comment stating why: repo and master layers may describe broader project extensibility but must not implicitly change generated agent configuration. `cfs resolve-vars` calls it too and looks at the master layer specifically. The order matters because `merge_components` in `utils/manifest.py` iterates the list and applies last-writer-wins — its docstring calls it "inner-scope-wins" — so a later layer overrides an earlier one on the same component ID, and provenance records both the winner and everything it overrode.

   (b) A `manifest.toml` that exists but does not parse. It becomes a `ManifestLayer` with `state = ManifestLayerState.PARSE_ERROR` and `manifest = None`, and it stays in the returned list. This is the detail Chapter 4 praises, and it deserves one qualification after you follow it downstream.

   Every consumer reads that state the same way: as a filter. `merge_components` skips any layer that is not `LOADED`; `agents.py` checks `layer.state == _ManifestLayerState.LOADED` in three places; `resolve_vars.py` does the same for the master layer. Nothing branches on `PARSE_ERROR` to *report* it. The only place a human learns about a broken manifest is the `logger.warning` in `_load_manifest_layer`, and even that only covers kit layers — `_load_repo_layer` and `_load_master_layer` swallow the exception silently. So the state is preserved as data, and no diagnostic currently reads it. The design is genuinely better than crashing or dropping the layer; it is one small function short of being as good as its docstring implies.

   (c) Renaming "layers" to "precedence sources". Nothing behavioural. The identifiers `ManifestLayer`, `ManifestLayerState`, `discover_layers`, and `scope` would all change spelling; the call sites would change spelling; no control flow, no ordering, no filtering, and no output would change. That is the whole argument of the chapter compressed into one refactor: if a word can be swapped out with no effect on any computation, the word was never carrying a guarantee. Contrast `gears-frontx`, where deleting the layering claim means deleting three dependency-cruiser rules and the build stops failing on real violations.

2. **Trace one PDSL routing decision.**

   `analyze.md` contains only two units. Everything else it names is resolved through `LOAD` of a module file, which is the first thing the exercise teaches: the graph is not in the file.

```
AnalyzeBootstrap
  --RUN--> WorkflowBootstrapRouterPrelude (workflow-bootstrap.md)
  --RUN--> WorkflowBootstrapSimpleModeGate (workflow-bootstrap.md)
           --RUN--> SimpleModeGate (gates/simple-mode.md)
  --RUN--> WorkflowBootstrapCommandWorkflowResolution
           --RUN--> WorkflowBootstrapCommandResolution --RUN--> CommandResolution
  --CONTINUE (unconditional)--> AnalyzeRoute

AnalyzeRoute
  --RUN--> WorkflowResolution (runtime/workflow-resolution.md)
           --RUN--> WorkflowResolutionCommandSetup
```

```
          --RUN--> WorkflowResolutionEnumerate
  --CONTINUE WHEN ANALYZE_INTENT_CAPTURE_STATE == resume--> AnalyzeDescribeIntentResume
  --CONTINUE WHEN AVAILABLE_SKILLS is empty-------------> AnalyzeNoMatch
  --CONTINUE WHEN ORIGINAL_INTENT != unset--------------> AnalyzeRouteIntentFlow
  --CONTINUE WHEN ORIGINAL_INTENT == unset--------------> AnalyzeRouteLoadFlow

AnalyzeRouteIntentFlow
  --CONTINUE WHEN at least one relevant skill matched--> AnalyzeRouteIntentMenu
  --CONTINUE WHEN no relevant skill matched-----------> AnalyzeNoMatch
```

State: `AnalyzeRoute` declares `ORIGINAL_INTENT`, `AVAILABLE_SKILLS`, and `ANALYZE_INTENT_CAPTURE_STATE`, all scoped `workflow_run`, and writes the first two. `AnalyzeDescribeIntent` writes `ORIGINAL_INTENT` and `ANALYZE_INTENT_CAPTURE_STATE`. Everything else reads.

Counting the units you actually traverse: two in `analyze.md`, four bootstrap and command-resolution units, three workflow-resolution units, and six routing and fallback units (`AnalyzeDescribeIntent`, `AnalyzeDescribeIntentResume`, `AnalyzeNoMatch`, `AnalyzeOtherSkills`, `AnalyzeRouteIntentFlow`, `AnalyzeRouteIntentMenu`, `AnalyzeRouteLoadFlow` — seven, if you count the intent menu separately). Fifteen or sixteen nodes depending on how you count, across five files.

Where the first `WAIT` or `STOP_TURN` appears is the surprise. It is not in the routing at all: it is in `SimpleModeGate`, reached from `AnalyzeBootstrap`'s *second* `RUN`, and guarded — `EMIT_MENU`, `WAIT user.reply`, and `STOP_TURN` all carry `WHEN SIMPLE_MODE == unset`. So on the first invocation in a session the workflow stops for a human before it has resolved a single skill, and on every later invocation in the same session it does not stop there at all. The second candidate is `WorkflowResolution`, which `STOP_TURN`s when zero cf-* skills are discovered — a broken install.

And the answer to "which unit performs actual work rather than routing" is: none of them. Every terminal node either emits a menu and waits, or invokes some other skill entirely. That is the chapter's dispatch-table finding, confirmed by walking one instance of it.

3. **Write a minimal PDSL unit.**

```
UNIT ImplementationRequirementsGate
PURPOSE: Refuse to begin implementation until an approved requirements
        document is identified.
STATE:
  SET REQUIREMENTS_DOC: string (default unset, scope workflow_run)
  SET REQUIREMENTS_APPROVED: yes | no | unset (default unset,
                                          scope workflow_run)
WHEN:
  REQUIRE implementation work is about to begin
DO:
  SET REQUIREMENTS_DOC = the requirements document named by the user
  EMIT_MENU MissingRequirements WHEN REQUIREMENTS_DOC == unset
  WAIT user.reply WHEN REQUIREMENTS_DOC == unset
  STOP_TURN WHEN REQUIREMENTS_DOC == unset
  CONTINUE ImplementationPlan WHEN REQUIREMENTS_APPROVED == yes
RULES:
  NEVER write, edit, or create a source file while
       REQUIREMENTS_APPROVED != yes

MENU MissingRequirements:
  TITLE: No approved requirements document was named
  OPTIONS:
    1 name the document -> SET REQUIREMENTS_DOC; SET REQUIREMENTS_APPROVED
    2 write one first -> CONTINUE RequirementsAuthoring
    3 cancel -> EMIT "Implementation not started."; STOP_TURN
  INVALID:
    EMIT "Choose 1, 2, or 3."
    WAIT user.reply
    STOP_TURN
```

Twenty-four lines. Run it through `cfs pdsl validate` and the findings you are most likely to get first are structural rather than semantic: an unfenced block (the validator extracts fenced `pdsl` blocks, so the unit has to be inside one), a section header it does not recognise, or menu option numbering that is not consecutive from 1.

What the validator will *not* tell you is the interesting part, and you should notice it while fixing the things it does report. It has no idea whether `ImplementationPlan` exists. It has no idea whether the `NEVER` is enforceable, or whether the guarded `STOP_TURN` can be reached, or whether option 1 actually sets both state variables it claims to. It checks that the block is well-formed PDSL. Exercise 5 is about that gap.

4. **Audit a name in your own code.**

   One worked example, on a real shape you are likely to have.

   The name: a package called `middleware` in an HTTP service.

   The defining property, written before looking: in the Chain of Responsibility sense, each handler either handles the request or passes it to the next handler, and — the part people forget — the sender does not know which handler will service the request. In the pipeline-of-middlewares sense that web frameworks use, each stage receives the request and a `next` continuation, may transform either direction, and the composition order is explicit.

   The code that would have to exist: a link to the next handler that the current handler decides whether to invoke, and no stage reaching around the chain to call a specific other stage by name.

   The verdict, in the case I checked: split. Four of the five stages were genuine — they took `next` and either called it or short-circuited. The fifth mutated a shared request-scoped context object that two later stages read by key, which is the disqualifying evidence Chapter 6 lists for pipe-and-filter and is a weaker version of the same problem here: stage three and stage five are coupled through a channel that is not the chain.

   What it actually is: a chain of four handlers plus one context-mutating stage. Which is fine, and would have been fine under any name — the cost was that a new engineer reordered stages three and five, the compiler was happy, the tests passed, and an authorisation attribute silently became absent for one route.

   The transferable move is the ordering. Write the property down before reading the code. If you read first, you will find a property the code satisfies and call that the definition.

5. **The interpreter you cannot test.**

   You get stuck immediately, and precisely: there is nothing to import.

   Take `STOP_TURN`. To test what it means you would need a function that implements it, a state object it acts on, and an assertion that the state changed. None of the three exists. Grep the platform and the only module that knows the token is the validator, `utils/pdsl.py`, and it treats it as a string appearing in a block. There is no evaluator, no environment, no program counter. The nearest thing to a runtime is a language model reading markdown, and you cannot instantiate that in a test process, cannot assert on its internal state, and cannot get the same answer twice from it by construction.

   A dishonest version of this test is easy to write — prompt a model with a PDSL block, check that its reply stops where you expected — and it is worth articulating why that is not a unit test of `STOP_TURN`. It tests one model, at one temperature, on one input, on one day. Change the vendor and the test may fail without a single character of PDSL having changed. A test whose result depends on a component you did not write, cannot version, and cannot inspect is a monitoring probe, not a specification.

The test you *can* write:

```
def test_menu_without_invalid_branch_is_reported():
    findings = validate_pdsl_block(MENU_MISSING_INVALID)
    assert any(f.rule_id == "menu-missing-invalid" for f in findings)
    assert findings[0].line == 4
```

Deterministic, fast, and genuinely useful — it will catch the malformed menu before it reaches a user.

The difference between the two tests locates PDSL's semantics exactly. The validator test asserts on `PdslFinding`, whose fields are `rule_id`, `severity`, `line`, `column` — a linter's data model, coordinates in a text file. A semantics test would need to assert on an execution state that no artifact in the codebase holds. So the notation's *shape* lives in the repository and is testable, and the notation's *meaning* lives in model weights and is not. That is what "fully externalized interpreter" means when you try to write the test.

A.5 Chapter 5: Gears, the Plugin Runtime

1. **Find the contracts file and explain the cardinality split.**

   `libs/toolkit/src/contracts.rs` declares eight traits:

   - `Gear` — required of every gear. Many.
   - `SystemCapability` — `pre_init` and `post_init` hooks, both defaulting to no-ops. Internal; a small number of gears.
   - `DatabaseCapability` — returns migration definitions. Many.
   - `RestApiCapability` — registers routes, optional healthcheck. Many.
   - `ApiGatewayCapability` — the `rest_host`. **Exactly one.**
   - `RunnableCapability` — start and stop for stateful gears. Many.
   - `GrpcServiceCapability` — registers gRPC services. Many.
   - `GrpcHubCapability` — the gRPC hub. **Exactly one.**

   Two are singular and six are plural, and the two singular ones are the two that own a listening socket. That is the whole of the cardinality split: many things can add to a server, one thing can be the server.

   If two gears both declared `rest_host`, the failure is not at link time, not at `init`, and not at `start`. It is in the REST wiring phase, in `libs/toolkit/src/runtime/host_runtime.rs`, which counts the gears whose capability set answers to `ApiGatewayCap` and matches on the count: zero means either proceed or `RegistryError::RestRequiresHost` if any gear declared `rest`; one proceeds; anything else returns `RegistryError::MultipleRestHosts`, whose message is "multiple 'rest_host' gears detected; exactly one is allowed". The identical shape guards gRPC with `GrpcRequiresHub` and `MultipleGrpcHubs`.

   Two things worth taking from that. The check is a `match` on a count, which is about as cheap as an invariant gets — and the alternative failure, had nobody written it, is two gears racing to bind the same port with an error message from the operating system rather than from the runtime. And the failure arrives before the server starts, not after, so a misconfigured deployment refuses to come up rather than coming up half-wired.

2. **Trace a request from attribute to running route.**

   `#[toolkit::gear(name = "api-gateway", capabilities = [rest_host, rest, stateful], deps = [grpc_hub, authn_resolver], lifecycle(entry = "serve",`

`stop_timeout = "30s", await_ready))]` expands into a registrator function that knows how to add `ApiGateway` to a `RegistryBuilder`, submitted via `inventory::submit!`. At startup `discover_and_build` iterates every submitted `Registrator`, lets each fill the builder, and calls `build_topo_sorted`, which orders gears by the declared `deps` — so `grpc_hub` and `authn_resolver` are guaranteed to have been initialised before `api-gateway`.

Then, phase by phase, against the nine-line order in `host_runtime.rs`:

- **pre_init** — skipped. Only gears with the system capability participate, and this one declares `rest_host`, `rest`, `stateful`.
- **DB migrations** — skipped. No database capability declared.
- **init** — participates, in topological order after its two dependencies. This is where it resolves its `AuthNResolverClient` out of `ClientHub`, and where it fails loudly if auth is enabled and no `authn_resolver` gear is linked.
- **proxy-wiring** — skipped. This gear declares no `#[toolkit::consumes]` contracts; it resolves its one client directly.
- **post_init** — skipped. System gears only.
- **REST wiring** — participates twice, and this is the part worth being careful about. As the single `rest_host` it is the gear the phase validates the count of, and its `ApiGatewayCapability` `rest_prepare` builds the base router and `rest_finalize` attaches `/openapi.json`, `/docs`, and the health endpoints. As a `rest` gear it also takes its turn in the `register_rest` fold like any other route provider.
- **gRPC registration** — skipped. It declares neither `grpc_hub` nor `grpc`; it merely depends on the hub gear.
- **start** — participates. `stateful` plus `lifecycle(entry = "serve")` means the runtime calls `serve`, and `await_ready` means it waits for the readiness signal before moving on. This is where the socket finally binds.
- **stop** — participates, in reverse order, with the 30-second `stop_timeout` governing the two-phase shutdown.

Five phases skipped, four joined. The skips are the point: nothing was registered, nothing was stubbed, and no abstract method was overridden with a body that throws. The gear answered "no" to five questions by not mentioning them in an attribute.

3. **Write a minimal gear.**

```
/// Serves the build-info endpoint. Depends on tenant-resolver only so
/// its context is available before routes are registered.
#[toolkit::gear(
    name = "build-info",
    capabilities = [rest],
    deps = [tenant_resolver]
)]
pub struct BuildInfo {
    version: &'static str,
}

impl toolkit::contracts::Gear for BuildInfo {
    fn init(&self, _ctx: &toolkit::context::GearCtx) -> anyhow::Result<()> {
        Ok(())
    }
}

impl toolkit::contracts::RestApiCapability for BuildInfo {
    fn register_rest(
        &self,
```

```
        _ctx: &toolkit::context::GearCtx,
        router: Router,
        _openapi: &dyn OpenApiRegistry,
    ) -> anyhow::Result<Router> {
        let v = self.version;
        Ok(router.route("/build-info", get(move || async move { v })))
    }
    // healthcheck() not implemented: the trait default returns None.
}
```

When does its code run? In two phases only. `init`, once, guaranteed after `tenant-resolver`'s `init` because of the declared dependency. And REST wiring, once, when the fold reaches it — and that phase runs after DB migrations, as the trait's own doc comment promises, so any gear that needed a migrated schema has had it. It never starts or stops, because it declared no `stateful`; it is never asked for a healthcheck result, because the default `None` opts it out of the aggregate.

What happens if `tenant_resolver` is not linked? The `inventory` collection only contains registrators for crates actually linked into the binary, so the dependency names a gear the builder never sees. The failure comes from `build_topo_sorted`, at startup, as a named `RegistryError` — before any gear's `init` runs. That is the right shape: an unsatisfiable dependency graph is a configuration error, and it is reported as one rather than as a `None` somewhere in `init` or, worse, a service that starts and returns wrong answers.

There is a second thing that will happen and is easy to misread as a bug. If you build a deployment that includes `build-info` but not `tenant-resolver`, `cargo-shear` may also flag the crate as unused, because the macro's re-exports that keep the registration alive only exist in expanded code. The documented remedy is the workspace ignore list, not disabling the checker.

4. **Settle a pattern question the hard way.**

   Worked on the healthcheck registry, `libs/toolkit/src/healthcheck.rs`.

   Step 1, the defining property, before reading. Fowler's Registry is a well-known object that other objects use to *find* common objects and services. The operative verb is find. A Registry exists so that a caller who holds a key can obtain the thing.

   Step 2, what would have to exist. A lookup operation: some `get(key)`, `find(key)`, or index that returns a stored item to a caller who did not put it there.

   Step 3, the code. `RestHealthcheckRegistry` holds a read-write-locked `Vec` of `RegistryEntry`, a cached report, and a cancellation token. Its public surface is `new`, `with_cancellation`, `register`, and `report`. `register` pushes onto the vector. `report` clones the whole vector, runs every check concurrently via `join_all`, aggregates the statuses, and caches the result for two seconds.

   There is no lookup. Nothing keyed, nothing retrieved, no caller ever asks for one gear's healthcheck. The gear name is stored purely so it can be printed in the report.

   Verdict: rejected. What it actually is: a subscriber list with a fan-out aggregator. Registration is one-way, the collection is only ever traversed in full, and the interesting logic is in `compute_aggregate` — deciding what one composite status is, given several component statuses. That is much closer to a Composite over an Observer-style subscription list than to a Registry, and the substantive design content is the aggregation policy that Chapter 5 quotes from the trait doc comment: degraded keeps you in rotation, unhealthy does not.

   `OpenApiRegistry` lands the same way if you check it. Its trait is `register_operation`, `ensure_schema_raw`, `as_any` — an accumulator that builds one document, with `ensure_`-

`schema_raw` returning a canonical component name rather than a stored object. Also not a lookup.

Which is not a criticism of either. "Registry" is being used in the plain English sense of "the place things are registered", and both types do that well. It just is not the pattern, and the exercise is worth doing precisely because the name is so plausible that nobody checks.

5. **Compare two layer claims.**

`.dependency-cruiser.cjs` states three rules, all `severity: 'error'`, all expressed as forbidden import edges between path globs: L1 (`state`, `screensets`, `api`, `i18n`) may not import L2 or L3; L2 (`framework`) may not import L3 (`react`); and L3 may not import L1 directly, with the comment naming the replacement — go through `@gears-frontx/framework`'s re-exports.

Against the two criteria: strict one-way dependency direction between adjacent layers is present and machine-checked. A defined service interface at each boundary is present in the third rule specifically — `react-no-sdk` is what turns `framework`'s re-export surface into a boundary rather than a convenience, because it is the only permitted path from L3 to L1. The rules run in CI. Confirmed.

`discover_layers` satisfies neither. There is no dependency direction between its layers because there are no calls between them at all — the function returns a `List[ManifestLayer]` and `merge_components` folds it by precedence. Layer n offers layer $n + 1$ nothing, because layer $n + 1$ is a list element, not a caller. Rejected; it is a config-precedence merge cascade with inner-scope-wins semantics and a provenance record.

What Studio would have to add. The honest first answer is: rename it, because the mechanism does not want to be layered and forcing it would be worse. But the exercise asks for the check, so here is a concrete one that would make a narrower, defensible claim true.

The claim to defend: a kit-scope manifest may not reference paths outside its own kit root, and a repo-scope manifest may not reference paths inside a kit root. That is a real isolation property, it is the property a reader actually assumes when they see the word "layer", and today nothing enforces it — `_resolve_registered_kit_path` checks only that a kit path stays inside the project root.

The check: extend `cfs validate-kits` with a scope-containment rule that, for every component in every discovered layer, resolves its effective source path and asserts it is inside that layer's own root. Report a violation with the layer scope, the manifest path, the component ID, and the offending path — all four are already available, because `merge_components` records exactly that in its provenance table.

Where it runs: in CI, on every pull request that touches a manifest, at error severity. Locally as part of `cfs validate-kits`.

The violation it catches: a kit manifest that reaches into {`studio_root`}`/config/` to pick up a project rule file, which works on the machine where it was written, silently changes meaning in the next repository, and is the exact class of change a reviewer currently has no grounds to block.

One check, one CI job, and the word starts carrying a guarantee.

A.6 Chapter 6: Insight, Data and Lineage

1. **Find the invariant and its enforcement.**

What enforces it: nothing executable. There is no test, no lint, no dbt selector, and no CI job asserting that Gold never reads Bronze. What there is, is the property holding: grep `src/ingestion/gold/` for the string `bronze` and you get zero hits across all twenty models. Every Gold model reaches Silver through {{ `ref('class_...')` }} or through another Gold model, and {{ `source('bronze_...', ...)` }} appears only in the per-connector staging models that are themselves the Bronze-to-Silver step.

So the invariant is true today and is held up by convention plus code review. Given that the document calls it a hard constraint and that reason 3 for its existence is tenant isolation — a security property — that gap is worth naming rather than glossing. This is Chapter 7's framed sentence arriving in a different subsystem: a Gold model that added `source('bronze_slack', ...)` would violate no stated rule, trip no check, and look like every other model in the directory.

The check I would add.

Artifact: the dbt manifest, which already records every model's resolved dependencies — so nothing needs parsing by hand.

Condition: for every node whose `fqn` or tags place it in the Gold layer, assert that its `depends_on.nodes` and `depends_on.sources` contain no entry resolving to a `bronze_*` schema. One traversal, no SQL parsing, no false positives from the word "bronze" appearing in a comment.

Where: a dbt singular test or a small stdlib-only script run after `dbt parse`, wired into the existing `ci.yml`, failing the build. Not a warning — the invariant's own document says "not a recommendation", and a warning would make that a recommendation.

Bonus, and cheap: the same traversal can assert the documented exception is still an exception, by allowing exactly the named Silver bridge models such as `class_ai_org_usage` and failing on any new one added without a review.

2. **Trace one row from source to metric.**

   Walking `collab_metric_observations` backwards:

   (a) `gold/collab_metric_observations.sql` reads exactly one relation: `ref('collab_metric_evidence')`. It groups to canonical grain — one row per person, day, measure, subject, and dimension set — and drops rows where `entity_id = ''`, which is how an unresolved identity stays out of every serving relation.

   (b) `gold/collab_metric_evidence.sql` reads five Silver `class_*` relations: `class_collab_chat_activity`, `class_collab_meeting_activity`, `class_collab_email_activity`, `class_collab_document_activity`, and `class_focus_metrics`, each `FINAL`.

   (c) `silver/collaboration/class_collab_chat_activity.sql` is a `ReplacingMergeTree` incremental model whose entire body is `union_by_tag('silver:class_collab_chat_activity')` — a union over every per-source staging model carrying that tag. Its `depends_on` comments name the three: `m365__`, `slack__`, and `zulip_proxy__collab_chat_activity`.

   (d) `connectors/collaboration/slack/dbt/slack__collab_chat_activity.sql` reads `source('bronze_slack', ...)` directly, and depends on `slack__bronze_promoted` — the promotion model from the chapter, which carries the `@cpt-principle:cpt-dataflow-principle-promote-bronze` marker.

   (e) The Bronze tables themselves are written by the Slack connector, whose `connector.yaml` stamps `tenant_id`, `source_id`, `collected_at`, and `data_source` onto every record at extraction.

Forward to the evidence counterpart: `collab_metric_evidence` is the row-level relation the observations model aggregates. Given one number on a dashboard, an auditor can select the evidence rows sharing its `tenant_id`, `source_key`, `entity_id`, `metric_date`, `measure_key`, `subject_key`, and `dimensions`, and get back the individual contributions that summed, maxed, or minned into it — including which source system each came from. That is what makes "here are the fourteen items it counted" an answerable question.

On `workspace_id`: check this one carefully, because the shipped SQL and the decision record do not use the same word. The tenancy column in these models is `tenant_id` — stamped onto Bronze rows at extraction as `insight_tenant_id`, carried through the Silver `class_*` relations, and present on both the Gold evidence and observation rows. `workspace_id` as a literal column name appears in only a handful of places in the whole dbt tree: the three HR-directory `__to_class_people` models, which compute it as `coalesce(tenant_id, '') AS workspace_id`; the `class_people` DDL; one HR assertion test; and `gold/metric_entity_cohorts_current.sql`, which reads it back and aliases it *to* `tenant_id`.

So the ADR's "`workspace_id` on every Bronze, Silver and Gold table" is implemented under a different name on this path, with `workspace_id` surviving as a people-dimension alias. The isolation property is real and the column is present at every stage; the vocabulary drifted. Finding that is the exercise doing its job — it is a smaller cousin of the Unifier, caught by tracing rather than by reading.

3. **Apply the ten-minute test.**

   Your document, your verdict. Two things to hold yourself to.

   First, run the steps in the given order and do not skip step one. Reading the subtitle and status line first is what the original study failed to do, and the Unifier document said "Conceptual architecture" on line three. A document that labels itself conceptual has told you the answer before you have grepped anything, and every minute you spend after that is confirmation.

   Second, when you report which step was decisive, be honest about the common outcome: for most documents, steps two and three both return results and the verdict is "partially built." The clean binary of the Unifier — thirty-two mentions in markdown, zero in code — is unusually crisp. The more typical finding is that the mechanism exists under a different name, does two of the five things the document describes, and has no test for either. In that case the decisive step is usually step four: ask what would break if it vanished. If the answer is "the three things it actually does", you have located the real boundary between the design and the system.

4. **Write a minimal promotion model.**

```
{# Bootstrap model for GitHub bronze -> RMT promotion.
   Counterpart of `slack__bronze_promoted` for GitHub. See ADR-0002.
   The `promote_bronze_to_rmt` macro is idempotent - tables already
   on a Replacing engine are detected and skipped on later runs. #}

-- @cpt-principle:cpt-dataflow-principle-promote-bronze:p1
{{ config(materialized='view', schema='staging', tags=['github']) }}

{% do promote_bronze_to_rmt(
     table='bronze_github.pull_requests',
     order_by='(insight_tenant_id, insight_source_id, id)') %}
{% do promote_bronze_to_rmt(
     table='bronze_github.reviews',
     order_by='(insight_tenant_id, insight_source_id, id)') %}

SELECT 1 AS promoted
```

The `order_by` argument is the part to get right rather than copy: its prefix is the natural dedup key, so it must be the tenant, the source, and whatever identifies the entity in that table. Get it wrong and `ReplacingMergeTree` collapses rows that were not duplicates.

Why the final `SELECT` exists: dbt models are relations, and a model that compiles to no query is not a model dbt can materialise. The two {`% do %`} calls are pure side effects — they issue DDL against ClickHouse and return nothing — so the file has to end with something selectable for dbt to build a view over. `SELECT 1 AS promoted` is the smallest honest way to satisfy the tool while making it obvious to a reader that the value is a formality rather than data.

5. **Score the marker convention.**

   First, the count, because the chapter states it two ways and only one of them is right. Grep `@cpt-principle` across Insight and you get 26 markers across **five** distinct identifiers, not six:

   - `cpt-dataflow-principle-promote-bronze` — 22 occurrences
   - `cpt-dataflow-principle-staging-then-union` — 1
   - `cpt-dataflow-principle-rmt-with-version` — 1
   - `cpt-dataflow-principle-ephemeral-passthrough` — 1
   - `cpt-dataflow-principle-unique-key-formula` — 1

   Five is the number to work from. And the chapter's other claim holds exactly: grep every markdown file in the repository for `cpt-dataflow-principle` and you get zero. Not one of the five is defined in prose anywhere.

   Taking `cpt-dataflow-principle-promote-bronze`, the twenty-two claiming models are twenty-one per-connector `*__bronze_promoted.sql` files plus the shared macro `dbt/macros/promote_bronze_to_rmt.sql` itself. Inferred from those alone, the definition it ought to have:

   > Airbyte writes Bronze tables as plain `MergeTree` with `destinationSyncMode='append'`, so a full-refresh stream appends every row on every sync and Bronze accumulates N copies of each entity. Before any Silver model reads a Bronze table, that table is promoted once to `ReplacingMergeTree`, versioned on `_airbyte_extracted_at` and ordered by a natural key whose prefix is `(insight_tenant_id, insight_source_id, ...)`, so deduplication is a property of the storage engine rather than a `QUALIFY ROW_NUMBER` paid on every downstream read. The promotion is idempotent — a table already on a Replacing engine is detected and skipped — and is performed by the shared `promote_bronze_to_rmt` macro through an atomic create-and-exchange, so it is safe to re-run after a failure. Every connector owns exactly one bootstrap model that calls the macro once per Bronze table it produces.

   Would writing that down have helped? Yes — and the argument is narrower than "documentation is good."

   Three facts in that paragraph are load-bearing and none of them is inferable from the models that claim the principle. The natural-key prefix convention is visible only in the arguments individual models pass. The idempotence guarantee and the create-and-exchange race window are stated only in the macro's own header comment. And the caveat that replicated engines are not handled is one comment away from being lost entirely.

So the choice is not really between a definition and nothing. It is between a definition and a macro docstring that already does most of the job. Which suggests the smallest useful change: make the marker resolvable to that docstring — one short markdown file per principle, five files, each pointing at the one place the reasoning already lives — rather than restating the reasoning and creating a second copy to keep current. That is the version that survives, because it adds a name for something rather than a duplicate of it.

The version that would genuinely be worse than nothing is a prose definition maintained separately from the macro, drifting from it, and looking authoritative. Chapter 6 makes that argument about the Unifier; it applies just as well to a principle document nobody re-reads.

A.7 Chapter 7: Security and the Bigger Picture

1. **Find the clippy configuration and extend it.**

   First, correct the premise, because the file has changed since the excerpt in the chapter: `gears-rust/clippy.toml` bans **five** methods, not four. The fifth is `sea_orm::Select::count`, with the reason "Use SecureSelect::count() after .secure().scope_with()" and `allow-invalid = true` — the flag that tells Clippy not to fail if the path cannot be resolved in the current dependency graph. Worth noticing before you go looking for four call sites and find five wrappers.

   The corresponding secure wrappers are all in `libs/toolkit-db/src/secure/`. The call chain every one of them requires is the same three steps, visible throughout `libs/toolkit-db/tests/`:

```
entity::Entity::find() // an ordinary sea_orm Select
    .secure() // -> SecureSelect<E, Unscoped>
    .scope_with(&scope) // -> SecureSelect<E, Scoped>
    .all(&runner).await? // only exists on the Scoped type
```

   The type parameter is doing the work: `all`, `one`, and `count` are implemented only for `SecureSelect<E, Scoped>`, so skipping `scope_with` is not a lint failure but a compile failure. The lint exists to close the other door — calling `.all()` on the raw `Select` and never entering the secure types at all. `SecureUpdateMany` and `SecureDeleteMany` follow the same shape with `exec`.

   A sixth entry worth adding:

```
{ path = "sea_orm::SelectTwo::all",
  reason = "Use SecureSelect::find_also_related(..).all() after \
            .secure().scope_with()" },
```

   Why it is needed. `SecureSelect` already provides `find_also_related` and `find_with_related`, returning `SecureSelectTwo` and `SecureSelectTwoMany` with their own scoped `all` and `one`. So the secure path for joins exists and is supported. But the moment a developer writes `Entity::find().find_also_related(Other)` on the plain SeaORM types, they hold a `sea_orm::SelectTwo`, and `SelectTwo::all` is not on the banned list — so the existing five do not stop it. The result is an unscoped join executing against two tables, which is a strictly worse outcome than the unscoped single-table select the lint already blocks.

   What it would break. Any legitimate unscoped join: migrations, system tables that carry no tenant column, and test fixtures. Each of those needs a local `#[allow(clippy::disallowed_methods)]` with a comment, which is the correct cost — it makes the exception visible at the call site instead of invisible everywhere. The honest

downside is that a bare `#[allow]` is easy to paste, so the entry buys you a reviewable artifact rather than an impossibility.

The harder candidate, worth naming and not adding lightly, is `sea_orm::Select::paginate`. It is the most on-point gap in the whole list — this chapter is about authorisation that composes with pagination — and it cannot responsibly be banned until `SecureSelect` grows a scoped paginator, because today there is nothing to point people at. Banning a method with no replacement teaches people to allow the lint.

2. **Trace an authorised query end to end.**

   The path: the domain gear acting as PEP calls the AuthZ Resolver gear, which forwards to the vendor's PDP; the PDP returns an AuthZEN response extended with constraint predicates; the PEP compiles those predicates into SQL through the derive-macro property-to-column mapping; the scoped query executes and the database returns only rows the policy allows — so the count the database reports is already the authorised count, and page three means page three.

   Where each of the four failures is caught:

   A missing decision field — at the PEP, immediately on parsing the decision-point response, before any SQL is built. Decision false or missing denies everything. This is the earliest of the four and the cheapest.

   An unknown predicate type — at the PEP, during constraint interpretation, after the response has parsed. The rule is narrower than the first: the unknown predicate is treated as *false* rather than the whole request denied. Since constraints are OR-ed, one unusable predicate removes one disjunct, and if it was the only one the OR over an empty set is false, which denies. So the same conservative direction reached one step later and with finer granularity.

   An `In` predicate with an empty value list — at the same step, and it is the one with two independent reasons. Semantically an empty set means match nothing, which is a deny. Mechanically, handing it to the ORM would generate `WHERE col IN ()`, which is a syntax error on some engines — so even a system that wanted to be permissive could not be. It is classified as a decision-point contract violation, which means the PEP logs an error naming the PDP as the faulty component before failing closed. That distinction is the difference between a debuggable outage and a mysterious empty list.

   A scope with no tenant identifiers at all — not caught by a check at all, which is the point. An empty access scope compiles to a predicate that is unsatisfiable by construction, so the database returns zero rows because the SQL cannot match anything. No branch ran and failed; there is nothing to bypass. Compare the ordinary arrangement, where an empty filter list means no filtering and therefore everything, and you have the same code path with the opposite default.

   Note the progression across the four: deny at parse, deny at interpretation, deny plus attribution at interpretation, and finally deny by algebra. The last one is the cheapest to run and the hardest to remove by accident.

3. **Write a minimal fail-closed rule set.**

   Worked on a licence check: a service that asks an external entitlement API whether a workspace may use a paid feature.

   (a) Response absent (no HTTP response at all) → feature off.

   (b) Connection refused or DNS failure → feature off.

   (c) Timeout → feature off. Do not retry inline; the caller is holding a request.

(d) HTTP 5xx → feature off, log as provider fault.

(e) HTTP 4xx other than 404 → feature off, log as *our* fault: a 401 means our credentials are wrong and somebody has to know.

(f) HTTP 404 for the workspace → feature off. Unknown workspace is not entitled.

(g) Body is not valid JSON → feature off, log as contract violation naming the provider.

(h) Body is valid JSON but the `entitled` field is absent → feature off, log as contract violation.

(i) `entitled` present but not a boolean → feature off, log as contract violation.

(j) `entitled: true` with an `expires_at` in the past → feature off. An expired grant is not a grant.

(k) `entitled: true` with a well-formed unexpired grant → feature on. One branch out of eleven.

Then the honest part. Counting the branches the code actually had before writing this list: three. Case 11, a generic "request failed" catch that covered cases 1–5, and an implicit case for everything else — which fell through to a truthiness check on a possibly-absent field, meaning case 9 with a non-empty string body would have turned the feature *on*.

That is the value of the exercise, and it is the same value the eleven rules have in the original. None of the branches is clever. What is hard is being certain you have enumerated every way the input can be wrong, and a numbered list is how you find out you have not. Write the list before you read your code, or you will enumerate the cases your code already handles.

4. **Re-verify two rows.**

   The current state of any vendor's documentation is not something this book can supply, and inventing it would be exactly the failure Chapter 6 is about. What can be said in advance is what your findings will look like, and the shape is predictable enough to be useful.

   The openness column re-verifies in seconds and is the only one that ever does. The control-flow column is where you will find the most drift, because "spec-driven" and "agentic" are marketing positions as much as architectural ones, and vendors move between them. The federation column is the one you will most often be unable to confirm: absence of a claim is not a claim of absence, and "not documented" in Table 7.1 means exactly that.

   Sort your findings into the three buckets the exercise names, and be strict about the boundary between the second and third. "Supports multi-repo workflows" on a features page is marketing copy. A documented command, configuration key, or API that operates across repositories is a confirmation.

   On shelf life, the paragraph to write is the one Chapter 7 already earns with the Windsurf and Devin entry: a table in this category has a half-life measured in months, and the fastest-moving cell is not any technical axis but the company name in column one. The practical conclusion is that a comparative table should be treated as a record of what was true on a date — which is why the chapter states its date — and re-verified for the two or three rows that matter before any decision rests on it. Copying such a table forward without re-checking is how a rebrand becomes a fact.

5. **Audit one claim in your own system.**

   One worked example, on a claim of the kind that is easy to make and rarely checked.

   The claim: a module's README states that a service is "stateless, so it can be scaled horizontally."

The defining property: no request's outcome depends on anything the process retains from a previous request. Equivalently: any request can be served by any instance, and restarting an instance mid-traffic loses nothing but the in-flight requests.

What must exist for it to hold: no mutable module-level or instance-level state read across request boundaries; no in-process cache whose absence changes an *answer* rather than a latency; no sticky-session assumption anywhere in routing.

The verdict: split. Two of three held. The third was an in-process memoisation of a permission lookup with a five-minute expiry — which does change answers, because a revoked permission stayed effective on whichever instance had cached it, for up to five minutes, and only on that instance. So the same user got different answers from different pods, which is precisely what statelessness is supposed to rule out.

The remedy, and why the cheap one is the right one. A rename — "mostly stateless, with a five-minute permission cache" — would be honest and would fix nothing. Moving the cache to shared storage is the correct engineering answer and is a week of work. The smallest check that makes the claim true is neither: a test that constructs two instances, revokes a permission through one, and asserts the other denies immediately. It runs in a second, it fails today, and it converts a paragraph in a README into something a reviewer can point at.

That is the general shape worth taking away. When a claim fails, the choice is usually between renaming it, rebuilding to satisfy it, and adding the smallest artifact that would have caught the violation. The third is almost always available and almost never the one people reach for first.

A.8 Chapter 8: The Kit-Authoring Mechanism

1. **Read a real manifest and account for every resource.**

   `studio-kit-sdlc/.cf-studio-kit.toml` declares **39** `[[kits.resources]]` entries under a single `[[kits]]` block with `slug = "sdlc"`, `version = "1.0"`. By kind:

Kind	Count	What they are
`skill`	11	the eleven workflow files
`other`	11	5 `rules.md` + 5 `examples/example.md` + `codebase/rules.md`
`template`	7	5 artifact templates + 2 PR report templates
`checklist`	6	5 artifact checklists + `codebase/checklist.md`
`script`	1	`scripts/change-impact-analysis.toml`
`rule`	1	`agents-res.md`
`directory`	1	`scripts`
`constraints`	1	`constraints.toml`

   Two of the nine kinds are unused: `agent` (the kit configures no workers) and nothing else — unlike `compete`, this kit does declare one `script` resource alongside its `directory`.

   Twelve resources are public: the eleven `skill` entries (`workflow_doc_prd`, `workflow_doc_adr`, `workflow_doc_design`, `workflow_decompose`, `workflow_doc_feature`, `workflow_implement`, `workflow_change_impact_analysis`, `workflow_reverse_engineer`, `workflow_pr_review`, `workflow_pr_status`, `workflow_migrate_openspec`) and the one `rule`, `agents` — which is exactly the three kinds the validation table permits `public = true` on.

Thirteen carry `user_modifiable = false`: those same twelve, plus `constraints`. One sentence each for why:

The eleven workflows are the kit's control flow. A user editing one is editing the process, not configuring it, and their edit would be a silent fork of a file the kit will update. If you want a different process, that is your own kit.

`agents` is the generated `AGENTS.md` entry point — a file the host tool reads to find the kit at all. Editing it is how you make the kit stop being discoverable.

`constraints` is the deterministic contract that `cfs validate` reads. It is the one file in the kit whose meaning is enforced mechanically, and letting a user relax it locally would mean the same repository validates differently on two machines.

Everything else — every template, every checklist, every `rules.md`, every example — is `user_modifiable = true`. That is the kit author saying: the process is ours, the content is yours.

Where `source` **and** `install_path` **differ**: exactly one resource, `agents`, authored as `agents-res.md` and installed as `AGENTS.md`. The rename exists because a file literally named `AGENTS.md` sitting at the kit root would be picked up as an agent entry point in the kit's *own* repository, which is not what the author wants while developing the kit. All 39 entries do carry an explicit `install_path`; for the other 38 it is identical to `source`, which is why both install modes behave the same way and why the distinction is easy to miss until you meet this one entry.

2. **Write the smallest legal kit and prove it is legal.**

   The two files:

```
kits/scratch/.cf-studio-kit.toml
kits/scratch/templates/NOTE.md
```

```
manifest_version = "1.0"

[[kits]]
slug = "scratch"
name = "Scratch"
version = "0.1"

[[kits.resources]]
id = "note_template"
kind = "template"
source = "templates/NOTE.md"
```

   A clean run reports the kit registered, the integration files written, and `cfs validate-kits` passing with no findings. Record the exact output for your version; what follows are the four failures, whose messages come from `utils/kit_model.py` and are stable.

   Remove `manifest_version`**:**

```
.cf-studio-kit.toml: missing required manifest_version. Update
Constructor Studio with ‘pipx upgrade constructor-studio‘ and retry.
```

   Set `type = "folder"`**:**

```
Resource ’note_template’: type must be ’file’ or ’directory’
```

   Set `public = true` **on the template:**

```
Resource ’note_template’: public=true is only allowed for skill,
agent, and rule resources
```

   Add an `artifacts.PRD` **binding to it:**

```
Resource 'note_template': artifacts bindings are only allowed for
constraints resources
```

Three of those four are model answers for what a validator message should be: they name the offending resource, state the rule, and enumerate the legal values, so the fix is obvious without opening documentation.

The hardest to diagnose is the first, and interestingly it is hardest *because of* its remedy text rather than despite it. "Update Constructor Studio with `pipx upgrade`" is good advice for the neighbouring failure — an unsupported `manifest_version` value, which really does mean your tooling is behind — and actively misleading for this one, where the fault is a missing line in a file you just wrote. Without this chapter you would plausibly spend ten minutes upgrading `cfs`, confirming the version, and re-running before looking at the first line of your own manifest.

3. **Predict the public names.**

   With `slug = "acme"`, the prefix rule in `utils/kit_model.py` leaves a name alone if it equals `cf-acme` or begins with `cf-acme-`, and otherwise prepends `cf-acme-`:

Frontmatter `name`	**Published as**	
`review`	`cf-acme-review`	correct
`cf-review`	`cf-acme-cf-review`	the trap
`acme-review`	`cf-acme-acme-review`	clumsy
`cf-acme-review`	`cf-acme-review`	correct, idempotent

   Ship the first or the fourth; they produce the same string. The fourth is the convention both real kits follow, and it is marginally better because the file's own frontmatter then tells you the name a user will type, with no mental substitution.

   `cf-review` is the trap, and it is a good one because it looks already-prefixed. The guard checks for `cf-<slug>-`, not for `cf-`, so a name that carries the generic prefix but not the kit's gets the kit's prefix prepended to it and you publish `cf-acme-cf-review`. The same logic makes `acme-review` into `cf-acme-acme-review`. Neither is an error and neither is caught by any validator — you find out when you type the name you expected and nothing happens.

   To verify the no-double-prefix rule against a real kit: `studio-kit-sdlc` ships `workflows/doc-prd.md` with frontmatter `name: cf-sdlc-doc-prd` under `slug = "sdlc"`, and `studio-kits-pm` ships `kits/compete/workflows/request.md` with `name: cf-compete-request` under `slug = "compete"`. Both already start with their prefix, both install unchanged, and neither becomes `cf-sdlc-cf-sdlc-doc-prd`.

4. **Register mode against copy mode, empirically.**

   In **register** mode, `core.toml` gains a `[kits.scratch]` table carrying `format = "CFS"`, the kit `path`, the `version`, `install_mode = "register"`, and the source provenance. What it does *not* gain is the thing to look for: the registration code explicitly removes any `resources` table for a register-mode kit. There are no per-resource bindings recorded at all.

   In **copy** mode the same table appears with `install_mode = "copy"`, and a `[kits.scratch.resources]` table is written with one binding per declared resource, each pointing at the copied file inside the setup directory.

That single difference explains everything else. In register mode there are no persisted bindings because there is nothing to bind to — effective resource paths are re-derived at run time from the manifest `source` plus the registered kit root. So editing `templates/NOTE.md` in your kit directory is live immediately, with no reinstall, because the file being read is the file you edited. In copy mode the persisted binding points at a snapshot, so your edit changes a file nothing reads until you reinstall.

For a company kit, register mode, and the reason that is not "the guide says so": there is exactly one copy of every file, so there is nothing to drift. Copy mode gives you two files with the same content and no mechanism telling you when they stop agreeing — and the failure mode is not an error, it is a document generated from last month's template while this month's sits in git looking authoritative. The live-editing convenience is pleasant; the absence of a second copy is the actual argument.

5. **Find something the validator cannot catch.**

   One worked example. This workflow file passes `cfs validate-kits` cleanly — it is a well-formed `skill` resource with valid frontmatter — and violates the first behaviour check:

```
---
cf-studio: true
type: workflow
name: cf-scratch-note
description: Invoke when the user asks to record a scratch note.
version: 1.0
purpose: Create a NOTE artifact.
---

## Step 1: Draft the note
Write {artifacts_dir}/NOTE/{yyyy-mm-dd}-{slug}.md from
{note_template}, filling every section with your best reading of the
user's request.

## Step 2: Confirm the target
Ask the user which system the note belongs to and correct the front
matter if the answer differs from your assumption.
```

   It writes a file in step 1 and asks its clarifying question in step 2.

   No file-format check could detect that, and the reason is worth being precise about rather than hand-waving. The validator's entire model of this file is a resource entry plus YAML frontmatter: does `kind` take a legal value, is `source` resolvable, is `public` allowed for this kind, does the frontmatter parse. The body is prose, and the property being violated — "ask before you write" — is a claim about the *order in which a model will do things when it reads this prose*. There is no artifact in the file that encodes ordering; there is only text that a reader happens to process top to bottom. Checking it would require executing the workflow, and Chapter 4 established that the thing that executes it is a language model.

   The smallest addition that makes it visible to a reviewer: a `Route` section, borrowed from `compete`, declaring the step count and the question budget before step 1 — for example "2 steps → NOTE. Questions: 1 confirmation (Step 1). Step 2 runs without questions." It does not stop the violation. What it does is put the intended interaction contract in the file, next to the steps, where a reviewer reading the diff can see that the declaration and the steps disagree. A checklist item saying "confirm the route declaration matches the step order" then has something concrete to point at, which is the difference between a rule a reviewer can enforce and a rule they have to invent on the spot.

6. **Choose the mechanism.**

   (a) Data-protection impact assessment before implementation. Kit. A DPIA is a document kind with its own structure, its own reviewer, and its own claim boundary —

and the gate is "the implementation workflow must not proceed without one", which is a constraints-and-workflow concern, not a runtime one. Decisively: the person who owns this requirement is a privacy officer who cannot compile the project, and that settles it.

(b) Read-only HTTP endpoint exposing requirement approval status. Gear. Serving HTTP is not something a kit can do at all; kits are markdown and TOML read by an agent. This needs a crate implementing `RestApiCapability`, registering a route, and reading the artifact state.

(c) ADRs must carry a mandatory "options rejected" section with at least two entries. Kit — and specifically, this splits across two levels. The required heading is a `constraints.toml` entry with `required = true`, which `cfs validate` enforces mechanically. The "at least two entries" part is not a heading contract; it is a counting problem, so it belongs either in a small script or, if you are willing to accept judgement, in the ADR checklist. Note which half is deterministic: the section existing is checkable, the section being substantive is not.

(d) CPT identifiers in a database, queryable across forty repositories. Both, and this is the case the exercise is really about. The boundary falls where the data stops being text.

The kit side owns the identifiers themselves: the CPT patterns in `constraints.toml`, the artifact kinds that declare them, the rule that they appear verbatim in documents and code. That is where the identifiers are defined, and nothing about it changes.

The gear side owns everything that makes them queryable at scale: ingestion, storage, an API. And there is already a boundary artifact between the two — `cfs map` produces the graph as data, so the gear consumes map output rather than re-implementing the scanner. Getting that seam right is the whole answer. If the gear parses markdown itself you now have two scanners that will disagree, and the disagreement will surface as a dashboard that is confidently wrong.

The general test, from the end of Chapter 8: kits change what the agent knows, gears change what the system does. When you want both, the kit defines the vocabulary and the gear serves it.

A.9 Chapter 9: A Real Kit, Walked Through

1. **Verify the counts and find the drift.**

   The counts themselves hold. `kits/compete/artifacts/` contains eleven directories: `CANDIDATE-REGISTER`, `COMPANY-DOSSIER`, `COMPANY-PROFILE`, `COMPARISON-MATRIX`, `FEATURE-TAXONOMY`, `MECHANISM-STUDY`, `ORIENTATION-CARD`, `OSS-REVIEW`, `RESEARCH-REQUEST`, `RESEARCH-REVIEW`, `WATCH-REPORT`. `kits/compete/workflows/` contains fourteen files. `kits/compete/scripts/` contains eight Python files. The root manifest declares 63 resources.

   `SKILL.md`'s frontmatter `description` agrees on both lists: it names all eleven artifact kinds and all fourteen workflows explicitly.

   Two disagreements, both in prose, both on the README side.

   First, the version. `kits/compete/conf.toml` and the repository-root `.cf-studio-kit.toml` both say `1.15.0`. `kits/compete/README.md` says "Version 1.14.0" in its Status section. Two machine-read copies agree; the prose copy is a minor version behind.

   Second, the flow table. `README.md`'s table lists thirteen skills. The one it omits is `cf-compete-taxonomy`, and with it the `FEATURE-TAXONOMY` artifact kind — which is the same kind Chapter 9 singles out as a normalisation contract the comparison matrix binds to.

Search the README for "taxonomy" and you find one passing mention in a paragraph of release notes, and no row. So a reader who learns this kit from its README does not learn that a fourteenth workflow exists.

Which source to trust, and why: the directory listing first, because it is the thing that is actually installed; then the manifest and `SKILL.md`, because both are read by machines and both would break visibly if wrong; then the README, which is read only by people and therefore fails silently. Note that the two drifts are the same failure — a fact duplicated into prose — and that the prose copy lost both times.

The smallest change that makes one of them impossible to get wrong: for the version, delete it from the README. Not update it — delete it. The README can say "see `conf.toml`" or nothing at all, and the number then exists in exactly the two places that are checked. Chapter 10's instruction to keep exactly one authoritative copy of a version number is this finding stated as advice; here is the case it came from.

For the flow table, the equivalent move is generation rather than deletion: a CI step that regenerates the table from the workflow files' own frontmatter `name` and `purpose` and fails if the committed README differs. That is a slightly larger change and it earns its keep, because the flow table is genuinely useful to a reader in a way a version string is not.

2. **Trace one rule through four files.**

   In `template.md`, eight times, once per cell of the Request Fields table:

```
| Target | {A named company, a product, a required capability, ...
        - or "unspecified"} |
```

   In `rules.md`, twice. As a step in `RequestAuthoring`'s `DO` block:

```
- RUN set every field the requester did not state to the literal value
  "unspecified" - never infer, default, or borrow a value
```

   and as an invariant in the same unit's `RULES` block:

```
- ALWAYS render an omitted field as the literal "unspecified" so the
  omission stays visible in every later artifact
```

   plus, in `RequestOmissions`, the prohibition with its identifier:

```
- NEVER CPT-REQ-NO-001 (CRITICAL): silently defaulting or inferring an
  omitted field instead of recording the literal "unspecified"
```

   In `checklist.md`, three times — in the applicability context at the top, as a MUST HAVE item ("Every field carries either a stated value or the literal `"unspecified"` — no blank cells, no dashes standing in for omission"), and as the MUST NOT HAVE entry carrying the same identifier, `CPT-REQ-NO-001 (CRITICAL)`.

   In `workflows/request.md`, three times: in the frontmatter `description` the router matches against, in Step 2 ("Set every field the requester did not state to the literal `"unspecified"`. Present the field table — including every `"unspecified"` ..."), and in the Validation Criteria checklist at the end of the file.

   What each catches that the others cannot. The template catches it at the moment of authoring: the instruction is in the cell where the decision is made, competing with nothing. The rules file catches it at the moment of *planning* the write — it is what stops a model from deciding, before authoring, that a sensible default would be more helpful. The workflow's Step 2 catches it at the one moment a human who could supply the missing value is present and paying attention, which is why "including every unspecified" matters: showing only the filled fields would let an omission pass unremarked in front of the one

person who could fix it. And the checklist catches it after the fact, in review, when the artifact already exists and the requester has gone.

Which could a deterministic check enforce? Read `constraints.toml` carefully and the answer is: none of them. The rule appears there once, in the `description` field of the `req-fields` heading constraint — as documentation for a human reading the contract, not as anything the validator evaluates. `cfs validate` confirms that a level-2 heading matching `Request Fields` exists. It does not look inside the table. The kit's own architecture guide says so in as many words: `cfs` "cannot see table rows."

So the kit's most important epistemic rule — the first of its four design principles — has *zero* deterministic enforcement. What holds it up is four expressions of the same sentence at four moments, plus a user gate that displays the gaps. That is worth sitting with, because it inverts the natural assumption. The deterministic layer guarantees the document's shape. The guarantee that the document is honest comes from redundancy and from showing a human the thing they would otherwise not look at.

3. **Read the second half of a script.**

   The classification, from the register next-action block of `main`:

   For each register row, `expected` is the list of artifact kinds whose `ACTION_KINDS` pattern matched the row's next-action text, deduplicated with order preserved. If `expected` is empty the row prints as `MANUAL` and the loop moves on. Otherwise each expected kind is looked up among the artifacts of that kind, matched against the candidate's name tokens, and sorted into `fulfilled` or `missing`. Then: nothing missing is `done`; something missing *and* something fulfilled is `partial`; everything missing is `open`. Both `partial` and `open` increment `open_actions`.

   So in your own words: `partial` means the row asked for more than one follow-up and some but not all of them were produced. `open` means the row asked for follow-ups and none of them exists. The distinction only arises because a next action may name several things.

   Why `ACTION_KINDS` is checked in order with every match collected, rather than stopping at the first hit: because that is exactly the case the table exists for, and the code comment says so — "an action may name several follow-ups: 'Orientation card; verify open-source posture'". Stopping at the first match would classify that row as needing only an `ORIENTATION-CARD`, report `done` once the card existed, and silently lose the OSS review. Collecting every match is what makes `partial` a reachable state at all. The ordering is preserved so the report reads in the order the analyst wrote the action, not in dictionary order.

   Two constructed rows.

   `MANUAL: | Kiro | Ask legal whether the EULA permits benchmarking |`. No pattern in `ACTION_KINDS` matches — there is no "card", "profile", "dossier", "compare", "mechanism", "watch", or open-source phrasing anywhere in it — so the script prints `MANUAL Kiro: no artifact kind recognized in next action`.

   `partial: | Kiro | Orientation card; verify open-source posture |`, in a run where `ORIENTATION-CARD/2026-03-04-kiro.md` exists but no OSS review mentions Kiro. Two kinds expected, one fulfilled, one missing, so: `PARTIAL Kiro: fulfilled: ORIENTATION-CARD (...); missing: OSS-REVIEW`.

   Which is the better outcome for a reviewer: `partial`, without much argument. It tells you the exact artifact kind that is missing and the exact candidate it is missing for, which is a to-do item. `MANUAL` tells you the script gave up and a human must read the row — honest, and better than a wrong classification, but it is work the tool has handed

back. A run with many `MANUAL` rows is a signal about the register's phrasing rather than about the research: the analyst is writing next actions in vocabulary `ACTION_KINDS` does not recognise, and either the phrasing or the table should change.

4. **Install it and produce one artifact.**

   Your run, your output, but the expected shape is determinate enough to check yourself against.

   With a deliberately sparse ask — a target and nothing else — seven of the eight fields should come back `unspecified`. Target is the one you supplied; decision question, scope, comparison frame, depth, requested output, evidence cutoff, and access boundary have nothing to draw on. If any of those seven came back with a value, you have just observed `CPT-REQ-NO-001` in the wild, and that is a more interesting result than a clean run — record which field it was and what the model inferred it from.

   The route should be announced before Step 1, because the workflow's `Route` section commits to it: "6 steps → RESEARCH-REQUEST. Questions: 1 confirmation (Step 1: the restated ask), 1 decision (Step 2: the eight request fields), 1 closing confirmation (Step 6). Steps 3-5 run without questions." Three questions declared.

   Compare against what you actually got. A count higher than three usually means Step 2's field table was split across turns; a count lower usually means a confirmation was skipped, which is the more serious of the two because Step 2 is the gate where the `unspecified` fields are shown to the one person who can fill them in.

   `cfs validate --artifact` on the result should report PASS on structure, because Step 4 loops until it does. If it does not, the loop was not run — which is itself worth reporting, since Step 4 carries the `// turbo` marker precisely so it can run unattended.

5. **Break the deterministic contract on purpose.**

   The four edits, against `constraints.toml`'s `RESEARCH-REQUEST` section, which declares five required headings (`req-h1-title`, `req-fields`, `req-boundary`, `req-evidence`, `req-traceability`) and one identifier kind, `req`, with `template = "cpt-{system}-req-{slug}"` and `headings = ["req-h1-title"]`.

   Rename the H1 to just "Research Request" → `heading-missing`. The `req-h1-title` pattern is `'Research Request\s*[em/en/hyphen]\s*.+'`, so a bare title matches nothing and the required level-1 heading is reported absent, with the pattern and the constraint's description quoted back at you. Caught deterministically, and note what is being caught: not a missing heading but a heading with no content after the dash.

   Delete the `## 4. Traceability` heading → `heading-missing` for `req-traceability`. If you regenerated the table of contents beforehand you will also collect a `toc-stale` or `toc-heading-not-in-toc` finding, which is a second, independent check noticing the same edit.

   Change the ID to `cpt-myresearch-request-something` → two findings. `id-kind-not-allowed`, because `request` is not a kind this artifact declares (the allowed set is `['req']`); and `required-id-kind-missing`, because with the only `req` identifier malformed the artifact now has none and `required = true`.

   Add `TODO` to Section 2 → *nothing*. This is the one that is not caught, and it is worth checking rather than assuming: the placeholder regular expression in Studio's validator (`_CDSL_PLACEHOLDER_RE`, matching `TODO`, `FIXME`, `XXX`, `TBD`) fires only on CDSL step lines, and `RESEARCH-REQUEST` contains no CDSL. A `TODO` in ordinary prose passes structural validation cleanly.

What would have caught it: `CPT-REQ-NO-005 (HIGH)`, which exists in both `rules.md` and `checklist.md` and is enforced by the semantic review in Step 5, not by Step 4. If you wanted it deterministic, it is four lines of Python and belongs in the kit's scripts directory next to the other eight — a counting problem, in Chapter 10's terms, not a language problem.

That the kit's authors left it to the review layer rather than scripting it is a defensible call, and the reason is that placeholder detection has false positives — a research request may legitimately quote a requester who wrote "TODO" — and a checker that fails a build for something that is not wrong gets disabled.

6. **Judge the directory declaration.**

The eight replacement entries:

```
[[kits.resources]]
id = "script_build_evidence_index"
kind = "script"
source = "kits/compete/scripts/build_evidence_index.py"
install_path = "scripts/build_evidence_index.py"
type = "file"
user_modifiable = false
description = "Builds the EVD-### evidence index from a retained-evidence folder"

# ... and, with the same shape:
# script_check_evidence check_evidence.py user_modifiable = false
# "Checks the evidence layer: locators resolve, dates are ISO, enums are legal"
# script_check_artifact_refs check_artifact_refs.py user_modifiable = false
# "Validates one artifact's claim rows, evidence locators, and back-references"
# script_check_coverage check_coverage.py user_modifiable = false
# "Reports a research run's cross-artifact coverage; a report, not a gate"
# script_mine_terms mine_terms.py user_modifiable = true
# "Extracts candidate feature terms from evidence text for taxonomy drafting"
# script_render_matrix_deck render_matrix_deck.py user_modifiable = true
# "Exports a COMPARISON-MATRIX as a slide deck"
# script_render_pricing render_pricing.py user_modifiable = true
# "Renders the pricing benchmark view"
# script_render_views render_views.py user_modifiable = true
# "Exports csv and html views of an artifact set"
```

The `user_modifiable` split is the interesting part of writing them out, and it is a distinction the directory declaration cannot express at all. The four checkers are `false`: they are mandatory members of every workflow's deterministic gate, and a user editing a threshold in `check_coverage.py` is changing what "covered" means for everyone reading the report. The four renderers are `true`: an export format is presentation, and a team that wants its own slide layout should have it.

The case for per-file. You get eight accurate descriptions instead of one that names two files and has not been updated to mention the other six. You get the `user_modifiable` distinction above, which is a real promise to users that currently cannot be made. And you get a manifest that answers "what does this kit ship" without a directory listing.

The case for the directory. Adding a ninth checker requires no manifest change, no review of a TOML diff, and no chance of the manifest and the folder disagreeing — which is a category of drift that simply cannot occur. For a kit under active development, where scripts are added and renamed often, that is worth more than eight descriptions that will each rot at their own rate. And the current declaration is not silent about the cost: the `guides/` entry next to it uses its description to record a runtime dependency (`CAPTURE.md` and `EVIDENCE.md`) precisely because there is nowhere structural to put it.

What I would ship, and what would change my mind: the per-file version, for the `user_modifiable` reason alone — a promise about what survives an update is not a description, it is a contract, and one cannot be made for a folder. What would change

my mind is evidence that the script set is still churning: if three scripts were added or renamed in the last six months, the directory declaration is the right call and the honest fix is to correct that description to say "eight deterministic checkers and renderers" rather than naming two.

7. **Find the boundary between rules and scripts.**

   Three checks, one from each layer.

   By `cfs validate` via `constraints.toml`: the `req-h1-title` heading pattern, requiring "Research Request", a dash, and at least one more character. *Right place.* It is a regular expression over a heading; it needs no context, no judgement, and no evidence index. It is also the kind of check that must be fast and unconditional, because it runs in the Step 4 loop until it passes.

   By a PDSL `NEVER` rule: `CPT-REQ-NO-004 (HIGH)`, more than one comparison frame in a single request. *Right place, and interestingly so.* Detecting "more than one frame" would be easy if frames were an enum in a cell. They are not — the rule's own parenthesis says frames may be *mapped*, never silently mixed, so the violation is a semantic judgement about whether two things named in one cell are one frame expressed two ways or two frames smuggled together. No script can make that call. A reviewer with the checklist can.

   By a Python script: the evidence-density floors in `check_coverage.py` — one distinct `EVD` for an orientation card, two for a profile, four for a dossier. *Right place, decisively.* It is counting, it requires joining the artifact text against the evidence index, and it must filter out cited records that do not exist. Written as a checklist item it would read "the dossier should cite enough sources," which is unactionable, and a reviewer counting distinct `EVD` tokens by eye across a long document will get it wrong.

   One check in the wrong place: `CPT-REQ-NO-005`, the placeholder prohibition (TODO, TBD, FIXME). It currently lives in `rules.md` as a `NEVER` and in `checklist.md` as a MUST NOT HAVE, so it is caught by the semantic review in Step 5 and by nothing else — as Exercise 5 above demonstrates, `cfs validate` passes an artifact with `TODO` in its prose.

   This is a pure counting problem wearing a language problem's clothes: four string literals, case-insensitive, word-boundary matched. It should move into the scripts directory, called from the Step 4 deterministic gate alongside `cfs validate` and `cfs validate-toc`.

   What it would cost. About twenty lines of Python plus the argparse boilerplate the other eight scripts already share, and one line in each workflow's Step 4. The real cost is the false-positive case: a request that quotes a requester who literally wrote "TODO: decide the depth" in their original text, which Section 3 preserves verbatim. So the script has to exclude the verbatim-quote region, which is why this is a fifteen-minute job rather than a five-minute one — and why it is a reasonable thing to have deferred. Worth doing anyway: a placeholder shipping in a research request is a defect every time, and it is the kind of defect a reviewer stops seeing after the third document.

A.10 Chapter 10: Building Your Own Kit

The exercises in this chapter are the chapter. Every solution below is one worked example, on one invented but realistic domain, carried through all nine exercises so the parts fit together. Your domain is not this one, and a kit built by substituting your nouns into these files would be worse than nothing. What is transferable is the sequence of decisions and the reasoning at each fork.

The running example. A university research group that runs empirical software-engineering studies. The undocumented process is the one every such group has: what has to be written down before you touch data, what a study protocol must commit to before results exist, and which claims a given study design licenses. It lives in the head of whoever has been there longest, and it leaves when they do.

1. **Write the design brief before any code.**

   Started with six kinds: `RESEARCH-QUESTION`, `STUDY-PROTOCOL`, `DATA-STATEMENT`, `ANALYSIS-PLAN`, `RESULTS-REPORT`, `THREATS-REVIEW`.

   Applying the three tests:

   Identifier segment. All six named one easily: `rq`, `proto`, `data`, `plan`, `res`, `thr`. No casualties. This test is a filter for kinds that are secretly two kinds, and none of these were.

   Distinct reviewer. `ANALYSIS-PLAN` and `STUDY-PROTOCOL` are reviewed by the same person against the same criteria — the methodologist asking whether the design answers the question. Merged: the analysis plan became section 4 of the protocol. Six down to five.

   Distinct claim boundary. This is where it got uncomfortable, and the exercise warns you it should. `THREATS-REVIEW` does not establish anything a reader can conclude *from*; it qualifies what `RESULTS-REPORT` establishes. Its boundary statement would be identical to the results report's, negated. So it is not a kind — it is a required section of the results report, and making it a separate document is exactly how a threats-to-validity section ends up written last, by whoever is least invested in the finding. Five down to four.

   Survived: `RESEARCH-QUESTION` (`rq`), `STUDY-PROTOCOL` (`proto`), `DATA-STATEMENT` (`data`), `RESULTS-REPORT` (`res`).

   Claim boundaries, one sentence each:

   - `RESEARCH-QUESTION` establishes what is being asked and what an answer would have to look like. It establishes nothing about whether the question is answerable.
   - `STUDY-PROTOCOL` establishes the design, the population, the measures, and the analysis committed to *before* data exists. It establishes nothing about results.
   - `DATA-STATEMENT` establishes what data was collected, from where, under what consent, and what may be published. It establishes nothing about what the data shows.
   - `RESULTS-REPORT` establishes what was observed under one protocol, with its threats section bounding generalisation. It does not establish that the effect exists outside the studied population.

   The root: `RESEARCH-QUESTION`. Every other artifact cites the question it serves, and the reverse obligation is deliberately not required — a question may sit unanswered for two years, and demanding a protocol for every recorded question would make the validator warn permanently.

   Six started, four survived, and the claim-boundary test did the killing — once directly, on `THREATS-REVIEW`, and arguably a second time, since the reviewer test that merged the analysis plan was really the boundary test in disguise.

2. **Find your "never silently default" rule.**

   In one sentence: a measure that was decided after the data was seen is recorded as `post-hoc`, never presented as though it had been in the protocol.

   That is this domain's version of the `compete` kit's unspecified rule, and it has the same character: a careful researcher does it automatically, a careless one violates it invisibly,

and the violation is undetectable from the finished document unless the document was built to make it visible.

As placeholder text, in `STUDY-PROTOCOL`'s measures table:

```
| Measure | Operationalisation | Decided |
| --- | --- | --- |
| {Name} | {How it is computed from the raw data}
       | {pre-registered | post-hoc - never leave blank} |
```

As an `ALWAYS` in the PDSL `RULES` block:

```
- ALWAYS mark a measure added after data collection as post-hoc in the
  Decided column, and state in Section 5 what prompted adding it
```

As a MUST NOT HAVE with an identifier and a severity:

```
- NEVER CPT-PROTO-NO-001 (CRITICAL): presenting a post-hoc measure as
  pre-registered, or leaving the Decided column blank
```

What each catches that the other two cannot. The placeholder catches it at the moment the row is written, when the author is holding the measure in their head and knows perfectly well when they thought of it — there is no cell to leave blank without noticing. The `ALWAYS` catches the second half of the rule, which the cell cannot express: not just labelling the measure but recording *why* it was added, which is the part that lets a reader judge whether the addition was reasonable. And the MUST NOT HAVE catches it in review, when a colleague reads a finished protocol months later and can compare the Decided column against the data-collection dates in the `DATA-STATEMENT` — a cross-document check neither of the other two expressions can reach.

3. **Write the manifest.**

   One domain, one kit, so a single `[[kits]]` block with the manifest at the kit root. Slug `empirical`, which produces `cf-empirical-question`, `cf-empirical-protocol`, and so on — said out loud without wincing, which is the test Step 3 gives.

```
manifest_version = "1.0"

[[kits]]
slug = "empirical"
name = "Empirical Study Kit"
version = "0.1"

[[kits.resources]]
id = "question_template"
kind = "template"
source = "artifacts/RESEARCH-QUESTION/template.md"
install_path = "artifacts/RESEARCH-QUESTION/template.md"
type = "file"
user_modifiable = true
description = "RESEARCH-QUESTION artifact template"

[[kits.resources]]
id = "question_rules"
kind = "other"
source = "artifacts/RESEARCH-QUESTION/rules.md"
install_path = "artifacts/RESEARCH-QUESTION/rules.md"
type = "file"
user_modifiable = true
description = "Authoring procedure and prohibitions for RESEARCH-QUESTION"

# ... the same four entries for each of the four kinds ...

[[kits.resources]]
id = "workflow_question"
kind = "skill"
source = "workflows/question.md"
```

```
install_path = "workflows/question.md"
type = "file"
user_modifiable = false
public = true
description = "Record one research question (root of the chain)"
generated_targets = ["installed"]

[[kits.resources]]
id = "constraints"
kind = "constraints"
source = "constraints.toml"
install_path = "constraints.toml"
type = "file"
user_modifiable = false
description = "Heading and identifier contract for all four kinds"

[kits.resources.artifacts]
[kits.resources.artifacts.RESEARCH-QUESTION]
template = "question_template"
examples = "question_example"
# ... one binding per kind ...

[[kits.resources]]
id = "agents"
kind = "rule"
source = "agents-res.md"
install_path = "AGENTS.md"
type = "file"
user_modifiable = false
public = true
description = "Kit AGENTS.md for agent integration entry points"
generated_targets = ["installed"]
```

Tabulated: 4 `template`, 4 `checklist`, 8 `other` (four `rules.md`, four `examples/example.md`), 4 `skill`, 1 `constraints`, 1 `rule`, 1 `script` — 23 resources.

The `false` justifications, one line each: the four workflows, because a user editing one is forking the process rather than configuring it; `constraints`, because it is the one file whose meaning is enforced mechanically and a local relaxation would mean the repository validates differently on two machines; `agents`, because it is the generated entry point the host reads to find the kit at all.

Kinds not used: `agent`, because the kit configures no workers and Chapter 10 is right that getting a document flow correct is enough for a first version; and `directory`, because with one script there is nothing to bundle — the moment there are four, that decision should be revisited, and the argument for revisiting it is Chapter 9's Exercise 6.

4. **Draft the template and the rules for one kind.**

   `artifacts/RESEARCH-QUESTION/template.md`, abridged to the parts that carry the design:

```
---
date: {YYYY-MM-DD}
status: {draft | active | answered | abandoned}
---

# Research Question - {Concise question name}

**ID**: `cpt-{system}-rq-{slug}`

## Table of Contents

<!-- generated by `cfs toc` -->

## 1. The Question

{One sentence, interrogative, naming the population and the outcome.
2-4 supporting sentences on why it is worth asking now.}
```

```
## 2. What an Answer Would Look Like

{1-2 paragraphs. State the form of a satisfying answer BEFORE any study
exists: an effect direction, a magnitude with units, a taxonomy, a
counterexample. If you cannot describe the shape of an answer, the
question is not yet a question.}

## 3. Boundary

{What this question can and cannot settle. State explicitly that a
question establishes nothing about feasibility, that answering it does
not license a practice recommendation, and which adjacent questions it
is deliberately not asking.}

## 4. Prior Work Consulted

- {Citation, with what it settles - or "none consulted"}

## 5. Traceability

No studies have been derived from this question yet.

<!--
Once downstream artifacts exist, their workflows replace the line above
with one back-link per derived artifact:

- `cpt-{system}-proto-{slug}` - {Study protocol title}
- `cpt-{system}-data-{slug}` - {Data statement title}
- `cpt-{system}-res-{slug}` - {Results report title}
-->
```

Three things are borrowed deliberately. Length expectations live in the prompts. Section 3 is the required boundary section, and its content is prescribed rather than left open, because "state the limits" without saying which limits produces a paragraph of hedging. And Section 5 ships a default line with the back-link forms in an HTML comment, so a fresh question carries no dangling references and the next workflow finds the expected shape waiting for it.

`artifacts/RESEARCH-QUESTION/rules.md`, three of the five units:

```
UNIT QuestionAuthoring

PURPOSE:
  Author one RESEARCH-QUESTION: what is being asked, what an answer
  would look like, and what the question cannot settle. The question is
  the root of the traceability chain - every protocol, data statement,
  and results report is attributable to one.

STATE:
  - SET question_source: file | inline
    default: inline when the user stated the question in chat

WHEN:
  - REQUIRE recording a new research question, or revising a draft one

DO:
  - LOAD {cf-studio-path}/config/artifacts.toml to resolve the
    RESEARCH-QUESTION directory
  - LOAD {question_template} for structure
  - LOAD {question_example} for content depth reference
  - RUN capture the question verbatim as the user stated it
  - RUN author Section 2 before Section 1 is finalized: if the shape of
    an answer cannot be stated, return to the user
  - RUN author Section 3 (Boundary): what the question cannot settle
  - SET ID cpt-{system}-rq-{slug}
  - RUN verify ID uniqueness with cfs list-ids
  - LOAD {question_checklist} for self-review of semantic quality

RULES:
  - ALWAYS keep the question interrogative and name both the population
```

```
    and the outcome
  - ALWAYS keep Section 5 free of downstream references in a fresh
    question
  - ALWAYS treat {question_checklist} as the semantic source of truth
  - NEVER include placeholder content (TODO, TBD, FIXME)
  - NEVER duplicate {question_checklist} semantic criteria in this file
```

```
UNIT QuestionOmissions

PURPOSE:
  Enforce MUST NOT HAVE content; report any occurrence as a violation.

RULES:
  - NEVER CPT-RQ-NO-001 (CRITICAL): stating a hypothesis, an expected
    result, or a preferred answer in the question artifact
  - NEVER CPT-RQ-NO-002 (HIGH): more than one question in one artifact
    (compound questions may be split, never silently merged)
  - NEVER CPT-RQ-NO-003 (HIGH): a practice recommendation derived from
    an unanswered question
```

```
UNIT QuestionErrorHandling

PURPOSE:
  Recover from input failures and escalate ambiguous situations.

ON_ERROR:
  answer_shape_unstatable ->
    EMIT "Section 2 cannot be written: no shape of an answer was given"
    RUN present the candidate readings and WAIT for the user's choice

  duplicate_question ->
    EMIT "A RESEARCH-QUESTION with a matching population and outcome
          already exists: {id}"
    RUN ask whether to revise the existing question or record a new one

RULES:
  - ALWAYS ask the user rather than guess when the question is ambiguous
```

Note that no path appears anywhere in the rules file — every load is by resource identifier — and that `CPT-RQ-NO-002` borrows `CPT-REQ-NO-004`'s shape exactly: the prohibition is on the silence, not on the operation.

5. **Normalize, register, and fix what breaks.**

   What the normalizer and a hand-written manifest disagree about is predictable, and three disagreements are worth expecting because they recur:

   The normalizer classifies `rules.md` *as* `other` *and so did you* — agreement, and worth confirming rather than assuming, because `rule` is the tempting wrong answer and Chapter 10 explains why it is wrong.

   The normalizer sets `user_modifiable = true` *on the workflows.* Its guess is reasonable and yours is a promise. You are right; take your value. This is the single most common place the normalizer's output should not ship.

   The normalizer may or may not find `examples/example.md` depending on your layout. If it declared them and you forgot them, it is right.

   Then the flow. Expect `cfs generate-agents` to be forgotten once — Chapter 10 predicts this and the prediction is accurate — with the symptom that `cf-empirical-question` appears not to exist. Expect `cfs validate-kits` to report something on the first run; the entries above were written after reading the validation rules, which is not the order you will work in.

   A realistic iteration count for a first kit of four artifact kinds is three or four rounds: one for manifest shape, one for constraints not matching the templates you actually wrote,

and one or two for headings whose numbering or level disagrees with the contract. Record the message and the fix each time. The list you end up with is the most useful artifact of this exercise, because it is your kit's error vocabulary and you will see every entry again.

6. **Write the constraints and prove they bite.**

```
[[artifacts."RESEARCH-QUESTION".headings]]
id = "rq-h1-title"
level = 1
required = true
multiple = false
numbered = false
pattern = 'Research Question\s*[---]\s*.+'
description = "Question title (H1): 'Research Question - {name}'."

[[artifacts."RESEARCH-QUESTION".headings]]
id = "rq-question"
level = 2
required = true
multiple = false
numbered = true
pattern = "The Question"
description = "The question itself: interrogative, naming population and outcome."

[[artifacts."RESEARCH-QUESTION".headings]]
id = "rq-answer-shape"
level = 2
required = true
multiple = false
numbered = true
pattern = "What an Answer Would Look Like"

[[artifacts."RESEARCH-QUESTION".headings]]
id = "rq-boundary"
level = 2
required = true
multiple = false
numbered = true
pattern = "Boundary"

[[artifacts."RESEARCH-QUESTION".headings]]
id = "rq-traceability"
level = 2
required = true
multiple = false
numbered = true
pattern = "Traceability"

[artifacts."RESEARCH-QUESTION".identifiers.rq]
name = "Research Question"
description = "One empirical research question and the shape of a satisfying answer."
required = true
task = false
priority = false
to_code = false
template = "cpt-{system}-rq-{slug}"
headings = ["rq-h1-title"]

[artifacts."STUDY-PROTOCOL".identifiers.proto.references."RESEARCH-QUESTION"]
[artifacts."DATA-STATEMENT".identifiers.data.references."RESEARCH-QUESTION"]
[artifacts."RESULTS-REPORT".identifiers.res.references."RESEARCH-QUESTION"]
```

The regular expression heading is `rq-h1-title`, requiring a dash and at least one character after it, so "Research Question" alone fails deterministically.

`to_code = false`, and the justification is not that these documents are prose. It is that nothing in this domain produces a CPT marker in a source file: the end of the chain is a dataset and an analysis script, and there is no mechanism binding an identifier to a row in a CSV. Setting it `true` would make `cfs validate` demand code markers that will

never exist, and Chapter 10's failure mode 5 — the validator that always warns — arrives within a week.

Note that the three `references` entries are one-directional. The question carries no reverse obligation, for the `compete` kit's reason: a question may sit unanswered for two years and that is not a defect.

The four breaks:

Remove `## 3. Boundary` → caught, `heading-missing` for `rq-boundary`, plus a stale-TOC finding if the table of contents was current.

Malform the identifier to `cpt-esg-question-x` → caught twice: `id-kind-not-allowed` (`question` is not `rq`) and `required-id-kind-missing`.

Satisfy the regex heading with a bare title, "# Research Question" → caught, `heading-missing` for `rq-h1-title`. This is the one worth breaking deliberately, because it is the only one where the heading is present and the failure is about content.

Insert `TODO` *into Section 2* → **not caught**, for the reason established in Chapter 9's Exercise 5: placeholder detection fires only on CDSL step lines, and this artifact has none. What would have caught it: the checklist's MUST NOT HAVE item in semantic review, or the script in the next exercise.

7. **Write one deterministic checker.**

The counting problem in this domain: every measure named in a `RESULTS-REPORT` must appear in the measures table of the `STUDY-PROTOCOL` it cites, and every measure marked `pre-registered` must be present in a protocol version dated before the data-collection start date in the `DATA-STATEMENT`.

```
#!/usr/bin/env python3
"""Report measure conformance between a results report and its protocol.

Reads one RESULTS-REPORT, the STUDY-PROTOCOL it cites, and the
DATA-STATEMENT for the same question, and reports:
  - measures reported but absent from the protocol's measures table
  - measures marked pre-registered whose protocol row is dated on or
    after the data-collection start date
The summary line is stable for scripting:
``check_measures: N reported, M unlisted, K mislabelled``

This is a gate, not a report: an unlisted or mislabelled measure is a
defect, not routing. Exit codes: 0 = clean, 1 = usage error,
2 = findings.
"""
import argparse, datetime as dt, re, sys
from pathlib import Path

ISO_DATE_RE = re.compile(r"\d{4}-\d{2}-\d{2}")

def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("results", type=Path)
    parser.add_argument("--protocol", type=Path, required=True)
    parser.add_argument("--data-statement", type=Path, required=True)
    parser.add_argument("--as-of", type=str, default=None,
                        help="reference date (ISO; default: today)")
    args = parser.parse_args()
    if args.as_of and not ISO_DATE_RE.fullmatch(args.as_of):
        print(f"error: --as-of must be an ISO date: {args.as_of}",
              file=sys.stderr)
        return 1
    as_of = (dt.date.fromisoformat(args.as_of) if args.as_of
             else dt.date.today())
    ...
    print(f"check_measures: {n} reported, {m} unlisted, {k} mislabelled")
    return 2 if (m or k) else 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

Standard library only. The docstring doubles as `--help`. The summary line is stable. `--as-of` exists so the date comparison is reproducible under test, which is the difference between a checker you can run in CI against a fixture and one whose output changes at midnight.

And it declares itself a gate, which is the opposite of `check_coverage.py`'s decision and for a defensible reason: an unexecuted research step is routing, but a measure reported that was never in the protocol is the thing this whole discipline exists to prevent. Getting that call right in the docstring, before anyone runs it, is what stops the argument later.

Why this could not have been a checklist item. A reviewer reading a results report with eleven measures against a protocol with fourteen rows is performing a set difference by eye, across two documents, with the names sometimes phrased slightly differently. They will get it right the first time, approximately right the fifth time, and skip it the tenth. The date comparison is worse: it requires holding three dates in mind and knowing which comparison matters. Both are exactly the kind of check a human does badly and a script does identically every time — and, crucially, the failure mode of the human version is silent. Nobody reports "I did not really check the measures table."

8. **Run it with somebody watching.**

   The workflow file's `Route` section, declared before Step 1:

```
**Type**: Intake + Structuring
**Role**: Research question analyst
**Output**: `{artifacts_dir}/RESEARCH-QUESTION/{yyyy-mm-dd}-{slug}.md`

## Route

5 steps -> RESEARCH-QUESTION. Questions: 1 confirmation (Step 1: the
restated question), 1 decision (Step 2: the shape of an answer), 1
closing confirmation (Step 5). Steps 3-4 run without questions.
Announce the route before Step 1.
```

   What a first run typically produces, and what to record:

   The route announcement is the easiest thing to get right and the easiest to forget, because it is one instruction competing with the rest of the file. Record whether it appeared verbatim.

   The question count is where declarations and reality part company. Three declared; a first run often asks four or five, because Step 2's decision gets split — the model asks about the answer shape, then separately about the population. That is not wrong behaviour, it is a wrong *declaration*: your Route section said one decision and your Step 2 contains two. Fix the declaration or merge the questions; do not leave them disagreeing.

   The deterministic gate is the one that either loops or does not, visibly. If it ran once and moved on with a failure outstanding, the step's wording is too weak — "fix and re-run until all report PASS" is the phrasing that works, and anything softer gets treated as advice.

   The confusion worth acting on, in the run this example is based on: the colleague could not tell whether Section 2 had been written from their answer or from the model's own reading. Both were plausible and the document did not say.

   The smallest fix: add to Section 3 of the template a required line recording whether the answer shape was supplied by the requester or proposed by the workflow and confirmed — the same move as `RESEARCH-REQUEST`'s Section 3 preserving the verbatim original.

Why a validator could not have found it. Every check passes. The headings are present, the identifier is well formed, the TOC matches, the document says what a good document says. The defect is that a reader six months later cannot distinguish a requester's commitment from a model's suggestion, which is a property of what the document *fails to record* rather than of anything it contains. No file-format check can notice a missing distinction it was never told to look for — only somebody reading the output and asking "where did this come from" will.

9. **Decide the extension level for three real requirements.**

   Taking the house-overlay brief, and the answer is genuinely different for each of the three, which is the point of the exercise.

   (1) A security sign-off section in every DESIGN. *Artifact level.* This is one required heading on an existing kind. Add it as a `constraints.toml` heading entry with `required = true` plus a matching checklist item for whether the content is substantive. You commit to maintaining two small entries and to the fact that every DESIGN in every repository using your constraints now fails validation until it has the section — which is what you wanted, but say it out loud before you ship it, because somebody has forty existing designs.

   (2) ADRs must carry at least two rejected options. *Resource level, then script.* The heading already exists in the shipped ADR template; the requirement is about content, which is a counting problem. The `user_modifiable = true` on `adr_checklist` means you can add the criterion to the shipped checklist in place and it survives kit updates — that is precisely the guarantee the flag encodes. The count itself belongs in a small script called from the ADR workflow's deterministic gate. You commit to one checklist edit and about thirty lines of Python, and to the risk that a future SDLC-kit release rewrites the checklist enough that your criterion sits oddly in it.

   (3) A mandatory data-classification field in every FEATURE. *Artifact level, and this is the one that tempts you into a fork.* A field inside a document is not a heading, so a heading constraint cannot reach it, and the natural instinct is to copy `feature_template` into your own kit and add the field. Resist it if you can: the shipped FEATURE template is the largest and most actively maintained file in the SDLC kit, and forking it means inheriting every future improvement by hand.

   The better shape is a required subsection — a level-3 heading "Data Classification" with a constraints entry — plus a checklist item for the value being one of your permitted classes. That gets you a deterministic check on presence and a semantic check on validity, without owning anybody else's template.

   If you do fork a shipped template, decide now what happens when it changes upstream, because "we will deal with it" is how a fork becomes permanent. The workable answer is a scheduled diff: on each SDLC kit release, diff the shipped template against your fork, and treat every upstream hunk as a change to accept or explicitly decline with a comment. It takes twenty minutes a release and it is the only thing that keeps a fork from silently becoming a different document with the same name.

A.11 Chapter 11: Mental Models and the Shape of a Kit

These exercises ask you to practise one skill: finding the place where two people are running different private models of the same work, and building something external that makes the difference visible. As in Chapter 10, the solutions below are one worked example carried through all seven exercises, so that the mechanism designed in Exercise 2 is the one attacked in Exercise 6. Your team is not this team. What transfers is the sequence: name the divergence, name

the artifact that failed to surface it, build one thing, decide what it cannot do, and decide in advance when to remove it.

The running example. A payments team of nine engineers with a two-person on-call rotation, an incident tool, and a postmortem practice about eighteen months old. The postmortem template was copied from a well-known public one, lightly edited, and has not been touched since.

1. **Diagnose one recurring disagreement.**

   The disagreement. Every few weeks somebody asks why an incident is still open, or why one was closed. It has been read as a personality problem for a year. It is not.

   The two columns.

   - *On-call engineer.* An incident is over when customer-visible impact has stopped. Everything after that is ordinary engineering work, tracked where ordinary work is tracked. This is so obvious to them that they have never said it out loud.
   - *Support lead.* An incident is over when the thing that caused it can no longer cause it. Until then the incident is a live commitment to customers who have been told it is being handled. Equally obvious, equally unsaid.

   The one sentence they differ on: "An incident is resolved when customer-visible impact has stopped." The on-call engineer affirms it, the support lead denies it, and neither has ever heard the other say it.

   Which Norman property produced it: fuzzy boundaries. The word `resolved` was imported from the ticket tracker, where it has meant "the defect is fixed" for years, into the incident tool, where it means "impact has ended." The two systems look similar enough that nobody noticed the term had changed meaning on the way across. Parsimony is the secondary cause: one word is cheaper than a distinction, and nobody works out in advance which stopping condition they mean because they have never needed to.

   Where it could have surfaced and did not. Four occasions, and the progression is the interesting part.

   The incident channel required a status word drawn from an enumeration whose members are nowhere defined. Both parties read `resolved` and each saw their own meaning. An enumeration without definitions does not converge models; it disguises the divergence as agreement.

   The daily incident review required a verbal update. Verbal updates are where this kind of ambiguity survives indefinitely, because each listener supplies the missing definition from their own model and no artifact records that they supplied different ones.

   The postmortem template has a Timeline section with a resolution timestamp. This is the occasion that looks like it should have worked, and it is the worst of the four: the divergence became a *number*, and a number looks like a settled question. Two incidents of the same kind, written up by the two parties, produced resolution times differing by eleven days, and nobody compared them because nobody had a reason to.

   The monthly metrics report aggregates those timestamps into a mean-time-to-resolve figure that is a mixture of two different quantities. The divergence is now invisible in principle, not just in practice.

   The closing sentence: nothing in our process ever asks anyone to state which of the two stopping conditions the resolution timestamp refers to.

2. **Design exactly one externalizing mechanism, in a new domain.**

   Domain: the incident postmortem. One field.

   (a) The private model failure it targets. Fuzzy boundaries, as diagnosed above: a term carried between two systems with different meanings, so that each participant's model of "resolved" is locally coherent and mutually incompatible. The field does not target the disagreement about which definition is right. It targets the fact that neither party can currently discover that a disagreement exists.

   (b) The literal text. The postmortem template already has impact-start and impact-end timestamps. One field is added under them:

```
### Impact window

- **Impact began**: {YYYY-MM-DDThh:mm:ssZ}
- **Impact ended**: {YYYY-MM-DDThh:mm:ssZ}
- **Impact end established by**: {monitoring-signal | customer-confirmation
  | manual-check | assumed - never leave blank}
- **Evidence**: {permalink to the query, alert, or message that shows
  impact had stopped at the time above}
```

   The enumeration and the permalink are one field between them; the permalink without the enumerated value is unclassifiable, and the enumerated value without the permalink is an assertion.

   (c) Who fills it in, and when. The incident commander, at the moment they set the incident to resolved in the tool — not the postmortem author three days later. This matters more than the wording. At the moment of resolution the person knows exactly what made them believe impact had stopped, because they are looking at it. Three days later they are reconstructing, and a reconstruction of a criterion is the criterion the template suggests.

   (d) What a reader can now check. Given a quarter of postmortems: which criterion each incident actually used; whether the two parties' incidents cite different kinds of evidence, which is the direct test of the Exercise 1 diagnosis; and whether the linked evidence covers the timestamp claimed. Before the field, none of those questions had an answer anywhere.

   Why this mechanism and not the other obvious one. The obvious alternative is a second timestamp: keep `impact_ended_at` and add `defect_removed_at`. It fails, because the same person with the same private model fills in both, and the result is two numbers nobody can interpret — exactly the failure the existing single timestamp already has, doubled. Requiring the *observation* works because the two models cannot cite the same kind of thing: the on-call engineer links a latency panel back at baseline, the support lead would have to link a merged pull request. The divergence becomes visible in the shape of the citation, not in anybody's willingness to argue about definitions.

3. **Sort five checks into the three columns.**

   Check 1. The Impact window section exists, with all four lines, and `Impact end established by` is non-empty. **Contract.** A required heading plus a required-content pattern, the same construction as Chapter 9's boundary heading. Nothing is counted and nothing is judged.

   Check 2. The value is one of the four enumerated members. **Contract**, as a regular expression on the line. Worth separating from Check 1 because the failures are different: Check 1 catches deletion, Check 2 catches invention. In practice Check 2 fires on values like ``dashboard'' and ``eyeballs'', both of which are honest attempts at `monitoring-signal` and `manual-check`, and both of which would corrupt any later count.

Check 3. The evidence permalink resolves, and the time range it covers contains the claimed impact-end timestamp. **Counting problem, script** — it needs to fetch a URL and parse a range out of it, which no heading contract can do. Split verdict on gate versus warning, and the split is the interesting part. Link *presence and parseability* is a gate: a permalink that is not a permalink is a defect at the moment of writing, and the author can fix it in thirty seconds. The *range comparison* is a warning that returns zero, because dashboards get renamed and re-pointed, and a two-year-old postmortem whose link has rotted is not a defective postmortem. Making that a gate produces Chapter 10's failure mode 5 — the validator that always warns — within one dashboard migration.

Check 4. The proportion of postmortems in the last quarter whose value is `manual-check` *or* `assumed`*.* **Counting problem, script, and a warning that returns zero.** This is the `compete` kit's soft-floor pattern exactly. One `assumed` is honest and correct; a quarter where 40% are `assumed` is a monitoring gap, which is a finding about the service and not a defect in any document. Gating it would punish the individual author for a team-level problem, which is the fastest way to teach people to write `monitoring-signal` and link something vaguely relevant.

Check 5. Does the cited evidence establish impact end for every affected customer, or only in aggregate? **Judgement, needs a reader.** A latency panel returning to its median is compatible with one large tenant still failing every request, and no script knows which incidents were per-tenant. Checklist sentence: "If impact was partial or tenant-specific, does the cited evidence cover the affected tenants specifically, rather than the aggregate?"

Two contract checks, two scripted counts, one judgement. That split is roughly what a healthy mechanism looks like; if the exercise had produced five judgements, the honest conclusion would be that this was a training problem wearing a template's clothes.

4. **Stress-test the claim register on a real paper.**

   The paper. An empirical study reporting that repositories which adopted a particular static-analysis tool showed lower post-release defect density than those which did not. The disagreement surfaced in the second review round: one co-author had been treating the result as evidence that the tool reduces defects, the other as evidence that the tool is adopted by teams that were already more careful. Both had read the same table for four months.

   The register as it would have stood. Reconstructed at the point the disagreement existed and was invisible:

```
cpt-sa-clm-tool-reduces-defects
  cites: cpt-sa-evd-t3-density-by-repo
         cpt-sa-evd-f2-adoption-timeline

cpt-sa-evd-t3-density-by-repo
  from: analysis/density_by_repo.R, dataset v4, run 2024-03-11

cpt-sa-evd-f2-adoption-timeline
  from: analysis/adoption.R, dataset v4, run 2024-03-11
```

   Which of the seven checks fire. Walking them honestly: the identifiers are well formed and unique — pass. The claim cites evidence, two records — pass. Both cited records exist — pass. Neither is orphaned — pass. Every section maps to a claim and every claim appears in a section — pass. The headings including the boundary section — pass, the paper had a threats section.

   Six of seven pass. The mechanism as specified would not have caught the thing that mattered, and saying so is the correct answer.

The seventh check is a partial hit, and it is instructive. "Every claim in the abstract appears in the claim register." The abstract said *reduces*; the body's own careful sentence, once it was extracted into the register, read *is associated with lower*. As specified, the check is an identifier match and would pass, because the authors would naturally register one claim and let the abstract paraphrase it. It fires only if the register requires each abstract claim to name the body claim identifier it restates — at which point the causal verb and the associational verb sit on adjacent lines and the difference is visible without anyone being clever. That is a real strengthening of the chapter's proposal, found by testing it against a case it did not anticipate, and it is cheap: one required column, no new machinery.

What the disagreement was actually about, and whether anything deterministic could reach it. It was about what the evidence cannot rule out — specifically, that adoption was not randomly assigned. That is a claim about a *missing comparison*. Every deterministic check in the chapter's list inspects things that are present: an identifier, a link, a heading, a count. No check that inspects present things can report an absent confounder, because there is no finite list of comparisons a study should have made.

The nearest mechanism is not a validator at all, it is the boundary section, and its division is exactly the one the chapter is careful about: that the section exists is deterministic and would have passed, and that the section names selection into adoption as an unaddressed alternative explanation is a judgement, made by a reader. The mechanism would have made the disagreement cheap to *have* — both authors reading one line and one linked table rather than four months of prose — and it would not have found it.

5. **Name one failure a template cannot fix.**

   The failure. The postmortem that is accurate, complete, well-evidenced and about the wrong boundary. A retry storm takes down the settlement path for two hours. The postmortem is exemplary: a minute-by-minute timeline, an impact window with a `monitoring-signal` citation and a live permalink, a contributing-factors section, five action items with owners. It passes every check in Exercise 3, including the judgement one — the evidence does cover the affected tenants. And it does not say that the same fragility was raised in a planning meeting fourteen months earlier and dropped for capacity reasons.

   Why no template prevents it. The obvious repair is a field: "was this risk previously known?" It does not work, for a reason worth stating exactly. The person filling it in is the person who deprioritised the risk, or reports to them, and `known` is precisely the word whose private model is at stake. They can write `no` in complete good faith, because in their model what happened in that meeting was a discussion of a general class of concerns and not knowledge of this failure. There is nothing to cite against them, because the March decision was never written down anywhere with an identifier. A template can only externalise models about the object it covers. This model lives in a different process — planning — and was never externalised there. No amount of postmortem design reaches into it.

   What does address it, and what it costs. Two things, and only together.

   An outside reviewer: one senior engineer from another team who reads each SEV-1 and SEV-2 postmortem and has read the last two quarters of planning decisions. Cost: roughly half a day per incident, plus the standing cost of that person being expensive and interruptible. It works because they hold neither party's model and have no stake in the answer.

   And the artifact the reviewer needs to exist: a written decision record in the planning process, with a date, a named decider and a stable identifier, for every risk that is raised

and not scheduled. Cost: real, and it belongs to a process that did not ask for it, which is why this is the hard half. Without it the reviewer is relying on somebody's memory of a meeting, which is the same private model with a bigger audience.

One thing that does *not* address it, listed because it is tempting: a mandatory 72-hour delay before a postmortem can be closed. It is cheap, it is a genuine improvement, and it fixes a different failure — the exhausted author writing a thin timeline. It has no purchase on this one at all. Being clear about which failure a mechanism addresses is the whole discipline here.

6. **Break your own mechanism.**

   The cheapest string. `manual-check`, with a permalink to the incident's own chat channel. It satisfies the enumeration, it satisfies "permalink", it takes about four seconds, and it carries no information whatsoever — the channel is where somebody said "looks fine now". It will be the first workaround, it will appear within three weeks, and it will spread by copy-paste from the most recent postmortem, because that is where everybody looks for the expected shape.

   Does it still pay at a third? Yes, but for a reason that was not the design intent, and the distinction matters. At a third `manual-check`, the field has stopped being a record of the impact-end criterion and become a measurement of how often the team has no monitoring signal for impact end. That is a more valuable finding than the one the field was built for. But it is only valuable if somebody looks at the number. The mechanism pays for its friction as long as Check 4 from Exercise 3 is read by a human once a quarter, and it stops paying the day that stops — at which point it is nine engineers spending four seconds each on a field nobody reads, which is worse than not having it, because it certifies that the criterion was recorded.

   What to change so the cheap string is unavailable. Three candidates, and the first is wrong.

   Removing `manual-check` from the enumeration is the instinct and it is wrong: sometimes a manual check genuinely is what happened, and an enumeration that omits the truthful answer produces `monitoring-signal` plus a vaguely relevant link. That is not a stricter mechanism, it is a mechanism that now lies.

   Change one, kept: when the value is `manual-check` or `assumed`, one additional sentence is required, naming the signal that *would have* established impact end and stating whether it exists. This does not make the cheap answer harder to give. It makes the cheap answer productive: the field turns into a monitoring-gap register maintained as a by-product of incidents, which is a thing no team ever has time to build deliberately.

   Change two, kept: reject evidence permalinks whose host is the chat tool. One line in the Check 3 script. It does not stop anybody — it converts a four-second escape into a ten-second one and an honest `assumed`, which is the outcome we want anyway.

   The retire clause, written down because the exercise is right that choosing afterwards is not choosing. If the combined `manual-check` plus `assumed` rate is still above 40% two quarters after change one ships, *and* no monitoring work has been scheduled as a result, the field is removed — not tightened. At that point it has demonstrated that the team knows about the gap and has decided not to close it, and a field that records a decision nobody intends to revisit is a tax. Named owner, named date, in the same document as the field.

7. **Find the one you already route around.**

   The mechanism. The postmortem template's Action Items table, with columns for the item, an owner and a target date. It is filled in every single time. The owner is almost

always whoever was on call, and the target date is almost always "next sprint". Pulling the last twenty postmortems: fifty-eight action items, and thirty-four were never done, of which nineteen have no corresponding ticket anywhere in the tracker.

The private model it was built to expose. That an incident implies specific work, owned by a named person and scheduled somewhere. More precisely, it was built to close the gap between the responder's model — "this is handled now" — and the planning process's model — "nothing entered the backlog". Those are the two models, and they are the same shape as the Exercise 1 pair: each is coherent, neither is stated, and nothing brings them into contact.

Which of the two things happened. Diverged silently, and the mechanism is now worse than nothing. A filled Action Items table is a signed statement that the follow-up is owned, and that statement is precisely what stops anyone asking again. Without the table, "who is fixing this?" remains an open question that somebody eventually asks in a planning meeting. With it, the question looks answered. The table converts an uncomfortable unknown into a comfortable falsehood, which is the specific way an externalising mechanism can do damage rather than merely waste time.

Verdict: rewrite. Deleting it loses the one thing it does well, which is producing the list at the moment the incident is fresh. The rewrite is a cross-reference requirement, and it is the chapter's dangling-reference check moved into a different system: an action item is not valid unless it carries the identifier of a ticket in the tracker, and a postmortem cannot be closed while any action item's ticket is neither in a scheduled sprint nor marked `wont-do` with a named decider. Both halves are deterministic. Both are one query against an API the team already calls.

Applying Exercise 6 to the rewrite, before shipping it. The cheapest string is now obvious: create a ticket titled the same as the action item, leave it in the backlog, mark the action item `wont-do`, and name yourself as the decider. Four clicks. The change that removes it: `wont-do` requires a decider who is not the author of the postmortem, which is checkable in one comparison and forces a thirty-second conversation with somebody who has capacity authority — which is the conversation the whole mechanism exists to cause. And the retire clause: if more than half of action items resolve to `wont-do` in a quarter, do not tighten the rule. The mechanism is correctly reporting that the team has no capacity for follow-up work, that is a staffing question, and a stricter postmortem template is the wrong instrument for it.